FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

Multi-Agent Reinforcement Learning Control of Renewable Energy Communities

Tiago Carlos Caló Fonseca



Programa Doutoral em Engenharia Informática

Supervisor: Luís Miguel Moreira Lino Ferreira, PhD

Co-Supervisor: Armando Jorge Sousa, PhD

December 2026

"For the things we have to learn before we can do them, we learn by doing them."

Aristotle

Nicomachean Ethics

© Tiago Carlos Caló Fonseca, 2026

Declaration of Honour

I declare that this thesis is my own work and has not previously been submitted and assessed at this or any other institution. References to other authors (statements, ideas, thoughts), as well as the use of published work, strictly comply with the rules of attribution and are duly identified in the text and bibliographical references in accordance with the applicable referencing standards. I am aware that plagiarism and self-plagiarism constitute academic misconduct, as established in the University of Porto Code of Ethics.

I further declare that artificial intelligence tools were used in the preparation of parts of this thesis, and that their use and the resulting material are duly identified and have been carefully checked for errors, inaccuracies, unfounded attributions, or other forms of bias in content and arguments, as well as for the determination of authorship.

Porto, _____ of _____ of 20_____

Tiago Carlos Caló Fonseca
(Signature of the Student)

Declaration on the Use of Generative Artificial Intelligence

Generative Artificial Intelligence (AI) supported the discussion of research and technical decisions throughout this work. Its use helped examine assumptions and compare alternative designs, particularly when translating research goals into software requirements and interfaces. It also assisted in identifying inconsistencies and edge cases before implementation. Further uses included research organisation, the preparation and revision of text, and support for software development, documentation, debugging, and testing.

Software development followed requirements and interfaces established before implementation, which provided the basis for reviewing proposed designs and code. Suggested references, numerical values, and interpretations were checked against the relevant primary sources and the project's data, code, tests, and experimental records. Material that did not withstand this review was revised or discarded.

The research questions and methods, experimental design and verification, and interpretation of results remained the author's responsibility, as did all final decisions concerning the software and the manuscript. The author is fully responsible for the originality and accuracy of this thesis, including its arguments and conclusions.

Statement on Co-Authored Work

The components presented in this thesis form part of a research architecture conceived and defined by the author. Several were developed in collaboration with supervisors, research colleagues, students, open-source contributors, and industrial partners. The author proposed and formulated the corresponding lines of work and contributed to their technical direction, development, supervision, and validation. Their integration into the wider project connected these collaborative efforts to the research questions addressed by the thesis.

The nature and extent of the collaboration varied between components. The following table identifies the collaborators, the author's contribution, and the location of each component in the thesis. The associated publications are listed separately under Publications and Research Outputs Arising from the Thesis.

Component or tool	**Co-author(s) or contributor(s)**	**Author's contribution**	**Thesis location**
[Title or name]	*[Name(s)]*	*[Concise contribution]*	*[Ch./Sec.]*
[Title or name]	*[Name(s)]*	*[Concise contribution]*	*[Ch./Sec.]*
[Title or name]	*[Name(s)]*	*[Concise contribution]*	*[Ch./Sec.]*
[Title or name]	*[Name(s)]*	*[Concise contribution]*	*[Ch./Sec.]*

Responsibility for the selection and verification of the material, the integration and interpretation of the collaborative work, and the scientific arguments and conclusions presented in this thesis rests with the author.

Funding

This work was supported by Fundação para a Ciência e a Tecnologia, I.P. (FCT), under the supervision and superintendence of the Ministry of Education, Science and Innovation, through doctoral grant 2024.00855.BD, financed by the Portuguese State Budget and co-financed by the European Social Fund Plus (FSE+) through the Programa Demografia, Qualificações e Inclusão (PDQI/PESSOAS 2030).

Part of the research presented in this thesis was developed within the Optimisation of Electric Vehicle Autonomy (OPEVA) project, which received funding within the Key Digital Technologies Joint Undertaking (KDT JU) from the European Union's Horizon Europe Programme and the National Authorities of France, Belgium, Czechia, Italy, Portugal, Turkey, and Switzerland, under grant agreement 101097267.

This work was also supported by the DEMFLEX operation (COMPETE2030-FEDER-01657200 – 19215), supported by the Innovation and Digital Transition Programme (COMPETE 2030), under Portugal 2030, and co-financed by the European Union.

The views and opinions expressed in this thesis are those of the author alone and do not necessarily reflect the views of FCT, the European Union, KDT JU, or the respective funding and granting authorities. Neither these institutions nor the granting authorities can be held responsible for them.

Acknowledgements

Sincere thanks go to the supervisor, Luís Miguel Moreira Lino Ferreira, PhD, and the co-supervisor, Armando Jorge Sousa, PhD, for their guidance and trust. Throughout many meetings, they made time to listen to my ideas, discuss different possibilities, and return to questions as the research developed. These conversations helped shape the work and were among the most meaningful parts of the doctoral experience.

Particular thanks are due to the industrial partners CleanWatts, especially Eng. Jorge Landeck, and i-Charging, especially Eng. Paulo Rodrigues. Their experience with energy systems and electric-vehicle charging brought a practical understanding of the constraints encountered during the work in OPEVA.

The students whose theses and research projects developed alongside this work made supervision a source of learning. Their questions opened up different ways of approaching related problems, and the opportunity to accompany their projects from an initial idea through development and validation was especially valuable.

Friends and colleagues at FEUP made the day-to-day experience of the doctorate easier to share. João Vitorino and Tiago Dias deserve a special mention for the conversations, shared frustrations, and laughter that accompanied these years.

My parents, Vera and Abílio, deserve a particularly warm word of thanks. They provided the stability and opportunities on which this path was built, long before the doctorate began. Their encouragement sustained choices that sometimes demanded patience and sacrifice from them, and their example taught the value of work, responsibility, and perseverance. The affection and presence of my grandparents were equally important, giving a sense of family that extended far beyond the years devoted to this thesis.

The final and most personal thanks go to Inês. She shared the achievements of the doctorate and lived closely with the pressure and exhaustion it brought into everyday life. There were periods when the work took up too much space, and its frustrations followed it home as impatience, distraction, or withdrawal. Her support continued through those periods, even when it was difficult to give. She made time to listen and helped restore perspective when a research problem seemed to occupy everything. She celebrated progress that was too easily dismissed, offered encouragement when confidence faltered, and knew when a challenge would help more than reassurance. Her belief remained steady through long periods of doubt and slow progress. The patience and affection behind that support meant a great deal to the completion of this work and to the shared life that continued around it. The order of a finished thesis conceals much of the uncertainty and repetition involved in writing it. Inês accompanied that less orderly part of the work too, with a patience and affection for which these pages offer only a small acknowledgement.

Nes, Pai, Mae, Aida, Vo Ek e Vo Fonso.

Abstract

Optimising the operation of Renewable Energy Communities (RECs) requires the coordinated management of distributed generation, storage and flexible demand, including electric-vehicle charging. The objective is to reduce energy costs and increase the use of local renewable generation while satisfying participant requirements and electrical constraints. Variability in production and consumption, temporal dependencies and changes in asset availability shape this optimisation problem. Community-wide decisions introduce further complexity by coupling the operation of individual buildings through shared resources and collective objectives.

Despite advances in optimisation and learning-based energy management, the application of Multi-Agent Reinforcement Learning (MARL) to real-world systems, such as renewable energy communities, raises unresolved challenges of scalability, deployability, safety and robustness. This thesis investigates how these requirements can be addressed jointly when system configurations change, measurements are delayed or incomplete, and policies execute on heterogeneous computing resources.

The proposed approach adopts decentralised control, with a local agent assigned to each participating building. An integrated research platform (EnergAIze) provides the experimental and operational foundation for the investigation, connecting data, simulation, controller development and deployment. A simulation environment (CityLearn v3), together with annual datasets at 15-minute resolution, supports reproducible evaluation of heterogeneous RECs under changing configurations and operating conditions. A common evaluation protocol compares rule-based control, optimisation, single-agent Reinforcement Learning (RL) and MARL across these settings.

The principal algorithmic contribution is a learning method that accommodates changes in the control interface, termed Typed-Interface Multi-Agent Reinforcement Learning (TI-MARL). A versioned typed contract specifies the entities observed and controlled; a deterministic compiler translates this description into the current observations and action groups, including their dependencies and bounds. This representation enables a policy shared across agents to operate locally as assets are reconfigured within known semantic types. An analytic projector enforces local action feasibility, while a variant with centralised training, Typed-Interface Multi-Agent Proximal Policy Optimisation (TI-MAPPO), uses a critic over the active agent population to support coordination during training.

The evaluation combines energy performance with assessments of reconfiguration, fault response, computational scalability and deployment compatibility, using two energy demonstrators (OPEVA and DemFlex) as application contexts. These contributions integrate REC energy optimisation with the preservation of control semantics from simulation to policy execution, supporting the application of MARL to distributed, heterogeneous and reconfigurable energy systems.

Keywords Multi-Agent Reinforcement Learning; Cyber-Physical Systems; Renewable Energy Communities; TI-MARL; Scalability; Safety and Robustness; Policy Lifecycle.

Resumo

A otimização do funcionamento das comunidades de energia renovável (RECs, do inglês Renewable Energy Communities) exige a gestão coordenada da produção distribuída, do armazenamento e do consumo flexível, incluindo o carregamento de veículos elétricos. O objetivo é reduzir os custos de energia e aumentar o aproveitamento da produção renovável local, respeitando as necessidades dos participantes e as restrições elétricas. A variabilidade da produção e do consumo, as dependências temporais e as alterações na disponibilidade dos ativos condicionam esta otimização. As decisões à escala da REC acrescentam complexidade ao interligarem o funcionamento dos edifícios através de recursos partilhados e objetivos coletivos.

Apesar dos avanços na otimização e aprendizagem para gestão de energia, a aplicação de aprendizagem por reforço multiagente (MARL, do inglês Multi-Agent Reinforcement Learning) a sistemas reais, como as RECs, enfrenta desafios de escalabilidade, capacidade de implantação, segurança e robustez. A tese investiga como satisfazer conjuntamente estes requisitos perante reconfigurações, medições tardias ou incompletas e execução em recursos computacionais heterogéneos.

A abordagem adota controlo descentralizado, com um agente por edifício participante. Uma plataforma integrada de investigação (EnergAIze) articula dados, simulação, desenvolvimento de controladores e implantação. Um ambiente de simulação (CityLearn v3) e dados anuais com resolução de 15 minutos permitem avaliar RECs heterogéneas, de forma reprodutível, sob diferentes configurações e condições de funcionamento. Um protocolo comum compara controlo baseado em regras, otimização, aprendizagem por reforço com um único agente e MARL.

A principal contribuição algorítmica, Typed-Interface Multi-Agent Reinforcement Learning (TI-MARL), acomoda alterações na interface de controlo. Um contrato tipado e versionado especifica as entidades observadas e controladas; um compilador determinístico traduz essa descrição nas observações e nos grupos de ação disponíveis, com as respetivas dependências e limites. Uma política partilhada entre agentes atua localmente perante reconfigurações entre tipos semânticos conhecidos. Um projetor analítico assegura a viabilidade local das ações, enquanto uma variante com treino centralizado, Typed-Interface Multi-Agent Proximal Policy Optimisation (TI-MAPPO), cujo crítico considera os agentes ativos, apoia a coordenação durante o treino.

A avaliação combina o desempenho energético com a análise da reconfiguração, da resposta a falhas, da escalabilidade computacional e da compatibilidade de execução em diferentes plataformas, tendo dois demonstradores energéticos (OPEVA e DemFlex) como contextos de aplicação. Estas contribuições articulam a otimização energética das RECs com a preservação da semântica de controlo, desde a simulação até à execução das políticas, sustentando a aplicação de MARL a sistemas energéticos distribuídos, heterogéneos e reconfiguráveis.

Palavras-chave Aprendizagem por Reforço Multiagente; Sistemas Ciberfísicos; Comunidades de Energia Renovável; TI-MARL; Escalabilidade; Segurança e Robustez; Ciclo de Vida de Políticas.

Publications and Research Outputs Arising from the Thesis

The following references group the publications associated with this thesis by publication type and identify the chapters to which they principally relate. Submitted manuscripts, preprints, planned articles, software and datasets are listed separately. Yellow rows identify planned articles that have not yet been completed or submitted. Their titles are provisional, with Tiago Fonseca as first author and the remaining authors and publication venues to be confirmed.

ID	Bibliographic reference	Chapter
Journal articles		
P1	**T. Fonseca**, L. L. Ferreira, J. Landeck, L. Klein, P. Sousa and F. Ahmed (2022). Flexible Loads Scheduling Algorithms for Renewable Energy Communities. *Energies*, 15(23), 8875. DOI.	2, 6
P2	**T. Fonseca**, L. L. Ferreira, B. Cabral, R. Severino, K. Nweye, D. Ghose and Z. Nagy (2025). EVLearn: Extending the CityLearn Framework with Electric Vehicle Simulation. *Energy Informatics*, 8, 16. DOI.	5
P3	K. Nweye, K. Kaspar, G. Buscemi, **T. Fonseca**, G. Pinto, D. Ghose, S. Duddukuru, P. Pratapa, H. Li, J. Mohammadi, L. L. Ferreira, T. Hong, M. Ouf, A. Capozzoli and Z. Nagy (2025). CityLearn v2: Energy-Flexible, Resilient, Occupant-Centric, and Carbon-Aware Management of Grid-Interactive Communities. *Journal of Building Performance Simulation*, 18(1), 17–38. DOI.	5
P4	**T. Fonseca**, C. Sousa, L. Ferreira, P. Rodrigues, P. Paiva, R. Venâncio, R. Severino and L. Matos (2026). Can Intelligent Renewable Energy Communities Deliver on Equity for a Just Energy Transition? A Policy Oriented Demonstrator Analysis. *Energy Research & Social Science*, 133, 104584. DOI.	8
Conference papers and communications		
P5	**T. Fonseca**, L. L. Ferreira, L. Klein, J. Landeck and P. Sousa (2022). Flexigy Smart-grid Architecture. In *Proceedings of the 11th International Conference on Sensor Networks (SENSORNETS)*, pp. 176–183. SciTePress. DOI.	4, 6
P6	**T. Fonseca**, L. L. Ferreira, B. Cabral, R. Severino and I. Praça (2024). EnergAIze: Multi Agent Deep Deterministic Policy Gradient for Vehicle-to-Grid Energy Management. In *IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm)*, pp. 232–237. IEEE. DOI.	4, 6, 8
P7	B. Cabral, R. Venâncio, P. Costa, **T. Fonseca**, L. L. Ferreira, R. Severino and A. Barros (2024). Multiprotocol Middleware Translator for IoT. In *27th Euromicro Conference on Digital System Design (DSD)*, pp. 327–334. IEEE. DOI.	4

ID	Bibliographic reference	Chapter
P8	**T. Fonseca**, C. Sousa, R. Venâncio, P. Pires, R. Severino, P. Rodrigues, P. Paiva and L. L. Ferreira (2025). Control of Renewable Energy Communities using AI and Real-World Data. In *IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA)*, pp. 1–8. IEEE. DOI.	6, 8
P9	C. Sousa, **T. Fonseca**, L. L. Ferreira, R. Venâncio and R. Severino (2025). Percepta: High Performance Stream Processing at the Edge. In *IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA)*, pp. 1–4. IEEE. DOI.	4, 8
P10	R. Venâncio, C. A. Sousa, P. Pires, **T. Fonseca** and L. L. Ferreira (2025). PulseCharge: Enabling Interoperability in EV Data. *16th INForum – Simpósio de Informática*, Évora, Portugal. Conference communication. Programme; Manuscript.	4, 8
Submitted manuscripts and preprints		
P11	**T. Fonseca**, L. L. Ferreira, A. Sousa, A. Mohammadi and Z. Nagy (2026). CityLearn v3: A Configurable Simulation and Evaluation Framework for Realistic Control Studies of Renewable Energy Communities. Submitted manuscript; arXiv:2609.21570. Preprint.	5
P12	B. Cabral, **T. Fonseca**, C. Sousa and L. L. Ferreira (2024). FlexiGen: Stochastic Dataset Generator for Electric Vehicle Charging Energy Flexibility. Preprint, arXiv:2411.07040. Preprint.	5
Planned articles		
P13	**T. Fonseca** (co-authors to be confirmed). TI-MARL: Typed-Interface Multi-Agent Reinforcement Learning for Reconfigurable Cyber-Physical Systems.	7
P14	**T. Fonseca** (co-authors to be confirmed). Comparative Evaluation of Control Strategies for Renewable Energy Communities.	6
P15	**T. Fonseca** (co-authors to be confirmed). EnergAIze: An Integrated Architecture for Renewable Energy Community Control.	4, 8
P16	**T. Fonseca** (co-authors to be confirmed). Datasets for Control of Heterogeneous Renewable Energy Communities.	5
P17	**T. Fonseca** (co-authors to be confirmed). Offline Reinforcement Learning and Policy Reuse in Renewable Energy Communities.	6
P18	**T. Fonseca** (co-authors to be confirmed). Multi-Agent Control of Renewable Energy Communities with Variable Asset Configurations.	6
P19	**T. Fonseca** (co-authors to be confirmed). Hierarchical Coordination of Local Controllers in Renewable Energy Communities.	6
P20	**T. Fonseca** (co-authors to be confirmed). Multi-Agent Reinforcement Learning for Renewable Energy Communities: A Review of Simulation and Real-World Operation.	3
Software and datasets		
P21	CityLearn Project (2026). *CityLearn v3*. Open-source simulation software, documentation and benchmarks. Release history.	4–8
P22	*Renewable Energy Community datasets*. Heterogeneous assets and operating scenarios at 15-minute resolution. Dataset release; creators and public record to be confirmed.	5–7

Provide the creators and public location of the datasets in P22.

Contents

Declaration of Honour i

Declaration on the Use of Generative Artificial Intelligence iii

Statement on Co-Authored Work v

Funding vii

Acknowledgements ix

Abstract xi

Resumo xiii

Publications and Research Outputs Arising from the Thesis xv

List of Acronyms xxxiii

Glossary xxxix

1 Introduction 1

1.1 Context and Motivation . . . 1

1.1.1 Multi-Agent Learning and Control . . . 2

1.1.2 Energy Management and Flexibility in Homes . . . 3

1.1.3 Renewable Energy Communities . . . 4

1.1.4 Institutional Setting and Partnerships . . . 5

1.2 Research Problem . . . 6

1.2.1 Real-World Challenges . . . 6

1.2.2 Lexical Grounding . . . 9

1.3 Thesis Statement and Research Questions . . . 10

1.3.1 Thesis Statement . . . 10

1.3.2 Research Questions . . . 10

1.4 Research Contributions and Outcomes . . . 11

1.4.1 Scientific Contributions . . . 11

1.4.2 Software and Data Contributions . . . 13

1.4.3 Societal and Environmental Relevance . . . 14

1.5 Thesis Organisation . . . 14

2 Foundations **15**
2.1 Renewable Energy Communities . . . 15
2.1.1 Community Members . . . 15
2.1.2 Asset Operation and Flexibility . . . 18
2.1.3 Energy Markets and Decision Signals . . . 21
2.1.4 Information and Operational Conditions . . . 22
2.2 Neural Networks and Learned Representations . . . 23
2.3 Reinforcement Learning . . . 25
2.3.1 Sequential Decision-Making and Partial Observability . . . 25
2.3.2 Returns, Episodes, Value Functions, and Policies . . . 27
2.3.3 Reward Design and Objective Specification . . . 28
2.3.4 Learning from Experience . . . 29
2.3.5 Prior Knowledge and Constrained Decisions . . . 31
2.4 Multi-Agent Reinforcement Learning . . . 32
2.4.1 Markov Games and Multi-Agent Objectives . . . 32
2.4.2 Learning and Execution Organisation . . . 33
2.4.3 Coordination and Credit Assignment . . . 35
2.5 Offline Reinforcement Learning . . . 35
2.6 Hierarchical Reinforcement Learning and Coordination . . . 37
2.6.1 Decision Levels and Temporal Abstraction . . . 37
2.6.2 Goals, Skills, and Coordination Signals . . . 37
2.7 Structured Policy Architectures . . . 39
2.7.1 Entity and Relational Representations . . . 39
2.7.2 Attention and Transformer Architectures . . . 41
2.7.3 Modular Policies and Action Decoding . . . 42
2.8 Chapter Summary . . . 44

3 Related Work and Research Gaps **45**
3.1 Scope and Review Method . . . 45
3.1.1 Search Terms and Sources . . . 46
3.1.2 Eligibility Criteria and Study Selection . . . 47
3.2 MARL Applications and Real-World Challenges . . . 47
3.2.1 Simulation-Based Evidence . . . 48
3.2.2 Real-World Evidence . . . 49
3.2.3 Remaining Research Gaps . . . 52
3.3 Scalability and Dynamic Reconfiguration . . . 55
3.3.1 Learning and Coordination at Scale . . . 55
3.3.2 Architectures for Variable Agent and Resource Configurations . . . 58
3.3.3 Comparative Analysis and Remaining Questions . . . 61
3.4 Deployability across Heterogeneous Installations . . . 62
3.4.1 Learning from Available Data and Prior Knowledge . . . 62
3.4.2 Policy Reuse and Adaptation across Installations . . . 64
3.4.3 System Integration and Controller Lifecycle . . . 66
3.4.4 Comparative Analysis and Remaining Questions . . . 69
3.5 Physical and Operational Safety . . . 70
3.5.1 Constraint Representation in Learning and Control . . . 70
3.5.2 Action Protection and Supervisory Enforcement . . . 71
3.5.3 Coordinated Physical Limits and Service Feasibility . . . 74

3.5.4 Comparative Analysis and Remaining Questions . . . 75
3.6 Robustness, Degraded Operation, and Recovery . . . 77
3.6.1 Learning under Uncertainty and Disturbance . . . 78
3.6.2 Fault Handling, Degraded Operation, and Recovery . . . 79
3.6.3 Evidence under Operational Disturbances . . . 81
3.6.4 Comparative Analysis and Remaining Questions . . . 84
3.7 Chapter Summary . . . 84

4 An Architecture for Real-World Community Energy Control: EnergAIze 87
4.1 Energy Control in Buildings and Communities . . . 87
4.2 Runtime Architecture . . . 90
4.2.1 The Controller and Its Inference Service . . . 92
4.2.2 Data Integration and Routing: Percepta . . . 93
4.2.3 Vehicle Information: PulseCharge . . . 93
4.2.4 Production and Consumption Forecasting . . . 93
4.2.5 Flexibility Forecasting: Flexon . . . 94
4.2.6 Action Validation: Fail-Safe Supervisor . . . 94
4.2.7 Service Monitoring: Watchdog . . . 95
4.2.8 Operational Records, User Interfaces, and KPI Manager . . . 95
4.3 Controller Development . . . 96
4.3.1 Data Preparation and Simulation Scenarios . . . 96
4.3.2 Controller Training and Evaluation . . . 98
4.4 Experiment Orchestration and Controller Deployment . . . 99
4.4.1 Infrastructure and Experiment Configuration . . . 99
4.4.2 Execution on Heterogeneous Computing Resources . . . 101
4.4.3 Job Progress, Results, and Recovery . . . 102
4.4.4 The Model Contract . . . 103
4.4.5 Controller Deployment and Updates . . . 104
4.5 Chapter Summary . . . 105

5 Simulation and Evaluation of Renewable Energy Communities: CityLearn v3 107
5.1 Development of the Simulation Environment . . . 107
5.2 Community Modelling and Simulator Architecture . . . 108
5.2.1 Scenarios, Time, and the Simulation Step . . . 109
5.2.2 Resources, Electrical Limits, and Participant Services . . . 110
5.2.3 Local Sharing and Demand-Response Services . . . 112
5.2.4 Changing Participation and Operating Conditions . . . 114
5.2.5 Controller Interfaces and Multiple Communities . . . 116
5.3 Community Datasets and Experimental Scenarios . . . 117
5.3.1 Construction and Common Assumptions . . . 117
5.3.2 Community Bases and Their Experimental Roles . . . 118
5.3.3 Scenario Variants and the Questions They Address . . . 121
5.3.4 Data Characterisation and Quality . . . 122
5.4 Performance Indicators and Evaluation Protocol . . . 123
5.4.1 Energy, Cost, and Community Outcomes . . . 123
5.4.2 Service Delivery, Safety, and Robustness . . . 125
5.4.3 Scorecard, References, and Aggregation . . . 127
5.4.4 Temporal Protocol and Reproducible Comparisons . . . 129

5.5 Chapter Summary . . . 131

6 Development and Evaluation of Community Energy Controllers 133
6.1 Business-as-Usual References . . . 133
6.1.1 BAU with and without Storage . . . 134
6.1.2 Annual Operation across Community Scenarios . . . 135
6.2 Rule-Based Control and Optimisation . . . 137
6.2.1 Rule-Based Control and Its Variants . . . 138
6.2.2 Optimisation over a Scheduling Horizon . . . 139
6.3 Local Reinforcement Learning . . . 142
6.3.1 The Building as a Learning Agent . . . 142
6.3.2 PPO and TD3 as Local Alternatives . . . 144
6.4 Multi-Agent Reinforcement Learning . . . 145
6.4.1 Local Actors and Centralised Training . . . 145
6.4.2 Multi-Agent Actor–Critic Methods . . . 147
6.4.3 Comparative Evaluation . . . 148
6.5 Controller Design and Training Strategies . . . 151
6.5.1 Observation and Action Design . . . 151
6.5.2 Guided Exploration, Demonstrations, and Residual Control . . . 151
6.5.3 Reward Design and Experience Sampling . . . 153
6.5.4 Controller Variants and Ablation Studies . . . 154
6.6 Offline Reinforcement Learning . . . 155
6.6.1 Operational Data for Policy Learning . . . 156
6.6.2 Offline Value Learning and Dataset Coverage . . . 156
6.7 Hierarchical Community Coordination . . . 157
6.7.1 Separating Community and Building Decisions . . . 158
6.7.2 Community Benefits and Their Distribution . . . 159
6.8 Control under Resource Reconfiguration . . . 160
6.8.1 Availability Changes and Structural Changes . . . 161
6.8.2 Variable-Size Policies and Their Operational Scope . . . 162
6.9 State-of-the-Art Methods for Real-World Control . . . 163
6.9.1 Task-Open Coordination and Compositional Learning . . . 164
6.9.2 Constraint Handling and Fault-Tolerant Learning . . . 165
6.9.3 Common Control Task and Comparative Assessment . . . 165
6.10 Comparative Discussion . . . 167
6.10.1 Comparison Scope and Interpretation . . . 167
6.10.2 Economic Performance, Service Delivery, and Safety . . . 169
6.10.3 Computational Demand, Policy Reuse, and Deployability . . . 171
6.10.4 Robustness and Resource Reconfiguration . . . 173
6.10.5 The Remaining Research Problem . . . 176
6.10.6 Chapter Summary . . . 178

7 Typed-Interface Multi-Agent Reinforcement Learning: TI-MARL 179
7.1 Problem Formulation and Method Overview . . . 179
7.1.1 Typed-Interface Decision Processes . . . 180
7.1.2 Method Overview . . . 182
7.2 Typed Interfaces and Runtime Compilation . . . 183
7.2.1 Schemas, Entities, and Dynamic Module Instances . . . 183

7.2.2 Typed Observation Parts and Community Context . . . 184
7.2.3 Dynamic Action Groups and Typed Parameterised Ports . . . 185
7.3 Policy Architecture and Multi-Agent Learning . . . 187
7.3.1 Local Information and Hierarchical Encoding . . . 187
7.3.2 Dynamic Grouped Action Decoding . . . 188
7.3.3 Centralised Training and Local Execution . . . 189
7.3.4 Proximal Policy Optimisation with Typed Interfaces . . . 190
7.3.5 Training Data and Time Resolution . . . 191
7.4 Constraint Handling and Degraded Operation . . . 192
7.4.1 Health-Aware Causal Compilation . . . 192
7.4.2 Constraint-Aware Local Bundle Construction . . . 193
7.4.3 Shared Limits and Operational Supervision . . . 194
7.5 Compatibility and Deployment . . . 195
7.5.1 Interface Snapshots and Admission . . . 195
7.5.2 Checkpoint, Export, and Actor-Only Handoff . . . 196
7.5.3 Execution-Level Decision Traces . . . 196
7.5.4 Formal and Operational Properties . . . 197
7.6 Experimental Methodology . . . 198
7.6.1 Reference Methods and Attribution . . . 198
7.6.2 Annual Scenarios and Event Comparisons . . . 198
7.6.3 Measurements across the Four Research Dimensions . . . 200
7.6.4 Training, Selection, and Paired Evaluation . . . 200
7.7 Results and Comparative Analysis . . . 201
7.7.1 Interface and Learning Implementation . . . 201
7.7.2 Comparative Questions and Evidence Scope . . . 202
7.8 Discussion . . . 202
7.8.1 Relationship to State-of-the-Art Methods . . . 203
7.8.2 Scalability and Deployability . . . 203
7.8.3 Safety and Robustness . . . 204
7.8.4 Scope of the Contribution . . . 205
7.9 Chapter Summary . . . 205

8 Real-World Deployment and Validation 207
8.1 Evidence Levels and Comparative Reading . . . 207
8.2 OPEVA Demonstrator . . . 208
8.2.1 Control Problem and Physical Configuration . . . 208
8.2.2 Integration with the Energy-Control Framework . . . 209
8.2.3 Scenarios and Validation Strategy . . . 210
8.2.4 Results and Demonstrated Boundary . . . 211
8.3 DemFlex Demonstrator . . . 212
8.4 Cross-Demonstrator Lessons and Simulation-to-Operation Differences . . . 212
8.5 Contribution to the Research Questions and Chapter Summary . . . 214

9 General Discussion and Answers to the Research Questions 215
9.1 What the Combined Evidence Establishes . . . 216
9.2 Answer to RQ1: Scalability of Heterogeneous Multi-Agent Control . . . 217
9.3 Answer to RQ2: Deployability and Governed MARL Lifecycle . . . 219
9.4 Answer to RQ3: Physical and Operational Safety . . . 220

9.5 Answer to RQ4: Robustness to Real-World Disturbances . . . 220
9.6 Interactions among the Four Requirements . . . 221
9.7 Scientific Contributions, Generality, and Responsible Use . . . 222
9.8 Limitations and Threats to Validity . . . 223
9.9 Implications and Open Research Problems . . . 224

10 Conclusions and Future Research 225
10.1 What the Research Established . . . 225
10.2 Answers to the Research Questions . . . 227
10.3 Principal Contributions . . . 228
10.4 Assessment of the Central Statement . . . 230
10.5 Limits of the Conclusions . . . 230
10.6 Future Research Directions . . . 231
10.7 Closing Remarks . . . 232

References 233

A Simulation Studies of Community Energy Management 253

B Reproducibility and Supplementary Material 259
B.1 Claim and Evidence Inventory . . . 259
B.2 Scenario and Environment Manifests . . . 259
B.3 Portugal 2023–2024 Dataset Analysis and Audit . . . 259
B.3.1 Source Data, Preparation, and Reproduction . . . 260
B.3.2 Resource Diversity and Energy Profiles . . . 261
B.3.3 Charging and Appliance-Service Distributions . . . 262
B.3.4 Events, Forecasts, and Experimental Boundaries . . . 263
B.3.5 Audit Results and Evaluation Record . . . 265
B.4 Historical Annual Dataset Contract . . . 265
B.4.1 Calendar, price forecasts, and settlement . . . 266
B.4.2 Demand, PV, and stationary storage . . . 266
B.4.3 Electrical connections and phase allocation . . . 267
B.4.4 EVs, sessions, and deferrable services . . . 269
B.4.5 Topology and robustness calendars . . . 270
B.4.6 Matched annual baseline scorecard . . . 272
B.5 Simulator Audits and Demonstrative Results . . . 273
B.5.1 Software and contract verification . . . 274
B.5.2 Selected capability demonstrations . . . 274
B.5.3 Performance and scalability smoke benchmarks . . . 275
B.6 Simulator KPI Catalogue . . . 276
B.7 Controller, TI-RL, and TI-MARL Configurations . . . 278
B.7.1 Executable Controller Inventory . . . 278
B.7.2 Observation, Action, and Training Record . . . 279
B.7.3 TI-MARL Reproducibility Contract . . . 280
B.7.4 Reward Profile Ledger . . . 281
B.8 Hyperparameters, Seeds, and Statistical Procedures . . . 284
B.9 Supplementary and Negative Results . . . 284
B.10 Lifecycle and Deployment Records . . . 284

B.11 Software, Data, and Archival Record 284
B.12 Annual BAU Comparisons and Reference Diagnostics 284
B.12.1 Development-Year Annual Comparison 285
B.12.2 Companion Service and Execution Records 287
B.12.3 Real Short-Window Reference Diagnostics 287
B.12.4 Complete Annual Controller Scorecards 288

List of Figures

2.1 Levels of organisation around a REC . . . 17
2.2 Start-time and energy flexibility in a FlexOffer . . . 19
2.3 Fixed, shiftable, and elastic assets . . . 20
2.4 A feed-forward network mapping three charging-related features through a learned representation to one output. The illustrated layer widths are architectural choices. Original illustration based on Goodfellow et al. (2016). . . . 24
2.5 Agent–environment interaction in a fully observable MDP. The reward r_t follows action a_t, and the next state s_{t+1} becomes available to the agent. Original illustration based on the formulation of Sutton and Barto (2018). . . . 26
2.6 Centralised training and decentralised execution for two sites. Joint observations and actions support policy updates during training; local inputs determine charging decisions during execution. Original illustration based on Lowe et al. (2017). . . . 34
2.7 Offline learning uses a fixed dataset collected beforehand. The trained policy subsequently receives new observations and selects actions. . . . 36
2.8 One REC signal guides successive local decisions. Each site combines it with its own charging conditions to select a power request. . . . 38
2.9 Three representations of the same site resources. Vector positions identify feature groups, a set groups attributes by entity, and a graph includes declared relationships. The lower row illustrates adding another charger record. . . . 40
2.10 Attention and a Transformer block. An attention head computes weights and combines values. Multiple heads form the attention sublayer of the illustrated block; residual paths feed the additions before layer normalisation. Original illustration based on Vaswani et al. (2017). . . . 42
2.11 Shared decoding with resource-specific outputs. The same charger decoder processes the representations of C1 and C2, producing a separate power request for each. A different decoder produces the appliance's discrete choice. Repeated decoder symbols denote shared parameters. . . . 43

3.1 Review stages and methodological sources. . . . 45
3.2 Critic information and actor updates in MADDPG, MATD3, and MAPPO, followed by their common local execution pattern. Critic-to-actor arrows indicate learning signals. Original schematic based on Ackermann et al. (2019), Lowe et al. (2017), and Yu, Velu, et al. (2022). . . . 56

4.1 Information and actions in EnergAIze. . . . 88
4.2 Building controllers within a REC. . . . 89
4.3 Runtime architecture of EnergAIze. . . . 91

4.4 Controller development through simulation and evaluation. 97
4.5 Computing infrastructure for experiment orchestration. 100
4.6 Model deployment and the runtime control loop. 105

5.1 Representation of one or more RECs in CityLearn v3. Reproduced from Fonseca, Ferreira, Sousa, et al. (2026). 109
5.2 Peak visibility at three temporal resolutions of the same synthetic annual trace. . 111
5.3 A complete appliance programme scheduled within its admissible start window. . 112
5.4 REC and participant outcomes under two accounting rules for the same historical trajectory. 113
5.5 Propagation of membership and equipment events through the simulated resource collection. 115
5.6 Separate observation, forecast, command, and equipment-availability disturbances in the diagnostic run. 116
5.7 Monthly daily-average input demand and PV production in the four static REC bases. 120
5.8 Core-30 daily input and vehicle-presence profiles: Monday–Friday means in winter (December–February) and summer (June–August) of 2024, in Lisbon local time. 121

6.1 The building-level learning loop. Resource commands share one local decision process; learning uses the resulting experience. 144
6.2 Joint information supports critic learning; the trained actors make building-level decisions from local inputs. 146
6.3 **Illustrative values; invented.** Cost comparison for selected controllers in the Core-30 Normal manuscript example. 150
6.4 **Illustrative values; invented.** MATD3 configurations with different uses of prior knowledge. Correction denotes an eligible building decision with at least one actually adjusted command. All outcomes use the full Core-30 Normal 2024 test-window illustration. 153
6.5 A REC signal influences local scheduling. Resource commands remain with the building controllers. 158
6.6 A change in resource count requires matching observations and commands. The same learned representation can be reused for compatible instances of known resource types. 161
6.7 **Illustrative values; invented.** Peak training memory in GB and full 2024 test-window cost in EUR. Methods are paired within Core-30 Normal and Premium-100 Normal; the absolute magnitudes are invented. 171

7.1 TI-MARL decision path within each building. Local construction precedes execution; the REC critic participates only in training. 182

8.1 Validation evidence levels and the claims supported by each level. 208
8.2 OPEVA Demo 9 sites, assets, platforms, and EnergAIze integration path. 210
8.3 Timeline of one audited OPEVA control or integration period. 212
8.4 Observed differences between simulation assumptions and demonstrator conditions. 213
8.5 Verified OPEVA and DemFlex results by evidence level. 213

9.1 Research-question, contribution, finding, and evidence map. 221
9.2 Interactions among scalability, deployability, safety, and robustness. 222

10.1 Contribution stack and cumulative thesis argument. 229

B.1 Member-level non-shiftable demand and PV production in 2024. Each point represents one member of a complete static base; the dotted line denotes equal annual demand and production. 262
B.2 Complete-session departure targets and appliance start-window slack in the two-year collection. Boxes show the interquartile range and median; whiskers reach the furthest values within 1.5 interquartile ranges. Outliers are omitted. 263

List of Tables

2.1 Energy-management objectives at different organisational levels 17
2.2 Availability, health, dependencies, and degraded operation 22
2.3 Three organisational choices in multi-agent learning and execution. 34

3.1 Term families used in the application and mechanism searches. 46
3.2 Eligibility criteria for the chapter's review protocol. 47
3.3 Real-world evidence in community-relevant energy applications 50
3.4 Established evidence and remaining questions for community application. 54
3.5 Capabilities established by the studies reviewed in Section 3.3.2. 61
3.6 Mechanisms for reducing controller preparation and the work retained at a target installation. 67
3.7 Evaluation settings for controller preparation, reuse, and integration. 69
3.8 Mechanisms for representing and enforcing control constraints. 73
3.9 Energy-control evidence for physical protection and service delivery. 76
3.10 Mechanisms for maintaining and recovering control under disturbance. 81
3.11 Evidence for continued operation and recovery under disturbance. 83
3.12 Literature-based answers to the RQs and remaining gaps pursued in the thesis. . . 86

4.1 Computing resources and their execution adapters. 102
4.2 Information carried by the model contract. 104

5.1 Complete physical inventories in the Portugal 2023–2024 collection. 119
5.2 Purpose of the scenario variants and their reference comparisons. 122
5.3 Common scorecard. Physical and monetary quantities retain their absolute units; percentages describe service, utilisation, or intervention directly. 127
5.4 Temporal separation for the principal controller comparisons. 129

6.1 Conventional operation represented by the two BAU variants. 134
6.2 **Illustrative values; invented.** Energy and economic outcomes for the two BAU variants over the complete 2024 window. 136
6.3 **Illustrative values; invented.** Departure service, safety, and storage use for the two BAU variants over the complete 2024 window. 137
6.4 Rule-based family: additional information, purpose, and operational dependence. 139
6.5 Interpretation of the optimisation references. Information horizon is declared separately from decision scope. 141
6.6 **Illustrative values; invented.** Perfect-information scheduling over the complete Core-30 Normal test year. 141

6.7 **Illustrative values; invented.** Full 2024 test-window illustration, Core-30 Normal. Ordinary operation and rule-based references. . . . 142
6.8 Local observations and their control purpose. The selected feature profile and normalisation are retained with the experiment. . . . 143
6.9 **Illustrative values; invented.** Full 2024 test-window illustration, Core-30 Normal. Local learning against the strict local rule. . . . 145
6.10 Learning alternatives and the mechanism each comparison examines. All actors retain their declared local execution inputs. . . . 148
6.11 **Illustrative values; invented.** Full 2024 test-window illustration, Core-30 Normal. Common scorecard, energy and economic outcomes. . . . 149
6.12 **Illustrative values; invented.** Full 2024 test-window illustration, Core-30 Normal. Common scorecard, service, safety, and resource use. . . . 150
6.13 **Illustrative values; invented.** Full 2024 test-window illustration, Core-30 Normal. MATD3 configurations using different forms of prior knowledge. . . . 152
6.14 Learning difficulties and the mechanisms investigated to address them. Configuration versions and numerical parameters belong to Appendix B.7. . . . 154
6.15 Mechanism comparisons that separate controller design choices. These are comparison definitions, not completed numerical results. . . . 155
6.16 **Illustrative values; invented.** Offline-policy illustration against its behaviour-policy reference, Core-30 Normal. . . . 157
6.17 **Illustrative values; invented.** Full 2024 test-window illustration, Core-30 Normal. Hierarchical coordination, energy and economic outcomes. . . . 159
6.18 **Illustrative values; invented.** Full 2024 test-window illustration, Core-30 Normal. Service and distribution of coordination benefits. . . . 160
6.19 **Illustrative values; invented.** Full 2024 test-window illustration, Core-30 Normal. Nominal performance before a change of resource configuration. . . . 162
6.20 **Illustrative values; invented.** Annual operation through Core-30 Topology changes. 163
6.21 Algorithmic references for the real-world control problem and their comparative role. 166
6.22 **Illustrative values; invented.** Annual energy-purchase cost [EUR] across the seven scenarios. . . . 168
6.23 **Illustrative values; invented.** Annual minimum EV service [%] across the seven scenarios. . . . 169
6.24 **Illustrative values; invented.** Full 2024 test-window illustration, Core-30 Safety. Nominal correction rates retain the matched Normal reference. . . . 170
6.25 **Illustrative values; invented.** Missed EV service over the full 2024 test-window illustration. Paired Core-30 Normal and Safety settings. . . . 170
6.26 **Illustrative values; invented.** Computational demand in the two Normal RECs. . 172
6.27 **Illustrative values; invented.** Preparation and execution of the three controller packages in the same Core-30 installation. . . . 173
6.28 **Illustrative values; invented.** Full 2024 test-window illustration. Paired Core-30 Normal and Robustness settings. . . . 174
6.29 **Illustrative values; invented.** Missed EV service over the full 2024 test-window illustration. Paired Core-30 Normal and Robustness settings. . . . 175
6.30 **Illustrative values; invented.** Runtime and service outcomes over the complete Core-30 Robustness year. . . . 175
6.31 **Illustrative values; invented.** Operational DR in Premium-100 All-in, complete 2024 illustration. . . . 176

6.32 Method assumptions that motivate the next stage of the investigation. These are qualitative design questions, independent of the illustrative numerical rankings. . 177

7.1 Main TI-MARL components and their interpretation in the running energy example. 183
7.2 Representative typed observation parts for a building with two chargers. Values are illustrative. 185
7.3 Representative action ports for two charger instances and one stationary battery. . 186
7.4 Matched comparisons for attributing TI-MARL behaviour. 199
7.5 Mechanisms and complementary evidence for the four research dimensions. . . . 200
7.6 TI-MARL behaviour established by the focused 139-test regression suite. 202
7.7 Relationships to neighbouring mechanisms and the specific TI-MARL comparison. 204

8.1 OPEVA Demo 9 scenarios and the evidence boundary reported in deliverable D6.3. 211

9.1 Role of the principal evidence layers in the thesis argument. 217

A.1 Simulation studies of community energy management 253

B.1 Generation, storage, and appliance inventories in the complete static bases. 261
B.2 Annual input energy in the complete static bases, in MWh. 261
B.3 Charging-session counts and service distributions across the two-year catalogues. 263
B.4 Selected forecast mean absolute errors in the stored inputs. 264
B.5 Verification records for the current Portugal collection. 265
B.6 Static-asset generation contract by member class. PV and BESS entries apply only to members that own the corresponding asset. 267
B.7 Routine archetypes used to construct heterogeneous demand and flexibility. . . . 267
B.8 Electrical-service and charger contract. Charger powers are compatible with the charging classes described by ERSE (Entidade Reguladora dos Serviços Energéticos 2026b). 268
B.9 Generated EV identities, annual session counts, and topology-observable departure obligations. A censored session is absent from the controller's service denominator because its member or physical charger does not exist at the relevant boundary. . 270
B.10 Core-30 dynamic topology calendar. Paired dates denote removal and restoration. 271
B.11 Premium-100 dynamic topology calendar. Member ranges denote simultaneous first-entry cohorts. 271
B.12 Annual matched cost and service scorecard. Paired entries are BAU/RBC Smart. . 272
B.13 Annual safety, demand-response, and robustness scorecard for the stress variants. Paired entries are BAU/RBC Smart. 273
B.14 Verification evidence for the June 2026 simulator-study snapshot and the separately identified historical annual suite. 274
B.15 Selected application evidence and the bounded claim supported by each example. 275
B.16 Computational-cost evidence from matched simulator configurations. 276
B.17 Expanded CityLearn v3 KPI catalogue. 276
B.18 Executable Algorithms registry grouped by scientific role. 278
B.19 TI-MARL v1 configuration and evidence fields retained for reproduction. 280
B.20 Implemented Chapter 5 reward-profile lineage. 281
B.21 **Illustrative values; invented.** Energy and economic outcomes for the two BAU variants over the complete 2023 window. 285

B.22 **Illustrative values; invented.** Departure service, safety, and storage use for the two BAU variants over the complete 2023 window. 286
B.23 **Measured short-window BAU diagnostic.** Portugal v3.1, 671 executed fifteen-minute steps from 1 January 2023 00:00 through 7 January 2023 23:30. Cost and import sum member-level purchases; peak uses REC net power. 287
B.24 **Illustrative values; invented.** Annual energy-purchase cost [EUR] of the additional training and hierarchy variants. 288
B.25 **Illustrative values; invented.** Core-30 Normal. Energy and economic outcomes. 289
B.26 **Illustrative values; invented.** Core-30 Normal. Service, safety and resource use. 289
B.27 **Illustrative values; invented.** Core-30 Topology. Energy and economic outcomes. 290
B.28 **Illustrative values; invented.** Core-30 Topology. Service, safety and resource use. 290
B.29 **Illustrative values; invented.** Core-30 Safety. Energy and economic outcomes. . 291
B.30 **Illustrative values; invented.** Core-30 Safety. Service, safety and resource use. . 291
B.31 **Illustrative values; invented.** Core-30 Robustness. Energy and economic outcomes. 292
B.32 **Illustrative values; invented.** Core-30 Robustness. Service, safety and resource use. 292
B.33 **Illustrative values; invented.** Core-30 Combined. Energy and economic outcomes. 293
B.34 **Illustrative values; invented.** Core-30 Combined. Service, safety and resource use. 293
B.35 **Illustrative values; invented.** Premium-100 Normal. Energy and economic outcomes. 294
B.36 **Illustrative values; invented.** Premium-100 Normal. Service, safety and resource use. 294
B.37 **Illustrative values; invented.** Premium-100 All-in. Energy and economic outcomes. 295
B.38 **Illustrative values; invented.** Premium-100 All-in. Service, safety and resource use. 295

List of Acronyms

AACFT Fault-Tolerant Model with Attention on Actor and Critic

AC Alternating Current

AI Artificial Intelligence

AMP Automatic Mixed Precision

APA Agência Portuguesa do Ambiente

API Application Programming Interface

BACnet Building Automation and Control Network

BAU Business as Usual

BC Behaviour Cloning

BCM Batch Constrained Munchausen Reinforcement Learning

BCQ Batch-Constrained Q-Learning

BEMOSS Building Energy Management Open Source Software

BESS Battery Energy Storage System

BOPTEST Building Optimization Testing Framework

BTE Baixa Tensão Especial

BTN Baixa Tensão Normal

CA Controllable Asset

CBF Control Barrier Function

COIN Compositional Interfaces

COMA Counterfactual Multi-Agent

CPO Constrained Policy Optimisation

CPS Cyber-Physical System

CPU Central Processing Unit

CQL Conservative Q-Learning

CSV Comma-Separated Values

CTDE Centralised Training with Decentralised Execution

CVaR Conditional Value-at-Risk

CW Cleanwatts

DC Direct Current

Dec-POMDP Decentralised Partially Observable Markov Decision Process

DER Distributed Energy Resource

DOI Digital Object Identifier

DQN Deep Q-Network

DR Demand Response

DRQN Deep Recurrent Q-Network

DSO Distribution System Operator

EMS Energy Management System

EPOpt Ensemble Policy Optimisation

ERSE Entidade Reguladora dos Serviços Energéticos

EV Electric Vehicle

EVSE Electric Vehicle Supply Equipment

FACMAC Factored Multi-Agent Centralised Policy Gradients

FMI Functional Mock-up Interface

G2V Grid-to-Vehicle

GAE Generalised Advantage Estimation

GDPR General Data Protection Regulation

GNN Graph Neural Network

GNN-DT Graph Neural Network Enhanced Decision Transformer

GPU Graphics Processing Unit

GRAS-AVC Graph Residual Actor with AC Screening for Active Voltage Control

HAPPO Heterogeneous-Agent Proximal Policy Optimisation

HATRPO Heterogeneous-Agent Trust Region Policy Optimisation

HELICS Hierarchical Engine for Large-scale Infrastructure Co-Simulation

HIL Hardware-in-the-Loop

HPC High-Performance Computing

HRL Hierarchical Reinforcement Learning

HTTP Hypertext Transfer Protocol

HVAC Heating, Ventilation, and Air Conditioning

IEEE Institute of Electrical and Electronics Engineers

INESC TEC Instituto de Engenharia de Sistemas e Computadores, Tecnologia e Ciência

IoT Internet of Things

IPPO Independent Proximal Policy Optimisation

IQL Implicit Q-Learning

ISEP Instituto Superior de Engenharia do Porto

JSON JavaScript Object Notation

JSONL JavaScript Object Notation Lines

KL Kullback–Leibler

KPI Key Performance Indicator

LSTM Long Short-Term Memory

MACPO Multi-Agent Constrained Policy Optimisation

MADDPG Multi-Agent Deep Deterministic Policy Gradient

MADRL Multi-Agent Deep Reinforcement Learning

MAPPO Multi-Agent Proximal Policy Optimisation

MARL Multi-Agent Reinforcement Learning

MARTA Multi-Agent Robust Training Algorithm

MAS Multi-Agent System

MASAC Multi-Agent Soft Actor–Critic

MATD3 Multi-Agent Twin Delayed Deep Deterministic Policy Gradient

MDP Markov Decision Process

MILP Mixed-Integer Linear Programming

ML Machine Learning

MLOps Machine Learning Operations

MLP Multilayer Perceptron

MPC Model Predictive Control

MSE Mean Squared Error

NFC Non-Flexible Consumption

OCPP Open Charge Point Protocol

OMAR Offline Multi-Agent Actor Rectification

OMIE Operador del Mercado Ibérico de Energía

ONNX Open Neural Network Exchange

OOD Out of Distribution

OPEVA Optimisation of Electric Vehicle Autonomy

OSSC Open-Source Software Contribution

PC Personal Computer

PER Prioritised Experience Replay

POMDP Partially Observable Markov Decision Process

PPO Proximal Policy Optimisation

PV Photovoltaic

PVGIS Photovoltaic Geographical Information System

PyPI Python Package Index

RAL Runtime Assurance Layer

RARL Robust Adversarial Reinforcement Learning

RBC Rule-Based Controller

REC Renewable Energy Community

REE Red Eléctrica de España

ReLU Rectified Linear Unit

REN Redes Energéticas Nacionais

REST Representational State Transfer

RL Reinforcement Learning

RLOps Reinforcement Learning Operations

RMA Rapid Motor Adaptation

RNG Random Number Generator

RQ Research Question

SAC Soft Actor-Critic

SC Scientific Contribution

SDG Sustainable Development Goal

SMAC StarCraft Multi-Agent Challenge

SoC State of Charge

SoftCPS Software Technologies for Cyber-Physical Systems

SRO Shared Read-Only

SSH Secure Shell

TCG Typed Constraint Graph

TCP/IP Transmission Control Protocol/Internet Protocol

TD3 Twin Delayed Deep Deterministic Policy Gradient

TI-MAPPO Typed-Interface Multi-Agent Proximal Policy Optimisation

TI-MARL Typed-Interface Multi-Agent Reinforcement Learning

TI-MDP Typed-Interface Markov Decision Process

TI-PPO Typed-Interface Proximal Policy Optimisation

TI-RL Typed-Interface Reinforcement Learning

TIC Typed Interface Compiler

TIMG Typed-Interface Markov Game

TIP Typed-Interface Policy

ToU Time of Use

TSO Transmission System Operator

UAS Unified Action Space

UI User Interface

UPDeT Universal Policy Decoupling with Transformers

UTC Coordinated Universal Time

V2G Vehicle-to-Grid

VAT Value Added Tax

VPP Virtual Power Plant

YAML YAML Ain't Markup Language

Glossary

action group A set of mutually exclusive action ports representing alternative operating modes for one controllable function or entity, while compatible action groups may be controlled simultaneously within the same local action bundle.

action port A callable typed action exposed by the current agent interface, including its semantic name, parameters, bounds, validity conditions, and any projection or safety rule applied before execution.

action-group instance A runtime controllable group bound to a specific module or entity instance, such as the operating mode of one particular EV charger; the same shared decoder can be applied to multiple instances of the same group type.

agent A decision-making entity that receives observations and selects actions according to a policy. An agent is distinct from the member, site, and assets whose operation it manages; the thesis generally assigns one local agent to each participating building.

behaviour cloning Supervised policy learning from state–action demonstrations produced by an expert, rule-based controller, optimiser, or previously deployed policy.

capability schema A versioned description of the potential observations, action groups, action ports, constraints, compatibility requirements, and fallback semantics that an agent or module type can expose.

community In the thesis application, a Renewable Energy Community (REC): an autonomous legal entity with open and voluntary participation, organised around renewable energy activities and benefits for its members and local area. A REC membership boundary need not coincide with a shared electrical connection.

compatibility signature A versioned summary of interface and capability requirements used to determine whether a policy, compiler, schema, and runtime target can operate together safely.

control semantics The correspondence between a physical control problem and its representation and execution in software. Observations retain their identity, provenance, and validity; actions refer to the available capabilities and control authority; and constraints remain attached to the decisions and service obligations they govern. Deployment preserves this correspondence by reconstructing the preprocessing, action encoding, compatibility assumptions, and software context used during evaluation.

controller The software or physical component responsible for producing control actions. A controller may implement a rule-based strategy, an optimisation method, or a learned policy.

coordination-aware monitoring Operational monitoring that detects failures in the relationships among agents and policies, such as incompatible versions, oscillating or conflicting actions, stale messages, unfair shared-resource allocation, or degradation of a REC-level objective despite healthy processes.

deployability The ability to introduce and maintain a learned controller in real-world installations with manageable preparation, training, adaptation, and integration effort. Its scope includes reuse across installations and the complete model lifecycle: data collection and preparation, training, validation, deployment, monitoring, maintenance, and updates; see Section 1.2.2.

dynamic reconfiguration Adaptation to changes in membership, equipment, or coordination relationships during operation. This includes changes to the resources and capabilities available within a building, such as EV connection and departure or equipment becoming unavailable; see Section 1.2.2.

EnergAIze The integrated research platform and energy-domain reference implementation through which the methods, software components, lifecycle mechanisms, and demonstrators developed in this thesis are connected and evaluated.

energy asset An individual unit of energy equipment at a site, such as a charger, battery, PV panel, or heat pump. A connected EV is an asset with its own availability and service requirements. An asset is distinct from its owner, its site, and the software agent controlling it.

energy resource An energy supply, storage capability, or demand available to the control problem. Assets provide these resources; resource availability can change with equipment status, EV sessions, and service requirements. The term describes what can contribute to energy management without equating it with a member, site, or agent.

entity A runtime object that gives observations or actions a concrete identity and operational meaning, such as a connected EV session, charger, battery, or other controlled or observed object within an agent.

entity-bound typed interface A runtime observation–action interface in which observation parts, action groups, and ports explicitly identify the module and controlled entity that give them meaning.

experiment manifest An immutable or content-addressed record that links an experiment to its source revision, environment, scenario and data, reward, observations and actions, constraints, seeds, algorithm configuration, dependencies, resources, and resulting artefacts.

fail-safe mode A controlled operating state in which a learned controller is constrained, bypassed, or replaced by a conservative controller because data, communication, policy compatibility, confidence, or safety conditions are not sufficient for normal operation.

health closure The deterministic propagation of source or module health through declared dependencies to observation validity, action-port availability, parameter bounds, action groups, constraints, and operational mode.

interface snapshot An immutable record of the observation parts, action ports, dependencies, validity states, bounds, health, and resource couplings exposed to an agent at one decision step.

local action bundle The structured action produced by one local typed-interface policy, containing one selected port and its parameters for each active action-group instance after local hard-feasibility construction.

local feasibility The construction or adjustment of an agent's local action bundle so that simultaneous actions remain within non-negotiable local physical, device, and service constraints while preserving policy intent as far as possible.

member A person or organisation participating in a REC under its membership arrangements. A member may consume, produce, or both consume and produce energy, and may be associated with several sites.

module instance A concrete runtime instance of a capability-bearing component associated with an agent, such as one charger, battery, or controllable subsystem; module instances may appear, disappear, or change availability without redefining the complete policy architecture.

observation part A typed block of information exposed to a policy, such as self state, scanner status, battery state of charge, deadline, entity relation, or measurement-quality indicator.

offline reinforcement learning Reinforcement learning from a fixed dataset of previously collected transitions or trajectories without additional environment interaction during the offline training stage.

physics-informed reinforcement learning A family of approaches that incorporates physical models, invariants, residuals, constraints, or model-based structure into policy learning, value estimation, action validation, or training objectives.

policy A mapping from the information available to an agent to the action selected by that agent. A policy may be rule-based, optimisation-based, or learned.

policy registry A service that manages policies throughout their lifecycle by recording their identity, versions, provenance, evaluation evidence, and compatibility. It also tracks approval for deployment, active deployments, and the version history needed for rollback.

policy set A governed collection of interdependent policies intended to operate together, including their agent-role mappings, observation and action schemas, communication assumptions, safety dependencies, compatibility rules, and shared deployment version.

real-world operation Operation of a controller connected to an actual installation, with measurements obtained from its equipment and commands applied to that equipment. Simulation using real-world data, and hardware coupled to a simulated environment, are identified separately from this operating setting.

resilience The capacity to recover useful operation after a disruption. Recovery complements the retention of performance and service examined under robustness.

robustness Retention of acceptable control performance and service under disturbances and conditions outside the training experience. Its scope includes changing operating patterns, sensor and data faults, communication delays or losses, unavailable equipment, and execution failures; see Section 1.2.2.

runtime health state The current quality and availability state of observations, modules, communication channels, or controlled entities used by the Typed Interface Compiler to determine validity, bounds, degraded modes, and fallback behaviour.

safety Compliance of applied actions and resulting states with the physical, electrical, and service constraints specified in the thesis. These include equipment and connection limits and service commitments such as departure energy. Constraint violations and supervisory interventions are assessed separately; see Section 1.2.2.

scalability The ability to sustain useful control across increasing numbers and diverse combinations of agents and resources, with manageable growth in computational demand. Together with dynamic reconfiguration, it covers variation in REC populations and in the resources within each building, as defined in Section 1.2.2.

shared hard-constraint monitor An exceptional runtime assurance mechanism that detects and, when necessary, minimally corrects or rejects joint actions that would violate non-negotiable constraints shared across agents; it is not the normal REC optimisation mechanism.

site A physical installation, such as a dwelling, commercial building, or industrial facility, with one or more connections to the electricity grid. A site contains assets; several members may share a site, and a member may have several sites.

transferability The extent to which knowledge, representations, or policies learned in one setting can be reused or adapted in a different agent, configuration, scenario, or environment.

typed constraint graph A runtime graph connecting agents, modules, entities, action ports, resources, topology, and local or shared constraints through typed relations; it supports local bundle feasibility, training-time graph critics, hard shared checks, compatibility, and decision traces.

typed decision trace An execution record grounded in the actual interface snapshot, including active entities, observations and health, available and invalid ports, selected actions, local feasibility adjustments, shared safety interventions, and final commands.

typed interface A runtime contract describing the observations and actions available to an agent, including their types, parameter bounds, validity conditions, and associated capabilities and constraints. The interface can change as entities connect, leave, or become unavailable.

Chapter 1

Introduction

In many real-world systems, participants make decisions about their own resources while sharing the consequences with others, so a choice that serves one participant well may limit what another can achieve. A shared goal therefore has to accommodate local interests that sometimes pull in different directions. The same difficulty appears in robot teams, networks of road junctions, and groups of buildings managing electricity together, where local decisions contribute to an outcome that no participant determines alone. Multi-Agent Reinforcement Learning (MARL) offers a way to learn these relationships through experience, allowing agents to improve their decisions in response to both their own actions and those of others (Gronauer and Diepold 2022; Keren et al. 2024).

This thesis investigates how such learned coordination can be put to use in real-world systems by developing multi-agent learning and control methods alongside the mechanisms needed to apply their decisions in practice. The central problem is to translate MARL's promising results in simulation into controllers that organisations can deploy and maintain, with Renewable Energy Communities (RECs) providing the setting for this investigation. The following sections establish the context and motivation for the work, formulate the research problem, and set out the thesis statement and Research Questions (RQs). The contributions and organisation of the thesis complete this introductory chapter.

1.1 Context and Motivation

To place this investigation in context, the discussion begins with MARL and the ways in which agents learn to coordinate their decisions. It then turns to energy management, starting with the needs and flexibility of a single household before considering how its choices interact with those of other members of a REC. These interactions bring shared objectives into contact with equipment limits and individual service requirements, revealing the coordination challenges that make this setting well suited to investigating MARL under real-world conditions.

1.1.1 Multi-Agent Learning and Control

Control involves choosing actions that guide a system towards an intended outcome, using the available information about its current condition and how it may evolve. When several controllers act within the same environment, each decision can influence both the part of the system being controlled and the conditions faced by others. Coordination requires these effects to be considered together (Albrecht et al. 2024).

Neighbouring road junctions, for example, illustrate how an action can affect conditions elsewhere in the system. Keeping a traffic light green for longer may clear the queue at one junction while sending more vehicles towards the next, so the value of the decision depends on both the traffic that can move immediately and the conditions created further along the road.

The way a controller responds to these conditions can be specified through different approaches. It may operate with rules defined by an expert, optimise an explicit model, or search among candidate schedules. This thesis focuses on learning because experience can help improve decisions whose consequences emerge over time and depend on other participants (Albrecht et al. 2024; Sutton and Barto 2018). Chapter 3 compares learning with other approaches to these control problems, including rule-based and optimisation-based methods.

Reinforcement Learning (RL) formalises this use of experience through the interaction between a decision-maker, termed an *agent*, and its environment. The agent receives *observations* and selects an *action* according to a *policy*, while a numerical *reward* expresses how the outcome relates to the intended objective. By seeking to improve the reward accumulated over time, learning gives consequences that emerge later a role in the present decision (Sutton and Barto 2018). At a road junction, observations might describe the number of waiting vehicles and the current signal phases, while actions adjust the timing of the lights. Through repeated interactions, the controller learns how these adjustments affect subsequent traffic.

With several learning controllers, the experience of one agent also reflects the decisions of others, whose behaviour may change as they learn. Traffic arriving at a junction, for example, depends on what neighbouring controllers have released. With the objective of reducing delay across the road network, MARL seeks to learn how local decisions can contribute to this shared outcome. Each controller must account for these wider consequences while observing and acting on its own part of the system (Albrecht et al. 2024; Gronauer and Diepold 2022).

The prospect of learning coordinated behaviour has encouraged research on MARL in cooperative robot navigation, task allocation, traffic control, and complex strategy games, with surveys reporting promising results across these domains (Gronauer and Diepold 2022; Orr and Dutta 2023). Once trained, a policy can draw on experience gathered through many interactions to select an action from current observations. Translating experimental success into reliable real-world operation nevertheless remains a research challenge, since the learned behaviour must function with the information, constraints, and equipment of an actual installation (Dulac-Arnold et al. 2021; Orr and Dutta 2023). This is the central problem pursued in this thesis and developed in Section 1.2.

Energy management provides a setting in which both the promise of coordination and the

demands of practical control can be examined. Decisions have consequences over time and across participants, as electricity stored in one building can meet a later need, while electricity consumed there is no longer available for another use (Keren et al. 2024). Understanding these relationships begins with the choices available within a single household.

1.1.2 Energy Management and Flexibility in Homes

For a household, energy management is closely tied to the rhythm of daily life. Rooms should be warm when occupants return, a dishwasher may have until morning to finish its cycle, and a vehicle needs enough energy for the next journey. These requirements leave different amounts of freedom over when equipment operates. Solar panels and batteries extend that freedom by allowing some electricity to be produced at home and retained for later use. The controller's task is to arrange these choices around the services the occupants expect.

Consider an Electric Vehicle (EV) that returns home at 18:00 with its battery at a State of Charge (SoC) of 30%. The driver plans to leave at 08:00 the following morning and requests a departure SoC of 80%. This specifies the service to be delivered, but leaves fourteen hours in which to arrange charging. Charging immediately would draw power alongside cooking, heating, and other evening uses, adding to demand at a time when the household connection may already be heavily used. If many residents charge in this way, the concentration can also increase pressure on the local network and the need for costly infrastructure reinforcement (Hodge 2017).

Those fourteen hours allow charging to be arranged around the household's other needs. EV charging power can be reduced during the evening peak and increased after demand has fallen, or part of the required energy can be delivered during lower-price hours. This freedom to adjust consumption, storage, or supply within operating limits while preserving the required service is termed *energy flexibility* (Hussain et al. 2023). One representation, the *FlexOffer*, describes flexible energy amounts and timing and is discussed in Section 2.1.2 (Šikšnys et al. 2019).

For the vehicle, the available flexibility depends on how much energy is still needed, how long it will remain connected, the vehicle and charger ratings, and the capacity available at the household connection. As departure approaches, there is less room to postpone charging. Other appliances offer different choices, with a dishwasher allowing its start time to move while a heat pump must keep the indoor temperature within an acceptable range. Each resource therefore combines an opportunity to adjust energy use with a requirement that the adjustment must preserve.

Local energy production changes which of those choices are attractive. Rooftop Photovoltaic (PV) panels may produce a surplus around midday, several hours before the household's evening demand. For the vehicle arriving at 18:00, a stationary battery offers a way to retain some of that production for later use. On a different day, when the same vehicle remains at home until late afternoon, charging can coincide directly with solar generation while still meeting its next departure target. A flexible appliance provides another use for the surplus. The timing of demand and storage therefore helps determine how much renewable electricity is used in the home and how much electricity must be bought or exported (Hodge 2017; Hussain et al. 2023).

The relationships introduced in Section 1.1.1 are visible in these household choices, where the vehicle, storage, and flexible appliances share the available energy and connection capacity over time. A household may prioritise lower expenditure, greater use of locally produced electricity, reduced demand peaks, or a chosen combination of these aims. Whichever objective is selected, comfort and the energy needed for the next journey remain service requirements. The preferred timing then depends on what the present decision makes possible later. Filling the stationary battery with cheaper grid electricity overnight may reduce subsequent purchases but leave less room for the next day's solar surplus; using its stored energy early in the evening leaves less for a later demand peak. A household controller must relate these consequences to the chosen objective, using measurements and forecasts of demand, generation, and prices (Chen, Qu, et al. 2022).

For a single household, these decisions can be learned through RL by one agent managing its controllable resources (Vázquez-Canteli and Nagy 2019). In practice, meters, sensors, and device interfaces connect that agent to the home's equipment, providing measurements and carrying its commands. Section 1.2 discusses the practical challenges of this sensing and control architecture.

Some opportunities extend beyond the household. While its occupants are away and its panels produce a surplus, a neighbour may have an EV ready to charge. Using that surplus introduces a coordination problem between households, extending the decisions previously considered within a single home.

1.1.3 Renewable Energy Communities

For neighbouring households to make use of such opportunities, they need an arrangement through which they can participate, share energy, and agree on the benefits they seek. RECs provide this setting. They are autonomous legal entities with open and voluntary participation, bringing together individuals, small and medium-sized enterprises, and local authorities around renewable generation, consumption, storage, and sharing. Their primary purpose is to provide environmental, economic, or social benefits to their members and the local area (European Parliament and Council of the European Union 2018; Presidência do Conselho de Ministros 2022). Differences in members' routines and equipment then become a resource for coordination, allowing production at one building to meet demand at another (Reis et al. 2021).

The composition of a REC can change both through members joining or leaving and through changes to the equipment within each household. Existing members may install new chargers, batteries, or other assets, remove equipment, or face temporary unavailability when a device fails. Vehicles connecting and disconnecting also change which resources can participate at a given time. These changes in resources, capabilities, and relationships constitute *dynamic reconfiguration*, altering the opportunities for sharing and the actions available for coordination.

Returning to the example of the vehicle that remains at home until late afternoon, its charging can draw on another member's solar surplus where sharing arrangements permit. Battery Energy Storage Systems (BESS) at participating buildings can retain production for later needs, extending the opportunities to use renewable electricity within the REC (Faia et al. 2023; Srithapon and

Månsson 2023). An agreed local price can make sharing beneficial to both households, giving the producer a better return than exporting the surplus and the consumer a lower cost than buying from the grid, after applicable charges (Moret and Pinson 2019; Presidência do Conselho de Ministros 2022). Matching demand to this production can also reduce upstream power flows and electrical losses (Pohl et al. 2024), and make use of electricity that export limits would otherwise require to be curtailed (Hodge 2017).

Realising these benefits requires attention to how members' decisions interact. If every EV owner independently chooses the cheapest charging period, their combined demand may create a new peak in grid imports (Hodge 2017). Spreading charging across the night can avoid this concentration, but must accommodate different departure times; for example, a vehicle leaving at 06:00 may need to charge sooner than one leaving at 08:00. Coordination therefore has to reconcile the REC's objective with each member's service requirements. Where participation is possible, this flexibility can also support *Demand Response (DR)*, adjusting consumption in response to prices or incentives to serve needs beyond the REC (Entidade Reguladora dos Serviços Energéticos 2024; European Parliament and Council of the European Union 2019).

This need to reconcile individual requirements with shared objectives connects the household decisions in Section 1.1.2 to the multi-agent coordination introduced in Section 1.1.1. In a MARL formulation with one agent per building, each agent manages its local resources while learning how its decisions contribute to the REC's objective. A charging decision can then account for the driver's needs together with the availability of another household's surplus or the demand created by other vehicles. Experience of these interactions provides a basis for learning coordinated behaviour, with surveys of simulated energy systems reporting promising reductions in costs and demand peaks (Keren et al. 2024; Sajid, Li, Qi, et al. 2026).

RECs therefore provide a setting for investigating the central problem of this thesis, which concerns putting MARL to use in the real world. Individual and shared objectives must be reconciled across members with service requirements and changing resource configurations. These and other challenges of applying MARL in real-world installations are examined in Section 1.2.

1.1.4 Institutional Setting and Partnerships

The research was conducted at the Software Technologies for Cyber-Physical Systems (SoftCPS) laboratory of Instituto Superior de Engenharia do Porto (ISEP), part of the Technical University of Porto. SoftCPS belongs to the HumanISE centre of Instituto de Engenharia de Sistemas e Computadores, Tecnologia e Ciência (INESC TEC), the thesis host institution (SoftCPS 2026b).

The Optimisation of Electric Vehicle Autonomy (OPEVA) and DemFlex projects supported work on energy management and EV charging with partners including Cleanwatts and i-charging (Fonseca, Sousa, et al. 2025; SoftCPS 2026a). These partnerships connected controller development to the data, equipment interfaces, and service requirements of real-world installations.

1.2 Research Problem

Promising simulation results in robotics, traffic control, complex strategy games, and energy management have encouraged interest in the wider use of MARL (Gronauer and Diepold 2022; Keren et al. 2024; Orr and Dutta 2023). Organisations seeking to bring these methods beyond simulation and into real-world applications need controllers that work with the equipment and information available, deliver the services people rely on, and remain useful as operating conditions change. Applying MARL under these conditions requires the learning methods and the systems supporting their operation to be developed together. The following subsections discuss the challenges involved and explain the choice and meaning of scalability and dynamic reconfiguration, deployability, safety, and robustness as the four dimensions that guide this thesis.

1.2.1 Real-World Challenges

Applying MARL in the real world involves several challenges identified in the literature but only partly examined in simulation studies. In energy management, reviews report more attention to costs and demand peaks than to robustness, transferability, and deployment conditions (Keren et al. 2024; Sajid, Li, Qi, et al. 2026). These real-world challenges include scaling coordination to larger groups, adapting policies to different and changing resources, obtaining sufficient learning experience, reusing learned behaviour across installations, and connecting decisions to physical equipment (Gronauer and Diepold 2022; Kannari et al. 2025; Orr and Dutta 2023). Respecting physical limits and individual service requirements while pursuing shared objectives, and maintaining control under disturbance, add further demands (Dulac-Arnold et al. 2021; Gu et al. 2023), alongside questions about fairness, explainability, privacy, and cybersecurity (Bekkemoen 2024; Farokhi 2020; Moret and Pinson 2019; Stouffer et al. 2023). Chapter 3 examines these gaps in detail. The discussion below summarises them to frame the research problem through three perspectives, (i) the practical perspective, concerning the organisations that must install, operate, and maintain controllers across different application domains, (ii) the REC perspective, concerning their consequences for the members, equipment, and energy services introduced in Section 1.1.3, and (iii) the scientific perspective, concerning MARL algorithms and their associated challenges.

Scalability and dynamic reconfiguration become practical concerns when organisations extend coordinated control to more participants or adapt it to changing resources. Expanding a robot fleet, for example, increases the number of agents whose movements and tasks must be coordinated (Orr and Dutta 2023). The training time, memory, communication, and decision time required by MARL algorithms can limit how widely an organisation can apply them (Charbonnier, Morstyn, et al. 2022b). From the REC perspective, this difficulty appears in RECs with different numbers of members and different combinations of resources, whose coordination must remain useful as the population grows (Almilaify et al. 2023). These conditions raise scientific challenges for MARL. A larger population introduces more combinations of decisions, while each agent learns in an environment partly shaped by the changing behaviour of the others (Gronauer and Diepold 2022;

Orr and Dutta 2023). If a method uses a separately trained policy for each building, adding a member may also require training another policy and adapting coordination with the existing agents (Wu, Qiu, et al. 2024).

Dynamic reconfiguration requires control to adapt as resources, capabilities, and relationships change over time. In a REC, this includes members joining or leaving, as introduced in Section 1.1.3, and changes within a building even when the number of agents stays the same. A new charger gives its building's agent another service to manage, while removing a battery takes away actions that an earlier policy could use. The scientific difficulty lies in accommodating these changes within the learned model. If it expects a fixed collection of inputs and outputs, routine equipment changes may require modifying it and repeating part of the training. Generalisation to new actions is an established RL problem, and energy studies examine representations for changing building populations and equipment configurations (Jain et al. 2020; Orfanoudakis, Panda, et al. 2026; Wu, Qiu, et al. 2024).

Deployability concerns the effort an organisation needs to put a learned controller into service and maintain it across installations. Obtaining suitable learning experience may be costly or difficult, and exploring new actions on physical equipment can interfere with services already being provided (Dulac-Arnold et al. 2021). From the REC perspective, a new member may have little usable historical data, while differences in equipment, demand, tariffs, and occupants' routines can limit the value of experience collected elsewhere. These practical conditions raise scientific questions about how much experience MARL algorithms require and how well learned behaviour generalises beyond its training conditions. Transfer studies in simulated building populations examine reuse of learned behaviour, while reviews of real building control identify generalisation between installations as an outstanding need (Kannari et al. 2025; Nweye, Sankaranarayanan, et al. 2023).

The practical demands of deployability also extend across the model's lifecycle, from collecting and preparing sensor data to training, validation, deployment, monitoring, maintenance, and updates (Kreuzberger et al. 2023; Li, Thomas, et al. 2022). These stages must work together with manageable configuration and intervention effort. Integration with operational data and control systems is also needed in domains such as traffic management and telecommunications (Li, Thomas, et al. 2022; Zhang, Zhang, et al. 2026). In a REC, the sensing and control architecture introduced in Section 1.1.2 must obtain usable measurements from different devices, associate them with the right resources, and deliver commands to the equipment. Because installations differ, deploying a controller at a new site may require repeating data preparation, model validation, and device integration (Fonseca, Sousa, et al. 2025; Kannari et al. 2025). For the MARL formulation, this raises the question of whether its assumed observations and actions retain their meaning when the policy operates through these interfaces.

Safety requires operators to keep applied decisions within physical limits while preserving the services expected by users. In a robot fleet, coordinated movement must avoid collisions (Orr and Dutta 2023); in the REC charging example, decisions must respect electrical limits while delivering

the energy needed before departure. Representing the driver's request alongside the equipment's capabilities determines how much flexibility can actually be used (Hussain et al. 2023; Šikšnys et al. 2019). For MARL algorithms, a reward may favour an economical schedule, but learning and execution must also account for actions that are physically inadmissible or leave too little time to complete charging. Safe MARL and shielding methods address how constraints enter learning and how proposed actions can be corrected before execution (Elsayed-Aly et al. 2021; Gu et al. 2023). The coordination problem also includes how those corrections affect other agents sharing the available capacity.

Robustness matters to organisations that need control to continue under disturbances and changing operating conditions, since interrupted service can require manual intervention. A mobile robot, for example, may have to continue navigating when its sensor readings become unreliable (Dulac-Arnold et al. 2021). In a REC, a missing measurement or failed charger removes information or actions on which a learned policy normally relies, while changes in occupants' routines can alter demand even when all equipment works correctly. Building-control reviews and operational REC studies document the effects of data and interface problems (Fonseca, Sousa, et al. 2025; Kannari et al. 2025). An unavailable charger may affect one household while other resources remain usable. From the scientific perspective, these situations require learned behaviour to remain useful with incomplete observations or changes in the effects of actions, including conditions outside the training experience (Dulac-Arnold et al. 2021). Continued service then depends on how the policy responds and how the surrounding system recognises the disruption and maintains or restores control.

Fairness concerns how the benefits and burdens of coordination are distributed among participants. In a REC, a lower collective cost may be accompanied by unequal savings or repeated inconvenience for some members (Moret and Pinson 2019). If the same household's charging is repeatedly postponed, participants and operators need to understand the reason and whether intervention is appropriate. For MARL, these concerns involve both how the shared reward encourages particular outcomes (Albrecht et al. 2024) and how the resulting decisions can be interpreted, motivating research on *explainability* (Bekkemoen 2024).

Privacy and *cybersecurity* require organisations to protect the information and interfaces used for coordination against unauthorised access or alteration, as in other operational control systems (Stouffer et al. 2023). In a REC, detailed energy measurements can reveal household routines, affecting what data can be made available for learning and exchanged during operation (Farokhi 2020). Protection also covers the links carrying measurements and equipment commands. From the scientific perspective, restrictions on information must be reflected in the observations and communication assumed by the MARL formulation.

Subsection 1.2.2 identifies and defines the dimensions selected for investigation in this thesis.

1.2.2 Lexical Grounding

The thesis examines scalability and dynamic reconfiguration, deployability, safety, and robustness as four dimensions of applying MARL in the real world. Their selection draws on the literature reviewed in Chapter 3 and experience of connecting learned controllers to real-world installations in the supporting research projects. These dimensions guide the RQs about learning methods and their supporting systems; the definitions below establish their meaning.

Scalability and dynamic reconfiguration. In this thesis, scalability denotes the ability to sustain useful control across increasing numbers and diverse combinations of agents and resources, with manageable growth in computational demand (Bondi 2000; Charbonnier, Morstyn, et al. 2022b). Dynamic reconfiguration denotes adaptation to changes in membership, equipment, or coordination relationships during operation (Orfanoudakis, Panda, et al. 2026; Wu, Qiu, et al. 2024). These terms jointly cover variation in REC populations and in the resources within each building.

Deployability. Deployability is used here for the ability to introduce and maintain a learned controller in real-world installations with manageable preparation, training, adaptation, and integration effort. Its scope covers reuse across installations (Nweye, Sankaranarayanan, et al. 2023) and the model lifecycle, from data collection and preparation to training, validation, deployment, monitoring, maintenance, and updates (Kreuzberger et al. 2023; Li, Thomas, et al. 2022).

Safety. Safety denotes compliance with the physical, electrical, and service constraints specified in this thesis, for both applied actions and resulting states (Elsayed-Aly et al. 2021; Gu et al. 2023). These constraints cover equipment and connection limits and service commitments such as delivering the required energy before departure.

Robustness. Robustness refers here to retaining acceptable control performance and service under disturbances and conditions outside the training experience (Dulac-Arnold et al. 2021). Its scope covers changing operating patterns, sensor and data faults, communication delays or losses, unavailable equipment, and execution failures. The related term *resilience* denotes the capacity to recover useful operation after a disruption.

These dimensions are closely related, and improvements in one can also benefit another. Greater scalability, for example, can make a system easier to apply in installations of different sizes, contributing to deployability. The thesis therefore considers these dimensions together when assessing the suitability of MARL for real-world control.

1.3 Thesis Statement and Research Questions

The challenges discussed in Section 1.2 lead to the thesis statement and RQs presented below. Together, these guide the investigation of how MARL can be applied in the real world.

1.3.1 Thesis Statement

The central statement examined in this thesis is:

> *MARL can support scalable, deployable, safe, and robust control of RECs through the integration of reusable learning across compatible resources, explicit constraint handling, and execution supervision.*

1.3.2 Research Questions

The four RQs examine this proposition along the dimensions defined in Subsection 1.2.2.

- **RQ 1: Scalability and Dynamic Reconfiguration** – How can MARL accommodate increasing populations and changes in agents and local resources, and how do these changes affect control performance, computational demand, and the need for further learning?
- **RQ 2: Deployability of Learned Controllers** – Which parts of controller preparation can be reused across heterogeneous installations, and what training, adaptation, integration, and maintenance effort is required at each installation to retain control performance?
- **RQ 3: Physical and Operational Safety** – How can physical and service constraints be represented and enforced in MARL-based control, and how does their enforcement affect coordination, individual service delivery, and the need for protective intervention?
- **RQ 4: Robustness to Real-World Disturbances** – How do learned policies and execution supervision contribute to service continuity and sustained recovery under changing operating conditions and failures in data, communication, or equipment?

These questions are addressed through comparisons of energy performance and service delivery alongside the computation, preparation, and intervention required to obtain them. Reconfiguration studies distinguish immediate policy reuse from performance obtained after further learning. Deployability studies separate preparation shared across installations from work repeated at each site. Safety and robustness assessments follow individual services through protective intervention, degraded operation, and recovery, accounting for the feasibility of the requested service under the available resources. The evaluation methodology in Chapter 5 defines the corresponding indicators and comparison conditions.

1.4 Research Contributions and Outcomes

This research advances the study and real-world application of MARL through contributions to learning methods, control architectures, simulation, and experimental evaluation. Developed in REC energy management, the work connects the coordination of heterogeneous resources with the conditions required to apply learned decisions in practice. Subsection 1.4.1 presents the scientific contributions, Subsection 1.4.2 describes the software and data outputs, and Subsection 1.4.3 discusses the societal and environmental relevance of the work.

1.4.1 Scientific Contributions

The Scientific Contributions (SC) developed in this thesis concern the analysis of existing research, the organisation and evaluation of control, and learning under changing configurations and operating conditions.

SC1 **A review and gap analysis of MARL for real-world control.**
A systematised review connects evidence from REC applications with methods addressing scalability and dynamic reconfiguration, deployability, safety, and robustness. Its comparative synthesis identifies demonstrated capabilities, limitations in their evaluation, and unresolved questions that guide the investigation (Chapter 3).

SC2 **An architecture for multi-agent REC control.**
This thesis presents EnergAIze, a distributed architecture that connects controller development, REC management, and supervised operation. EnergAIze separates control decisions from installation-specific data and equipment interfaces, supporting heterogeneous installations through common descriptions of resources and services. Its modular organisation connects local control with shared data, deployment, and monitoring services (Chapter 4).

SC3 **A simulation model for changing REC configurations and services.**
This thesis presents CityLearn v3, a simulation environment that represents RECs, their members, energy assets, and service requests. The model connects changes in resource relationships and availability to energy exchanges, electrical constraints, and service delivery, supporting controlled studies of reconfiguration and disturbances before real-world application (Chapter 5).

SC4 **A methodology for comparing learned REC controllers.**
This thesis proposes a set of Key Performance Indicators (KPIs) and a methodology for comparing controllers during development and real-world application. The assessment combines energy performance, service delivery, computational demand, and protective interventions to identify regressions and operational difficulties. Common definitions support comparisons between learned controllers and other control approaches (Section 5.4).

SC5 **A formal model for MARL under dynamic reconfiguration.**
This thesis presents a Markov-game formulation that links changes in agent participation, resource capabilities, and information availability to the observations and actions available to each agent. It distinguishes persistent equipment from temporary service requests and explicitly associates observations, actions, and constraints with the resources they concern. This formulation provides a common description of reconfiguration for learning and execution (Section 7.1.1).

SC6 **A modular learning method for reconfigurable multi-agent control.**
Building on SC5, this thesis presents Typed-Interface Multi-Agent Reinforcement Learning (TI-MARL), which uses the current resource description to construct each agent's decision interface and jointly select actions for its active equipment. The method supports investigation of policy reuse under combined changes in community membership and local equipment (Sections 7.2–7.3).

SC7 **Safety and robustness for real-world multi-agent control.**
This thesis extends TI-MARL by propagating changes in equipment and information availability through declared dependencies to determine which control capabilities remain usable. Affected actions can be restricted or suspended while other resources continue under their own operating conditions, with explicit conditions governing their return (Section 7.4).

SC8 **An experimental study of strategies for improving learned REC control.**
This thesis investigates hierarchical coordination, offline RL, Behaviour Cloning (BC), and the use of operating rules to guide learning through matched experiments. The comparisons examine their effects on learning effort, energy performance, and service delivery, identifying the conditions in which each strategy can support controller development (Sections 6.5–6.7).

SC9 **Application of learned control to heterogeneous real-world RECs.**
This thesis applies EnergAIze and its controllers to heterogeneous real-world REC installations, connecting learned decisions to operational data and equipment interfaces. Integration and operational studies examine service delivery, supervision, and the effects of imperfect data and equipment availability, establishing practical requirements for applying the developed methods (Chapter 8).

SC10 **Scientific publications.**
The publications associated with these contributions include nine published papers, comprising five conference papers and four journal articles. These works, together with a further conference communication, submitted manuscripts, preprints and planned articles, are listed in the front matter (p. xv).

1.4.2 Software and Data Contributions

In addition to the scientific contributions, this thesis produced Open-Source Software Contributions (OSSC) and reusable datasets. These code modules and experimental inputs help researchers and practitioners develop and evaluate REC control.

OSSC1 CityLearn v3 for REC simulation and evaluation.
The simulator development in this thesis resulted in CityLearn v3, distributed through the Python Package Index (PyPI) and the official CityLearn repository (CityLearn Project 2026b,c). Its support for EV charging, flexible loads, local sharing, and configuration changes implements the modelling and evaluation contributions in SC3–SC4 (Chapter 5).

OSSC2 A User Interface (UI) for constructing and analysing simulations.
CityLearn UI was also developed within this thesis to support scenario construction, inspection of building and equipment time series, and comparison of KPIs. It complements CityLearn and provides visual access to the evaluation outputs in SC4. Its source code is available in the CityLearn UI repository (SoftCPS Research Group 2026b).

OSSC3 REC datasets and scenario configurations.
Real-world measurements, public energy and weather records, and generated resource and service scenarios provide controller test data. The datasets cover demand, generation, prices, charging sessions, and service requirements, supporting the comparisons in SC3–SC4 while distinguishing measured and generated inputs (Section 5.3; Chapter 8).

OSSC4 Controller implementations and experiment software.
The controller implementations and configuration, training, and evaluation tools are available in the Algorithms repository (SoftCPS Research Group 2026a). They implement SC4–SC8, allowing others to inspect the procedures, repeat comparisons, and apply the controllers to their own scenarios (Chapters 6–7).

OSSC5 Supporting software for operating and supervising REC control.
Supporting work coordinated by the author provides the EnergAIze UI, backend services, database and domain models, data collectors, forecasting services, and operational tools. These components implement the architecture in SC2 and support the application studies in SC9. Public repositories include EnergAIze UI, Percepta, and EnergAIze Inference (SoftCPS Research Group 2026c,d,g) (Chapter 4).

OSSC6 Distributed orchestration of training and evaluation experiments.
The thesis also produced infrastructure for submitting, distributing, and monitoring experiments across local and remote computing resources. The Job Orchestrator manages jobs and results, while the Worker Agent executes workloads on heterogeneous machines (SoftCPS Research Group 2026e,f). Together, they support repeated comparisons and concurrent studies under the methodology in SC4 (Section 4.4).

1.4.3 Societal and Environmental Relevance

The thesis supports more efficient energy management by developing methods to coordinate local generation, storage, and flexible demand. In RECs, this coordination creates opportunities to use more locally produced renewable electricity, reduce energy expenditure, and manage demand peaks while preserving comfort and mobility (Hussain et al. 2023; Reis et al. 2021). Its environmental relevance lies in making distributed flexibility usable in everyday operation, supporting renewable integration and the transition towards low-carbon energy systems.

The societal contribution centres on the services that energy systems provide to people. Household needs and the distribution of benefits among REC members remain explicit considerations in control and evaluation (Moret and Pinson 2019). Open software, reusable datasets, and documented methods extend the value of the work to researchers and organisations developing their own energy-management solutions. Together, these contributions support the United Nations Sustainable Development Goals (SDGs) of affordable and clean energy (SDG 7), sustainable cities and communities (SDG 11), and climate action (SDG 13) (United Nations 2015).

1.5 Thesis Organisation

The thesis is organised as a connected investigation of the conditions required to apply MARL in real-world systems.

Chapter 2 establishes the concepts of energy flexibility, RECs, RL, and policy organisation used throughout the thesis. Chapter 3 compares existing control approaches and develops the research gaps that guide the subsequent chapters.

Chapter 4 presents the system architecture (EnergAIze), explaining how learning algorithms fit alongside the data, deployment, and supervision services needed for practical control. Chapter 5 then establishes the experimental environment in which control decisions can be studied under controlled, comparable conditions. This includes the simulator extensions developed throughout the research, the datasets, and the KPIs used to evaluate controllers.

On this basis, Chapter 6 investigates the methods identified in the state-of-the-art review, examining their capabilities and the requirements that remain unresolved. These studies motivate the method presented and evaluated in Chapter 7, which brings policy reuse, coordination, and operational constraints into an integrated design.

Chapter 8 follows with deployment and operational studies in real-world installations. These studies examine the practical implications of the earlier chapters, including the effects of data availability, equipment interfaces, and operating conditions on controller integration and use.

Chapter 9 draws together the findings to answer the RQs. Chapter 10 assesses the central thesis, summarises the contributions, and outlines future work. The appendices provide the detailed literature comparison and supporting experimental material.

Chapter 2

Foundations

Energy management combines physical assets, information about their condition, and decisions whose consequences unfold over time. The thesis investigates MARL for coordinating these decisions in RECs, bringing energy-domain concepts together with the foundations of learning and decision-making.

The chapter first establishes the energy-domain concepts and the neural networks used to represent learned functions. It then introduces single-agent RL and MARL before considering recorded experience, hierarchical decisions, and structured policy architectures. These foundations prepare the reader for the analysis of existing methods in Chapter 3 and for the controller formulations developed later in the thesis.

2.1 Renewable Energy Communities

This section introduces the energy-domain concepts used throughout the thesis to help the reader follow the later discussion. It begins with REC members and their organisation, then examines asset operation, flexibility, and market signals before describing the information and conditions needed for operation.

2.1.1 Community Members

A *REC member* is a person or organisation participating in the REC's activities under its membership arrangements. Members can take different energy roles. A *consumer* uses electricity, a *producer* generates it, and a *prosumer* combines both activities. A household with rooftop PV panels may therefore be a prosumer while drawing electricity from the grid during the evening. Membership describes participation in the collective; the energy role describes the member's activities (Reis et al. 2021).

Those activities take place at a *site*, which is a physical installation, such as a dwelling, commercial building, or industrial facility, with one or more connections to the electricity grid. The individual units of energy equipment within a site are termed *assets*, including chargers, batteries,

and PV panels. A member may be associated with several sites, while a shared building may accommodate several members. Assets can also be collectively owned or used, as with a REC battery or a charging facility serving several households (Moret and Pinson 2019; Reis et al. 2021).

Section 1.1.2 described a household scheduling the charging of an EV around its daily activities. In that example, the people living in the house participate as a REC member, and their home is the site. The charger is an asset installed at the site, and the EV connects to it during a charging session. These terms distinguish the participant, the installation, and the individual assets whose operation is managed there. The broader term *energy resource* refers to the supply, storage capability, or demand available to the control problem. Assets provide these resources, whose availability also depends on operating status and service requirements.

The organisation extends from these individual assets to the wider electricity system. Assets operate within sites, sites participate in RECs, and their exchanges with the grid involve the distribution and transmission networks. This hierarchy gives each level a different scope of operation, as illustrated in Figure 2.1.

Above the REC level, network operators manage the infrastructure through which sites exchange electricity. The Distribution System Operator (DSO) operates, maintains, and develops the distribution network serving connected users. The Transmission System Operator (TSO) performs these functions for the transmission network and its interconnections, supporting secure operation across the wider system (European Parliament and Council of the European Union 2019).

Each level has its own *operating limits*. An individual asset has technical limits associated with its physical design. For example, a charger can supply power within its rated range, and a battery can retain energy within its usable capacity. These limits describe the operation that the asset can support. Service requirements add conditions on its use, such as the energy a vehicle needs for departure or the temperatures occupants require in a room (Hussain et al. 2023).

At a site, several assets may share a *single-phase* or *three-phase* grid connection (E-REDES 2025). The power that can be supplied depends on the connection rating and the current limits of its conductors and protective devices. In a three-phase installation, the loading of each phase also matters, since one phase may reach its limit while capacity remains available on the others. The combined demand must respect these shared limits, even when each asset remains within its own rating. A charger and a heat pump, for example, may each operate at an admissible power while their simultaneous demand exceeds the capacity available to the home.

Similar relationships arise across sites connected to a common feeder or transformer. The scope of a shared electrical limit therefore follows the physical network connections, while REC membership identifies the participants whose activities are coordinated.

Within these operating conditions, energy management can pursue objectives at each level. Table 2.1 summarises examples of these objectives and the conditions to preserve at each level.

Asset characteristics and required services determine the operation available to meet these objectives. The next subsection explains how they define energy flexibility.

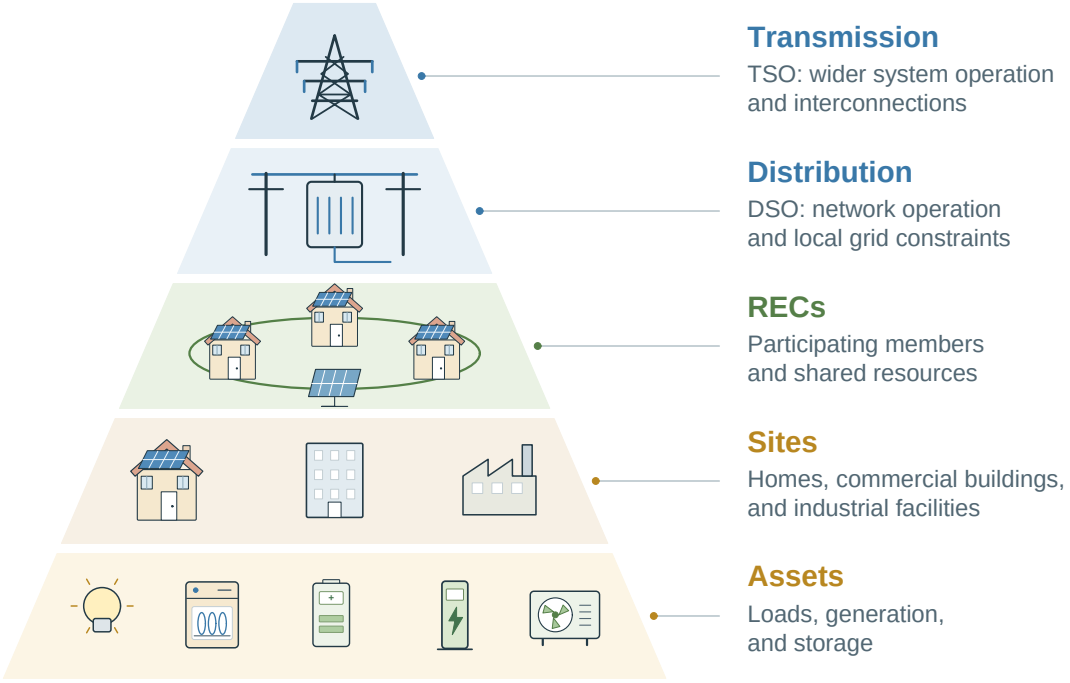


Figure 2.1 : Levels of organisation and coordination around a REC. The pyramid relates assets and sites to REC activities and the wider network context; membership arrangements and electrical connections determine the relationships in each installation.

Table 2.1: Examples of energy-management objectives and operating conditions at different levels, synthesised from European Parliament and Council of the European Union (2019), Hussain et al. (2023), Moret and Pinson (2019), and Reis et al. (2021).

Level	**Examples of objectives**	**Conditions to preserve**
Asset	Reduce energy losses or wear during operation.	Technical limits and requested service, such as departure energy or thermal comfort.
Site	Reduce expenditure and peak demand; increase local self-consumption.	Connection capacity and the service requirements of the site's assets.
REC	Increase shared use of renewable production; reduce collective expenditure; distribute benefits fairly.	Members' agreed requirements and applicable shared electrical limits.
Distribution (DSO)	Reduce network losses; use network capacity efficiently.	Voltage and loading limits, with continuity of supply.
Transmission (TSO)	Operate the wider system efficiently and coordinate exchanges between networks.	Supply–demand balance, frequency, and secure network operation.

2.1.2 Asset Operation and Flexibility

Assets perform different functions within a site. Appliances and heating equipment create demand, PV panels supply electricity, and a stationary battery retains energy for later use. An EV combines storage with a mobility requirement and is available to the installation only while connected. Consumption, production, and storage determine the site's exchanges with the grid and the opportunities to adjust its energy use (Hussain et al. 2023; Šikšnys et al. 2019).

Power describes the rate of energy transfer, usually expressed here in kilowatts (kW), whereas *energy* describes the amount transferred over an interval, in kilowatt-hours (kWh). A charger operating at 3 kW for one hour transfers 3 kWh at its electrical input. Losses mean that the increase in stored battery energy can be smaller. A storage asset's energy capacity and power rating express separate limits: capacity determines how much it can hold, while the power rating limits how quickly it can charge or discharge. Its SoC expresses the stored energy relative to a stated reference capacity. These distinctions matter when a service specifies an energy target but the asset accepts a power command.

At the building boundary, *import* is electricity drawn from the external network and *export* is electricity supplied to it. Local generation can meet demand directly, charge storage, or be exported. *Self-consumption* concerns the use of locally generated electricity within the accounting boundary being considered. That boundary may be a building or a REC, and the treatment of storage must be stated when the quantity is measured. REC sharing allocates locally produced energy among participating members under the applicable arrangement. The allocation is distinct from the physical paths taken by electricity through the network (Moret and Pinson 2019; Reis et al. 2021). A REC's membership and the assets behind one constrained electrical connection can consequently define different groups.

The timing of these exchanges affects the loading of the installation. A *power peak* is the highest power observed over a stated period, with its value depending on the measurement or averaging interval used. Starting the EV's charging while cooking and heating are already drawing power can raise the home's evening import peak. At REC level, the peak of the combined demand depends on how closely consumption at different sites coincides.

Ramping describes changes in power over time, and a *ramp rate* expresses how quickly power increases or decreases. A charger starting at full power produces an upward change in demand; a gradual increase spreads that change over a longer interval. The size of a peak and the speed of a change describe different features of the same power profile. Adjusting when charging begins and how its power develops can influence both, subject to the energy needed for departure.

Energy flexibility is the scope for changing consumption, production, storage, or exchange relative to a reference behaviour while preserving the required service. It depends on the adjustments that the asset permits, its availability, and the time remaining to deliver that service. In the example from Section 1.1.2, the EV arrives at 18:00 with 30% SoC and is required to reach 80% by 08:00. These fourteen hours define the period available to meet the charging request, while the charger's

power rating limits how quickly energy can be delivered. The session may allow several hours of delay early in the evening, but much less as departure approaches. Even a controllable charger offers no charging flexibility while the EV is absent (Hussain et al. 2023; Šikšnys et al. 2019).

A *FlexOffer* describes this scheduling freedom through an admissible start-time window and an energy profile divided into intervals (Fonseca, Ferreira, Landeck, et al. 2022; Šikšnys et al. 2019). Figure 2.2 illustrates the two choices: when the charging profile begins and how much energy is assigned to each interval.

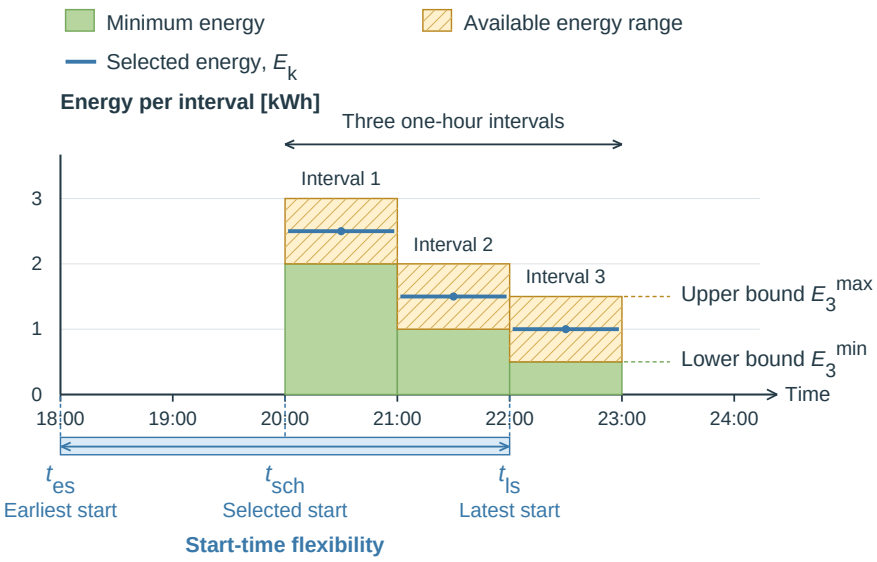


Figure 2.2: Start-time and energy flexibility in a FlexOffer. Redrawn and adapted from Fonseca, Ferreira, Landeck, et al. (2022); the values illustrate a profile rather than measured consumption.

For N intervals of duration Δt, a FlexOffer $\mathscr{F}$ is written as:

$$\mathscr{F} = \left([t_{\text{es}}, t_{\text{ls}}], \left\langle [E_k^{\min}, E_k^{\max}] \right\rangle_{k=1}^{N}, \Delta t\right). \tag{2.1}$$

Here t_{es} and t_{ls} are the earliest and latest permitted start times. The ordered profile contains N intervals, each with lower and upper energy bounds $E_k^{\min}$ and $E_k^{\max}$. The index k identifies an interval relative to the profile's start, and Δt specifies its duration.

Scheduling selects a start time t_{sch} and an energy amount E_k for each interval, subject to:

$$t_{\text{es}} \le t_{\text{sch}} \le t_{\text{ls}}, \qquad E_k^{\min} \le E_k \le E_k^{\max}, \quad k = 1, \ldots, N. \tag{2.2}$$

The selected time t_{sch} positions the whole profile, while each E_k fixes the energy transferred during its corresponding interval. Thus, the horizontal range in the figure represents a choice of start time, and the vertical ranges represent choices of energy. The profile lasts $N\Delta t$, so a service with deadline t_{dep} requires $t_{\text{sch}} + N\Delta t \le t_{\text{dep}}$. The latest permitted start must leave enough time for that duration.

Lower and upper bounds on total energy, $E_{\text{tot}}^{\min}$ and $E_{\text{tot}}^{\max}$, couple the interval choices through:

$$E_{\text{tot}}^{\min} \leq \sum_{k=1}^{N} E_k \leq E_{\text{tot}}^{\max}, \tag{2.3}$$

where $E_{\text{tot}}^{\min}$ and $E_{\text{tot}}^{\max}$ bound the energy transferred over the complete profile. An exact energy requirement E_{req} is represented by setting both bounds to E_{req}. In EV charging, this condition preserves the requested total while permitting energy to move between intervals. The target and interval bounds must refer to the same measurement boundary; a target expressed as stored battery energy must also account for charging losses. Together with the vehicle's availability, these conditions determine which schedules can deliver the service. Compatible offers can then be aggregated for scheduling and assigned back to individual assets (Šikšnys et al. 2019).

The flexibility available from an asset depends on which of these adjustments it supports. Figure 2.3 illustrates fixed, shiftable, and elastic operation through household assets, including stationary storage.

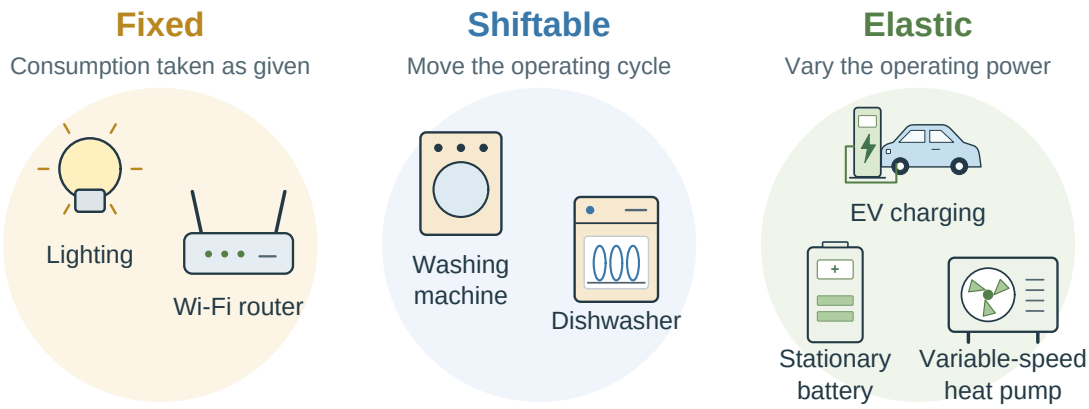


Figure 2.3: Household assets illustrating fixed, shiftable, and elastic operation. Stationary storage provides elastic operation through controllable charging and discharging.

Fixed or non-shiftable loads, such as lighting required while a room is occupied, contribute *non-controllable demand*: consumption taken as externally determined at the control timescale. Such demand can still be measured and included when scheduling controllable assets. In the FlexOffer description, a fixed profile has $t_{\text{es}} = t_{\text{sch}} = t_{\text{ls}}$ and $E_k^{\min} = E_k = E_k^{\max}$ for every interval k. Both its timing and its energy amounts are therefore specified in advance.

A *shiftable* appliance allows the start of a required operating cycle to move within a time window. A dishwasher loaded after dinner, for example, may begin at different times provided its cycle finishes by morning. Once started, the cycle may need to proceed without interruption. Its temporal flexibility is represented by $t_{\text{es}} < t_{\text{ls}}$, while $E_k^{\min} = E_k^{\max}$ retains the fixed cycle profile in each interval.

An *elastic* or modulable load allows its power to vary within an operating range. An EV charger may reduce power during an evening peak and increase it later, changing the interval energy amounts while retaining the departure target. Storage belongs to this category through its controllable charging and discharging power. A stationary battery, for example, can adjust its charging or discharging power within its power limits and usable energy range. This flexibility appears as $E_k^{\min} < E_k^{\max}$ in at least one interval, with Equation 2.3 expressing any total-energy requirement. For storage, the admissible interval choices also depend on the energy retained from earlier decisions and on conversion losses. A fixed start time can coexist with variable energy amounts, while the possibility of pausing operation depends on the asset and service (Fonseca, Ferreira, Landeck, et al. 2022; Hussain et al. 2023; Šikšnys et al. 2019).

The schedules permitted by this flexibility can lead to different energy bills and emissions. Their evaluation therefore also depends on the market arrangements and external signals associated with the REC's electricity use.

2.1.3 Energy Markets and Decision Signals

REC members can purchase electricity to meet their demand and sell production that is available for export, either through individual contracts or collective arrangements. Within the REC, energy-sharing agreements can allocate local production among participants, while a *local energy market* can organise transactions between buyers and sellers. These arrangements specify how exchanged energy is valued and how costs and revenues are distributed (Moret and Pinson 2019; Reis et al. 2021).

An *electricity tariff* specifies the prices and charges applied to consumption. A flat energy price applies the same unit rate throughout the day, while a *time-of-use tariff* assigns different rates to predefined periods. Under a *dynamic tariff*, the energy price follows variations in wholesale markets, such as the day-ahead market (Entidade Reguladora dos Serviços Energéticos 2026c; European Parliament and Council of the European Union 2019). The full bill may also include network charges, taxes, and fixed components.

Environmental information provides another basis for choosing when to use electricity. *Carbon intensity* expresses the greenhouse-gas emissions attributed to a unit of electricity, commonly in grams of CO_2 equivalent per kWh. It varies as the generation mix and exchanges between electricity systems change. A production-based value describes generation within an area, while a consumption-based value also accounts for imported electricity. The geographical boundary, time interval, and emissions-accounting method give the signal its meaning when comparing schedules (Electricity Maps 2026a,b).

Using these signals to guide asset operation requires information about the installation and its condition. The next subsection introduces how this information is acquired and the operational conditions that affect its use.

2.1.4 Information and Operational Conditions

Data acquisition obtains measurements and status information from the real-world installation through smart meters, sensors, and asset interfaces. These provide electricity readings, quantities such as room temperature, and operating states or diagnostics, with the measurement points determining which parts of the installation can be observed. Interfaces may also provide *control capabilities*, such as setting charging power, while feedback indicates whether a command was accepted and what the asset delivered (Gopstein et al. 2021).

The *sampling interval* is the time between successive measurements, and the *sampling frequency* is the number taken per unit of time. Acquisition, transmission, and storage can follow different schedules: a meter could sample every second, transmit a batch every minute, and retain quarter-hour aggregates, reducing data volume at the cost of temporal detail (Kelly and Knottenbelt 2015).

A *timestamp* identifies when a measurement or event occurred, while a separate reception time makes communication delays visible. Consistent clocks and time zones allow records from different assets to be aligned, and an aggregated energy value also needs the interval it covers (Stouffer et al. 2023).

A *forecast* estimates a future quantity from information available when it is issued; demand, PV production, and weather forecasts may be produced locally or obtained from external services (Nweye, Kaspar, et al. 2025). Each forecast needs an issue time and a *forecast horizon*, identifying how far into the future it extends, while retaining successive versions makes it possible to recover the information available at an earlier decision.

Data quality concerns the suitability of records for their intended use: *freshness* describes how recent a value is, *validity* concerns its expected format and physical interpretation, and *completeness* concerns the presence of required records.

Table 2.2 distinguishes availability, health, dependencies, and degraded operation through the charging example (Stouffer et al. 2023).

Table 2.2: Operational concepts and their interpretation in the charging example.

Concept	**Meaning**	**Charging example**
Availability	Whether an asset can be used for a service at a given time.	The vehicle is connected and charging is authorised.
Operational health	Condition of an asset or supporting component relative to its intended function.	A charger fault reduces the power it can deliver.
Operational dependency	An asset, component, or information source required by a function.	Charging adjusted to site headroom requires a site demand measurement.
Degraded operation	Continuation of a service with reduced capability or performance.	Charging continues under a local limit when demand-based adjustment is unavailable, where local protection supports this mode.

The preceding subsections establish the resources, signals, and information available for REC energy management. The next section introduces neural networks, explaining the computational models used to represent learned predictions and decisions before turning to RL.

2.2 Neural Networks and Learned Representations

The preceding section established the resources, signals, and operating conditions involved in REC energy management. Neural networks provide one way to transform this information into predictions or decisions. This section introduces their basic computation and learned representations, preparing the policy and value models used in the RL sections and the structured architectures developed in Section 2.7.

A *neural network* represents a function f_θ with learned parameters θ. In the vector formulation used here, the input $x \in \mathbb{R}^{d_{\text{in}}}$ and output $y = f_\theta(x) \in \mathbb{R}^{d_{\text{out}}}$ are numerical vectors; d_{in} and d_{out} count their components. Measurements supply numerical features, while categories require a numerical encoding. For a charger, three input components might represent SoC, time until departure, and electricity price, with one output component representing a power request (Goodfellow et al. 2016).

A *feed-forward network* passes information from input to output through successive transformations. A Multilayer Perceptron (MLP) uses dense layers, in which each unit combines all components from the preceding layer. Between input and output, one or more *hidden layers* compute intermediate numerical vectors.

The transformation producing the vector $h^{(\ell)}$ at layer ℓ is:

$$h^{(\ell)} = \sigma_\ell\left(W^{(\ell)}h^{(\ell-1)} + b^{(\ell)}\right), \qquad h^{(0)} = x, \tag{2.4}$$

where $W^{(\ell)}$ contains the learned weights, $b^{(\ell)}$ is a bias vector, and σ_ℓ is the layer's activation function. The weights combine the incoming values, the bias supplies an offset, and the activation transforms the result before it passes to the next layer.

Nonlinear activations allow stacked layers to represent nonlinear relationships. A Rectified Linear Unit (ReLU), for example, replaces negative values by zero and leaves non-negative values unchanged. Other common activations include the sigmoid, with outputs between zero and one, and the hyperbolic tangent, with outputs between -1 and 1. These functions produce continuous values rather than binary on/off decisions (Goodfellow et al. 2016).

The output's interpretation depends on the task. A numerical output can specify a continuous quantity such as charging power. For a discrete choice, such as charge or wait, the network can produce a vector of scores; a softmax transformation converts them into probabilities, from which a choice is selected. Section 2.7.3 returns to these action mappings.

A dense network is configured for a fixed number of input and output components. In the charging example, changing the SoC changes one of the three input values. Adding battery temperature would instead require a fourth input and a corresponding change to the network's input connections. Section 2.7 introduces ways to organise changing collections of resources, including padding, entity representations, and attention-based architectures.

Figure 2.4 follows the charging example through three input components, four hidden components, and one output. The four hidden values form a *learned representation* of the input, computed inside the network and passed to its output layer. They combine the supplied features using learned parameters; each hidden value need not correspond to a named physical quantity.

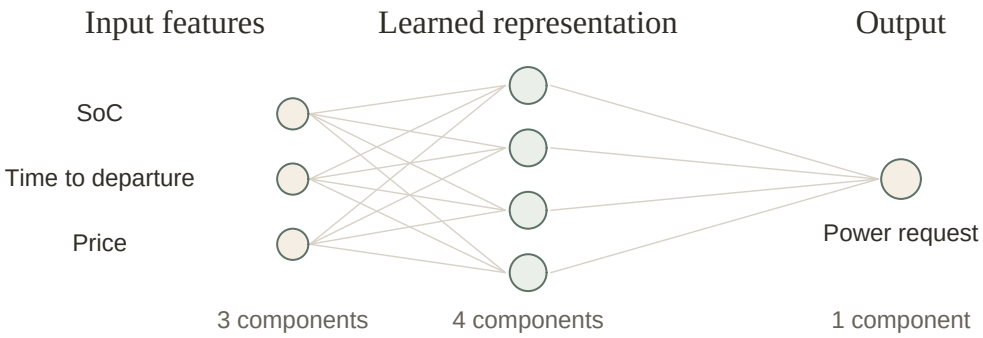


Figure 2.4: A feed-forward network mapping three charging-related features through a learned representation to one output. The illustrated layer widths are architectural choices. Original illustration based on Goodfellow et al. (2016).

Such a representation is often called an *embedding* when used as a numerical description for further computation. The part of the network that maps the inputs into this representation acts as an *encoder*; a *decoder* maps it into the required output. In the figure, the encoder produces the four hidden values and the decoder uses them to obtain the power request. Section 2.7 develops these roles for collections of resources.

Learning adjusts the parameters to minimise a *loss function*. With known target outputs, the loss can measure the discrepancy between predictions and targets. *Backpropagation* computes how the loss changes with the parameters, and an optimisation procedure uses these derivatives to update them (Goodfellow et al. 2016). Applying the trained network is *inference*. During fixed-policy execution, new measurements produce new outputs while the parameters remain unchanged.

This section established how a network transforms numerical inputs into outputs and how its parameters are learned. The next section introduces RL, where actions and their consequences supply the experience used to learn a decision rule.

2.3 Reinforcement Learning

RL is based on the idea that a decision-maker can improve its choices by experiencing their consequences. Some RL methods use the neural networks introduced in Section 2.2 to represent decision rules or estimates of future reward. This section introduces the foundations needed to understand the learning methods used in this thesis, covering the agent–environment interaction, the objective pursued over time, and the mechanisms used to learn a policy.

2.3.1 Sequential Decision-Making and Partial Observability

In sequential decision-making, an action can change both the immediate outcome and the choices available later. RL represents this process as an interaction between an *agent*, which selects actions, and an *environment*, which responds to them.

A Markov Decision Process (MDP) $\mathscr{M}$ formalises this interaction (Sutton and Barto 2018):

$$\mathscr{M} = (\mathscr{S}, \mathscr{A}, P, R, \gamma, \rho_0), \tag{2.5}$$

where $\mathscr{S}$ and $\mathscr{A}$ are the state and action spaces, $P(s' \mid s, a)$ is the transition distribution, and $R(s,a,s')$ is the expected immediate reward associated with a transition. The discount factor γ determines the weighting of future rewards, and ρ_0 specifies the initial-state distribution.

At decision step t, the *state* s_t describes the conditions relevant to the task, and the *action* a_t specifies the agent's choice. The *Markov property* means that s_t summarises the information from past interactions needed to predict the next state s_{t+1} and reward r_t, given a_t. Once s_t and a_t are known, the earlier sequence of states and actions adds no further information to that prediction. In the fully observable MDP formulation, the agent has access to s_t when choosing a_t (Sutton and Barto 2018).

A *policy* π determines how the agent selects a_t from s_t. Applying a_t leads to s_{t+1} and a *reward* r_t: a scalar signal that rates the outcome according to the task's objective, with larger values expressing more favourable outcomes. For a cost-minimisation objective, for example, the reward can be the negative cost incurred during the transition. Here r_t denotes the signal received after a_t, while $R(s,a,s')$ gives its expected value for a specified transition (Sutton and Barto 2018). Figure 2.5 illustrates this cycle, with s_{t+1} providing the information for the next decision.

EV charging illustrates this formulation within a REC. The agent controls charging at a site (for example, a household), while the environment includes the vehicle, charger, and the site's electrical installation. The state s_t includes the stored energy, departure target, time remaining, and relevant electrical conditions; a_t is the selected charging power. Including the remaining time distinguishes the same battery SoC early in the evening from that SoC shortly before departure. Charging increases the stored energy as the remaining time decreases, producing s_{t+1}, while r_t can reflect the cost of the energy delivered during the interval.

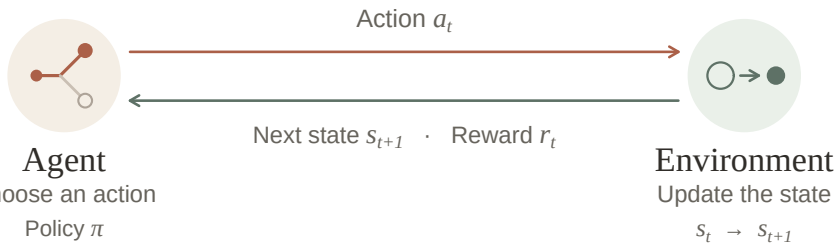


Figure 2.5: Agent–environment interaction in a fully observable MDP. The reward r_t follows action a_t, and the next state s_{t+1} becomes available to the agent. Original illustration based on the formulation of Sutton and Barto (2018).

In a real-world installation, the measurements available through sensors and interfaces determine which conditions the agent can observe, with noise or delays introducing further uncertainty (Dulac-Arnold et al. 2021). The agent receives an *observation* o_t containing the available information about s_t. The task is *partially observable* when this information provides an incomplete or uncertain description of the state relevant to its decisions, leaving some conditions affecting the outcome hidden from the agent.

A Partially Observable Markov Decision Process (POMDP) $\mathcal{M}_{\text{PO}}$ extends the MDP with an observation space and an observation model (Kaelbling et al. 1998):

$$\mathcal{M}_{\text{PO}} = (\mathcal{S}, \mathcal{A}, P, R, \mathcal{O}, Z, \gamma, \rho_0), \tag{2.6}$$

where $\mathcal{O}$ is the space of observations o_t and $Z(o_{t+1} \mid s_{t+1}, a_t)$ describes the distribution of the next observation after action a_t leads to state s_{t+1}. The remaining components retain their MDP meanings. The observation model describes how information about the underlying state becomes available to the agent.

For the charging agent, o_t might contain measured charging power and the reported departure time while omitting battery SoC. The agent can observe the rate of energy delivery, but the energy already stored remains hidden, leaving uncertainty about how much more charging is needed. The interaction still follows the cycle in Figure 2.5, with o_{t+1} replacing direct access to s_{t+1}; the agent can use past observations and actions to help infer the missing state information.

A *belief state* summarises this history as a probability distribution over possible states s_t, updated as actions are taken and new observations arrive. Given the POMDP model and an initial belief, it provides sufficient information for selecting actions, although exact updates can be computationally demanding (Kaelbling et al. 1998). Practical policies can instead retain a window of past observations or use recurrent memory to integrate information over time (Hausknecht and Stone 2015).

The next subsection explains how the rewards received across successive decisions are combined into a return, and how value functions use that return to assess a policy over the course of a task, including consequences that only emerge after several actions.

2.3.2 Returns, Episodes, Value Functions, and Policies

In the interaction described above, the agent receives a reward r_t after applying action a_t, giving feedback on that transition. Evaluating the policy requires following the consequences of successive choices, which the *return* G_t captures by accumulating rewards over the remainder of an episode or, for a continuing task, over a longer horizon.

An *episode* groups successive interactions over a defined period. In a REC, this may cover a day or a year of chronological data, containing many charging sessions and other asset operations. Lowercase t indexes a decision step, while uppercase T marks the end of the sequence, after the last action at $T-1$. A *terminal state* marks the end of the modelled task; a selected data window can also end while the real-world operation continues.

Over an episode ending at T, the discounted return G_t from step t combines the previously introduced rewards r_t as:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}. \tag{2.7}$$

Here r_{t+k} is the reward received k steps after t, weighted by γ^k, and the sum ends at the last transition before T. Continuing tasks can use an infinite discounted sum with $0 \leq \gamma < 1$, or an average-reward formulation. In finite tasks, $\gamma = 1$ retains equal weight for rewards throughout the remaining episode when the return is well defined (Sutton and Barto 2018).

If charging rewards are simply negative electricity costs and $\gamma = 1$, G_t is the negative total cost from the current step to the end of the episode. A discount factor below one reduces the contribution of distant rewards, with its effect depending on the decision interval. The same numerical value produces a different effective time horizon at one-minute and one-hour resolution.

A stochastic policy $\pi(a \mid s)$ gives a distribution over actions; a deterministic policy $\mu(s)$ selects one action. The *state-value function* $V^\pi(s)$ and *action-value function* $Q^\pi(s,a)$ describe expected returns under π:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s], \tag{2.8}$$

$$Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]. \tag{2.9}$$

Here $\mathbb{E}_\pi$ denotes expectation under policy π, conditional on the current state s or state–action pair (s,a). It accounts for transitions and, where applicable, policy randomness. For finite-horizon problems these functions can depend on the remaining time; the notation suppresses this dependence when time is included in the state. For the EV, $V^\pi(s)$ estimates the return from following the charging policy from the current conditions, while $Q^\pi(s,a)$ estimates the return from first choosing a particular charging power and then following that policy.

The Bellman relation expresses $Q^{\pi}(s,a)$ through the reward obtained now and the value of what follows:

$$Q^{\pi}(s,a) = \mathbb{E}\left[r_t + \gamma \mathbb{E}_{a' \sim \pi(\cdot|s_{t+1})} Q^{\pi}(s_{t+1}, a') \mid s_t = s, a_t = a\right]. \tag{2.10}$$

The outer expectation averages over the resulting reward r_t and next state s_{t+1}; the inner expectation averages over the next action a' drawn from π. At a terminal transition, the continuation value is zero. This recursion allows observed rewards and estimated future values to inform the same learning update. If the charging reward includes a penalty for energy missing at departure, an action with a small immediate cost can have a poor value because it leaves insufficient time to complete the service. The value function carries that later consequence into the present decision through the specified reward.

The *advantage* $A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$ compares an action with the policy's expected performance at the same state. For the EV, a positive advantage means that the selected charging power has a higher expected return than the policy achieves on average from those conditions. This relative learning signal can reduce variability when updating a policy.

These comparisons depend on what the reward treats as a favourable outcome, which leads to the next subsection's question of how to construct that signal from the task's objectives.

2.3.3 Reward Design and Objective Specification

The preceding subsection accumulated rewards into a return and used expected returns to evaluate a policy. The next step is to specify the reward function R introduced in the MDP, which determines what each transition contributes to that evaluation. In a REC, this means translating objectives such as reducing expenditure, delivering the requested departure energy, and maintaining thermal comfort into the learning signal. These objectives were introduced at the asset, site, and REC levels in Section 2.1.1 and Table 2.1.

Several such objectives can contribute to a single *composite reward* r_t through a weighted sum (Roijers et al. 2013):

$$r_t = \sum_{j=1}^{m} w_j f_j(s_t, a_t, s_{t+1}), \tag{2.11}$$

where m is the number of components, f_j is the signal for outcome j, and $w_j \geq 0$ sets its weight. Each f_j is expressed so that larger values are preferred; costs and shortfalls can therefore enter as negative terms. The components can conflict. Postponing EV charging may reduce expenditure while increasing the energy missing at departure, so a gain in one term can be offset by a loss in another. The weights express the relative importance of these outcomes in the combined signal.

The numerical scale of each f_j also affects this balance. For components measured in euros and kWh, the weights must account for both their numerical ranges and their intended relative importance. Normalisation can provide a reference scale, while changing either that scale or the weights changes the trade-off.

A service requirement can enter both the reward and the constraints governing the decision. A departure-energy penalty encourages learning to value readiness, while a constraint can require a charging schedule to deliver the requested energy whenever the available time and capacity permit. Electrical limits likewise restrict admissible actions. Reward terms guide the preference among outcomes; enforcing a requirement needs a mechanism that restricts the decisions applied, as introduced in Section 2.3.5.

Reward design also determines when feedback is supplied. A *sparse* reward occurs only at selected events, such as task completion or failure, whereas a *dense* reward provides more frequent feedback throughout the interaction. Sparse feedback can express the final outcome directly, although identifying which earlier decisions contributed to it becomes more difficult. In the charging example, expenditure may contribute at every interval while readiness is assessed at each departure within the episode; discounting then affects the relative influence of these contributions. Dense feedback can help learning recognise progress, but rewarding an intermediate quantity may favour behaviour that improves that quantity while weakening the final outcome (Sutton and Barto 2018).

Reward shaping supplies additional feedback during learning to make useful intermediate progress easier to recognise. For an EV, this can help relate successive charging decisions to the energy needed at departure, before that service is finally assessed. In potential-based shaping, a function Φ assigns a numerical measure of progress to each state and defines a shaped reward r_t^{Φ} (Ng et al. 1999):

$$r_t^{\Phi} = r_t + \gamma\Phi(s_{t+1}) - \Phi(s_t). \tag{2.12}$$

Here $\Phi(s_t)$ and $\Phi(s_{t+1})$ evaluate the current and next states, and γ is the discount factor already used in the return. Their discounted difference is added to the original reward r_t. In the charging example, Φ could assign higher values as the energy shortfall decreases, providing feedback about progress between successive decisions.

The reward function now specifies the feedback available to the learning process, while the return combines that feedback over time. The next subsection explains how experience from successive interactions is used to estimate values and improve the policy.

2.3.4 Learning from Experience

Learning uses the feedback defined above to improve subsequent decisions. Each interaction records the conditions faced by the agent, the action it applied, and the resulting reward and next state. In the REC example, a charging decision reveals how the selected power affects stored energy, electricity expenditure, and the time remaining to meet the departure target.

Value-based learning estimates the return associated with states or actions and uses those estimates to choose decisions. For the charging agent, this means learning how promising charging now, waiting, or selecting a lower power is from the current conditions, considering their consequences through the rest of the episode. Experience revises these estimates, helping the agent choose actions with higher expected return (Sutton and Barto 2018; Watkins and Dayan 1992).

Policy-based learning adjusts the rule that selects actions. For example, the charging agent can learn to assign a higher probability to power settings that perform better than expected in the conditions it encounters. The parameters θ control this rule, represented by a differentiable policy π_θ. The policy-gradient principle relates an improvement in expected return to terms of the form:

$$\mathbb{E}[\nabla_\theta \log \pi_\theta(a_t \mid s_t) A^{\pi_\theta}(s_t, a_t)], \tag{2.13}$$

where ∇_θ differentiates with respect to the policy parameters, $\log \pi_\theta$ is the log-probability of the selected action, and A^{π_θ} is its advantage. The expectation averages over the states and actions used by the method. The advantage guides the update towards making better-than-expected actions more likely and worse-than-expected actions less likely in comparable conditions (Sutton, McAllester, et al. 1999).

An *actor–critic* method brings these two roles together. The actor selects an action, while the critic estimates the values used to assess it and guide learning. In the REC example, the actor chooses charging power and the critic helps judge whether that choice improves the expected return.

Exploration allows the agent to learn about alternatives to its current preferred action. A charging agent might try two permitted power levels in similar conditions to learn how their consequences differ. A stochastic policy can sample different actions, while a deterministic policy can use a perturbation during experience collection. In a real-world installation, these choices must respect the available operating limits.

Entropy regularisation encourages a stochastic policy to retain variety in its action choices while improving return. For the charging agent, this can help avoid concentrating too early on a single power setting. Its coefficient controls the strength of that encouragement (Haarnoja et al. 2018).

On-policy and *off-policy* learning describe how the policy that collects experience relates to the one being learned. An on-policy charging agent learns from decisions made with the policy currently being improved. An off-policy method can also learn from decisions made by an earlier or different controller. The distinction identifies which policy generated the data; online and offline learning instead distinguish whether new interactions are collected during learning.

A *replay buffer* stores transitions so that an off-policy method can reuse them in later updates. A recorded charging decision can therefore contribute to learning more than once, even after the controller has changed. Learning entirely from a fixed archive is developed in Section 2.5.

Building on the learning methods described above, the next subsection explains how knowledge from an existing controller can help the agent learn useful behaviour, and how operational constraints govern the choices it can apply during learning and subsequent operation.

2.3.5 Prior Knowledge and Constrained Decisions

The learning process described above can begin with knowledge of how the installation is already operated. In a REC, a Rule-Based Controller (RBC) may already schedule charging using electricity prices, battery SoC, and departure requirements. This *prior knowledge* can help the agent learn useful behaviour through examples, guidance during exploration, or adjustments to an existing control rule.

BC learns to reproduce a controller's or expert's decisions from demonstrations that pair the conditions encountered with the actions chosen. For the charging agent, these examples associate the vehicle's needs and the electricity price with a selected charging power. Supervised learning fits the policy to these choices; when rewards and subsequent states are also recorded, the transitions can support RL (Ross et al. 2011).

Guided exploration uses existing knowledge to direct the agent towards promising actions while collecting experience. A controller's recommended charging power, for example, can provide a starting point for trying nearby permitted power levels. The resulting rewards and state changes then inform the learner's own decisions.

Residual learning retains the existing controller as part of the decision rule and learns an adjustment to its output. In the charging example, the controller supplies a reference power and the learned policy increases or decreases that value in response to the current conditions (Johannink et al. 2019).

These ways of using prior knowledge operate within the installation's physical and service requirements. *Constraints* express which actions or sequences of actions are admissible. A constrained MDP can maximise expected reward subject to budgets d_j on expected accumulated costs c_j, written generically as:

$$\mathbb{E}_\pi\left[\sum_t \gamma^t c_j(s_t, a_t)\right] \leq d_j, \qquad j = 1, \ldots, m, \tag{2.14}$$

where j indexes the m constraints, $c_j(s_t, a_t)$ is the cost at step t, and d_j is its permitted accumulated budget. The expectation averages discounted costs over trajectories generated by π (Achiam et al. 2017). Such a condition concerns an expectation over trajectories. An instantaneous equipment limit instead applies to every relevant command, and a deadline condition depends on a sequence of decisions. These meanings require different enforcement mechanisms.

An action mask can remove inadmissible discrete choices before selection. For continuous decisions, a projection or safety filter can map a proposed action into an admissible set determined by a constraint model. A supervisory controller may supply a fallback when the proposed decision cannot be used (Dalal et al. 2018; Elsayed-Aly et al. 2021).

This section introduced how an agent learns a policy from rewards and experience, with prior knowledge and constraints guiding its decisions. The next section extends these foundations to MARL, where agents interact within a shared environment such as a REC.

2.4 Multi-Agent Reinforcement Learning

The preceding section considered how one agent learns to act on its environment. In MARL, several agents learn while their decisions jointly affect that environment. Consider a REC in which each site has an agent controlling its EV charging. If two agents charge during the same low-price period, their simultaneous charging can create a REC demand peak. This section extends the decision model to such interactions, then explains how learning is organised and how individual decisions contribute to a shared outcome.

2.4.1 Markov Games and Multi-Agent Objectives

The MDP described one agent's action at each step. With several agents, the environment responds to their combined choices, represented by a *joint action*. A Markov game $\mathscr{G}$, also called a stochastic game, expresses this interaction (Littman 1994; Shapley 1953). A discounted formulation is:

$$\mathscr{G} = (\mathscr{N}, \mathscr{S}, \{\mathscr{A}_i\}_{i\in\mathscr{N}}, P, \{R_i\}_{i\in\mathscr{N}}, \gamma, \rho_0), \tag{2.15}$$

where $\mathscr{N} = \{1,\dots,n\}$ is the fixed agent set and $\mathscr{A}_i$ is agent i's action space. The joint action $a = (a_1,\dots,a_n)$ belongs to the product of those spaces. The transition distribution is now $P(s' \mid s,a)$, and each agent has an expected reward $R_i(s,a,s')$. As in the MDP, $\mathscr{S}$ is the state space, γ the discount factor, and ρ_0 the initial-state distribution.

In the REC example, $a_{i,t}$ is the charging power selected at site i, and a_t collects all sites' choices at that step. Their combined charging changes the vehicles' stored energy and the REC's total demand. Each agent's reward can therefore depend on both its own choice and the choices made elsewhere in the REC.

A *fully cooperative* game gives all agents the same reward and a common performance objective. For the charging agents, that reward could combine REC electricity expenditure, peak demand, and the energy missing at vehicle departures. Each site's decisions then contribute to the outcome that all agents learn to improve.

A *general-sum* game allows agents to have different rewards, expressing objectives that may partly coincide and partly conflict. In a REC, members may pursue individual expenditure targets alongside a REC objective. In a *zero-sum* game, the agents' rewards sum to zero for every transition. With two agents, one agent's reward is therefore the negative of the other's (Moret and Pinson 2019; Wang, Hong, et al. 2022).

The community-control problem studied in this thesis assumes agreement on the collective objectives and individual service commitments to be considered. Reducing a community peak, for example, is pursued alongside the energy required by each vehicle at departure. These commitments enter the control formulation through constraints, reward terms, and assessment of the service delivered. Cooperation is therefore a choice in the formulation of the control problem. A common reward defines the fully cooperative case, while formulations combining local and community rewards

retain agent-specific objectives whose effects on coordination must be assessed. Each controller's reward construction is specified in the corresponding method description. Assessing individual outcomes also remains necessary to establish how the benefits and burdens of coordination are distributed.

Under *partial observability*, each agent receives an observation $o_{i,t}$ that does not necessarily reveal all the state information relevant to its decisions. The charging agent might observe its own vehicle's SoC and departure time together with a published electricity price. Its action can depend on its local history $h_{i,t}$ of observations and previous actions.

The individual policies together define the joint policy π. When agents sample independently conditional on their local histories, it factorises as:

$$\pi(a_t \mid h_t) = \prod_{i \in \mathcal{N}} \pi_i(a_{i,t} \mid h_{i,t}). \tag{2.16}$$

Here $a_t = (a_{1,t}, \ldots, a_{n,t})$ and $h_t = (h_{1,t}, \ldots, h_{n,t})$ collect the agents' actions and local histories, and π_i is agent i's policy. The product combines the probabilities of their individual choices into the probability of the joint action. In the example, each site's agent chooses a charging power from its available information, and those choices are applied together.

A Decentralised Partially Observable Markov Decision Process (Dec-POMDP) formalises cooperation under a common reward, with agents acting from local observations and action histories (Oliehoek and Amato 2016).

2.4.2 Learning and Execution Organisation

The joint policy can be learned and applied through different organisational arrangements. Learning determines which experience and information support policy updates, while execution determines how actions are selected in operation. Parameter sharing concerns whether several agents use the same learned decision rule.

Independent learning lets each agent improve its policy while treating the other agents as part of the environment. In the REC example, each site learns from its available observations, charging decisions, and rewards (Claus and Boutilier 1998; Tan 1993).

Non-stationarity arises from the other agents' changing behaviour during learning. If one site learns to move its charging to an earlier period, the REC demand experienced by another site changes. The second agent is therefore learning while part of its environment is also adapting. Recorded experience can contain transitions from several stages of this adaptation (Foerster, Nardelli, et al. 2017).

Centralised execution selects the joint action through one decision process using combined information. A REC controller could, for example, select all sites' charging powers together. With *decentralised execution*, each site's agent selects its own action from its inputs, which can include local measurements and shared signals or messages.

Centralised Training with Decentralised Execution (CTDE) uses joint information during learning while retaining local action selection in operation (Albrecht et al. 2024; Lowe et al. 2017). Figure 2.6 illustrates how a centralised critic supports policy updates during training, while each site's actor selects charging power from its available inputs during execution.

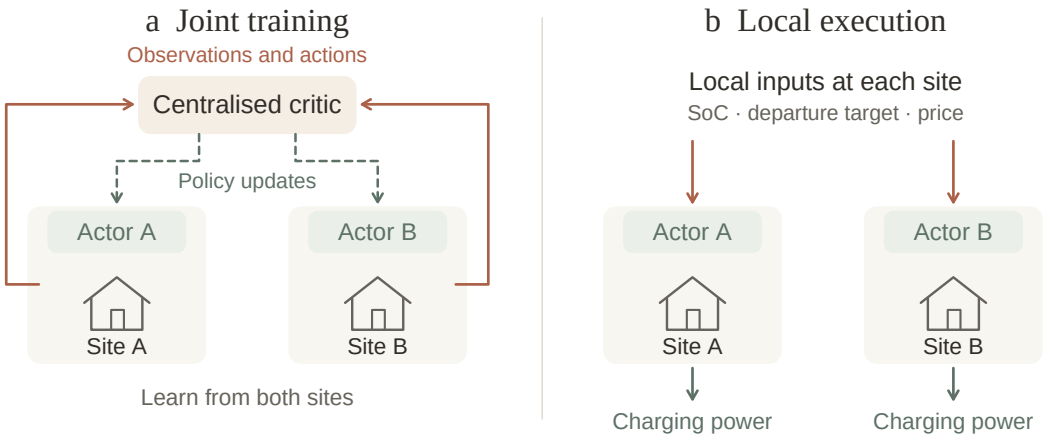


Figure 2.6: Centralised training and decentralised execution for two sites. Joint observations and actions support policy updates during training; local inputs determine charging decisions during execution. Original illustration based on Lowe et al. (2017).

Parameter sharing allows several agents to use the same policy parameters. Two sites can apply the same charging rule to different SoCs, prices, and departure times, producing different actions from that common model. Their experience can jointly train the shared rule. The policy uses compatible input and action representations, with asset capabilities or roles included when needed (Albrecht et al. 2024; Yu, Velu, et al. 2022).

Table 2.3 separates these three organisational choices. For example, sites can share policy parameters, train with joint information, and still choose their actions locally during operation.

The next subsection examines how these learning and execution arrangements support coordinated charging, and how a shared outcome can provide useful feedback for the individual decisions that produced it.

Table 2.3: Three organisational choices in multi-agent learning and execution.

Dimension	**Choices**	**REC-related interpretation**
Learning information	Local information or information from several agents.	Updates use records from one site or combine observations and actions from several sites.
Action selection	Centralised or decentralised execution.	One process selects all charging powers, or each site's agent selects its own.
Policy parameters	Separate or shared parameters.	Agents use separate charging policies or apply the same model to their respective inputs.

2.4.3 Coordination and Credit Assignment

The sites' separate charging choices combine into a REC demand profile. *Coordination* means that these choices account for their effects on the shared outcome. For example, agents can spread charging across the available low-price intervals while meeting departure targets, reducing the peak created when all vehicles charge together.

Credit assignment concerns how a collective result informs the learning of individual agents. If the REC peak falls, every agent may receive a better shared reward, although their charging decisions contributed differently to that reduction. Learning needs a way to relate the improvement to those individual choices.

Counterfactual credit evaluates one agent's alternatives while holding the other agents' actions fixed. A representative counterfactual baseline $b_i(s, a_{-i})$ is:

$$b_i(s, a_{-i}) = \mathbb{E}_{a_i' \sim \pi_i} \left[Q(s, (a_i', a_{-i})) \right], \tag{2.17}$$

Here a_{-i} contains all actions except agent i's, and Q evaluates a joint action in state s. The expectation averages over alternative actions a_i' drawn from π_i, retaining the other agents' actions and the same local information. The difference between the selected joint action's value and this baseline gives agent i an advantage signal (Foerster, Farquhar, et al. 2018).

Value decomposition relates individual utility estimates to a joint action value. Each utility provides a local score for an agent's action, and their combination estimates the collective return. When the combination is monotonic, increasing an individual utility cannot decrease the joint estimate. This property allows each agent to select its highest-scoring action consistently with maximising that estimate (Rashid et al. 2018). In the REC example, each site can select a charging action from its local scores while joint training relates their combination to the REC outcome.

The next section turns from how agents learn together to the experience available for learning, explaining how policies can be trained from a fixed archive of recorded interactions through offline RL.

2.5 Offline Reinforcement Learning

The preceding sections described learning through interaction with an environment. *Offline RL* instead learns from a fixed dataset of interactions recorded before training begins (Levine et al. 2020). The thesis uses this setting to study learning from existing controller records; Section 3.4.1 reviews the corresponding methods.

This dataset is an archive of past experience. Each recorded *transition* links an observation or state to the action taken, its reward, and the next observation or state. For a charger, a record might associate the vehicle's SoC and departure time with the selected power, the charging cost, and the resulting SoC. Consecutive records can form a trajectory.

The learner repeatedly uses these records to estimate values or improve a policy, without collecting new interactions during training. Figure 2.7 separates data collection, learning from the fixed dataset, and subsequent use of the policy.



Figure 2.7: Offline learning uses a fixed dataset collected beforehand. The trained policy subsequently receives new observations and selects actions.

Off-policy learning, introduced in Section 2.3.4, separates the policy being improved from the policy that generated the experience. A site can therefore learn from earlier charging decisions made by another controller while continuing to collect new interactions. Offline RL uses this separation within a fixed dataset: the learner revisits the same records throughout training, and its updated policy is not used to obtain additional experience during that phase (Levine et al. 2020).

The usefulness of the archive depends on its coverage of conditions and decisions. Suppose that the recorded charging controller always starts at maximum power when an EV arrives with a low SoC. Those records provide evidence about immediate charging in that condition, but little direct evidence about postponing it or selecting a lower power. Even a large archive can therefore leave some state–action combinations poorly represented. A new policy may select such combinations, and an inaccurate value estimate can make them appear more attractive than the recorded alternatives. Because offline training collects no additional interactions, the learner cannot test those choices and correct the estimate through new experience. This dependence on dataset coverage motivates the methods reviewed in Section 3.4.1 (Levine et al. 2020).

BC treats each recorded action as a training target. Given the vehicle's condition, the model learns to reproduce the power selected by the collecting controller. Offline RL instead uses the recorded rewards and transitions to learn about accumulated performance and improve the policy towards a return objective. Both approaches use past decisions, but their learning signals differ: cloning follows the demonstrated action, whereas offline RL evaluates decisions through their recorded consequences.

This section established how recorded experience supports policy learning. The next section turns to how decisions are organised, introducing a hierarchy between REC coordination and local control.

2.6 Hierarchical Reinforcement Learning and Coordination

REC coordination and local control concern different parts of the same charging process. A coordinator may guide when sites consume energy, while each site's controller selects the power requested from its chargers. A *hierarchy* links these responsibilities by making local decisions depend on a higher-level choice. Hierarchical Reinforcement Learning (HRL) learns policies within this organisation. The following subsections explain the levels, timing, and signals used in the thesis, providing the foundations for understanding the methods reviewed in Section 3.3.1.

2.6.1 Decision Levels and Temporal Abstraction

Section 2.1.1 organised the energy system into assets, sites, and RECs, each with its own scope of operation. A control hierarchy assigns decisions across such scopes. In the running example, the REC level supplies a coordination signal, and the site level uses that signal alongside its vehicles' requirements to choose charging powers. These responsibilities define the decision levels considered here.

The timing of a decision follows its role in this organisation. A coordinator might update its signal hourly while sites choose charging power every fifteen minutes, making four local decisions under the same signal. The levels can also share a decision interval while retaining different responsibilities.

Temporal abstraction groups several elementary steps into one extended behaviour. In the *options* framework, an option has an initiation set describing when it can begin, an internal policy selecting its actions, and a termination rule (Sutton, Precup, et al. 1999). A charging option might start when a vehicle connects and end when an energy target is reached or the vehicle departs.

For an option lasting τ steps, the discounted accumulated reward $R_t^{(\tau)}$ is:

$$R_t^{(\tau)} = \sum_{k=0}^{\tau-1} \gamma^k r_{t+k}. \tag{2.18}$$

Here t is the starting step, k counts steps within the option, r_{t+k} is the reward following that step's action, and γ discounts later rewards. Subsequent value is discounted by γ^τ. Variable option durations give the higher-level process a semi-Markov formulation (Sutton, Precup, et al. 1999).

These levels and intervals specify when decisions occur. The next subsection explains what the higher-level choice communicates to a local policy.

2.6.2 Goals, Skills, and Coordination Signals

A *goal* describes the condition that a local controller should reach. For example, the higher level may assign an amount of energy to deliver before departure. The local policy then determines how charging power develops over the available intervals to pursue that target.

A *skill* provides a policy for a particular behaviour, such as charging steadily or adjusting charging to local generation. Selecting a skill delegates the subsequent power choices to that policy, which responds to the observations it receives. The higher level therefore selects a behaviour whose individual actions are determined locally (Nachum et al. 2018; Vezhnevets et al. 2017).

A *power budget* specifies how much capacity a site can use for a given purpose during an interval. If two chargers share a 6 kW charging budget, the site controller can distribute that power between them according to their remaining energy needs and departure times. The budget sets the combined allowance while leaving the individual allocations to local control.

An *incentive signal*, such as a price multiplier, influences how the local controller assesses possible schedules. A signal that makes the current interval less costly can favour charging during that interval. The resulting power request still follows from the site's conditions, including whether a vehicle is connected and how much energy it requires.

A coordinator policy π_{H} can select a signal g_k at time t_k, which then conditions the local policy π_i:

$$g_k \sim \pi_{\mathrm{H}}(\cdot \mid o^{\mathrm{H}}_{t_k}), \qquad a_{i,t} \sim \pi_i(\cdot \mid h_{i,t}, g_k), \quad t_k \le t < t_{k+1}. \tag{2.19}$$

Here $o^{\mathrm{H}}_{t_k}$ is the coordinator's observation, $h_{i,t}$ is agent i's local information history, and $a_{i,t}$ is its action. The symbol $\sim$ denotes sampling, and the dot stands for the possible signal or action. The signal remains in use until the next coordinator decision at t_{k+1}.

The same incentive can produce different actions. A site with an unmet charging requirement may request power, while another with a fully charged vehicle selects zero. Figure 2.8 connects these local responses to the shared signal and its update interval.

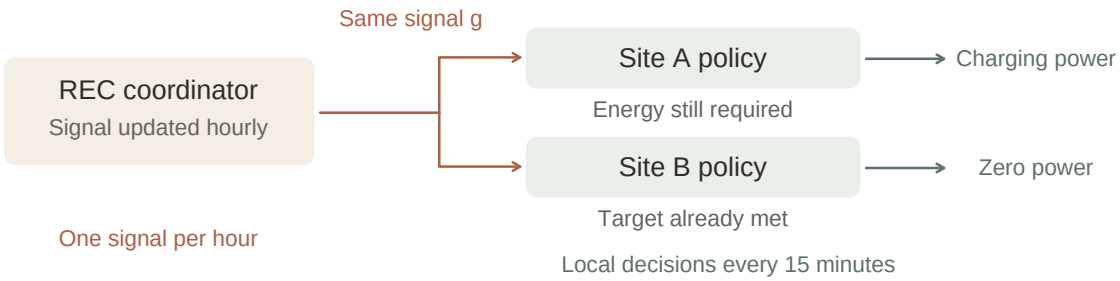


Figure 2.8: One REC signal guides successive local decisions. Each site combines it with its own charging conditions to select a power request.

Hierarchy establishes how decisions are distributed across levels. The next section examines the computation within each policy, from resource observations to the actions returned to individual assets.

2.7 Structured Policy Architectures

Section 2.2 introduced networks that transform numerical inputs into learned representations and outputs. Energy control also gives those quantities a physical organisation, with observations and actions belonging to particular sites and assets. This section develops the entity representations, attention mechanisms, and modular outputs needed to understand the architectures reviewed in Chapter 3 and the method formulations developed in Chapters 6 and 7.

2.7.1 Entity and Relational Representations

The numerical inputs introduced in Section 2.2 must describe the resources whose operation is being controlled. When a site contains several assets, their observations need an arrangement that preserves what each value refers to. A fixed observation vector provides one such arrangement by assigning a declared position to each feature.

For a site with a battery and two chargers, one group of positions can describe the battery and two further groups the vehicles connected to the chargers. The network receives the values in that order. Updating a vehicle's SoC changes a value; adding a third charger's observations changes the vector's length. The following representations organise these inputs around the resources themselves.

An *entity representation* groups attributes around the resource they describe. Let x_j contain the attributes of entity j. An encoder can map each x_j into an embedding, using the same parameters for entities with a common attribute format. Two chargers can therefore use the same encoder while producing different embeddings from their respective SoCs and departure requirements. The number of entities and the number of attributes describing each entity are separate dimensions.

A *set representation* treats the entities as an unordered collection. One construction uses a shared encoder ϕ and an aggregation to produce a site representation z:

$$z = \rho\left(\sum_{j=1}^{m} \phi(x_j)\right), \tag{2.20}$$

where m is the number of entities, $\phi(x_j)$ is the embedding of entity j, and ρ transforms the sum into the site representation (Zaheer et al. 2017). Each embedding has a common width, so the sum retains that width as m changes. Adding a compatible charger introduces another application of the encoder and another term in the sum, using the same learned functions.

Reordering the elements leaves this sum unchanged. This property is *permutation invariance*. A site-level value estimate, for example, can describe the same collection whether charger C1 is listed before or after charger C2. The ordering of the records does not change which physical resources are represented.

Resource-specific outputs use the complementary property of *permutation equivariance*. When the input entities are reordered, their corresponding outputs follow the same reordering. If the policy returns one power request per charger, the request for C1 remains associated with C1 after the list order changes. A model can preserve each entity's embedding alongside an aggregate site context to retain this association.

A *graph representation* additionally describes relationships between entities. Nodes can represent a battery, chargers, and their connection point; edges can indicate which assets share that connection. *Message passing* updates a node's representation using information from its neighbours through shared transformations (Gilmer et al. 2017). In the site example, the connection node can collect information about the assets that draw on its capacity. Figure 2.9 shows the same resources as a vector, a set, and a graph.

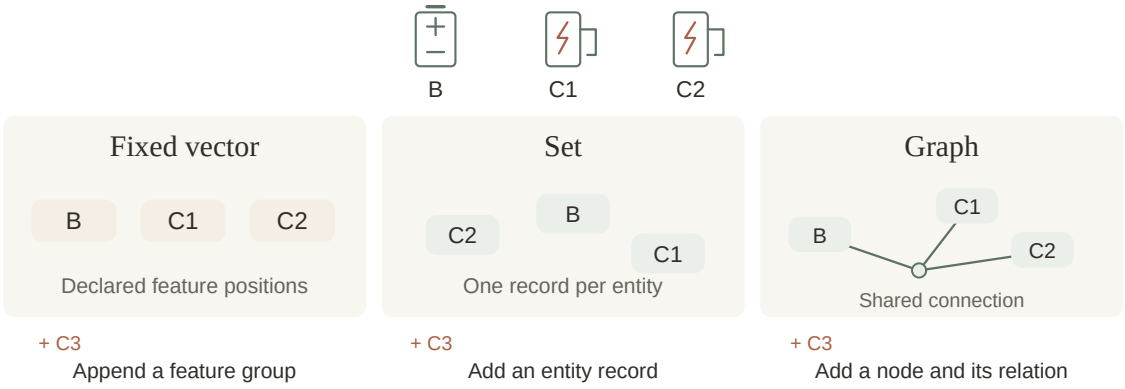


Figure 2.9: Three representations of the same site resources. Vector positions identify feature groups, a set groups attributes by entity, and a graph includes declared relationships. The lower row illustrates adding another charger record.

Padding fills unused positions when several collections are stored in an array with a common allocated size. A presence mask identifies which positions contain entities and which are padding. For example, a batch can allocate four charger positions per site while a mask marks the two populated positions at a particular site. The allocated length, the active entity count, and the embedding width describe different aspects of that representation.

These representations establish which elements enter the policy and how they are associated with resources. The next subsection introduces attention, explaining how the representation of one element can incorporate information from others before decisions are produced.

2.7.2 Attention and Transformer Architectures

An entity's decision can depend on information elsewhere in the site. A charger's remaining energy requirement, for example, can be considered alongside another charger's deadline and the available connection capacity. *Attention* forms a weighted combination of input representations, with the weights computed from learned comparisons between them (Vaswani et al. 2017).

The computation uses three projections of the available embeddings. A *query* participates in the comparison for the element being updated, while each *key* participates in the comparison for an element whose information may be used. The associated *value* contains the information to combine. For a charger query, matching it against keys from the site produces weights for the corresponding value vectors.

Scaled dot-product attention operates on query, key, and value matrices Q, K, and V:

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}} + M\right)V, \tag{2.21}$$

where rows of Q contain query vectors, rows of K contain key vectors, and rows of V contain their associated value vectors. The query and key width is d_k, and T denotes transposition. The optional additive mask M is zero for permitted entries and negative infinity for excluded entries.

The query and key dot products form comparison scores, scaled by $\sqrt{d_k}$. The row-wise softmax turns each query's scores into non-negative weights summing to one over the permitted keys. Multiplication by V then produces one weighted value combination for each query (Vaswani et al. 2017).

In *self-attention*, the queries, keys, and values are projected from the same collection. Each site entity can thus receive information from other entities. In *cross-attention*, the queries come from one collection and the keys and values from another. For example, queries associated with requested charging outputs can retrieve information from a collection of observation embeddings.

The number of rows and the width of each row play different roles. In self-attention over m entities, the query and key matrices each have m rows, while their projection widths are architectural choices. Changing m changes the number of comparisons and output representations. A later aggregation can produce a single site representation, or subsequent computation can retain one representation per entity (Lee, Lee, et al. 2019).

Multi-head attention performs several attention computations using separate learned projections. Their results are concatenated and projected into the output representation. Each head supplies its own combination of values. In the running example, the combined representation draws on several learned comparisons among the same site entities.

A *Transformer block* integrates attention with a feed-forward transformation applied to each element. The feed-forward part transforms the features within an element's representation after attention has combined information across elements. Stacking blocks repeats these operations, producing successive representations of the collection (Vaswani et al. 2017).

Residual connections add a sublayer's input to its transformed output, retaining a direct path for the incoming representation. *Layer normalisation* normalises features within each element and applies learned scale and offset parameters. Figure 2.10 relates an attention head to a Transformer block using residual addition and normalisation after each sublayer.

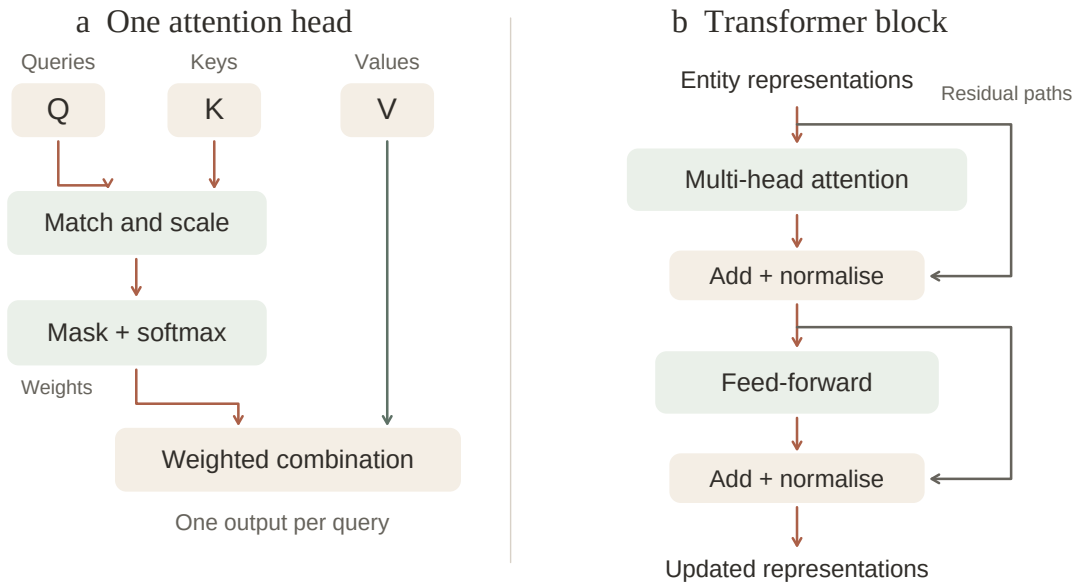


Figure 2.10: Attention and a Transformer block. An attention head computes weights and combines values. Multiple heads form the attention sublayer of the illustrated block; residual paths feed the additions before layer normalisation. Original illustration based on Vaswani et al. (2017).

Attention and Transformer blocks produce representations that combine the selected information. The next subsection explains how a policy organises those computations into modules and maps their outputs to actions with defined meanings for individual resources.

2.7.3 Modular Policies and Action Decoding

A *modular policy* divides its computation into components with specified input or output roles. An encoder can prepare observations, a shared component can combine their representations, and a decoder can produce the quantities used to select an action. Components may be specialised for different resource types or reused across several instances of the same type (Huang, Mordatch, et al. 2020; Luketina et al. 2025).

An *output head* maps a representation into a particular group of outputs. A fixed head for two chargers can produce a two-component vector with one power request in each declared position. The number and meaning of those positions belong to the head's specification. This is the output-side counterpart of the fixed feature vector introduced in Section 2.2.

A shared resource decoder applies the same function to each resource representation. For three compatible chargers, it produces three outputs, each associated with its source charger. Each evaluation has a fixed output width, while the number of evaluations follows the charger count. Figure 2.11 illustrates this reuse alongside a separate decoder for another action type.

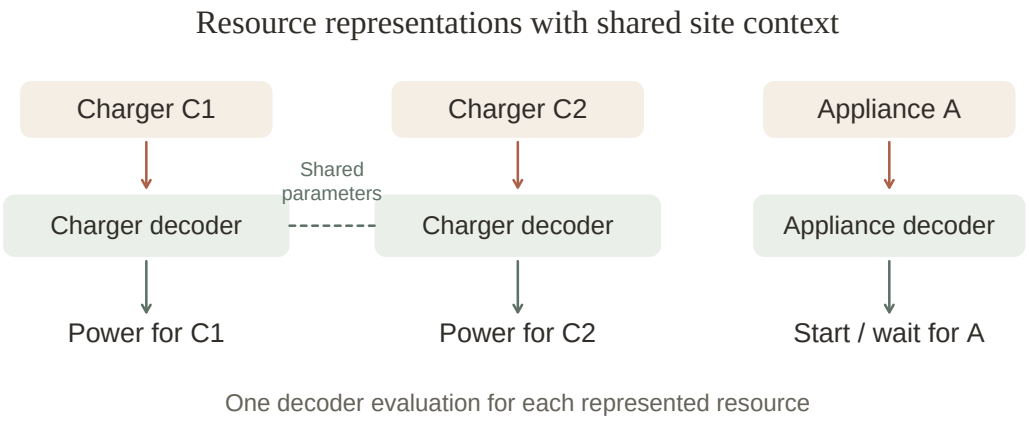


Figure 2.11: Shared decoding with resource-specific outputs. The same charger decoder processes the representations of C1 and C2, producing a separate power request for each. A different decoder produces the appliance's discrete choice. Repeated decoder symbols denote shared parameters.

A *parameterised action* combines a discrete choice with numerical parameters (Li, Tang, et al. 2022). A charger might select idle, charge, or discharge, with a power magnitude attached to an active mode. The decoder associates the mode and magnitude with the charger, while conversion to a physical request establishes its units and permitted range.

Resource count and resource type also describe different changes. Adding another charger with the same observation and action format adds another instance of the existing computation. Introducing an asset with a different sensor description or command family changes the input or output mapping required by that type. Type-specific encoders and decoders declare those mappings, while the resource association identifies which instance supplies or receives each quantity.

Structural compatibility concerns whether the architecture can process observations and produce actions for a changed configuration. Control generalisation concerns decision quality and any further learning needed, while computational scalability concerns time and memory requirements as resources increase. A shared decoder can serve a third compatible charger, but its vehicle adds a service request competing for connection capacity. The resulting schedule and additional processing must therefore be assessed. Section 3.3 examines these distinctions through the capabilities and evaluation conditions reported in the literature.

The representations, information exchange, and output mappings described here complete the architectural foundations used in the thesis. The chapter summary brings them together with the energy and learning concepts before the next chapter examines the methods built from them.

2.8 Chapter Summary

This chapter presented the energy and learning concepts needed to follow the controller formulations developed in the thesis. It began by distinguishing the members of a REC, the sites where energy is used, and the assets whose operation can be controlled. Flexibility described the adjustments available within physical limits and service requirements. The charging example illustrated how those adjustments depended on the vehicle's energy needs, connection period, and operating conditions.

Neural networks provided the computational basis for mapping observations to predictions or decisions. The chapter explained how numerical inputs pass through learned intermediate representations to produce outputs, distinguishing changes in input values from changes in the number of components expected by a dense network. These distinctions prepared the later discussion of resource representations.

RL placed those decisions within a sequence of actions and consequences. Rewards expressed the objective, returns combined rewards over time, and value functions assessed the expected consequences of following a policy. MARL extended this account to several interacting agents. In the REC example, individually selected charging powers combined into a shared demand profile, connecting local decisions with the need for coordination.

Offline RL explained how a policy could be learned from a fixed collection of recorded interactions. Hierarchical control then organised decisions across responsibilities and time scales, with a REC-level choice guiding local controllers. The same signal could produce different charging powers because each site also acted on its own vehicle requirements.

Structured policy architectures connected these decision mechanisms to the resources being represented and controlled. Entity-based inputs preserved the association between observations and assets, attention combined information across resources, and shared decoders produced actions for individual instances. The chapter distinguished adding another compatible resource from introducing a new resource type, establishing the concepts needed to discuss changes in a policy's inputs and outputs.

The chapter did not examine the detailed foundations of data acquisition and preparation, state estimation, flexibility prediction, or demand and generation forecasting, because its scope was the learning and decision mechanisms used for REC energy control. These subjects remain necessary for other parts of the project, including supplying controller inputs and constructing the experimental setting. Keeping their detailed treatment separate allowed the chapter to follow a continuous path from the energy problem to the formulation of learned decisions.

Chapter 3 now examines how existing methods use these foundations to control energy resources and identifies the gaps that underpin the research problems introduced in Chapter 1.

Chapter 3

Related Work and Research Gaps

This chapter examines how existing research addresses the difficulties of using MARL in real-world control. It develops the problem introduced in Chapter 1 by analysing the challenges reported in the literature, the methods proposed to address them, and the gaps that remain. The concepts established in Chapter 2 support the comparison of learning mechanisms, policy architectures, and applications. The review focuses on control in RECs and draws on studies from other domains where needed to examine related problems.

3.1 Scope and Review Method

This chapter uses a systematised review, incorporating explicit search, selection, and synthesis procedures as characterised by Grant and Booth (2009). Figure 3.1 presents the adopted stages and their methodological basis.

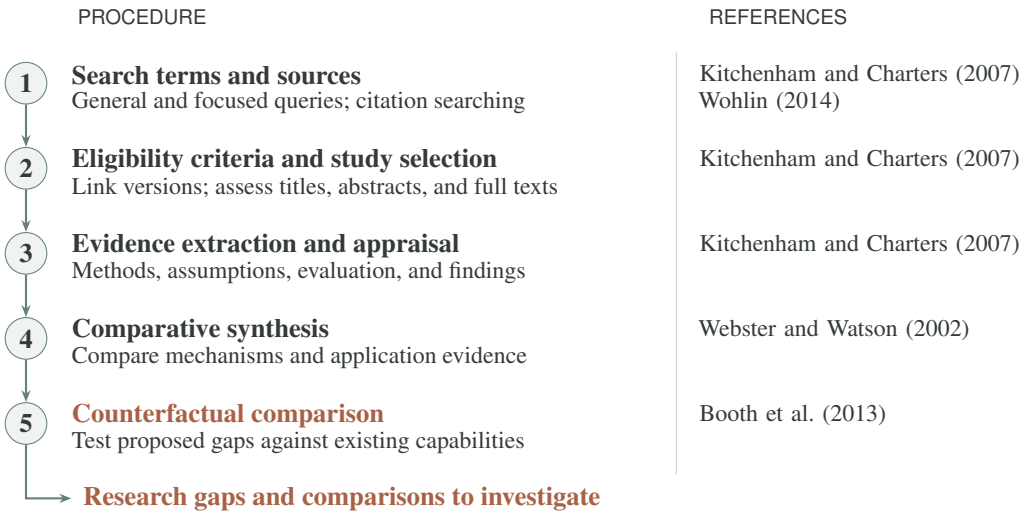


Figure 3.1: Review stages and methodological sources.

The search terms, eligibility criteria, and common extraction form adapt the procedures described by Kitchenham and Charters (2007). Section 3.1.1 presents the search terms and sources, including citation searching through reference lists using the backward-search procedure described by Wohlin (2014). Section 3.1.2 sets out the criteria used to select studies.

Comparative synthesis uses the concept matrices proposed by Webster and Watson (2002) to organise findings across studies. Counterfactual comparison examines how existing approaches could address a proposed gap under their reported conditions, informed by the use of disconfirming cases in qualitative evidence synthesis described by Booth et al. (2013). These comparisons can narrow or reject proposed gaps and are applied in each review section.

3.1.1 Search Terms and Sources

The search combined two routes. The application review in Section 3.2 focused on community energy management, including local trading, coordinated buildings, and microgrids. Queries combined learning terms with energy settings and resources; further searches sought physical experiments and deployments. Sections 3.3–3.6 examined mechanisms for each dimension, including single-agent RL, conventional control, and supporting infrastructure. The searches include Model Predictive Control (MPC) and Hardware-in-the-Loop (HIL) testing. Table 3.1 groups the search terms.

Table 3.1: Term families used in the application and mechanism searches.

Search purpose	Terms used in focused queries
Energy applications (Section 3.2)	multi-agent reinforcement learning; MARL; energy community; prosumer; peer-to-peer; multi-building; microgrid; shared storage; electric vehicles
Physical and operational evidence (Section 3.2)	real-world; field; deployment; pilot; experimental; hardware-in-the-loop; physical; occupied
Scalability and dynamic reconfiguration (Section 3.3)	parameter sharing; dynamic population; heterogeneous agents; graph policies; entity Transformers; modular policies; variable action space
Deployability (Section 3.4)	offline RL; behavioural cloning; policy transfer; adaptation; semantic models; application portability; RL operations
Safety (Section 3.5)	safe reinforcement learning; constrained learning; predictive safety filter; model predictive control; barrier functions; shielding; runtime assurance
Robustness and recovery (Section 3.6)	fault-tolerant MARL; adversarial learning; communication failures; delay; state estimation; fault mitigation; supervisory switching; recovery

Searches located papers on publisher websites, in conference proceedings, institutional repositories, and arXiv. Reference lists and research dossiers supplied further candidates; original methods and evaluation passages were examined before inclusion.

3.1.2 Eligibility Criteria and Study Selection

Eligibility covered community-energy applications and studies of the four research dimensions, with sufficient detail to assess their assumptions and evidence. Titles and abstracts guided initial screening, followed by examination of the original methods and evaluation against the criteria in Table 3.2. Studies were considered whether their results showed benefits, limitations, or both.

Table 3.2: Eligibility criteria for the chapter's review protocol.

Criterion	Inclusion	Exclusion or separate treatment
Relevance	A relevant MARL challenge, research dimension, or control alternative.	Unrelated applications and supporting systems with no identified effect on control decisions.
Contribution	Theory, methods, architectures, applications, and reviews with assessable assumptions or evidence.	Promotional or unsupported accounts; abstract-only candidates await assessment.
Domain	Energy applications (Section 3.2); relevant mechanisms from any domain (Sections 3.3–3.6).	Domain similarity alone, without a relevant control contribution.
Full text	English or Portuguese full text, including author manuscripts.	Unavailable full texts await assessment; other languages are outside the reading scope.
Status	Journal and conference papers; identifiable preprints and technical reports where relevant.	Versions of the same study share a single review record.

3.2 MARL Applications and Real-World Challenges

MARL applications in community energy management involve building equipment, household flexibility, and local electricity networks. This section compares simulation studies with real-world evidence to examine how far learned controllers can be installed, operated, and maintained under real-world conditions, and which gaps remain.

The search, conducted in October 2026, identified 129 distinct publications: 100 initially identified and 29 added through complementary searches and snowballing. Of these, 59 were not included in the synthesis: 53 lacked sufficient full-text assessment, and 6 fell outside the scope of the external comparison or its language criteria. The 70 selected publications comprise 50 simulation studies, including independent learners in Multi-Agent Systems (MASs), 10 studies with real-world evidence, and 10 supporting reviews, frameworks, or benchmarks.

3.2.1 Simulation-Based Evidence

The simulation literature covers energy communities that share generation and flexibility, alongside related microgrid and building-control problems (Charbonnier, Morstyn, et al. 2022a; Keren et al. 2024; Sajid, Li, Qi, et al. 2026). These studies address different parts of REC operation, from exchanges between members to control of their equipment. Their learning arrangements range from cooperative MARL to independent RL agents interacting through a common market or energy system.

Local energy-trading studies address exchange between community members directly. MARL is used to learn renewable-energy sharing and the purchases and sales of consumers and prosumers (Prasad and Dusparic 2019; Pu et al. 2022; Qiu, Wang, et al. 2021). Later applications combine trading with battery decisions and network conditions, extending the comparison to larger groups and more varied resources (Feng and Liu 2025; Lou et al. 2026; Ye et al. 2025). Reported outcomes include greater local exchange and lower expenditure relative to the reference strategies. The community market of Pereira et al. (2022) instead contains independently trained RL traders alongside untrained participants, with different outcomes across these groups. A multi-agent market can therefore contain learning agents without a shared cooperative objective.

Storage studies examine how household and shared assets contribute to community objectives. Nweye, Sankaranarayanan, et al. (2023) use independent RL controllers for residential batteries, whereas Savino et al. (2025) use coordinated MARL for electrical and thermal storage across buildings. Shared-storage control also couples the decisions of several users through one resource (Liu and Chen 2023). Microgrid applications extend coordination to electricity and heat, or to several storage technologies and external trading partners (Harrold et al. 2022; Wang, Cui, et al. 2023). Other work combines household battery trading with frequency regulation or studies exchanges between dairy farms (Shah, Cruz Victorio, et al. 2026; Xu, Zhao, et al. 2026). These settings inform REC resource coordination, while differing in the ownership, services, and market arrangements represented.

Residential demand-control studies use MARL to schedule EV charging, heating, and household loads against renewable availability and prices (Charbonnier, Morstyn, et al. 2022b; Dusparic, Harris, et al. 2013; Dusparic, Taylor, Marinescu, Cahill, et al. 2015; Dusparic, Taylor, Marinescu, Golpayegani, et al. 2017; Kaewdornhan et al. 2023). Evaluations include 200 modelled homes with PV, batteries, and EVs, and fast DR from up to 1000 thermostatically controlled loads (Mai et al. 2024; Yang, Hao, et al. 2019). In a community with heating and vehicles, Wilk et al. (2024) compare cooperative and competitive control through collective performance and the distribution of benefits among buildings. These studies address a central REC control task, coordinating members' flexibility while retaining their charging and comfort requirements.

Building-control studies examine related coordination problems within and across buildings. Multi-agent controllers regulate energy use and thermal conditions, including under changing weather and future climate scenarios (Hosseini, Erba, et al. 2025; Nik and Hosseini 2023; Yu, Sun, et al. 2021). Their agents may represent zones within one building rather than autonomous

community members. The grid-aware experiments of Pigott et al. (2022) additionally assess how building decisions affect network voltages. This evidence informs thermal control and network interactions in RECs, while the community setting adds coordination between members with separate resources and service requirements.

The full comparison of the 50 selected simulation applications is provided in Table A.1, in Appendix A. The studies are grouped by their principal application, with the scenario, assessment, and main observation recorded for each.

The comparison also shows that simulation already addresses some operational difficulties. Communication failures, weather-related outages, forecast uncertainty, and changing demand are explicitly examined in several studies (Marinescu, Dusparic, et al. 2017; Shah, Barrett, et al. 2025; Wu, Chiu, et al. 2026; Zhou et al. 2022). Their results establish capabilities under the disturbances represented in those experiments. Other comparisons expose limitations in the evidence itself. Zhang, Qiu, et al. (2025) show that converged learning rewards can accompany inferior market equilibria, while Nweye, Liu, et al. (2022) find that the experience available for training affects subsequent building-control performance. These findings help identify which claims need to be tested when moving to an installation.

3.2.2 Real-World Evidence

The real-world evidence includes tests with physical equipment in hybrid and laboratory settings, as well as deployments in occupied buildings and residential networks.

Two studies test MARL through hybrid systems combining physical equipment with simulated resources. The EV experiment of Marinescu, Taylor, et al. (2019) represents residential charging through accelerated time and aggregate hardware, while the voltage-control experiment of Darbali-Zamora et al. (2026) couples physical generation equipment to a simulated network. These tests validate components of community control at the equipment interface. In the latter, a hardware step takes two to three seconds, compared with one second during training, showing that transfer also requires the controller to accommodate the installation's execution rate.

Three further studies apply MARL to heating and building control, covering thermal resources relevant to RECs. A two-zone laboratory exposes the controller to real thermal dynamics and outdoor weather (Blad et al. 2023).

The occupied-building studies additionally involve existing management systems and occupants. A building energy-management framework combining collective intelligence with RL (CIRLEM) reports successful controller interaction in about 59% of scheduled steps, with software, data-interface, and equipment problems affecting operation (Hosseini, Papadopoulos, et al. 2026). The Building Automation and Control Network (BACnet) protocol carries equipment commands in the radiant-heating study of Wang, Ren, et al. (2026). Execution and communication errors required manual reversion, and affected days were excluded from the performance comparison. These studies demonstrate learned control of building resources, with operational continuity depending on the surrounding software and equipment.

Among the reviewed MARL deployments, the residential network in Shenzhen reported by Fu et al. (2026) is the closest to REC-scale operation in its coordination of PV and storage serving 150 households. The system combines MARL with optimisation and conventional power control, and its reported improvements concern that integrated arrangement. The study does not isolate the contribution of MARL or document household-level coordination of vehicles, heating, and service requests. It therefore leaves open how learned coordination contributes to the gains and how the approach would support the wider resource combinations examined in simulated communities.

REC-oriented systems have also been deployed without MARL. Patrício et al. (2026) use rule-based agents to control residential batteries, with individual self-consumption tested in three homes and collective-sharing alternatives evaluated in simulation. The EV aggregation of Kamboj et al. (2011) likewise delivers a grid service without RL. Other residential and multi-building deployments use learning without establishing cooperative MARL in the implemented controller. The water-heating system of Kazmi et al. (2019) retained a learned tank model while simplifying the proposed RL cycle, and the heating study of Moshari et al. (2026) reports coordinated operation without sufficiently specifying cooperative learning. Their relevance lies in the community resources and control interfaces operated in practice, which can also support the deployment of MARL.

Of the 10 real-world studies, 6 report MARL across 2 hybrid tests, 3 building-control studies, and 1 residential-network deployment. The other 4 comprise 2 systems without RL and 2 learning-based deployments whose implemented control is not established as cooperative MARL. These studies include Distributed Energy Resources (DERs) and Heating, Ventilation, and Air Conditioning (HVAC) equipment, alongside the charging and storage assets introduced in Chapter 2. Table 3.3 brings together their controllers, equipment, scale, evaluation periods, and operational observations.

Table 3.3: Real-world evidence in community-relevant energy applications

Study	Control and setting	Equipment commanded	Scale and period	Operational observation
Physical equipment coupled to simulation				
Marinescu, Taylor, et al. (2019)	MARL; hybrid power test	Aggregate inverter and load representing EV demand	230 virtual homes, 90 virtual EVs; accelerated profiles	Learned aggregate charging is executed through physical power equipment.
Darbali-Zamora et al. (2026)	Multiple learning DER agents; hybrid power test	Physical PV inverter and wind emulator; other resources simulated	Four DERs represented; approximately 110 minutes	Voltage control operates with a slower hardware step than the training model.

Continued on the next page.

Table 3.3: Real-world evidence in community-relevant energy applications (continued).

Study	Control and setting	Equipment commanded	Scale and period	Operational observation
Laboratory and occupied-building MARL				
Blad et al. (2023)	MARL; unoccupied heating laboratory	Heating supply and valves	Two zones; January–March 2022	Physical thermal control under outdoor weather; no occupant interaction.
Hosseini, Papadopoulos, et al. (2026)	Collaborative learning; occupied living laboratory	HVAC setpoints	Eight rooms; April 2023–March 2024	Energy/flexibility evaluation; successful controller interaction in about 59% of scheduled steps.
Wang, Ren, et al. (2026)	MARL; occupied commercial building	Radiant-heating valves through BACnet	18 zones; 42-day alternating-controller campaign	Energy and comfort comparison; manual reversion and exclusion of faulty periods.
Residential-network MARL				
Fu et al. (2026)	MARL with optimisation and conventional control	Network power exchange, PV and storage	150 households; June–August 2024	Reports improved network operation; effects belong to the integrated system.
Operational comparators with a different control structure				
Kazmi et al. (2019)	Shared learning; simplified deployed controller	Heat-pump water heating	53 homes; 11 months of control	Learned tank model used in practice; complete proposed RL cycle omitted.
Moshari et al. (2026)	Multi-building RL; cooperative MARL unconfirmed	Heating offsets sent to existing management systems	13 occupied buildings; 138 days	Physical coordination and reported savings against historical references.
Patrício et al. (2026)	Rule-based MAS; no RL	Residential batteries	Three homes; seven days; five-minute commands	Physical individual self-consumption; collective-sharing alternatives simulated.
Kamboj et al. (2011)	Multi-agent EV aggregation; no MARL	Vehicle charging and discharging for grid regulation	Three, then five real EVs; 2010 operational trials	Vehicles deliver an actual grid service; establishes a non-learning precedent.

3.2.3 Remaining Research Gaps

Simulation studies are more numerous and cover more combinations of community resources than the reviewed real-world deployments. Few deployments address the community as a whole, and some use other control methods, including the REC-oriented rule-based system of Patrício et al. (2026). MARL has been used to control real equipment in hybrid tests, buildings, and a residential network. In the Shenzhen deployment, however, MARL operates alongside optimisation and conventional control, and the reported results do not isolate its contribution (Fu et al. 2026). Evidence is therefore still limited on the benefit attributable to MARL in sustained coordination of a community's generation, storage, and flexible demand.

The studies also show that the control approaches require different forms of preparation and operating information. In the rule-based deployment of Patrício et al. (2026), battery decisions follow configured conditions using measurements and equipment limits; implementation still requires data and device integration. The optimisation stage of Fu et al. (2026) uses an explicit network model and constraints to coordinate decisions, alongside learned proposals and conventional control. Learning instead prepares a policy from experience and evaluates it using the observations supplied during operation; the quality of that experience and the organisation of interaction affect what is learned (Nweye, Liu, et al. 2022; Savino et al. 2025). Rules can therefore supply operational knowledge, optimisation can reconcile modelled constraints, and learning can capture consequences observed across interacting decisions. This synthesis motivates assessing MARL with comparable information and control authority, accounting for model or forecast preparation, training, integration, and decision-time computation. The relevant comparison includes both alternative controllers and the contribution of learning within hybrid systems.

Scalability is supported by simulations in which MARL coordinates large populations of resources (Charbonnier, Morstyn, et al. 2022b; Mai et al. 2024; Savino et al. 2025; Yang, Hao, et al. 2019). The real-world cases evaluate specified installations, leaving less evidence about repeated additions or substitutions of members and equipment during operation. The remaining question extends from controlling a large population to retaining coordination when its composition changes.

Deployability depends on the preparation and integration required at each installation. Reported deployments require data alignment and, in some cases, simplification of the proposed controller (Hosseini, Papadopoulos, et al. 2026; Kazmi et al. 2019). Reusing a policy at another site therefore raises questions about the data, adaptation, and integration effort it requires, consistent with the implementation concerns identified in building-control and EV-coordination reviews (Michailidis et al. 2024; Nazloo and Musilek 2026).

Safety is already considered in simulation through equipment limits and service requirements (Chen, Yan, et al. 2026; Khan et al. 2026; Wilk et al. 2024). The remaining question concerns how these constraints are respected during coordinated execution on real equipment, where the action performed may differ from the action selected by the policy. Community control must account for the limits of individual resources and the effects of their combined actions on shared capacity and participants' service requirements.

Robustness has been tested against communication failures and outages in simulation (Wu, Chiu, et al. 2026; Zhou et al. 2022). Real-world studies also report execution errors, equipment problems, and manual intervention (Moshari et al. 2026; Wang, Ren, et al. 2026). The unresolved questions concern how useful service continues when resources or information become unavailable, and how coordinated operation is restored afterwards. This requires assessing degraded operation and recovery alongside performance under normal conditions.

Community-wide performance also leaves participant-level questions open. Lower aggregate expenditure can coexist with unequal benefits (Pereira et al. 2022; Wilk et al. 2024), while occupant intervention changes the actions actually executed (Hosseini, Papadopoulos, et al. 2026). Reviews of DR and resource coordination identify participation and information exchange as implementation concerns (Charbonnier, Morstyn, et al. 2022a; Olorunfemi and Nwulu 2023). Fairness, understandable decisions, privacy, and cybersecurity thus remain part of the wider application problem established in Section 1.2.

Finally, the reported gains are difficult to compare across deployments. Historical baselines, model predictions, alternating controllers, and hardware experiments answer different evaluation questions, and campaign duration can exceed the period of successful learned control. Reporting resource coverage, active control periods, interventions, and the basis of comparison would help determine which benefits persist across installations. This need for comparable operational evidence is consistent with the real-building RL review of Kannari et al. (2025), whose scope extends beyond MARL. It also requires distinguishing the contribution of learned decisions from that of the surrounding control system.

Table 3.4 distinguishes established capabilities from the application questions they leave open.

The following sections examine methods relevant to four aspects of these questions. The review begins with scalability and dynamic reconfiguration, then considers deployability, safety, and robustness.

Table 3.4: Established evidence and remaining questions for community application.

Concern	Evidence available	Remaining application question	Thesis connection
Population and resource changes	Large simulated populations; real-world operation in specified configurations (Fu et al. 2026; Mai et al. 2024; Savino et al. 2025).	How can control accommodate growing membership and changing equipment while retaining useful coordination?	Scalability and dynamic reconfiguration; Section 3.3.
Preparation and integration	Shared learning and physical actuation; site-specific simplification and data alignment (Hosseini, Papadopoulos, et al. 2026; Kazmi et al. 2019).	How much data, adaptation, and integration effort is required at a new installation?	Deployability; Section 3.4.
Physical limits and service	Constraints and comfort considered in simulation and occupied buildings (Chen, Yan, et al. 2026; Wang, Ren, et al. 2026; Wilk et al. 2024).	How are equipment limits and participants' requirements respected during coordinated execution?	Safety; Section 3.5.
Disruption and recovery	Simulated fault tests; operational failures and manual intervention (Wang, Ren, et al. 2026; Wu, Chiu, et al. 2026; Zhou et al. 2022).	How can useful service continue with impaired resources, and how is coordinated operation restored?	Robustness; Section 3.6.
Participation and fairness	Unequal trading outcomes; comfort and charging objectives; occupant overrides (Hosseini, Papadopoulos, et al. 2026; Pereira et al. 2022; Wilk et al. 2024).	Who benefits from coordination, and how are individual needs and interventions accommodated?	Wider concerns established in Section 1.2.1.
Information and trust	Reviews discuss information exchange and practical adoption of multi-agent control (Charbonnier, Morstyn, et al. 2022a; Olorunfemi and Nwulu 2023).	What information can be shared, protected, and explained sufficiently for dependable use?	Privacy, cybersecurity, and explainability in Section 1.2.1.
Comparable operational evidence	Different baselines, active periods, and reporting of implementation (Hosseini, Papadopoulos, et al. 2026; Kannari et al. 2025; Moshari et al. 2026).	Which benefits persist across sites and operating conditions, and what intervention do they require?	Evaluation across the four dimensions; Section 3.7.

3.3 Scalability and Dynamic Reconfiguration

The applications reviewed in Section 3.2 show that MARL can coordinate numerous energy resources, but provide limited evidence of continued control as community members and equipment change. Continued community control must accommodate both changes in membership, which alter the population to be coordinated, and additions or removals of equipment within a home, which change its observations and available actions.

Section 3.3.1 first examines learning and coordination across large populations. Section 3.3.2 reviews architectures for changing members and resources, including evidence of policy reuse. Section 3.3.3 compares their demonstrated capabilities and identifies the remaining question for REC operation.

3.3.1 Learning and Coordination at Scale

Scalability first concerns methods that support learning and coordinated decisions across a large number of agents. Their use of storage and flexible demand affects both other members and the community's exchange with the grid. The review introduces actor–critic methods, then examines how shared experience, value factorisation, and selective interactions support larger populations. Changes to members and equipment during operation are examined in Section 3.3.2.

Actor–critic methods

An actor selects an agent's action, while a critic estimates the return used to improve its behaviour. CTDE gives the critic information about several agents during learning while retaining local observations for each actor during operation. This arrangement supports learning from interactions across the population without requiring one actor to select every action.

Three established methods illustrate choices within this arrangement. They are Multi-Agent Deep Deterministic Policy Gradient (MADDPG), Multi-Agent Twin Delayed Deep Deterministic Policy Gradient (MATD3), and Multi-Agent Proximal Policy Optimisation (MAPPO). The last extends Proximal Policy Optimisation (PPO) to the multi-agent setting. Figure 3.2 compares their critic information, actor updates, and local execution.

Lowe et al. (2017) introduce MADDPG with a deterministic actor and a centralised action-value critic for each agent. Joint observations and actions let the critic assess a decision in the context of the others, while training reuses experience from a replay buffer. This distributes action selection across agents, although the critic's joint input grows with the represented population.

Ackermann et al. (2019) develop MATD3 to reduce overestimation in these value estimates, using twin critics, target-action smoothing, and delayed actor updates. The lower target estimate guides critic learning. Their particle and simulated robotic experiments address the reliability of learning among interacting agents. MATD3 retains MADDPG's joint-information requirement, so supporting larger populations still depends on how that information is represented and shared.

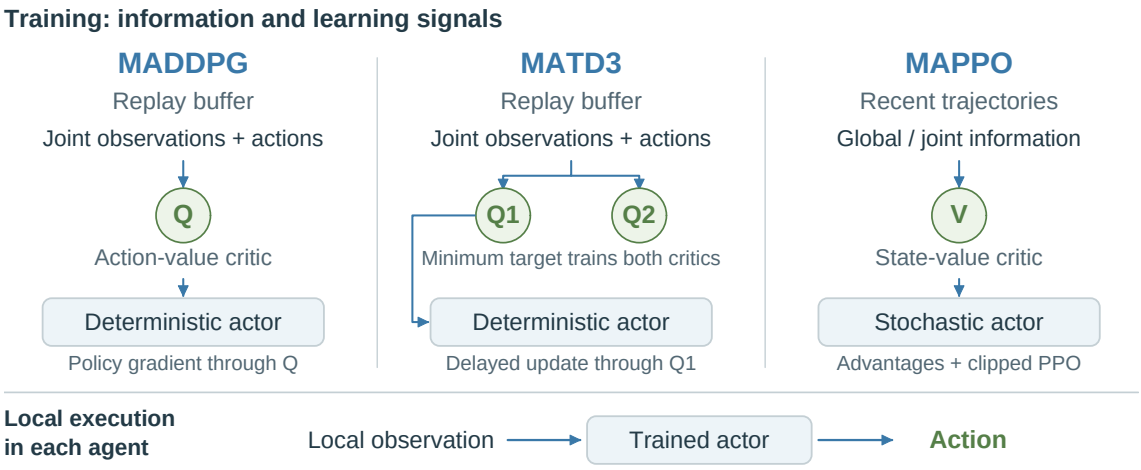


Figure 3.2: Critic information and actor updates in MADDPG, MATD3, and MAPPO, followed by their common local execution pattern. Critic-to-actor arrows indicate learning signals. Original schematic based on Ackermann et al. (2019), Lowe et al. (2017), and Yu, Velu, et al. (2022).

Yu, Velu, et al. (2022) evaluate MAPPO, which trains stochastic actors from recent trajectories using a centralised state-value critic and clipped policy updates. The critic supplies advantage estimates, while the clipped objective moderates changes to the policy. Their cooperative benchmarks show that this on-policy alternative can compete with more specialised MARL methods. Shared policies for homogeneous agents let several agents contribute experience to one learner. Critic inputs, value normalisation, and data reuse also affect learning performance.

Other actor–critic methods address credit assignment and the organisation of updates. Counterfactual Multi-Agent (COMA) policy gradients compare an agent's action with alternatives while retaining the others' actions (Foerster, Farquhar, et al. 2018). Heterogeneous-Agent Proximal Policy Optimisation (HAPPO) and Heterogeneous-Agent Trust Region Policy Optimisation (HATRPO) use sequential updates without requiring parameter sharing (Kuba et al. 2022).

Shared learning and value factorisation

The learning rule alone leaves open how experience and value estimates are organised across the population. One option is to share a policy, so that several agents contribute observations, actions, and outcomes to the same learning process. Each agent then uses that policy with its own observations. Terry et al. (2023) examine parameter sharing among heterogeneous agents, including differences in observation and action spaces. Agent characteristics or identifying information let the common policy distinguish their circumstances. This is relevant to communities whose members make similar decisions, such as charging batteries, but have different demands and storage capacities.

A complementary choice concerns how individual decisions contribute to the value of cooperation. Value factorisation constructs a joint value estimate from smaller components, making that structure explicit. Rashid et al. (2018) introduce a monotonic mixing network that combines individual value estimates into a team value, allowing locally selected greedy actions to agree

with the joint choice. For continuous actions, Peng et al. (2021) develop Factored Multi-Agent Centralised Policy Gradients (FACMAC), combining a factored critic with policy updates that account for the joint action. Sharing reuses learning across agents, while factorisation organises the joint value being learned. The two choices can support the same controller.

Residential coordination provides an application of this structured learning. Charbonnier, Peng, et al. (2025) apply FACMAC with optimisation-informed learning to communities of up to 30 homes. Their simulations report reduced learning effort and operating costs relative to the compared controllers, with training for the evaluated configurations. The study examines how value decomposition and optimisation guidance contribute to the results. It shows how organising the training procedure can make coordinated residential control more practicable as more homes are considered.

Organising interactions across the population

Beyond sharing experience and structuring value estimates, methods can simplify the interactions considered by each agent. Mean-field RL represents the influence of other agents through an aggregate, such as an average action (Yang, Luo, et al. 2018). Localised learning retains interactions within a neighbourhood, with approximation guarantees when the influence of distant agents decays sufficiently rapidly (Qu et al. 2020). These approaches describe the population at different resolutions. In community energy control, an aggregate may describe collective demand, whereas a neighbourhood can retain the relationships between members sharing a local network connection.

Graphs and hierarchies organise these interactions explicitly. Böhmer et al. (2020) factor the team value into individual and pairwise terms on a coordination graph, using message passing to select joint actions. In the feudal multi-agent approach of Ahilan and Dayan (2019), managers set subgoals and workers select actions to pursue them. Hierarchy offers a way to separate community allocation from local equipment decisions, provided that the subgoals convey the requirements each level must meet.

Attention learns which information matters within the represented population. Savino et al. (2025) use an attention-based central critic with a separate actor for each building. Their CityLearn simulations include districts of up to 73 buildings in California and 100 in Texas, with electrical and domestic-hot-water storage. The reported results improve the trade-off between operating cost and district load variation, connecting selective information use with control across a substantial population.

These methods organise learning for a specified population through shared experience, structured values, and selective coordination. Continued control after members or equipment change also requires suitable observation and action interfaces. Section 3.3.2 examines architectures that accommodate such changes.

3.3.2 Architectures for Variable Agent and Resource Configurations

When members or equipment enter or leave a community, the controller's observations and commands must reflect the resulting configuration. This subsection reviews complementary design choices for accommodating these changes, beginning with set aggregation and attention, followed by entity-to-action mappings, graph representations, and modular reuse.

Set aggregation and attention for entity representation

Representing resources as separate entities frees their measurements from permanent positions in a fixed input vector. In Deep Sets, Zaheer et al. (2017) apply a shared transformation to each element, followed by an aggregation independent of element order. The same operations process different numbers of elements. Retaining each resource's representation alongside the pooled summary makes individual circumstances and collective conditions available for control.

Pooling forms a common summary, while attention lets each entity incorporate information from others. The Transformer introduces attention over input representations (Vaswani et al. 2017), and the Set Transformer develops it for unordered collections (Lee, Lee, et al. 2019). In a REC, attention could relate a battery's available capacity to a charging request elsewhere. The supplied attributes and relationships, including resource ownership, determine what the policy can learn from that interaction.

Entity-to-action mappings and action semantics

Entity representations describe what is present, while the action interface determines which entities receive decisions and what those decisions mean. Universal Policy Decoupling with Transformers (UPDeT) connects entity observations to action groups (Hu et al. 2021). Contextual entities can inform decisions without requiring actions of their own. StarCraft experiments evaluate transfer between team sizes before and after additional learning. For a REC, this distinction separates context such as weather and prices from equipment requiring commands.

An explicit entity interface connects these representations to environment observations and actions. Entity Gym describes entity types and their features, variable collections of instances, and actions associated with acting or target entities. Availability masks restrict which entities and actions can be selected. RogueNet supplies a compatible Transformer policy with encoders shared within each entity type and heads for the declared action spaces. Symes Thompson et al. (2025) evaluate this combination with PPO in simulated cyber defence, including transfer to unseen network sizes. Their experiments concern variation across networks; changes in entity population within an episode are discussed as a further application. This is a close precedent for combining typed entities, variable observations, and resource-associated decisions.

Output queries provide another way to associate decisions with entities. In Perceiver IO, Jaegle et al. (2022) encode inputs into a latent representation and use queries to request outputs, allowing input and output layouts to vary independently. A resource-specific query could request an equipment command. This is a possible control application of the general-purpose mechanism.

Action meaning introduces a separate requirement when parameters are shared across heterogeneous agents. Yu, Lin, et al. (2024) construct a Unified Action Space (UAS) that distinguishes actions by their effects and targets, then use availability masks to recover each agent group's policy. An auxiliary prediction loss uses one group's trajectory to predict other groups' policies. The resulting U-MAPPO and U-QMIX methods are evaluated in StarCraft Multi-Agent Challenge (SMAC). Their relevance is the preservation of different action semantics within shared learning, such as distinguishing an attack from a healing action, rather than treating equal action indices as equivalent. In energy control, this motivates distinguishing commands with different effects even when their numerical representations have the same shape.

Graph representations for changing relationships

The associations considered so far connect entities to their observations and actions. Graphs also represent relationships among entities, allowing the controller to use the structure of the current configuration. In message-passing neural networks, shared functions update a node using information from neighbouring nodes and their connecting edges (Gilmer et al. 2017). A community graph can distinguish members from equipment and represent ownership or electrical connections. Adding a battery introduces its node and the relevant relationships; removing a member also removes the associated resources from the active representation. The policy can apply the same message-passing functions to the updated graph. Which relationships are included determines what information can influence each decision.

Changing tasks can also alter the actions available to otherwise familiar agents. Anil et al. (2025) model this setting with task-open Markov games and the MOHITO actor–critic method. Its evolving hypergraph links agents, tasks, and actions, allowing graph-based actors and critics to evaluate the current choices. Simulated ridesharing and wildfire-suppression experiments examine tasks appearing and disappearing during operation. For a REC, an arriving vehicle creates a service request, whereas installing a charger changes the equipment available to serve requests. MOHITO provides a precedent for dynamic task–action relations; relating temporary tasks to persistent equipment and community members requires those objects to be distinguished in the energy formulation.

Agent participation introduces another form of variation. The open-teamwork studies of Koh et al. (2025) and Rahman et al. (2021) consider agents entering and leaving, while Cai et al. (2024) evaluate unseen team compositions in robot cooperation. These studies complement MOHITO's task changes by examining changes to the agents available to perform the work.

Energy applications connect graph representations to variable participation. Orfanoudakis, Robu, et al. (2025) map active charging sessions to commands within the electrical infrastructure, reaching 1,000 charging points with separate training at the evaluated sizes. Orfanoudakis, Panda, et al. (2026) instead evaluate a Graph Neural Network Enhanced Decision Transformer (GNN-DT) trained for 25 stations on configurations with 5–100 stations without further training. The latter demonstrates reuse across infrastructure sizes, although performance deteriorates towards larger configurations.

Recent methods combine graph structure with cooperative learning or runtime protection. The HetGPS preprint uses shared MARL policies, a heterogeneous infrastructure graph, and a physical safety filter in simulations with 200–3,218 EVs, including transfer and limited adaptation (Wang, Nimalsiri, et al. 2026). In voltage control, Jin et al. (2026) evaluate a shared graph actor–critic with Alternating Current (AC) screening, termed Graph Residual Actor with AC Screening for Active Voltage Control (GRAS-AVC), on an unseen feeder, changes in active inverter participation, and an announced topology change. An updated graph and electrical model support a runtime comparison between the learned command and a droop fallback.

Real-world charging provides an example of managing changing participation through conventional control. Hermans et al. (2024) implement predictive control at an office complex, updating charging allocations as vehicles arrive and depart. Their trial demonstrates this response on operating chargers, complementing the simulated learning-based studies above.

Modular composition and reuse across resource types

The entity-to-action mappings reviewed above establish which resource receives each decision. Modular composition concerns which learned components can be reused when those resources change. The two choices can be combined, as the type-specific encoders in Entity Gym/RogueNet already illustrate. The studies below develop reuse across bodies, interfaces, and equipment configurations. In Shared Modular Policies, Huang, Mordatch, et al. (2020) assign shared control modules to robot limbs and exchange information according to the body structure, allowing one policy to control different morphologies. Each module remains responsible for its own actions. For energy systems, this suggests reusable control components associated with batteries, charging points, or other equipment. Their inputs and outputs must reflect the equipment type, since charging a battery and scheduling a flexible appliance involve different decisions and operating requirements.

Modularity can also separate how a system is observed from how it is controlled. Compositional Interfaces (COIN) connects specialised perception and action modules through a shared Transformer controller (Luketina et al. 2025). BC in a compositional grid-world evaluates held-out combinations of observation, action, and instruction spaces. Further experiments introduce a new observation modality by training its perception module while retaining the controller and action modules. This establishes reuse through recombination of learned interfaces and adaptation of selected components. Its learning setting is imitation rather than cooperative MARL.

Building2Building provides an energy example, connecting HVAC measurements and commands through typed graphs and type-specific encoders and decoders (Taboga et al. 2026). Its single-agent PPO controller reuses a Transformer policy across building configurations. EnergyPlus experiments evaluate temperature tracking in held-out buildings, with energy cost excluded from the objective.

Software typing and equipment descriptions support integration. The `marlenv` library provides strongly typed environment records and action-availability masks (yamoling 2026). Brick describes equipment, measurement points, and their relationships (Balaji et al. 2018). These supply inter-

face structure and resource semantics, while the policy architecture determines how they inform decisions.

The reviewed studies distinguish training for each configuration, applying a policy without further learning, and adapting it to a new setting. The charging transfer results illustrate why these conditions matter when comparing performance across configurations. The following synthesis brings these forms of reuse together with the capabilities of the reviewed architectures.

3.3.3 Comparative Analysis and Remaining Questions

Table 3.5 summarises the reviewed capabilities, the associated methods, and the settings in which they were evaluated.

Table 3.5: Capabilities established by the studies reviewed in Section 3.3.2.

Capability	Methods and references	Evaluation setting
Entity-associated decisions	UPDeT and Entity Gym/RogueNet (Hu et al. 2021; Symes Thompson et al. 2025).	Transfer across game teams and cyber networks.
Heterogeneous action meanings	UAS with U-MAPPO and U-QMIX (Yu, Lin, et al. 2024).	Heterogeneous game actions in SMAC.
Changing tasks and choices	MOHITO (Anil et al. 2025).	Tasks appearing and disappearing in simulated ridesharing and wildfire suppression.
Reuse of learned modules	Shared Modular Policies, COIN, and Building2Building (Huang, Mordatch, et al. 2020; Luketina et al. 2025; Taboga et al. 2026).	Robot morphologies, interface combinations, and buildings. COIN uses imitation and Building2Building uses single-agent PPO.
Reconfiguration in energy control	GNN-DT, HetGPS, GRAS-AVC, and predictive charging (Hermans et al. 2024; Jin et al. 2026; Orfanoudakis, Panda, et al. 2026; Wang, Nimalsiri, et al. 2026).	Simulated charging networks and inverter control. Real-world charging uses MPC.

The evidence reviewed in this section and summarised in Table 3.5 establishes several foundations for control across changing configurations. UPDeT and Entity Gym/RogueNet associate decisions with variable collections of entities, MOHITO updates relationships between agents, tasks, and actions, and Shared Modular Policies and COIN reuse learned components across different configurations. Graph-based energy controllers also demonstrate transfer across infrastructure sizes and operation under changes in participation. These capabilities provide concrete starting points for investigating how coordination can continue as a community changes.

The condition requiring further investigation combines member entry and departure with equipment additions and removals within members under cooperative MARL control of a REC. Within

the reviewed literature, no evaluation was identified that jointly examines these changes. A departing member removes demand and flexibility, while equipment added elsewhere introduces decisions and capacity associated with another member's services. The controller must preserve the associations between members, equipment, service requirements, and executable actions as both levels change. The question is how well existing approaches support this condition, considering community performance, individual service delivery, and computational demand. Performance immediately after reconfiguration must also be distinguished from performance recovered through further learning. This investigation concerns new instances and combinations of recognised equipment types.

A comparison under common operating conditions can establish how far the reviewed capabilities meet these requirements and where adaptations are needed. Differences in available information, action scope, execution protection, and learning resources must be accounted for when interpreting the results. Entity-based decisions, changing task relationships, and reusable modules offer alternative ways to organise this control problem, each with preparation and computational costs to assess. The absence of a joint evaluation motivates this comparison. Whether additional mechanisms are needed, and what they should change, must follow from the limitations observed in that comparison.

3.4 Deployability across Heterogeneous Installations

A controller prepared for one energy community may need new data, local adjustment, and different equipment connections before it can serve another. These activities take time and engineering work, even when the policy can represent the new configuration. Following Section 1.2.2, deployability concerns the effort needed to prepare, integrate, and maintain useful control. Section 3.3 examined the configurations that learning methods and architectures can accommodate. This section is concerned with how much of the preparation for control can be reused across installations.

Section 3.4.1 examines learning from operational records and prior control knowledge. Section 3.4.2 then considers policy reuse, selection, and adaptation, while Section 3.4.3 examines integrated control systems, from equipment interfaces and operational platforms to controller testing and updates. Section 3.4.4 compares the evidence and identifies the remaining question about their combined preparation effort.

3.4.1 Learning from Available Data and Prior Knowledge

Learning through repeated interaction with an occupied building takes time, and exploratory decisions affect energy use and the services delivered to occupants. Preparation can begin with information already available at the site. Records of earlier operation show how equipment responded to control decisions, while existing rules and predictive models provide knowledge that can guide learning. The review first examines imitation and offline RL from these records, then considers guidance from rules and models. It finally turns to community-level learning, where the recorded outcomes depend on decisions made by several agents.

Behaviour cloning and offline reinforcement learning

BC learns to reproduce recorded actions. Offline RL also uses their outcomes to seek improved behaviour from a fixed archive. As explained in Section 2.5, the difficulty is assessing actions for which that archive provides little evidence. A policy can otherwise favour actions whose estimated value is high because the estimate is unreliable.

Several methods restrict this improvement in different ways. Batch-Constrained Q-Learning (BCQ) generates candidate actions from a model of recorded behaviour and allows bounded adjustments (Fujimoto, Meger, et al. 2019). Conservative Q-Learning (CQL) penalises optimistic values for actions insufficiently supported by the data (Kumar, Zhou, et al. 2020). Implicit Q-Learning (IQL) learns values from observed actions and uses them to weight BC, avoiding unseen actions in its value-learning updates (Kostrikov et al. 2022). These choices control how far learning relies on information beyond the archive.

A simpler combination adds a BC penalty to Twin Delayed Deep Deterministic Policy Gradient (TD3) and normalises the observed states; the resulting method is denoted TD3+BC (Fujimoto and Gu 2021). Its competitive continuous-control results make it relevant where implementation and tuning effort matter alongside performance. These algorithms supply candidates for learning from energy records. Their general-control benchmarks do not establish a performance ranking for buildings.

Real-world room control illustrates why the application matters. Liu, Balaji, Gao, et al. (2022) extend BCQ with value regularisation to obtain Batch Constrained Munchausen Reinforcement Learning (BCM). The study uses a year of operational records and evaluates temperature and airflow control in 15 rooms on three floors, with individual runs lasting approximately one week. BCM gives more stable performance than the compared batch-learning alternatives, with a different method ranking from the simulated evaluation. Rule-based comparisons use historical weeks selected for similar outdoor temperatures. The study connects fixed-data learning to real-world execution while showing the importance of the records and comparison conditions.

The controller that collects those records also affects their usefulness. Nweye, Liu, et al. (2022) initialise policies from rule-based experience before continuing learning in CityLearn. More records from a basic controller can impair performance, whereas a stronger collecting policy supplies better preparation and remains competitive with the learned alternatives. The result distinguishes archive size from useful coverage. It also separates fully offline learning from initialisation intended to shorten subsequent interaction.

Rule-guided learning and model-based preparation

Existing control knowledge can guide updates as well as generate demonstrations. A rule-guided offline learning method (RUBICON) uses critic estimates over a minibatch to select either recorded behaviour or a rule-based reference for regularising offline actor updates (Liu, Balaji, Gupta, et al. 2023). Its online form regularises TD3 using the reference controller. Simulated HVAC experiments vary climate, data quality, and response uncertainty, showing how rules can supplement uneven experience without requiring equal imitation of every recorded action.

Prior knowledge can also shape the policy itself. Gnu-RL combines a differentiable predictive-control structure with imitation-based initialisation and subsequent policy-gradient learning (Chen, Cai, et al. 2020). Its conference-room study uses 20 selected historical days for training and 13 for testing, followed by three weeks of real-world airflow control. Temperature tracking improves and cooling demand decreases relative to the existing controller. Residual learning uses established control differently: a learned correction is added to a retained feedback controller, as demonstrated in real-world robotic assembly by Johannink et al. (2019). One approach incorporates planning into the learned policy, while the other preserves a baseline contribution to each action.

Records can instead prepare a model through which a policy learns. In domestic hot-water control, Kazmi et al. (2019) share information across 53 households to support thermal modelling. The field implementation uses a simplified controller, partly to make learning, control, and communication faults easier to diagnose. In a laboratory heating system, Blad et al. (2023) learn recurrent dynamics from operational data, train a multi-agent controller through that model, and continue learning on the testbed. Conventional control remains active outside the learned operating region. Model-based preparation reduces physical interaction while introducing identification and validation work.

Offline multi-agent learning and joint records

Community records contain actions whose outcomes depend on other members. Improving each policy independently can produce a joint behaviour poorly represented in the archive. Offline Multi-Agent Actor Rectification (OMAR) combines policy-gradient updates with a critic-guided search for better actions (Pan et al. 2022). An offline multi-agent method with implicit global-to-local value regularisation (OMIGA) derives local value regularisation from a global regularised objective and combines it with learning from observed actions (Wang, Xu, et al. 2023). Their multi-agent locomotion, particle, and StarCraft evaluations provide approaches to offline coordination. Community application requires records that preserve the relationship between local actions, shared conditions, and outcomes.

These learning routes need more than meter readings. Policy learning requires decision-time observations, executed actions, and information from which the objective can be reconstructed. Model learning requires data that reveal the relevant dynamics. Data access, alignment, and interpretation therefore contribute to preparation effort. Protection of the eventual actions is considered separately in Section 3.5. The next subsection examines how much of a prepared policy can be reused at another installation.

3.4.2 Policy Reuse and Adaptation across Installations

Once a policy has been prepared, its use at another installation offers a way to share that initial investment. Two homes may have similar batteries but different demand patterns, while buildings with comparable heating needs may use different equipment. Reuse therefore depends on both the behaviour learned and the observations and actions available at the new site. A policy may be applied directly, selected from a suitable group, or adapted through local learning. The review

examines policy transfer, preparation shared across groups, and adaptation to different interfaces, distinguishing reusable work from the preparation still required at each installation.

Policy transfer and local action models

Residential batteries provide a direct example of parameter transfer. MERLIN evaluates policy exchange among independent controllers using measured profiles from 17 zero-net-energy homes in CityLearn (Nweye, Sankaranarayanan, et al. 2023). A source policy receives a further simulated training episode at the target. Reuse can approach locally trained performance with fewer target episodes, although the outcome depends on the source building and performance indicator. Selecting a useful source is therefore part of preparation. Both battery operation and adaptation are simulated.

Training across several sources can spread that initial investment. Karimi Madahi et al. (2024) prepare a shared battery policy from 90 Belgian households and adapt its final layer to ten unseen households. Common preparation uses 40,000 episodes, followed by 100 adaptation episodes per target. All households share the battery model and action interface, and evaluation uses April 2023 data. The study demonstrates reduced target learning under different consumption and generation profiles. Reporting source training separately shows what must be reused for that reduction to be worthwhile.

A local model can handle installation differences while retaining a common decision policy. Xu, Wang, et al. (2020) separate an HVAC front-end network from an inverse building model that maps its decisions to equipment actions. The local model is fitted through supervised learning using approximately two weeks of operation under an existing or simple on–off controller. Simulated transfers vary buildings, equipment, and climate, with outcomes depending partly on the direction of climate transfer. This approach shifts target preparation from additional RL towards supervised identification.

Cohorts, meta-learning, and federated preparation

Another route is to select an existing policy for a group of similar installations, or cohort. Zangato et al. (2025) group buildings by non-shiftable demand and assign three groups to shared policies in CityLearn. Classification from one week of observations agrees with annual-record grouping in their dataset. Cost results improve, while carbon results are more mixed. This provides a way to select behaviour from short records instead of training a separate policy for each new member.

Grouping by behaviour complements the interface compatibility discussed in Section 3.3. Two homes may expose identical battery actions but have different demand patterns. Similar demand, in turn, does not ensure equivalent equipment actions. Policy assignment therefore needs both a compatible interface and evidence that the selected behaviour suits the target. Unfamiliar profiles may call for reassignment or adaptation.

Meta-learning prepares a policy specifically for such adaptation. MetaEMS learns an initialisation through building-level and group-level updates before adapting to target buildings (Zhang, Wu, et al. 2025). CityLearn experiments separate source and target buildings within four climate zones and report improvements over the compared initialisation and control methods. Target interaction

remains part of the procedure, but the shared preparation is organised to make those updates effective.

Federated learning allows sites to share preparation while retaining their operational records. Hagström et al. (2025) exchange model updates between Soft Actor-Critic (SAC) controllers for a simulated two-zone data centre under different climates, including an unseen evaluation climate. Learning and generalisation improve under selected optimisation and aggregation choices, with a trade-off between early comfort violations and final performance. Satisfactory behaviour takes three to 24 simulated weeks, depending on the configuration. Sharing updates thus changes data management as well as learning, while the required interaction remains consequential for real-world use.

Heterogeneous interfaces and real-world transfer

The preceding approaches reuse compatible policy structures or provide a local mapping. When observation spaces differ, the policy itself may need modification. Coraci, Brandi, et al. (2024) combine model slicing, imitation, and fine-tuning to transfer controllers with different observations and reward formulations. Their simulated cases retain chiller and thermal-storage control, while PV and battery systems use rules. Transfer improves on rule-based and initially untrained online controllers, although target-specific simulation training can retain an advantage. Interface adaptation and behavioural adaptation therefore involve distinct work.

Real-world transfer also requires model calibration and actuation access. In HiLo, Coraci, Silvestri, et al. (2025) transfer control between two offices in one building using a calibrated digital twin, 14 days of target records, and online fine-tuning. Matched controller comparisons use the twin, and two days affected by an equipment-mode change are excluded from the heating comparison. The trial establishes physical policy reuse alongside the local data and modelling needed to support it.

Table 3.6 brings together the preparation mechanisms in Sections 3.4.1 and 3.4.2. Its rows distinguish what can be reused from what a target still needs. These routes allow compatible installations to receive a policy through selection, local fitting, or adaptation, with different demands on data and training.

Reusing learned behaviour is one part of commissioning. The installation must also supply the expected observations and accept the resulting commands. The next subsection examines how software and equipment interfaces make that connection reusable.

3.4.3 System Integration and Controller Lifecycle

Putting a prepared policy into operation requires a working connection between measurements, decisions, and equipment commands. A battery may report through a local gateway while a heat pump uses a manufacturer's service, so the control system must bring these interfaces together with consistent meanings, units, and timing. The same connections must support subsequent controller versions as they are tested and introduced into operation. The review follows this progression

Table 3.6: Mechanisms for reducing controller preparation and the work retained at a target installation.

Mechanism	Reusable preparation	Target requirements
Offline learning and imitation	Recorded behaviour and its outcomes; trained policy or initialisation.	Action-labelled records, coverage assessment, and validation of the resulting behaviour.
Model-based preparation	Predictive structure, shared identification data, or a simulation workflow.	Local identification or calibration where needed; validation against operation.
Parameter transfer and fine-tuning	Source policy or representation.	Compatible inputs and actions; target data and the updates used for adaptation.
Transferable policy with local model	Common decision policy.	Supervised fitting of an installation-specific action mapping.
Cohorts and policy mapping	Policies shared by compatible groups.	Group assignment, interface checks, and assessment of unfamiliar profiles.
Meta-learning	Initialisation trained for adaptation across related tasks.	Target experience and adaptation steps.
Federated learning	Knowledge exchanged through model updates.	Local learning data, compatible models, and aggregation infrastructure.

from device adapters and semantic descriptions to integrated energy platforms, then examines the infrastructure for testing, deployment, and controller updates.

Device adapters and semantic descriptions

Protocol abstraction separates device communication from application logic. The Building Energy Management Open Source Software (BEMOSS) platform places application and data-management functions above VOLTTRON communication services (Khamphanchai et al. 2014). Device discovery and control agents expose supported interfaces, including BACnet and Modbus. Its architecture and laboratory testbed show how applications can share communication functions. A new device may need an adapter, which can subsequently serve several applications.

Semantic descriptions specify what those connected data mean. Brick represents equipment, measurement points, and relationships so that applications can find data by their roles (Balaji et al. 2018). Its evaluation covers six commercial buildings with more than 17,700 points and eight applications, four operating across all six buildings. Mortar combines records from 90 buildings and evaluates 11 portable analytics applications whose declared requirements determine usable sites (Fierro, Pritoni, et al. 2018). For learned control, the same principle can support consistent observations assembled from different site descriptions.

BuildingMOTIF checks whether a site's semantic description supplies the information an application needs (Fierro, Saha, et al. 2022). Templates reuse equipment structures, and validation

identifies missing properties or relationships. Evaluation uses 46 existing Brick models and control-sequence metadata for a reference building, supporting incremental preparation of the descriptions needed by each application.

Integrated control platforms and operational services

Energy platforms connect these interfaces to forecasting, scheduling, and actuation. Jelić et al. (2025) join gateways, time-series and semantic storage, forecasts, and scheduling in a cloud platform for island communities. Schedules become device commands, and acknowledgements return to the platform. Electrical control is evaluated in La Graciosa, with thermal applications also supported in participating communities. The pilot provides evidence of an operating acquisition-to-command chain. Comparisons across different periods make it harder to attribute energy differences to the platform alone.

The occupied-building CIRLEM deployment introduced in Section 3.2.2 makes site integration explicit. Its interoperability and coupling mechanisms translate names, types, and units between the controller and building-management system (Hosseini, Papadopoulos, et al. 2026). Separating the control service from local point mappings identifies which connections can be reused and which must be configured at another site.

An RL economiser recommendation service illustrates operation at a larger scale. Cui et al. (2023) report more than 10,000 rooftop units across over 200 sites, supported by Brick-based standardisation, time-series storage, and hourly cloud inference. One policy is trained per site and shared by its units, with quarterly retraining. Operators validate and enter recommended setpoints because security and trust requirements limit automation. Online performance results are withheld, and expected savings come from simulation. The evidence establishes service organisation and scale while identifying the human work retained in operation.

Testing, deployment, and controller updates

Once a controller is connected, subsequent versions need repeatable assessment and execution. The Building Optimization Testing Framework (BOPTEST) supplies simulation cases, a common interface, and performance indicators for comparing control changes (Blum et al. 2021). Kube-VOLTTRON explores container orchestration for distributing VOLTTRON services (Kempf 2022). These address complementary tasks: testing behaviour under controlled conditions and reproducing service execution. Each installation still needs suitable models, data connections, and command access.

Reinforcement Learning Operations (RLOps), developed for telecommunications, connects data preparation, training, testing, deployment, and monitoring (Li, Thomas, et al. 2022). Its relevance to energy control is the association between model versions, data processing, and evaluation results as controllers change. The economiser service provides an energy example of scheduled retraining, while the building and community platforms establish physical integration. Accepting an update, coordinating policy versions, and restoring a previous controller remain application-specific responsibilities.

Adapters, semantic descriptions, testing environments, and deployment services reduce different parts of commissioning and maintenance. Their connection matters because the observations and commands used in preparation must retain their meaning in operation. The following comparison brings this integration effort together with the data and adaptation requirements reviewed above.

3.4.4 Comparative Analysis and Remaining Questions

Deployability depends on the combined effort of preparation, training and adaptation, integration, and maintenance across the controller lifecycle, following Section 1.2.2. Table 3.7 summarises the reviewed approaches, the associated studies, and their evaluation settings.

Table 3.7: Evaluation settings for controller preparation, reuse, and integration.

Approach	Studies and references	Evaluation setting
Learning from operational records	BCM and Gnu-RL (Chen, Cai, et al. 2020; Liu, Balaji, Gao, et al. 2022).	Real-world HVAC control in a 15-room study and a conference room.
Policy transfer and selection	MERLIN, multi-source transfer, and cohort policy mapping (Karimi Madahi et al. 2024; Nweye, Sankaranarayanan, et al. 2023; Zangato et al. 2025).	Simulated residential batteries with measured demand profiles.
Policy transfer in operation	HiLo (Coraci, Silvestri, et al. 2025).	Two real offices, supported by a calibrated digital twin and local fine-tuning.
Semantic application reuse	Brick, Mortar, and BuildingMOTIF (Balaji et al. 2018; Fierro, Pritoni, et al. 2018; Fierro, Saha, et al. 2022).	Multi-building analytics and metadata validation.
Integrated energy control	Island-community platform and CIRLEM (Hosseini, Papadopoulos, et al. 2026; Jelić et al. 2025).	Operating community and building systems linking measurements to commands.
Controller services and updates	Economiser recommendation service (Cui et al. 2023).	More than 200 sites, with quarterly retraining and operator-applied recommendations.

The studies brought together in Table 3.7 establish several ways to prepare and deploy controllers, from learning with existing records to reusing policies and connecting them to equipment. Their results include real-world control and services operating across many sites. The reviewed literature, however, does not provide a common assessment of data preparation, policy adaptation, equipment integration, and controller updates together for communities with different member and resource configurations. A reduction in target training may nevertheless depend on a calibrated model, prepared metadata, or equipment connections established in advance. These activities

contribute to the effort of deployment even when they fall outside the learning procedure being evaluated.

That effort depends on what changes between installations. A compatible member may need policy assignment and data mapping, whereas a different thermal response may require local identification or fine-tuning. A new equipment interface can require an adapter as well as changes to the policy. Some of this work can support several installations, while local calibration, validation, and subsequent updates may recur at each site. Comparing the alternatives therefore requires relating retained performance to both shared preparation and repeated local effort, including unsuccessful transfers. The gap concerns how far these activities can be reused together while preserving community performance and individual services across heterogeneous installations.

3.5 Physical and Operational Safety

A charging command must respect the equipment's rating and the available connection capacity while leaving enough time to deliver the energy required at departure. Other resources may depend on that same capacity. Following Section 1.2.2, safety concerns these physical, electrical, and service constraints on applied actions and resulting states. Connecting a controller to equipment, as discussed in Section 3.4, makes their enforcement part of operation.

Section 3.5.1 examines how constraints enter learning and planning. Section 3.5.2 reviews mechanisms that adjust or replace proposed actions, and Section 3.5.3 considers shared limits and service commitments over time. Section 3.5.4 compares the evidence and identifies the remaining question about their combined effects on community services.

3.5.1 Constraint Representation in Learning and Control

A power rating limits an instantaneous action, whereas a departure request links several decisions to a future outcome. These requirements can enter the reward, constrain policy improvement, or restrict a planned trajectory. Building on Section 2.3.5, the discussion first considers learning objectives and then explicit models that connect present decisions to admissible future states.

Reward penalties, cost budgets, and risk criteria

Reward penalties make undesirable outcomes costly during learning. Exceeding a power threshold or leaving energy undelivered can reduce the reward alongside the benefit of economical operation. The weights determine the trade-off that the policy learns. Constrained learning instead separates the performance objective from an admissible cost budget, making that requirement explicit in policy improvement (Achiam et al. 2017; Gu et al. 2023).

Constrained Policy Optimisation (CPO) uses performance and constraint-cost bounds to construct updates within a trust region (Achiam et al. 2017). Its constraint concerns an expected accumulated cost. Lagrangian methods increase the pressure on costs that exceed their budgets through adjustable multipliers. In cooperative learning, Gu et al. (2023) account for successive

changes in the joint policy through Multi-Agent Constrained Policy Optimisation (MACPO) and MAPPO-Lagrangian. Their simulated multi-robot experiments provide methods for learning with individual or shared budgets while retaining interactions between agents.

Risk criteria address outcomes that an average can conceal. A chance constraint limits the probability of a specified event, while Conditional Value-at-Risk (CVaR) describes the costly tail of an outcome distribution. Chow et al. (2018) develop policy-gradient and actor–critic methods for percentile-risk criteria. For energy control, defining the event and horizon determines whether the criterion concerns one departure, repeated comfort losses, or an entire operating period. Penalties, expected-cost budgets, and risk criteria therefore express different requirements.

Predictive constraints and model-based verification

Explicit optimisation places constraints on the decisions being computed. MPC plans from the current state and forecasts, applies the first decision, and then replans. A charging formulation can impose power limits throughout the horizon and sufficient energy at departure. Real-world charging provides an operational example (Hermans et al. 2024), while predictive safety filters use related optimisation to check proposals from learned controllers (Wabersich and Zeilinger 2021).

Uncertainty determines which future conditions that plan covers. Nominal planning uses an anticipated trajectory, robust formulations impose requirements over a specified disturbance range, and probabilistic formulations associate compliance with a stated probability. Reserving capacity against more possible demand protects a wider range of conditions while reducing the flexibility available for economical scheduling (Muntwiler et al. 2020; Wabersich and Zeilinger 2021).

Learning can also supply proposals for model-based verification. Zhang, Dehghanpour, et al. (2021) combine supervised multi-agent policy learning, model-derived gradients, and consensus on shared constraint variables. A power-flow calculation checks actions, and backtracking tightens bounds and updates policy parameters when violations are found. Simulations of five interconnected microgrids demonstrate the combination for equipment and network constraints. The resulting decisions depend on both learning and verification. The next subsection examines how such checks modify the commands that reach equipment.

3.5.2 Action Protection and Supervisory Enforcement

A proposed action can be accepted, adjusted, or replaced. Protection mechanisms differ in how they find an admissible alternative and how far ahead they examine its effects. The review separates restrictions on policy outputs, checks of continued admissibility, and supervision using an established controller.

Admissible outputs and projection layers

Simple restrictions define available actions: a mask can remove charging when a vehicle is absent, and a bounded output can respect its charger's rating. State-dependent constructions go further. Tabas and Zhang (2022) map neural-policy outputs into actions that preserve a prepared

robustly controlled invariant set. Their frequency-control simulations use a closed-form mapping whose protection depends on the supporting model, controller, and set.

Projection preserves a proposal as closely as possible while imposing constraints. Dalal et al. (2018) learn a local approximation of how actions affect a safety signal, then use it to correct continuous-control decisions. Their analytical solution assumes feasibility and at most one active constraint. The approximation defines what the correction protects, making its accuracy central to the mechanism.

An optimisation layer can also connect correction to learning. Agrawal et al. (2019) differentiate through convex programmes so that upstream network parameters can be trained through the constrained solution. Gnu-RL provides a physical example of a differentiable MPC policy with bounded airflow actions and temperature tracking in its objective (Chen, Cai, et al. 2020). Its three-week conference-room experiment demonstrates structured commands for real equipment, with action bounds and thermal performance playing distinct roles.

Predictive filters, barrier functions, and shields

Predictive protection checks whether an action admits a suitable continuation. Wabersich and Zeilinger (2021) search for a trajectory satisfying state and input constraints and reaching a terminal set with a safe controller. When a new full-horizon plan is unavailable, a shortened-horizon route retains the previous certification. The probability of satisfying the constraints depends on initial feasibility, model-error assumptions, and the terminal construction. The check therefore concerns both the present command and the ability to continue operating within the specified constraints.

Control Barrier Functions (CBFs) express continued admissibility through a condition on state evolution. Ames et al. (2017) place this condition in a quadratic programme that combines a control objective with preservation of a specified set. Forward invariance means that a system starting in that set remains there under the stated dynamics and control conditions. Applying this principle to sampled energy control requires appropriate treatment of response times and actuation limits.

Formal shields instead use a permitted-behaviour specification and a system abstraction to replace inadmissible proposals. Elsayed-Aly et al. (2021) develop centralised shielding of joint actions and factored shielding over subsets of agents. Their simulated multi-agent experiments demonstrate protection during learning and improved scalability through factorisation. The abstraction and consistency between local shields determine the joint property being preserved.

Supervisory switching and reference controllers

Supervision can retain an established controller alongside the learned one. The Neural Simplex Architecture allows learned control while recovery by a baseline remains possible and switches before that option is lost (Phan et al. 2020). It also supports learning after intervention. The switching condition matters because the fallback controller protects a defined operating region.

Physical heating illustrates this division of responsibility. Blad et al. (2023) control the supply and valves of a two-zone underfloor-heating system with MARL and temperature-based supervisors. Intervention also affects the learning reward. Measured temperatures describe the delivered service, while heat-pump electricity cost is modelled because the testbed uses a different heat source.

The experiment establishes learned coordination alongside conventional protection on physical equipment.

A supervisor can alternatively compare candidate actions. In graph-based voltage control, Jin et al. (2026) use an AC power-flow model to evaluate the learned proposal and a droop-control candidate at the same state. The selector applies the action with lower modelled voltage risk. Simulations report reduced voltage exposure together with residual violations. This is a relative selection criterion, distinct from preserving a certified recovery region.

Table 3.8 compares the requirements and properties of these mechanisms with their information needs. It includes the learning formulations in Section 3.5.1, making explicit the distinction between guiding policy updates and enforcing conditions during execution.

Table 3.8: Mechanisms for representing and enforcing control constraints.

Mechanism	Requirement represented	Property or role	Information and preparation
Penalties and constrained policy learning (Achiam et al. 2017; Gu et al. 2023)	State, action, or accumulated costs.	Incentives or expected-cost budgets guide policy improvement.	Costs, budgets, experience, and update conditions.
Risk-constrained learning (Chow et al. 2018)	Event probability or the tail of accumulated cost.	A specified probabilistic or risk criterion.	Event, horizon, probability model, and risk threshold.
Admissible output construction (Tabas and Zhang 2022)	Actions preserving a prepared invariant set.	Constraint satisfaction in the certified domain.	Dynamics, disturbance bounds, and invariant set.
Projection and optimisation layers (Agrawal et al. 2019; Dalal et al. 2018)	Conditions defining an admissible action or plan.	Correction relative to an explicit constraint model.	Constraint model, feasibility, and numerical accuracy.
Predictive safety filtering (Wabersich and Zeilinger 2021)	Current action and a continuation to a safe terminal set.	Continued compliance under feasibility and uncertainty assumptions.	Dynamics, uncertainty bounds, terminal controller, and optimisation.
Barrier-based control (Ames et al. 2017)	A condition preserving an admissible state set.	Forward invariance under the stated dynamics and control conditions.	Barrier, state, actuation limits, and sampling conditions.
Centralised or factored shielding (Elsayed-Aly et al. 2021)	A formal safety specification over joint behaviour.	Adherence to the specification within the system abstraction.	Abstraction, shield synthesis, and consistent local composition.
Supervisory switching (Jin et al. 2026; Phan et al. 2020)	Recoverability by a baseline, or relative risk of candidate actions.	Recovery-region protection or lower-risk selection.	Baseline, switching condition, and state or model evaluation.

The next subsection examines how shared physical limits and future service requirements affect protection across resources.

3.5.3 Coordinated Physical Limits and Service Feasibility

The vehicle in Section 2.1.2 arrives at 18:00 with 30% SoC and needs 80% by 08:00. An admissible charging power now may still leave too little time to meet that request. This subsection examines how predictive protection preserves future options, how shared capacity couples local decisions, and how interventions affect delivered service.

Predictive service constraints and intervention feedback

A predictive constraint can require departure energy while respecting power and storage limits over the remaining intervals. Thermal control has a similar temporal relationship because current heating affects later comfort. Markgraf and Althoff (2023) use shrinking-horizon MPC filters for simulated households with batteries and air conditioning. Each filter imposes local temperature, power, and storage requirements, including terminal battery charge, with bounded forecast errors represented in the constraints.

Their five-household study also makes intervention visible to learning through a penalty proportional to the correction. The selected MAPPO models require 48 corrections with this penalty and 446 without it (Markgraf and Althoff 2023). Both variants retain the filter. The comparison isolates the benefit of teaching the policy to request fewer corrections while preserving the same protection mechanism.

Shared limits and distributed predictive protection

Individually feasible schedules may conflict when two vehicles rely on the same connection headroom. As established in Section 2.1.1, the relevant limit follows the physical site, phase, feeder, or transformer. Markgraf and Althoff's filters protect uncoupled household constraints, with transformer capacity identified as an extension. Joint protection instead considers combined actions, while hierarchical control can allocate compatible capacity budgets to local controllers.

Distributed predictive certification formalises this coordination. Muntwiler et al. (2020) construct local safety problems for coupled linear systems with bounded disturbances. Neighbours coordinate admissible trajectories and uncertainty bounds using a structured terminal set. In energy control, Tao et al. (2025) combine local RL with distributed MPC checkers for two interconnected microgrids. Their subsecond frequency-control simulations provide an electrical example of coordinated state and input protection.

CommonPower assigns control responsibilities to resource groups and balancing assets managed by an operator (Eichelbeck et al. 2026). Symbolic component models support optimisation and safeguards for learned actions. Building experiments combine a heat pump and battery under uncertain forecasts and different learning arrangements. Component constraints receive robust treatment, while power-flow constraints apply to the nominal trajectory. This distinction identifies which uncertainty each level of protection covers.

Intervention order and delivered service

Multiple safeguards can change the same command in different ways. HetGPS first applies voltage correction and then a mapping for availability, SoC, and departure rules (Wang, Nimalsiri, et al. 2026). The second step can alter the electrically corrected action, for example by increasing charging near departure. Simulations across 200–3,218 EVs report reduced voltage violations and departure outcomes assessed with a specified SoC tolerance. Service mapping remains active in all compared variants, so the voltage-filter comparison measures its added effect within an already supervised charging system.

This interaction connects instantaneous protection to future allocation. A projection addresses the present electrical condition, whereas predictive control can also reserve capacity for later obligations. Coordinated capacity budgets provide another way for local schedules to respect arrivals, deadlines, and competing demand. The predictive and distributed mechanisms above supply approaches to that problem, with the resulting service depending on the allocation across participants.

Physical charging reveals additional effects of execution. In the MPC study of Hermans et al. (2024), lower-level allocation and pausing required practical adjustments because two vehicles had difficulties with paused sessions. Average recorded energy per session also differed between comparison periods. These observations accompany the reported peak reduction and show why individual delivery matters alongside aggregate outcomes.

Interpreting a service deficit requires knowing whether the request was feasible. Repeated postponement can make an initially achievable departure target impossible, while another request may exceed available time or capacity on arrival. The former concerns control decisions during service. The latter requires admission, revised commitments, or explicit priorities. This distinction helps compare protection methods without attributing every deficit to the same cause. The final comparison brings together their enforcement scope and the services evaluated.

3.5.4 Comparative Analysis and Remaining Questions

The reviewed methods connect constraint-aware learning, execution protection, and coordinated service in different ways. A policy may learn to request fewer corrections, while a filter enforces its constraint model and conventional equipment control provides further protection. Table 3.9 compares these roles in energy applications, retaining the service conditions and the distinction between simulation and real-world operation. The batch-learning and real-world deployment settings are introduced in Sections 3.4.1 and 3.2.2.

Table 3.9: Energy-control evidence for physical protection and service delivery.

Study and setting	Enforcement mechanism	Demonstrated capability	Service or evaluation condition
Interconnected microgrids; simulation (Zhang, Dehghanpour, et al. 2021)	Supervised neural policy learning, shared constraint variables, power-flow checking, backtracking.	Coordination of equipment and network constraints in five microgrids.	Applied decisions depend on both the learned policy and model-based verification.
Household DR; simulation (Markgraf and Althoff 2023)	Local predictive filters and optional intervention penalty during MARL training.	Protection of battery, power, and thermal constraints over a shrinking horizon.	Terminal battery requirement; uncoupled household constraints. Both learning variants retain protection.
Building and grid control; simulation (Eichelbeck et al. 2026)	Symbolic models, predictive safeguards, resource groups and balancing assets.	Robust component protection and comparisons of learned controllers.	Heat-pump and battery case; nominal network constraints. The separate annual network case uses central optimisation.
EV charging; simulation, preprint (Wang, Nimalsiri, et al. 2026)	Graph MARL, voltage correction, and subsequent service mapping.	Reduced voltage violations across 200–3,218 EVs.	Availability and departure rules active in all variants; final actions determine electrical and service outcomes.
Voltage control; simulation (Jin et al. 2026)	AC-model comparison of a graph-policy proposal with droop control.	Lower voltage exposure through candidate selection.	Relative one-step risk criterion; residual voltage violations reported.
Microgrid frequency control; simulation (Tao et al. 2025)	Local RL with distributed MPC checking and backup control.	Coordinated state and input protection for two coupled microgrids.	Bounded coupling and disturbance assumptions; subsecond frequency-control task.
Conference-room HVAC; real-world operation (Chen, Cai, et al. 2020)	Differentiable MPC policy with action bounds.	Learned airflow control and improved temperature tracking over three weeks.	Thermal tracking enters the objective; comparison with existing control.
Occupied-building HVAC; real-world operation (Liu, Balaji, Gao, et al. 2022)	Batch RL with regularised policy learning and a defined action space.	Control of room temperature and airflow from recorded experience.	Empirical comfort assessment; approximately week-long algorithm trials and historical reference periods.

Continued on the next page.

Table 3.9: Energy-control evidence for physical protection and service delivery (continued).

Study and setting	Enforcement mechanism	Demonstrated capability	Service or evaluation condition
Two-zone heating; physical laboratory (Blad et al. 2023)	MARL supply and valve control with temperature-based supervisors.	Coordinated physical heating with conventional intervention.	Measured thermal response; modelled heat-pump cost. Reported temperature-distribution comparison excludes solar overheating.
EV charging at an office complex; real-world operation (Hermans et al. 2024)	Aggregate MPC with lower-level current allocation.	Three-week operation covering 1,226 sessions; reduced grid-power peaks.	Individual charging behaviour and pause compatibility affect delivered service.

The remaining question is how action protection can enforce shared physical limits while preserving individual services in a heterogeneous community. The reviewed studies establish local temporal protection, distributed certification, and combined electrical and charging safeguards. Their different horizons, allocation rules, and evaluation settings leave the consequences for community services only partly comparable. The gap concerns how these mechanisms work together and distribute the effects of intervention across participants.

An action corrected to satisfy one limit may change another member's available capacity or its ability to meet a later commitment. Resource changes also require the relevant constraints and obligations to remain associated with the decisions being controlled. These relationships motivate comparison of delivered service and repeated deficits alongside electrical compliance, with learned behaviour distinguished from the protection supplied by the surrounding system. Section 3.7 brings these findings together with the other research dimensions. The next section considers impaired measurements, communication, and equipment, and the recovery of coordinated control.

3.6 Robustness, Degraded Operation, and Recovery

A charging request remains relevant when a measurement stops arriving or a charger becomes unavailable. Information loss changes what the controller knows, while equipment loss changes what it can deliver. Other resources may continue operating and help preserve the service. Following Section 1.2.2, robustness concerns retaining acceptable performance under disturbance, and resilience concerns recovering useful operation afterwards. The constraints examined in Section 3.5 still apply, but the available information and capacity may now be reduced.

Section 3.6.1 examines policies prepared for uncertainty, faulty agents, and incomplete observations. Section 3.6.2 then considers diagnosis, fallback, and restoration. Section 3.6.3 compares

controlled disturbance tests with operational incidents, before Section 3.6.4 identifies the remaining question about retained service and sustained recovery.

3.6.1 Learning under Uncertainty and Disturbance

Training can expose a policy to varied demand, equipment behaviour, or observation quality. The chosen variation determines what it can learn to handle. A battery with lower usable capacity has less energy, whereas a missing SoC measurement limits knowledge of that energy. The methods below address variation in dynamics, impaired teammates, and estimation from incomplete information.

Randomised and adversarial training

Robust learning can emphasise difficult conditions within a training distribution. Robust Adversarial Reinforcement Learning (RARL) trains a control policy alongside an adversary that applies destabilising forces (Pinto et al. 2017). Ensemble Policy Optimisation (EPOpt) instead samples models with different parameters and uses lower-return trajectories for policy improvement (Rajeswaran et al. 2017). Their simulated continuous-control experiments illustrate two approaches: learning challenging disturbances and concentrating updates on poorly performing model conditions. Energy applications require disturbance ranges and objectives that represent the services to be maintained.

Fault-aware multi-agent learning

In cooperative control, a fault can alter another agent's behaviour. Zhao et al. (2022) use a coach to vary the proportion of crashed agents encountered during learning. The remaining agents cooperate despite teammates that remain still or act randomly. Crash status is assigned at the start of each episode and held fixed, allowing comparison of fixed, progressive, and adaptive training schedules for impaired teams.

Multi-Agent Robust Training Algorithm (MARTA) introduces faults selectively within a trajectory. A learned switching mechanism chooses when and which agent has its action replaced by a random or adversarial alternative, with a cost or budget regulating the interventions (Mguni et al. 2026). Multi-agent benchmarks include different training and evaluation fault regimes. Results favour selective preparation in several settings, although improved final return coexists with poorer cumulative training performance in a traffic-junction comparison. This distinguishes the behaviour obtained from the learning effort needed to obtain it.

Attention and prioritised experience sampling provide another approach. Fault-Tolerant Model with Attention on Actor and Critic (AACFT) uses these mechanisms in simulated tasks where surviving agents maintain communication or redistribute work after failures during an episode (Shi et al. 2024). One task involves retrieving resources from a failed teammate. Recovery therefore means restoring useful resources, rather than repairing and readmitting that agent. Results also depend on the scenario, with a multiple-critic baseline performing better in one recovery case.

Memory, state estimation, and runtime adaptation

When observations disappear, retained history may support decisions. A Deep Recurrent Q-Network (DRQN) integrates successive observations, with intermittently hidden game inputs used to evaluate partial observability (Hausknecht and Stone 2015). In energy control, measurements and executed actions may similarly help estimate the present condition during a temporary information gap. That estimate depends on what the remaining history reveals about the system.

Beliefs can represent missing coordination information explicitly. Zhou et al. (2022) combine recurrent value estimation with beliefs about other agents' actions and a correlated-equilibrium procedure in a microgrid. Simulations report improved economic outcomes under communication failures relative to the compared methods. The observation model, indirect information, and parameter-server architecture define the scope of this result. Identifying which links fail is therefore part of interpreting the retained performance.

Delay requires estimating the state affected by a command from older information. Cheng et al. (2024) construct a predicted state interval for imprecisely known delay and consider adverse states within it for MARL voltage control. Simulated distribution networks evaluate this treatment of delayed measurements. Memory, beliefs, and interval prediction consequently offer different ways to reconstruct decision-relevant information.

Runtime adaptation can also condition a fixed policy on changing equipment behaviour. Rapid Motor Adaptation (RMA) estimates a latent description of operating conditions from recent states and actions (Kumar, Fu, et al. 2021). Both networks are trained beforehand, and only the latent estimate changes during execution. Physical robot tests demonstrate adaptation to surfaces and loads. This distinguishes updating an operating-condition estimate from retraining policy weights. The next subsection considers the supervisory response when estimation and policy preparation alone are insufficient.

3.6.2 Fault Handling, Degraded Operation, and Recovery

An unacknowledged command may indicate communication delay, an unavailable interface, or equipment that cannot act. These cases require different responses. Operational fault handling must obtain that information from measurements and execution records, even when a learning experiment supplies fault indicators directly. The discussion follows diagnosis and mitigation, supervisory alternatives, and the restoration of service.

Fault diagnosis and predictive mitigation

Model-based diagnosis compares measured behaviour with expected relationships. Zafra-Cabeza et al. (2023) use residuals and stochastic thresholds to detect and isolate microgrid faults. A second predictive controller supplements the operating MPC, selecting mitigation actions according to demand satisfaction and resource use. Diagnosis thus changes the available decisions and how they are ranked, providing a conventional approach to allocating reduced capacity.

State estimation can support mitigation when information is incomplete. Qiu, Yang, et al. (2021) use set-membership estimation to bound possible microgrid states under bounded process and measurement disturbances. Networked MPC combines these bounds with forecasts and error compensation, accommodating unavailable devices or communication links. Tests pair simulated energy dynamics with physical wireless communication. Estimation represents what remains known, while planning allocates resources within that representation.

Supervisory switching and recovery controllers

The Neural Simplex Architecture separates continued control, policy improvement, and permission to resume (Phan et al. 2020). A baseline takes over before recovery becomes unavailable and a reverse-switching condition governs the return to neural control. During baseline operation, an adaptation module can retrain the neural policy to reduce later interventions. These functions extend the protective switching introduced in Section 3.5.2 into a process for returning control authority.

Distributed supervision coordinates this decision across agents. Mehmood et al. (2021) construct local switching conditions that account for neighbours and preserve joint safety relationships. Their simulated microgrid switches to baseline control as voltage approaches a boundary and returns after voltage is reduced. The advanced controller uses nominal power setpoints. Reversible supervision is therefore demonstrated independently of learning, providing a reference for assessing the contribution of a robust MARL policy.

Recovery RL learns the alternative controller itself. It estimates risk from experience of constraint violations and trains a recovery policy separately from the task policy (Thananjeyan et al. 2021). A selector invokes recovery when a proposed task action is judged too risky. Simulation and physical robotics evaluate task completion alongside violations. Recovery behaviour and its risk estimate are additional learned components, whose usefulness depends on the states from which recovery remains possible.

A reference action can also be considered at every decision. GRAS-AVC reconstructs degraded telemetry and uses an electrical model to select between a graph-policy action and droop control (Jin et al. 2026). Simulations include missing, noisy, delayed, and jointly impaired observations. The combined system reduces voltage exposure while retaining violations and increasing losses relative to droop. Its result depends on reconstruction and selection as well as the actor, including the state information available to the selector.

Service restoration and return to coordination

Reduced capacity makes service priorities consequential. Storage loss may leave enough energy for urgent charging while other flexible demand is postponed. Restoration methods make related allocation choices explicit. Vu et al. (2024) use deep MARL to select circuit-breaker operations after an outage, with masks restricting admissible actions. Simulations on three feeders evaluate weighted load restoration. Here, recovery has a service objective beyond returning a physical variable to its nominal value.

A repaired charger likewise restores an available action, but the remaining service depends on the energy and time left before departure. Resuming coordination may require updated state estimates and compatible allocations among resources that continued operating. Table 3.10 compares the mechanisms contributing to continued operation and return, including their information and preparation needs.

Table 3.10: Mechanisms for maintaining and recovering control under disturbance.

Mechanism	Contribution to continued operation	Information or preparation required
Randomised and adversarial training	Prepares a policy for variable dynamics and disruptive actions.	Perturbation distribution, severity, and a relevant training objective.
Fault-aware team learning	Supports compensation and task redistribution among surviving agents.	Representative faults, useful remaining observations, and sufficient capacity.
Memory, beliefs, and state estimation	Reconstructs decision-relevant information from incomplete or delayed data.	History, indirect observations, or a model with stated uncertainty.
Diagnosis and predictive mitigation	Identifies faults and reallocates available resources.	Measurements, residuals or fault indicators, models, and feasible alternatives.
Supervisory switching and recovery policies	Transfers control to a suitable alternative and governs the return.	Risk or recoverability criterion, available fallback, and switching conditions.
Online parameter adaptation	Changes learned behaviour in response to persistent operating differences.	New experience, learning time, and control during the update.
Service restoration	Selects actions that restore supply or task completion after disruption.	Resource availability, service priorities, and physical feasibility.

The mechanisms address different parts of recovery: estimating the current condition, retaining control, reallocating capacity, or restoring service. The next subsection examines their operational evidence, distinguishing controlled disturbances from incidents observed during use.

3.6.3 Evidence under Operational Disturbances

A repeated simulated fault can isolate the effect of a control component, whereas real-world operation exposes communication, equipment response, and intervention together. The distinction matters when comparing a fault-trained policy with a complete supervised system. The evidence below separates controlled communication and mitigation tests from building campaigns in which incidents arose during operation.

Controlled communication and fault-mitigation tests

The networked MPC study above uses Raspberry Pi devices for wireless communication while energy dynamics remain simulated (Qiu, Yang, et al. 2021). It therefore tests control with physical communication, with power-equipment response represented by a model. Identifying that boundary makes the demonstrated response more precise.

A Direct Current (DC) microgrid study combines an OPAL-RT simulator with power-supply and converter equipment in a reported HIL arrangement (Negahdar et al. 2024). RL adjusts event-triggered communication thresholds around conventional secondary control. Tests include load changes, unit withdrawal and reinsertion, partial line failure, and delays. Voltage regulation and load sharing recover in several cases, while the illustrated 750 ms delay case loses load-sharing stability. The evidence concerns the operating range of adaptive communication within that control system.

Physical predictive mitigation is demonstrated by Zafra-Cabeza et al. (2023). Their laboratory microgrid includes battery storage, a programmable supply reproducing renewable generation, and an electronic load representing demand. A five-hour experiment introduces a fault and activates mitigation according to demand satisfaction, grid use, and lithium-battery use. Diagnosis and explicit allocation support continued operation with the changed resources.

Supervised operation and occupied-building incidents

The heating experiment introduced in Section 3.2.2 places MARL alongside local controllers and temperature supervision (Blad et al. 2023). Measured thermal response establishes physical service under changing outdoor conditions, while heat-pump cost is modelled. The study demonstrates supervised operation, without a controlled communication- or equipment-fault campaign.

Occupied-building records reveal the work required when integration fails. CIRLEM reports interrupted controller interaction, repairs, and continued technical intervention (Hosseini, Papadopoulos, et al. 2026). Its interaction rate measures exposure to learned control, not heating availability or delivered comfort. The radiant-heating campaign of Wang, Ren, et al. (2026) reports manual reversion and sensor or hydraulic problems, with faulty days excluded from performance comparisons. Both accounts motivate following service through fallback and repair alongside outcomes on retained comparison days.

Table 3.11 compares the disturbances, responses, and service interpretations across these settings and the preceding simulations. Controlled tests establish responses to specified events. Operational campaigns reveal incidents arising in use, with manual or fallback control forming part of the operating record.

Table 3.11: Evidence for continued operation and recovery under disturbance.

Study and setting	Disturbance or operating condition	Response and observed capability	Service or recovery interpretation
Microgrid management; simulation (Zhou et al. 2022)	Communication failures and uncertain generation.	Bayesian beliefs and recurrent value estimation support coordinated decisions.	Economic outcomes; observation model and remaining information condition performance.
Voltage control; simulation (Jin et al. 2026)	Missing, noisy, delayed, and jointly degraded telemetry.	Reconstruction, graph policy, and model-based selection reduce voltage exposure.	Residual violations; protection and operating losses must be assessed together.
Distributed supervision; simulation (Mehmood et al. 2021)	Voltage approaching prescribed limits.	Local baseline intervention followed by reverse switching.	Demonstrates return of control authority in a conventional microgrid controller.
Load restoration; simulation (Vu et al. 2024)	Supply interrupted by an outage.	Multi-agent learning and action masking select restoration sequences.	Weighted load restored; decision computation and physical restoration time are distinct.
DC microgrid; reported HIL (Negahdar et al. 2024)	Unit withdrawal/reinsertion, line impairment, and delays.	Learned communication thresholds support voltage regulation and load sharing.	Includes a delay case with loss of stability; learning adjusts communication.
Microgrid; physical laboratory (Zafra-Cabeza et al. 2023)	Induced fault during a five-hour mitigation experiment.	Diagnosis and two MPC components reconfigure resource use.	Demand satisfaction is explicit; generation and demand are electrically emulated.
Heating; physical laboratory (Blad et al. 2023)	Actual thermal response and changing outdoor conditions.	MARL operates with local control and thermal supervisors.	Measured thermal service; modelled heat-pump cost; no dedicated fault campaign.
CIRLEM; occupied building (Hosseini, Papadopoulos, et al. 2026)	Software, data, and communication incidents during an annual campaign.	Learned control operates with technical intervention and system adjustments.	Algorithm–installation interaction in about 59% of scheduled steps.
Radiant heating; occupied building (Wang, Ren, et al. 2026)	Sensor, communication, execution, and equipment problems.	Learned control with manual reversion during affected periods.	Alternating-controller campaign; error-affected days excluded from performance comparisons.

3.6.4 Comparative Analysis and Remaining Questions

The remaining question is how robust policies and supervision can jointly preserve individual services during a disturbance and restore coordinated operation afterwards. The reviewed studies establish fault-aware learning, estimation, predictive mitigation, reversible supervision, and supply restoration. Their joint effects remain only partly measured against common service obligations in heterogeneous communities. The gap concerns the service retained and recovered by the complete system, with the contribution of learning distinguished from fallback and intervention.

Reduced capacity can be allocated in ways that improve aggregate performance while concentrating deficits in particular participants. Interpreting departure energy, discomfort, or unserved demand against the resources still available separates physical shortage from avoidable allocation losses.

Recovery extends beyond the first resumed action: measurements, actuation, and permission to coordinate may return at different times. Renewed peaks, repeated switching, and deficits transferred to others affect whether the return remains useful. Section 3.7 connects these questions to scalability, deployability, and safety, and summarises their implications for the RQs.

3.7 Chapter Summary

This chapter examined the research problem introduced in Section 1.2 through the evidence available for applying MARL to real-world community energy control. Section 3.1 established the search, selection, and comparison procedures, combining energy applications with methodological work on scalability, deployability, safety, and robustness. The review considered what existing methods achieve, the conditions supporting those results, and the questions that remain when learned controllers must serve changing communities. This provides the literature-based grounding for the four RQs defined in Section 1.3.2.

The application review in Section 3.2 compared simulation studies with real-world implementations. Simulations reported coordination of storage, flexible demand, and energy exchanges across varied community settings, including some disturbances and operational constraints. Deployments demonstrated that learned decisions can reach operating equipment, while exposing dependencies on data access, local control, equipment behaviour, and technical intervention. The reviewed evidence remains limited for sustained cooperative MARL control across a REC's changing members, resources, and service requirements. These findings substantiate the difficulty identified in Section 1.2 of carrying promising learned behaviour into continued operation. They also explain why the subsequent review examined the learning methods together with the systems supporting their use.

Section 3.3 examined learning across large populations and architectures for variable collections of agents and resources. For RQ1, shared experience, value factorisation, and selective interactions support coordination across larger populations, while entity-based, graph, and modular representations accommodate different configurations. These capabilities underpin the comparative

investigation outlined in Section 3.3.3. The remaining question concerns control performance and computational demand as populations increase and membership and equipment change together. Evaluation must distinguish performance retained immediately after reconfiguration from performance recovered through further learning, recording the learning effort required. This establishes the basis for assessing existing approaches and any necessary adaptations across new instances and combinations of recognised resource types.

The discussion then moved from representing a configuration to preparing a controller for an installation. Section 3.4 reviewed learning from operational records, policy transfer and adaptation, and the integration of equipment and controller services. In relation to RQ2, these approaches show how prior experience, reusable policies, and common interfaces can reduce parts of deployment effort. Real-world studies also reveal the local data, calibrated models, and equipment connections supporting that reuse. The remaining question is which preparation can be shared across heterogeneous installations and which training, adaptation, integration, and maintenance activities must recur at each one. Assessing both parts alongside retained control performance can establish whether greater initial preparation reduces subsequent local effort and under which installation conditions that investment remains useful.

Section 3.5 examined how constraints enter learning and how proposed actions are checked, corrected, or replaced before execution. For RQ3, the review establishes complementary roles for constrained learning, local and coordinated filters, and predictive protection. Their guarantees depend on the constraints, models, and horizons represented, while their interventions alter the decisions available to other participants. In a community, respecting an immediate power limit and retaining enough flexibility to meet a later service commitment are connected requirements. The remaining question therefore links compliance with physical and service constraints to coordination, individual service delivery, and the need for protective intervention. Evaluation must examine the services delivered to each member alongside constraint violations and the frequency and extent of intervention, so that aggregate compliance does not obscure persistent service deficits.

Robustness and recovery were considered in Section 3.6, which reviewed disturbance-aware learning, estimation, fault mitigation, and supervision alongside operational evidence. For RQ4, these studies show that both learned policies and execution supervision can contribute to continued or restored operation. Deployments also document fallback and human intervention. The remaining question concerns the services retained during disruption and the extent and duration of their recovery, distinguishing policy behaviour from the contribution of supervision. Recovery must be assessed beyond the first resumed command, since returning resources can create renewed peaks, repeated intervention, or deficits elsewhere. Comparisons must therefore identify which mechanisms remain active during each disturbance and recovery period to establish how learning and supervision contribute to service continuity and sustained recovery under changing conditions and data, communication, or equipment failures.

Table 3.12 consolidates these findings by relating the contribution of the literature to the gap investigated under each RQ.

Table 3.12: Literature-based answers to the RQs and remaining gaps pursued in the thesis.

RQ and review	What the literature establishes	Gap addressed by the thesis
RQ1 Section 3.3	Shared learning and structured representations support larger populations and variable configurations.	Control performance and computation under population growth and joint membership and equipment changes, distinguishing immediate operation from recovery through further learning.
RQ2 Section 3.4	Recorded experience, policy reuse, and common interfaces reduce parts of preparation, with local data and integration requirements.	Reusable preparation and recurring training, adaptation, integration, and maintenance effort at each installation, assessed alongside retained performance.
RQ3 Section 3.5	Constrained learning addresses expected-cost budgets, while execution protection restricts applied actions under specified model and feasibility assumptions.	Compliance with physical and service constraints, effects on coordination and individual services, and the frequency and extent of protective intervention.
RQ4 Section 3.6	Robust learning, estimation, mitigation, and supervision sustain or restore aspects of control.	Service continuity and sustained recovery under changing conditions and failures, distinguishing the contributions of learned policies and execution supervision.

The remaining gaps concern how learning and its supporting mechanisms work together as community resources and operating conditions change. Addressing them requires evaluating the benefits and trade-offs of the complete control system, considering community performance and the services delivered to individual members. More extensive protection may require additional computation, while greater reuse across installations may depend on more initial preparation. The investigation must establish when these costs lead to improvements in control and operation. Chapter 4 presents the architecture connecting controllers to data and equipment, while Chapter 5 introduces the simulation environment and evaluation methodology. Together, they support the comparisons in Chapter 6, which examine what learning and its supporting mechanisms contribute and the costs associated with those benefits.

Chapter 4

An Architecture for Real-World Community Energy Control: EnergAIze

The preceding chapter reviewed approaches to REC energy control and identified gaps in their use in real-world installations. This chapter presents the architecture of EnergAIze, connecting control in buildings and RECs to controller development, experiment orchestration, and deployment.

4.1 Energy Control in Buildings and Communities

Households need their vehicles ready for travel, their rooms at a comfortable temperature, and their appliances to complete their work. Meeting these needs leaves choices about when electricity is consumed or stored, especially where a building also produces some of its own energy. To make those choices, a controller needs information about the services people require, the resources available to deliver them, and the limits within which they can operate. EnergAIze connects this information to the equipment through which a decision takes effect.

Consider the vehicle introduced in Chapter 1, arriving home at 18:00 with a SoC of 30% and a target of 80% by 08:00. The charger reports that the vehicle is connected, vehicle information supplies its SoC, and the household meter shows how much power the other loads are using. Together with the departure requirement and the available charging capacity, these observations allow the controller to choose a charging power. That decision is sent through the installation's control interface, and subsequent measurements report the power delivered and the progress towards the target.

Figure 4.1 presents this flow. In the figure, *State of the Environment* denotes the observations assembled for a decision. EnergAIze uses them to determine actions for the connected resources. Measurements and actions pass through the local control system or external service provider associated with the equipment.

The inputs shown in Figure 4.1 fall into seven categories, according to the information they contribute to a decision:

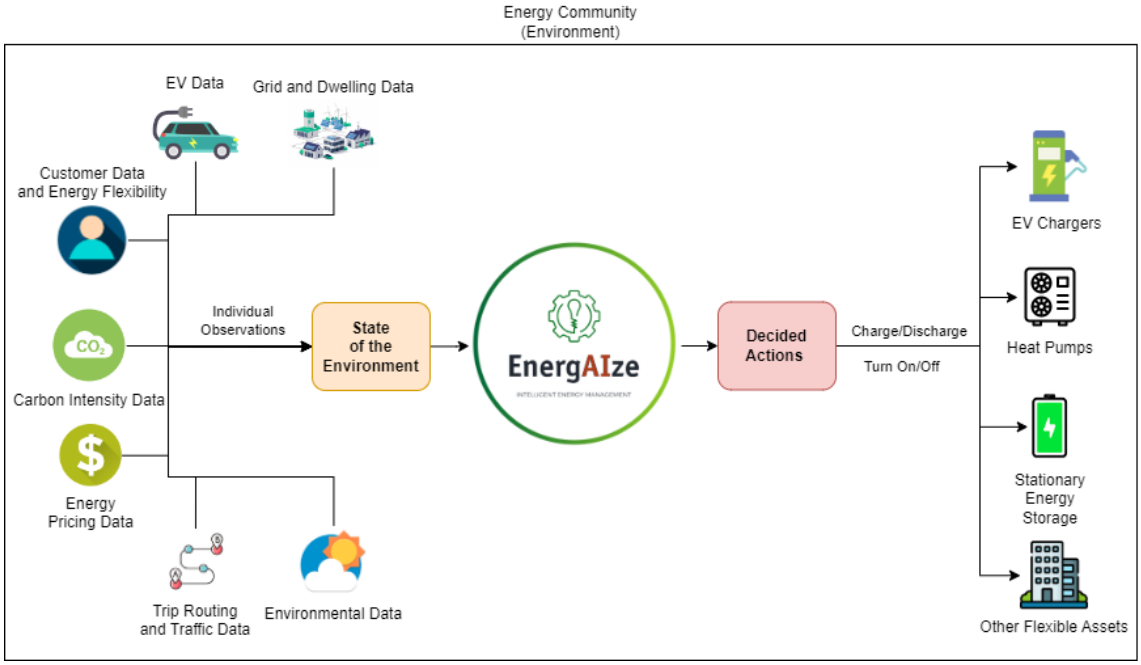


Figure 4.1: Information and action flow. Source: author's master's dissertation.

User requirements and flexibility. Requested services and the freedom to adjust their delivery, such as a vehicle's departure time and target SoC, a comfort range, or an appliance's operating window.

Vehicle data. Battery SoC, charging status, and battery capacity. These describe the vehicle's current condition and help determine the energy needed for its next use.

Building and grid data. Demand, PV production, storage state, grid exchange, and connection limits. These describe the installation's energy balance and the capacity available for flexible resources.

Electricity prices. Purchase prices and, where applicable, export remuneration or REC exchange prices. Their variation over time informs the cost of a charging or storage decision.

Carbon intensity. Emissions associated with electricity supply. This information allows the controller to consider the emissions consequences of using grid electricity at different times.

Environmental data. Weather conditions and forecasts, including temperature and solar irradiance. These help anticipate changes in renewable production and weather-dependent demand.

Travel and traffic data. Planned journeys, routes, and traffic conditions. These can inform estimates of vehicle arrival, departure, and energy use, complementing the requirements supplied by the user.

These categories can contain measurements, user declarations, configuration values, or forecasts. The vehicle's current SoC is a measurement; its required departure charge is a user requirement; its

expected return time may be a forecast. Keeping these roles distinct allows the controller to use each value appropriately. The input selection for each experiment is retained in the configuration records described in Appendix B.7.

The loop repeats as conditions change. Cooking and heating can reduce the power available for charging; when those loads subside, charging can increase. At each decision interval, the controller receives updated measurements and accounts for the time remaining before departure. The measurements also show how closely the delivered power followed the request.

Across a REC, this arrangement is repeated for buildings with different equipment and daily routines. One household may have PV panels and a battery, another a vehicle charger, and a workplace several charging points. Each building has a controller and associated data processing, configured for its resources. Measurements and actions remain linked to that building, while information relevant to the REC can be shared among the participating controllers.

Figure 4.2 extends the first view to several buildings. Each local controller receives observations from its building and returns actions to its equipment. In the illustrated deployment, a REC server supplies shared information, including time, weather, carbon intensity, and electricity prices. The actor network represents the learned controller executed for each building. This organisation also accommodates the hierarchical methods developed in Chapter 6, where a REC controller supplies guidance for local decisions.

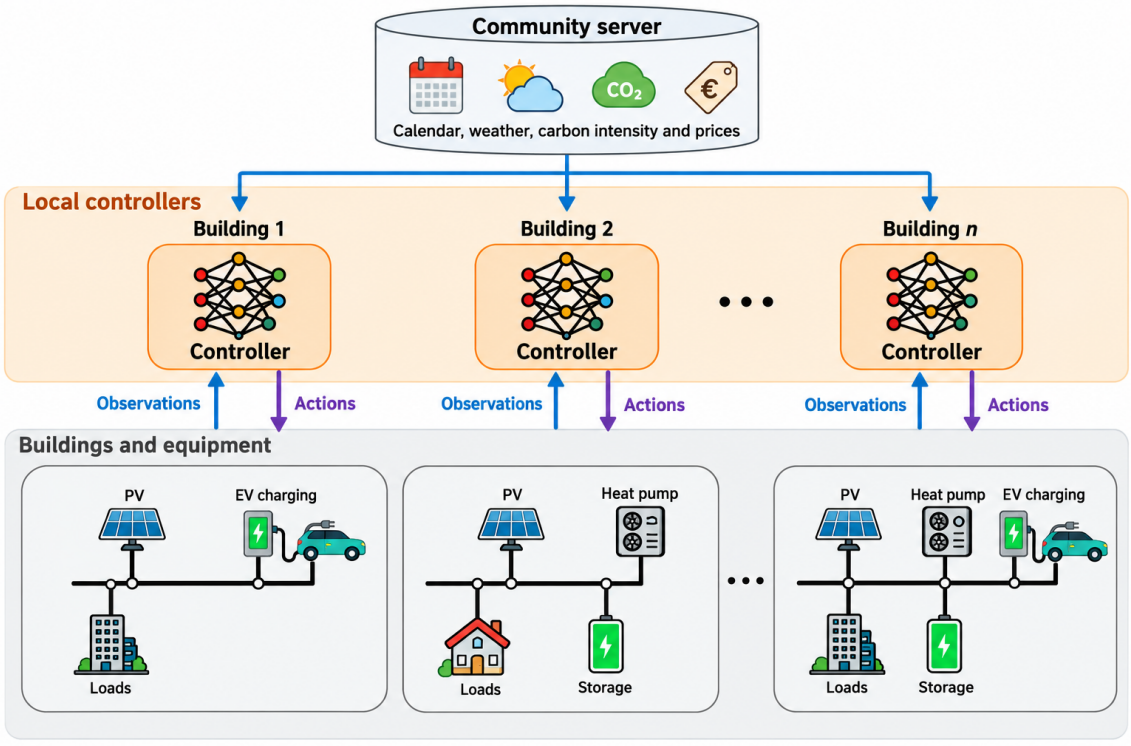


Figure 4.2: REC deployment. Adapted from the author's master's dissertation.

The blue arrows carry observations from each building and the shared services into its controller; the purple arrows carry the resulting actions to its equipment. For the charging example, local information establishes what the household can do, while REC information helps determine when that flexibility is useful. On a different day, when the vehicle remains connected until late afternoon, the controller can account for renewable production available elsewhere in the REC. When many vehicles arrive together, spreading charging over their connection windows can limit the combined demand.

Organising data processing and control around each building gives installation-specific work a clear scope. Connecting a new charger changes that building's configuration and available actions. Adding a building introduces another local loop and updates the REC information used for coordination. Processing close to the data source can reduce communication and keep detailed measurements within the local processing path. The controllers in Figure 4.2 coexist with shared services and provider platforms, with placement determined by each installation.

Putting these local control loops into operation involves two complementary paths. During *runtime*, incoming observations support a decision whose consequences are measured at the installation. During *development*, data and simulation provide the conditions in which a controller is prepared and evaluated for that role. The runtime path is considered first, establishing what the controller must receive and produce. The development path then explains how such a controller is obtained, followed by the infrastructure that runs the experiments and loads the selected model into service.

4.2 Runtime Architecture

The information and action flow described above requires several services to work within each decision interval. Measurements must reach the controller in a usable form, the controller must choose an action, and that action must reach the equipment with the applicable checks. The Cleanwatts (CW) cloud platform and other Internet of Things (IoT) interfaces connect the equipment to these services through Application Programming Interfaces (APIs). Figure 4.3 shows how EnergAIze assigns these responsibilities, expanding one building to expose its runtime components.

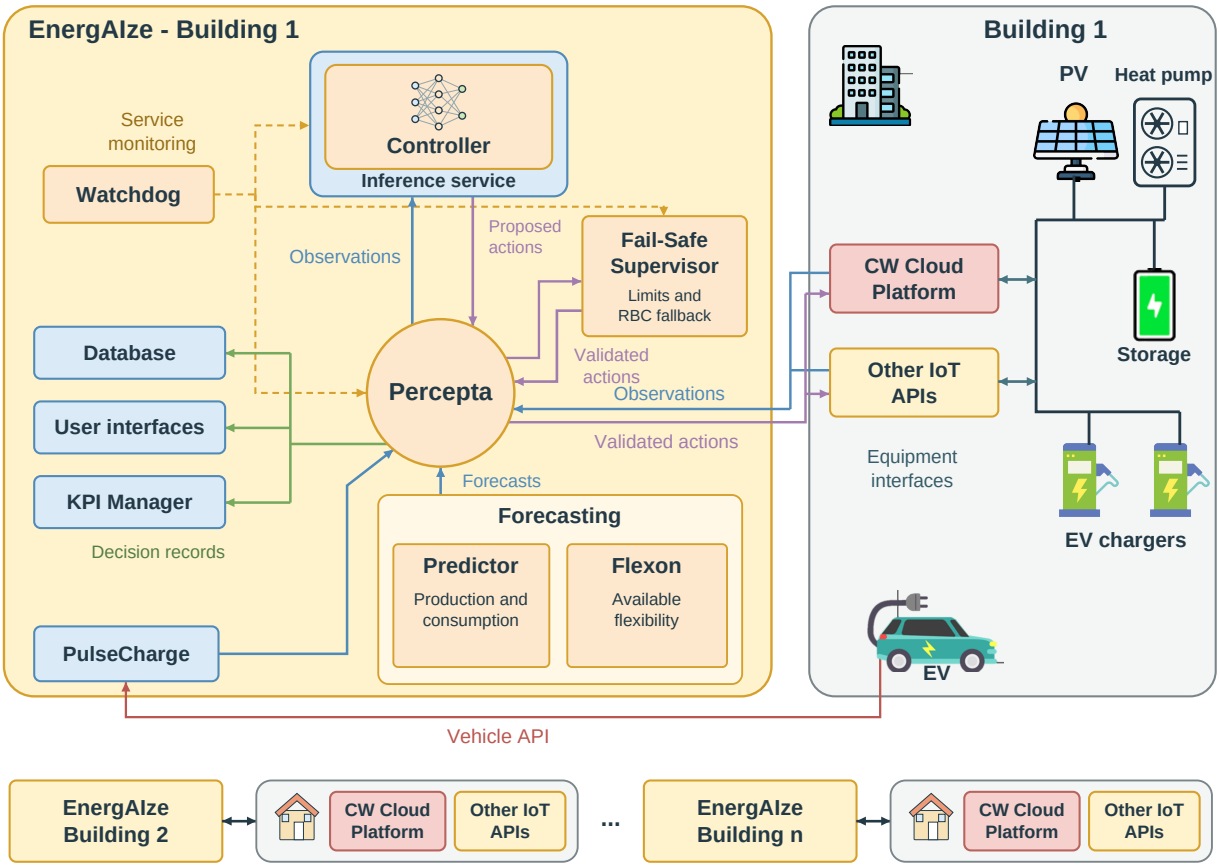


Figure 4.3: Runtime architecture of EnergAIze, expanded for one building.

On the right of the figure, the CW cloud platform and other IoT interfaces connect the physical equipment to EnergAIze. On the left, Percepta processes incoming observations and routes outgoing actions. Vehicle information and forecasts support the controller, while action validation, service monitoring, and operational records accompany its decisions. This organisation underlies the integrations described in the EnergAIze publications and OPEVA demonstrator (Fonseca, Ferreira, Cabral, Severino, and Praça 2024; Fonseca, Sousa, et al. 2025; Salvan and OPEVA Consortium 2026).

Installation configuration links these components to equipment, measurement sources, commands, and operating limits. A model contract specifies the controller's inputs and outputs; Section 4.4.4 describes its passage into runtime. The architecture and the roles of its components were conceived and designed by the author, who also guided the collaborative research and student projects through which several components were implemented. Their implementation contributions are acknowledged in the associated publications and the Statement on Co-Authored Work.

4.2.1 The Controller and Its Inference Service

At the centre of the runtime is the controller, shown within the inference service in Figure 4.3. It determines how the resources under its control should operate during the next interval. The surrounding services supply information, apply its decisions, and report their consequences. The control methods investigated in Chapters 6 and 7 occupy this position in the architecture.

For the household vehicle, the immediate choice is the charging power to request. Its consequences extend beyond the next interval: charging now contributes towards the departure requirement but also increases household demand, while delaying charging preserves capacity now and leaves less time to deliver the remaining energy. A stationary battery introduces a related choice. Discharging it may support the household during a busy period, although the energy then used will no longer be available later. The controller considers these resources together, using the observations and forecasts selected for its decision problem.

The control methods reviewed in Chapter 3 can occupy this role through a common observation and action interface. A RBC applies its configured rules, an optimisation method solves the current scheduling problem, and a learned controller evaluates its policy. The reinforcement-learning methods studied in this thesis use that interface during both development and operation, keeping decisions associated with the same resources and service requirements.

At REC level, the decisions made in one building influence the conditions experienced by others. Several households postponing charging until the same low-price interval can create a new peak. Coordination requires the controller formulation to represent the relevant REC information and relationships, whether through shared observations, the learning process, or guidance from a REC controller. The runtime provides the communication and resource associations needed by the selected method. The progression from building decisions to coordinated and hierarchical control is developed in Chapter 6; Chapter 7 examines how the controller can accommodate changes in the participating resources.

The inference service executes the model selected for this role. It receives the prepared observations, applies the input processing declared by the model contract, evaluates the model, and returns named actions. The algorithm determines the decision; the service supplies the software interface through which that decision becomes available to the rest of EnergAIze. A compatible rule-based policy can occupy the same interface. Keeping the observation and action interpretation with the model allows a selected controller to be loaded without reconstructing its processing by hand.

The returned actions are proposals for the current interval. Before they reach the charger or battery, they pass through the validation and provider interfaces described below. Subsequent measurements close the loop by revealing what the equipment actually did. Preparing the inputs for this sequence begins with the data-integration service.

4.2.2 Data Integration and Routing: Percepta

The measurements required for a decision can arrive through different interfaces and at different times. A building energy-management platform may expose meter readings and battery status, a charging platform may report connection and charging power, and a vehicle service may provide SoC. The stream-processing component, Percepta, handles the integration and processing of these sources so that the controller receives observations associated with the correct building and resources (Sousa et al. 2025).

Source-specific receivers obtain the incoming data, and translators map their fields into the representation used by the installation. Subsequent processing aligns the values used for a decision interval. For example, charging power reported in watts and household demand reported in kilowatts must be expressed consistently before they enter a common calculation. Their timestamps also matter: a charging measurement taken before the vehicle disconnected describes a different condition from the latest connection status. The data-processing path retains these associations when assembling the controller's observations.

In the OPEVA integration, the Data Provider implements this path through receivers, translators, queues, an accumulator, and a manager that prepares the inference request (Salvan and OPEVA Consortium 2026). The same integration work supports the retrieval of historical records for development. On the action path, a forwarder converts the validated action into the format and destination required by the equipment provider. This arrangement places provider-specific communication at the boundary of the system, allowing the controller to work with the observation and action names defined for the installation.

4.2.3 Vehicle Information: PulseCharge

Vehicle information extends what can be learned from the charger. The vehicle-data service, PulseCharge, retrieves information such as SoC and charging status through manufacturer interfaces, subject to the information available for the connected vehicle. It handles the corresponding authentication and periodic data retrieval, then makes that information available to the data-processing path.

In the running example, the charger establishes that a vehicle is connected, while PulseCharge can supply the 30% SoC needed to determine how much energy remains to be delivered. These sources describe the same charging session from different points: one reports the charger and its power, the other the vehicle and its battery. Associating them with the correct vehicle and charger allows the controller to interpret them together. The charging platform remains the interface through which the charging action is applied.

4.2.4 Production and Consumption Forecasting

The production and consumption forecasting services, represented by *Predictor* within the forecasting group in Figure 4.3, supply expected conditions over the period considered by the controller.

Production forecasts estimate PV output; consumption forecasts estimate the demand against which charging or battery operation will take place. Their inputs can include historical measurements, weather information, and calendar variables. Percepta makes the resulting forecasts available alongside the latest observations, with the forecast horizon and time interval identifying the period they describe.

For the vehicle arriving at 18:00, a forecast of high household demand can support spreading charging over the night. If the vehicle remains at home on another day, a forecast of substantial midday production may favour charging during that period. Each forecast concerns a future quantity whose realised value will later be measured. As new observations arrive, subsequent decisions can account for the difference between the expected and actual conditions. This connection keeps forecasts within the ordinary sequence of measurement, decision, and response.

4.2.5 Flexibility Forecasting: Flexon

The flexibility-forecasting component, Flexon, estimates the flexibility expected to be available from participating resources. Within the forecasting role shown in the runtime figure, it supplies information about opportunities to adjust consumption or storage. For vehicle charging, this includes anticipating when a vehicle will be available and the energy associated with its expected use. The controller can then consider whether the anticipated connection window offers room to move charging towards a more favourable period.

User requirements establish the service to be delivered during the current connection. In the example, the driver requests 80% by 08:00. If an earlier departure is specified, the available charging window becomes shorter even when previous journeys suggest a later departure. The controller receives both the forecast used to anticipate availability and the current requirement that its decisions must satisfy. Production and consumption forecasts describe the energy conditions around this opportunity; Flexon describes the flexibility expected to be available within it.

4.2.6 Action Validation: Fail-Safe Supervisor

The Fail-Safe Supervisor is a separate component between the controller's proposed action and the request sent to the equipment. It checks the proposal against the installation's operating limits and current resource information, including charger ratings, battery limits, and available connection capacity. A proposal within those limits passes through unchanged; where a request exceeds the available capacity, the supervisor limits it before forwarding. In the OPEVA middleware, validation precedes the forwarder that constructs the provider request (Salvan and OPEVA Consortium 2026).

Each part of this path has a distinct responsibility. The controller's feasibility mechanisms account for the constraints represented by its method, while the inference service applies the output bounds and resolution declared for the model. The supervisor checks the resulting request against the operating conditions reported by the installation. Provider platforms and equipment retain their own protections when executing the validated action. This separation preserves the

distinction between policy proposals, supervisory intervention, and equipment response developed in Section 3.5.2.

For the household vehicle, an increase in other loads may leave less power available than the controller requested. The supervisor reduces the charging command to the available capacity, and the next measurements show how much energy was delivered. Subsequent decisions then account for the energy still required before 08:00. The record of the original proposal and its correction makes it possible to examine both the immediate limit and the effect on departure readiness.

If the algorithm stops or produces no inference output for the current interval, the supervisor uses a lightweight RBC to provide a fallback action. Its configured rules use the available operating information and remain subject to the installation's limits. The supervisor logs these fallback decisions as well as adjustments to policy proposals. The resulting records show when the learned controller supplied the action, when supervision changed it, and when control continued through the fallback.

4.2.7 Service Monitoring: Watchdog

The Watchdog monitors the services involved in the control loop, including data processing, controller execution, and supervision. It identifies interruptions so that the affected service can be examined and restored. The supervisor provides the immediate fallback when an inference output is unavailable; the Watchdog makes the service interruption visible to the operator. Its scope follows the dependencies: losing one vehicle-data source affects that vehicle's observations, while losing a shared inference service may affect several buildings. Resuming learned control requires the relevant observations and inference outputs to be available again.

Data availability also matters while the services themselves remain running. In the documented OPEVA middleware, an absent measurement can be completed with the previous value when preparing an interval (Salvan and OPEVA Consortium 2026). Its source time remains important when deciding how that value can be used. A delayed price update and an unavailable battery SoC have different implications for the charging decision. The response depends on the information needed for the requested action and the alternative control behaviour configured for the installation.

4.2.8 Operational Records, User Interfaces, and KPI Manager

The database retains observations and decision records so that the course of operation can be examined. These records associate the controller version and its observations with the proposed action, any supervisory adjustment or fallback, the validated action, the provider response, and subsequent measurements. For the vehicle, they allow a reduction in charging power to be traced to the policy, the supervisor, or the equipment. Keeping that distinction through the session connects each intervention to the energy delivered before departure.

The graphical user interfaces give operators access to the current state of the installation and its operational records. They provide a way to inspect the participating resources, follow their

measurements and actions, and examine interruptions reported through monitoring. An operator investigating unexpectedly slow charging can therefore relate the requested power to the vehicle state and the limits applied along the control path.

The KPI Manager calculates KPIs from the recorded operation. Electricity cost and peak demand can be examined at building and REC level, alongside the vehicle's SoC at departure. The intervention records show when those outcomes were obtained with corrections or fallback control. The same operational records supply data and cases for subsequent training and evaluation.

4.3 Controller Development

The runtime architecture establishes what a controller receives, how its proposals reach the equipment, and how their consequences are recorded. Development prepares the controller for that role. Historical data and installation configuration provide the starting point, while simulation allows candidate policies to encounter successive conditions and be evaluated against the services they should deliver. The same process supports later retraining when operational records reveal conditions that call for a revised controller.

Figure 4.4 follows this controller through development. Installation records (1) and REC configuration (2) define the scenarios (3). Training (4) and evaluation (5) lead to the selected model bundle (6), which reappears at the start of the deployment view in Figure 4.6. The computing infrastructure in Section 4.4 executes the training and evaluation stages as experiments.

4.3.1 Data Preparation and Simulation Scenarios

The household meter, vehicle service, and charger introduced in the runtime view also provide records from which development scenarios can be constructed. These records describe when energy was consumed, when a vehicle was present, and how its charging progressed. Data preparation aligns timestamps, units, and sampling intervals, identifies missing periods, and separates the records used for training from those reserved for evaluation. The REC configuration identifies its buildings, equipment, capacities, and operating limits. These describe the setting in which the selected control method is developed.

The building-energy simulator, CityLearn, uses these inputs to represent the environment in which controllers act (Nweye, Kaspar, et al. 2025). Vehicle and charger modelling was extended through EVLearn (Fonseca, Ferreira, Cabral, Severino, Nweye, et al. 2025), and the CityLearn v3 development presented in Chapter 5 supplies the experimental environment used throughout this thesis. Historical demand and production describe the conditions around a decision, while resource models calculate how the controllable equipment responds to it.

The distinction matters for the charging example. Recorded household demand can supply the background load for the evening, but the vehicle's SoC must evolve according to the actions chosen by the controller being tested. Replaying the recorded charging trajectory would prevent a different decision from changing the outcome. The vehicle model instead updates the stored energy

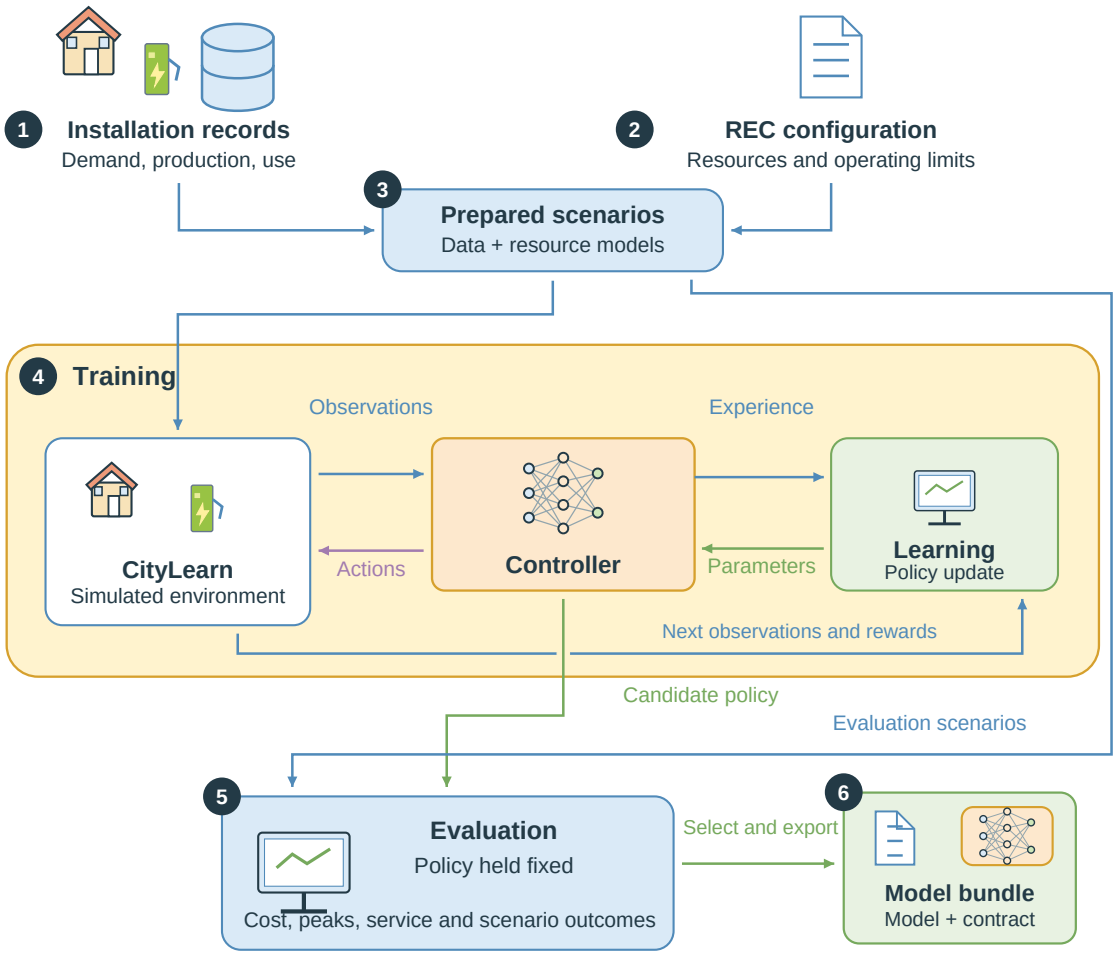


Figure 4.4: Controller development, with the model bundle (6) continuing into the deployment view.

as charging proceeds, subject to its parameters and the charger limits. The next observation reflects that response, allowing the controller to reconsider its decision as the departure approaches.

Supervisor records identify intervals in which a proposal was limited or a fallback supplied the action. These intervals help select conditions to reproduce during development, such as a recurring evening capacity shortage or an interruption in inference. When operational records are used directly as learning experience, the proposed action, validated action, and measured response remain associated. This retains the quantities needed to account for intervention in the learning method, following the distinction in Section 2.3.5.

A scenario fixes the conditions of this interaction. It identifies the buildings and resources, the available observations and actions, the external data, and the events encountered during the episode. The arrival at 18:00 and requirement of 80% by 08:00 can be held constant while several controllers are compared. An earlier departure, a lower connection limit, or an additional vehicle creates a

further scenario. These variations let the development process examine particular changes while preserving the settings needed to interpret their consequences.

The scenario also defines the correspondence with runtime. An observation of SoC belongs to a specified vehicle, and a charging action belongs to a specified charger with a defined unit and range. Forecasts describe information available when the decision is made. Maintaining these associations during simulation prepares the controller for the data and action interfaces through which it will later operate.

4.3.2 Controller Training and Evaluation

With the environment established, RL proceeds through repeated interaction. The controller receives the current observations and selects actions for the represented resources. The simulator applies those actions, advances the resource states, and returns the next observations and a reward. The learning procedure uses this experience to adjust the policy. Over successive episodes, the controller encounters the consequences of its choices across complete operating periods.

In the household example, selecting a low charging power may reduce demand during a busy interval, yet leave more energy to be supplied later. The simulated trajectory exposes both effects. The reward expresses the objectives used during learning, while the environment and the controller's feasibility mechanisms represent the applicable constraints. Costs, peaks, and service outcomes can then be examined over the resulting charging session. Their precise formulation depends on the method studied and is developed in the control chapters.

Multi-agent training extends this interaction to the REC. Each agent acts for the resources assigned to it, and the environment calculates the combined effect of those actions on buildings and shared quantities. The learning procedure may use information from several agents, even where the resulting controller uses local observations during execution. The development configuration records that distinction: information available to training and information required by the deployed policy have different roles. The runtime interface is determined by the latter.

Evaluation applies a candidate policy with its parameters fixed under scenarios kept separate from training. Common scenarios and information allow comparison with rule-based and other control methods. Energy and service outcomes are read alongside supervisory corrections and fallback use, with the same protection settings retained across a comparison. For the vehicle, a lower cost can then be related to departure readiness, household peak, and the interventions needed to obtain them. Following retraining, comparison with the previous controller also shows how behaviour changes under both the newly identified conditions and the previous evaluation scenarios. Repeated training runs with different random seeds expose variation introduced by learning.

The outcome of this process is a set of results associated with the configuration and model that produced them. Model selection uses those results to identify the controller intended for a particular runtime role. The selected model is exported together with its input and output description, forming the model bundle introduced in Section 4.4.4. Its observation order, processing, and action interpretation must agree with the controller evaluated in simulation.

These stages can require many executions: training several candidates, repeating seeds, and evaluating each controller under several scenarios. Their sequence establishes the work that the infrastructure must support. The next section follows an experiment from its configuration to execution on an available computing resource, then follows the selected model into the inference service.

4.4 Experiment Orchestration and Controller Deployment

Executing the development process requires a persistent account of each experiment and a way to run it on suitable computing resources. The experiment-management service, the Job Orchestrator, maintains the configuration, queue, and execution record. A Worker Agent connects that record to the machine or remote service that performs the computation. Once a model has been selected, the deployment backend manages its bundle and requests its loading at the chosen inference target. These services carry the controller from an experimental configuration to its place in the runtime architecture.

4.4.1 Infrastructure and Experiment Configuration

Figure 4.5 shows the computing arrangement behind this process. The web interface, backend, and orchestrator run as separate services on the OPEVA server, alongside persistent storage and the workers configured there. The web interface accesses jobs and experiment data through the orchestrator, while the backend provides the deployment interface. A local Worker Agent starts experiment containers through the host's Docker service. Other workers submit work to Deucalion or Union, where the experiment is executed remotely. Personal computers and the Jetson host run their own Worker Agent and experiment containers.

The execution targets provide Central Processing Units (CPUs) and, where required, Graphics Processing Units (GPUs); the worker selects an execution image compatible with the target hardware. Remote submission uses Secure Shell (SSH) or an API, according to the target. The Jetson target uses the 64-bit Arm architecture, labelled ARM64 in the figure.

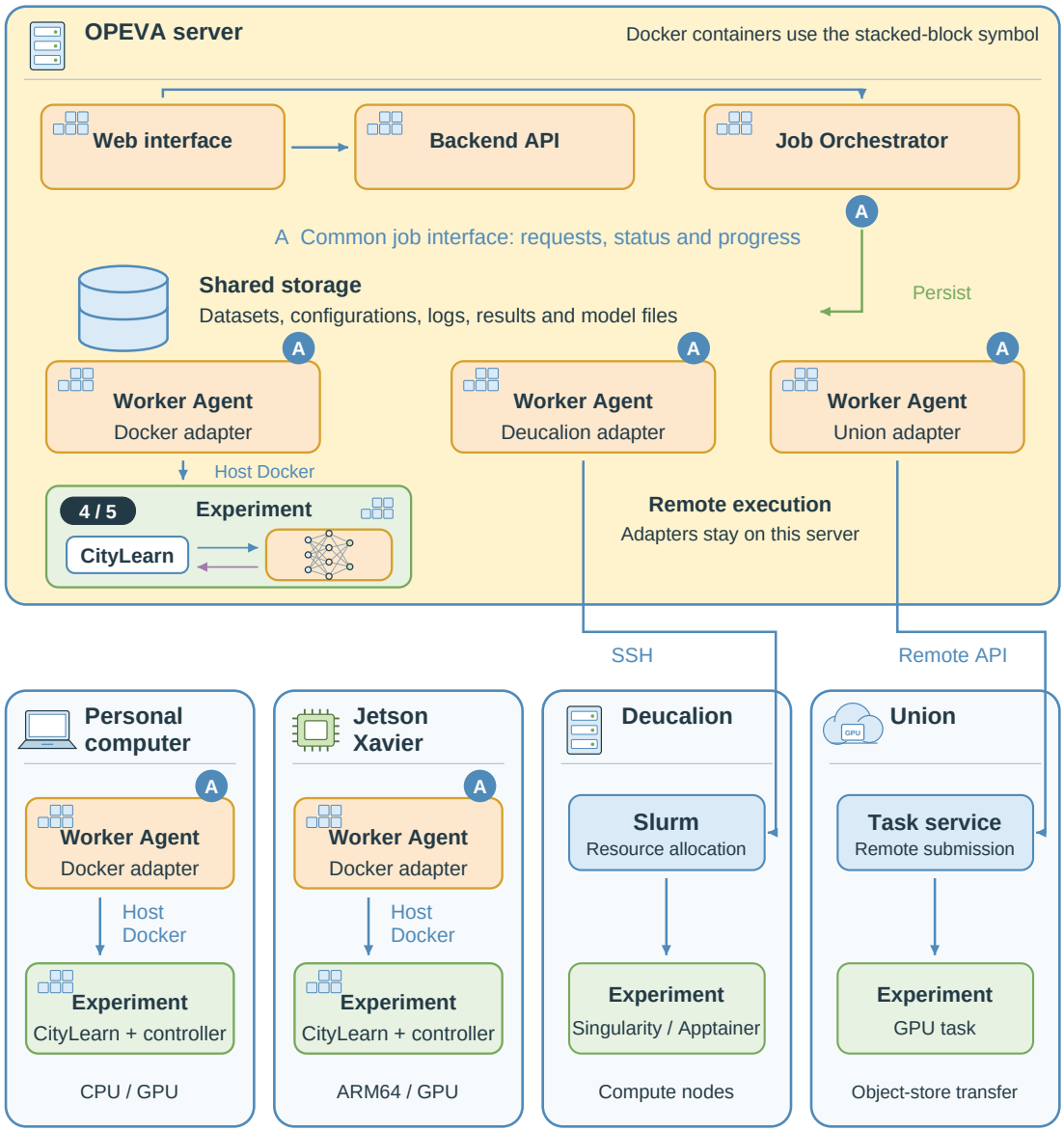


Figure 4.5: Worker Agents manage training (4) and evaluation (5) on local or remote resources. The expanded experiment retains the simulator and controller of Figure 4.4.

The worker is a persistent service responsible for obtaining work, launching it, and reporting its progress. The experiment is the process that runs the simulator and control method for one job. With Docker execution, the worker and experiment occupy separate containers on the same host; access to the host's Docker service allows the worker to start and stop the experiment. A remote adapter performs the equivalent management through the target's submission interface. This division allows the experiment software to retain its own dependencies and execution environment while the worker manages its lifecycle.

The experiment configuration combines the REC configuration and dataset with the selected

controller, observations, actions, reward, random seed, and training or evaluation period. The execution request adds the software image and computing requirements. Resolving these settings before dispatch gives the worker a complete description of the work to perform. The configuration actually executed is retained with the job, so that a later change to a reusable configuration file does not change the account of an earlier experiment.

For the charging example, two runs may use the same controller and household data while assuming different departure times or charging limits. Their interpretation depends on keeping those differences visible. A retraining run likewise has its own configuration and record, even when it continues from an earlier model.

Experiment tracking, model storage, and serving are established concerns in machine-learning operations (Kreuzberger et al. 2023; Zaharia et al. 2018). RL also requires the environment in which experience is generated (Li, Thomas, et al. 2022). The experiment record therefore retains the simulator and scenario configuration alongside the controller settings.

The orchestrator is implemented as a Hypertext Transfer Protocol (HTTP) service using FastAPI. Its interface supports experiment submission, job inspection, worker communication, and access to configurations and results. Shared storage retains each job's settings, state, and outputs under a common identity used by the web interface and workers. The experiment can therefore continue independently of the browser session through which it was submitted.

A worker periodically reports its availability, software version, active work, and relevant computing capabilities. When it has capacity, it requests a job that matches its execution profile. The orchestrator supplies the software image, resolved configuration, execution command, and required data locations. The worker then launches the simulator and controller using the execution mechanism of its target. The algorithms receive the configuration prepared for the experiment, while the worker handles the surrounding execution process.

This separation gives each service a clear responsibility. Queueing and job identity remain with the orchestrator; resource-specific execution remains with the worker. The training program can consequently run on a local machine or through a remote scheduler using the same experiment description. Additional workers make it possible to execute independent experiments concurrently, including different random seeds, controller settings, or scenarios.

4.4.2 Execution on Heterogeneous Computing Resources

The computing resources used during development have different execution environments. A workstation can start a container directly, a supercomputer admits work through a batch scheduler, and a remote service may require data to be transferred before execution begins. The Worker Agent accommodates these differences through target-specific adapters. Each adapter carries the common job description into the target environment and returns its state and outputs to the orchestrator.

Table 4.1 summarises the supported execution profiles. The distinction between them concerns how the experiment reaches the processor, obtains its data, and returns its results. The scenario and controller settings remain attached to the same job throughout this process.

Table 4.1: Computing resources and their execution adapters.

Resource	Execution mechanism	Responsibility of the adapter
Server	Docker with shared storage	Start the selected image, provide data access, and collect outputs.
Personal computer	Docker with optional GPU	Connect an intermittently available resource and complete active work before shutdown.
Deucalion	Slurm and Singularity/Apptainer	Stage inputs, submit the batch job, follow scheduler state, and retrieve results.
Union INESC TEC	Remote tasks with GPU resources	Transfer inputs, retain the remote run identity, and recover the returned results.
Jetson Xavier	Docker with a compatible NVIDIA image	Match the ARM64 architecture and installed GPU software to the execution image.

For server and personal-computer workers, the experiment container reads the configuration and data from mounted storage. For Deucalion, the Worker Agent remains on the submission side. Its adapter creates a remote workspace, stages the required inputs, and submits a Slurm request using a compatible container image. Slurm allocates the computing resources on which the experiment runs through Singularity or Apptainer. Scheduler limits and resource requests become part of that execution setup. Logs are synchronised while the job runs, and the final outputs return to the job's stored record.

The Union Worker Agent similarly remains on the OPEVA side and submits remote tasks. Inputs and results pass through object storage. The worker retains the remote run identity so that monitoring and result retrieval can continue across interruptions. The Jetson profile addresses another form of heterogeneity: its processor architecture and installed GPU software require a compatible image. These differences are handled at the worker boundary, where the information needed to execute the experiment is available.

Keeping the adapters behind the common worker interface allows the computing infrastructure to grow without introducing a separate experiment format for each resource. It also preserves information about where and how a result was produced. The software image, execution target, and returned logs remain available when interpreting differences between runs or preparing a further experiment.

4.4.3 Job Progress, Results, and Recovery

An experiment passes through several stages before its results are available. After submission and queueing, a worker accepts the job, prepares its execution environment, and starts the training or evaluation process. The job record follows this progression through explicit states. Remote preparation has its own place in that record because transferring a dataset or waiting for a scheduler is different from running the controller inside the simulator.

Progress updates and logs make this distinction useful to the operator. The EnergAIze web interface displays the queue, worker availability, execution state, training progress, and completed outputs. It also provides access to configurations and KPI comparisons, with links to MLflow

records where tracking is enabled. These views connect the visible result to the experiment that produced it. For a charging study, the operator can inspect cost and peak demand alongside the departure charge, then return to the corresponding scenario and controller settings.

Persistent state also supports recovery when communication or execution is interrupted. Each dispatched attempt can carry an identifying token that the worker returns with its updates. If a job is reassigned, updates from the superseded attempt are rejected. This prevents a delayed worker from replacing the state or results of a newer execution. A stop request follows the same division of responsibility: the orchestrator records the request, and the worker stops the container or remote job that it manages.

Remote result retrieval requires particular care. A training process may finish successfully while the connection needed to download its outputs is unavailable. In the Union adapter, a recovering state records this interval between completed computation and locally available results. The remote run identity and recovery progress are persisted, allowing the transfer to resume after the worker restarts. The completed training outputs can then be recovered without repeating the experiment. Deucalion similarly retains the connection between the local job and the remote scheduler record while its adapter monitors execution and retrieves the outputs.

Once execution and result retrieval are complete, model selection can use the recovered evidence. The selected controller then needs a description that allows the inference service to reconstruct the input processing and action interpretation used during evaluation. This is the role of the model contract.

4.4.4 The Model Contract

The selected model is exported with a model contract that preserves its input and output settings: observation names, ordering and encoding, returned actions, and agent–model associations. This manifest and the model files form the *model bundle* (6) shown in the development and deployment views. The full experiment configuration remains in the job record.

Table 4.2 groups the information passed through this interface. The installation configuration supplies the connection to physical equipment and its provider interfaces. The model contract supplies the interpretation required by the selected controller. Their correspondence is what allows an observation assembled by Percepta to reach the intended input and a returned action to be directed to the correct resource.

For the household vehicle, the contract determines where SoC enters the model and how the returned charging decision is named. If the model was trained with encoded calendar variables or a particular observation order, the inference service reconstructs that processing when it loads the bundle. A controller serving several agents also needs the association between each agent and its model. This association can point to separate model files or to a shared model used by several agents.

The general export path produces neural models in the Open Neural Network Exchange (ONNX) format and rule-based policies in JavaScript Object Notation (JSON). Before the bundle is finalised,

Table 4.2: Information carried by the model contract.

Part of the contract	Stored information	Use in runtime
Observations	Names, ordering, and input encoders	Reconstruct the input representation used during development.
Actions	Names, bounds, and output settings	Return identifiable actions in the form expected by the integration path.
Agents and models	Agent indices, model format, and file locations	Select the model associated with the requested agent.
Origin and configuration	Run identity and accompanying environment metadata	Associate the loaded controller with its development record.

validation checks the manifest structure, declared formats, referenced files, agent indices, and observation and action dimensions. The resulting package can be passed from the experiment implementation to the inference service without manually rebuilding its feature ordering or action mapping. The control methods developed in Chapters 6 and 7 supply any additional requirements for their runtime components.

4.4.5 Controller Deployment and Updates

The deployment backend maintains a catalogue of model bundles in shared storage. Each stored bundle retains its manifest and model files under a content-derived identity. The web interface allows the operator to inspect the available bundles and select a version for an inference target. A target is the service endpoint at which the selected controller will be loaded; the installation configuration determines which observations and equipment are associated with that service.

Figure 4.6 continues from the same model bundle (6), through catalogue storage (7), operator selection (8), and backend loading (9). The controller runs inside the inference service, using the runtime connections shown in Figure 4.3.

For the selected target, the backend resolves the stored bundle location and sends a loading request to the inference service. The service reads the manifest, locates the model files, prepares the input encoders, and creates the appropriate ONNX or rule-based executor. It then reports the loaded configuration and agent indices, making the active controller visible to the backend and web interface.

During operation, health information exposes the service's readiness, while request identifiers associate inference calls with their responses. The records described in Section 4.2.8 connect the loaded controller to supervisory interventions, provider responses, and subsequent measurements. An operator can consequently follow a charging request from the selected model to the power delivered at the installation.

Updates use the same loading path. The operator selects a new bundle for the target and inspects the reported active version after loading. Earlier bundles can remain in the catalogue and be selected again when a previous controller is required. The backend protects a bundle reported as active by a

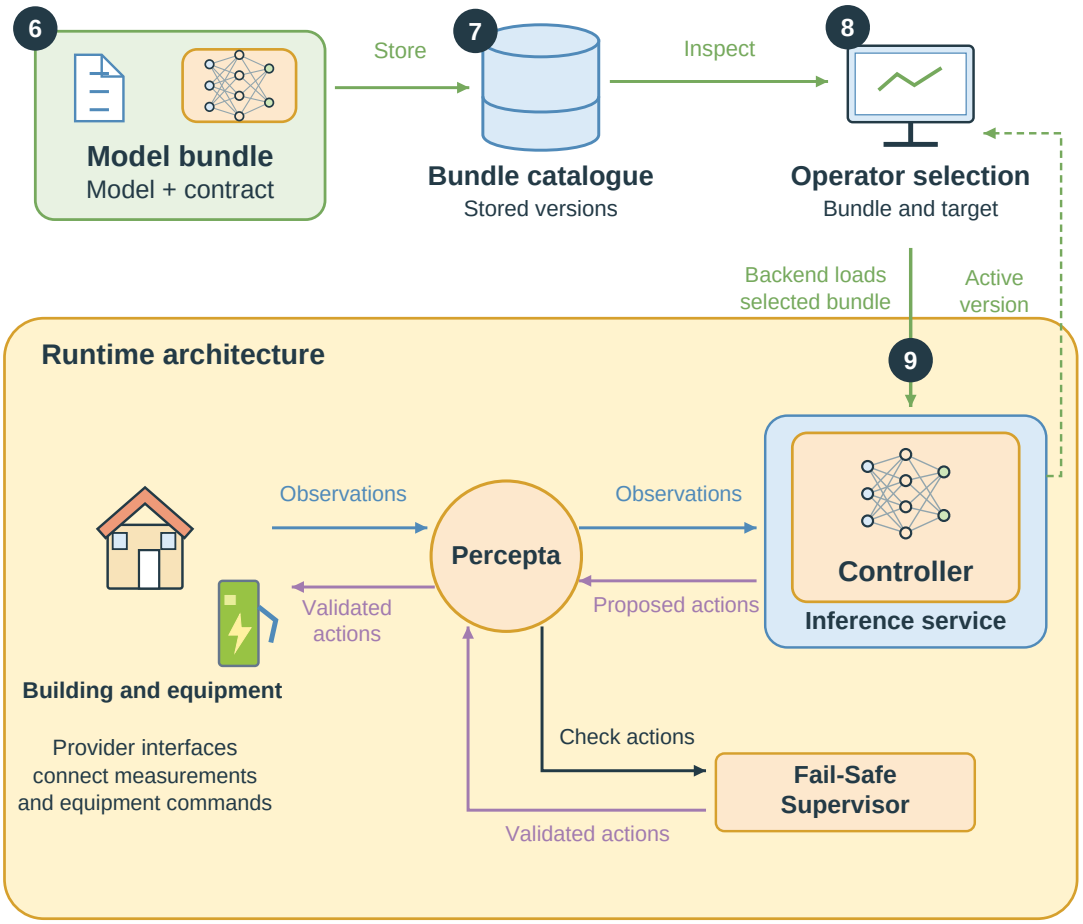


Figure 4.6: Model deployment (6–9) and the controller within the runtime control loop.

reachable target from deletion, preserving the files associated with that deployment. Each target exposes its own loaded state, allowing the operator to follow changes across installations.

The operational records retained during deployment supply the datasets and cases used for subsequent development. An operator can revisit intervals with repeated corrections or fallback use, together with changes in connection times, equipment, or operating limits. These findings guide scenario revisions, configuration updates, or controller retraining through the same orchestration service. The resulting candidate is evaluated through the process in Section 4.3.2, after which the operator can select its model bundle for deployment. Its development record retains the connection to the operating conditions that motivated the update.

4.5 Chapter Summary

EnergAIze connects REC energy control to the services needed to develop and operate its controllers. Runtime integrates observations, controller execution, independent supervision, and equipment

interfaces. Development uses data and simulation to train and evaluate candidates, with the Job Orchestrator and Worker Agent managing experiment execution. The deployment backend loads the selected model bundle into the inference service. Operational records return measured outcomes and supervisory interventions to development, supporting later retraining and evaluation.

These connections give architectural form to the four dimensions established in Chapters 1 and 3. For *scalability*, REC configurations and experiment records allow control outcomes to be related to the system's size, composition, and computing requirements. For *deployability*, the model contract, development services, and equipment interfaces organise preparation, training and adaptation, integration, and maintenance so that evaluated behaviour can be reproduced and sustained. *Safety* is addressed through constraint enforcement and separate records of proposals, supervisory intervention, and equipment response. *Robustness* draws on service monitoring, fallback control, and recovery, with retained service examined under the conditions recorded during operation.

Chapter 5 develops the simulation environment used to represent buildings and resources and compare their controllers. Chapters 6 and 7 develop the control methods, while Chapter 8 examines their integration and use in operating installations.

Chapter 5

Simulation and Evaluation of Renewable Energy Communities: CityLearn v3

The architecture in Chapter 4 places simulation between the preparation of REC data and the evaluation of a controller. This chapter develops that experimental environment: the extensions to CityLearn, the REC datasets, and the methods used to check the simulator and compare control decisions. Together, these provide the experimental foundation for the learning methods in Chapters 6 and 7.

5.1 Development of the Simulation Environment

Preparing a controller for operation requires an environment in which its decisions have consequences over time. Charging a vehicle now changes the energy still needed before departure; using a battery changes the flexibility available later; and sharing PV production changes the exchanges of several members. In the development process described in Chapter 4, simulation provides this continuity. It allows a controller to learn from repeated interactions and to be evaluated under specified REC conditions before it is selected for deployment.

An established building-energy environment, CityLearn, provided the starting point. Its building and storage models, common controller interface, and energy indicators already supported multi-building control studies (Nweye, Kaspar, et al. 2025; Vázquez-Canteli, Kämpf, et al. 2019). Shared tasks and datasets, including the CityLearn Challenge, had also established a common setting for comparing research methods (Nagy et al. 2021). Developing within this environment connected the work to an existing research community and allowed the new capabilities to be used alongside established models and controllers.

The contribution developed in this thesis extends that foundation with REC resources, operating events, controller interfaces, and evaluation methods. It began with the integration of EVs through EVLearn, which introduced chargers, connection schedules, departure requirements, and bidirectional charging (Fonseca, Ferreira, Cabral, Severino, Nweye, et al. 2025). Work on REC control,

informed by OPEVA and DemFlex, then required a broader representation of how installations operate. Members could join or leave, equipment could change, flexible appliances had complete programmes to deliver, and local sharing affected individual costs. Measurements and commands could also fail. These conditions shaped the subsequent development of CityLearn v3 and the experiments it needed to support (Fonseca, Ferreira, Sousa, et al. 2026).

Accommodating them involved changes across the simulation process. Installing a charger, for example, introduces a physical resource, an action that a controller can request, and future charging services whose outcomes must be recorded. Its identity and activation time therefore have to agree across scenario loading, state progression, controller inputs, and evaluation. Similar connections arise when a member leaves or an action cannot be applied. The work extended these parts together so that an experiment could follow a change from its declaration to its effect on energy use and participant service.

These extensions give the RQs in Chapter 1 concrete experimental settings. A changing resource collection exposes the adaptation and computational demands of scalability; electrical limits and participant services establish the conditions for examining safety; and repeatable failures make robustness comparisons possible. The controller interface also carries the resource and action descriptions needed by the deployment process in Chapter 4. The following control chapters use this foundation to investigate how the methods respond.

The resulting software was first distributed by the SoftCPS Research Group as the SoftCPS REC Simulator (SoftCPS Research Group 2026h,i). In September 2026, this development line was incorporated into the official CityLearn version 3, distributed as `citylearn` (CityLearn Project 2026a). Its Gymnasium interface, examples, documentation, and tests make the extensions available through the established project (CityLearn Project 2026b). The contribution thus provides both the experimental environment used in this thesis and reusable modelling and evaluation capabilities for other REC-control studies.

5.2 Community Modelling and Simulator Architecture

The simulator represents buildings and their resources within one or more RECs. In the scenarios used here, each member is represented by one building site, with a local agent managing that site's assets. This modelling convention instantiates the distinctions in Section 2.1.1; it does not equate members, sites, and agents in the general definition. A scenario supplies their data, connections, services, and operating events; a controller receives observations and proposes actions as the simulation advances. Figure 5.1 locates these interactions. Each REC retains its own members, local sharing, and outcomes, while a common environment can advance several RECs together.

The changes shown in the figure are part of the model: a member leaves, a charger is installed, and data or equipment become unavailable. Each changes a different aspect of the decision faced by the controller. The following subsections explain how these conditions are represented and how the implementation makes their consequences available for subsequent experiments.

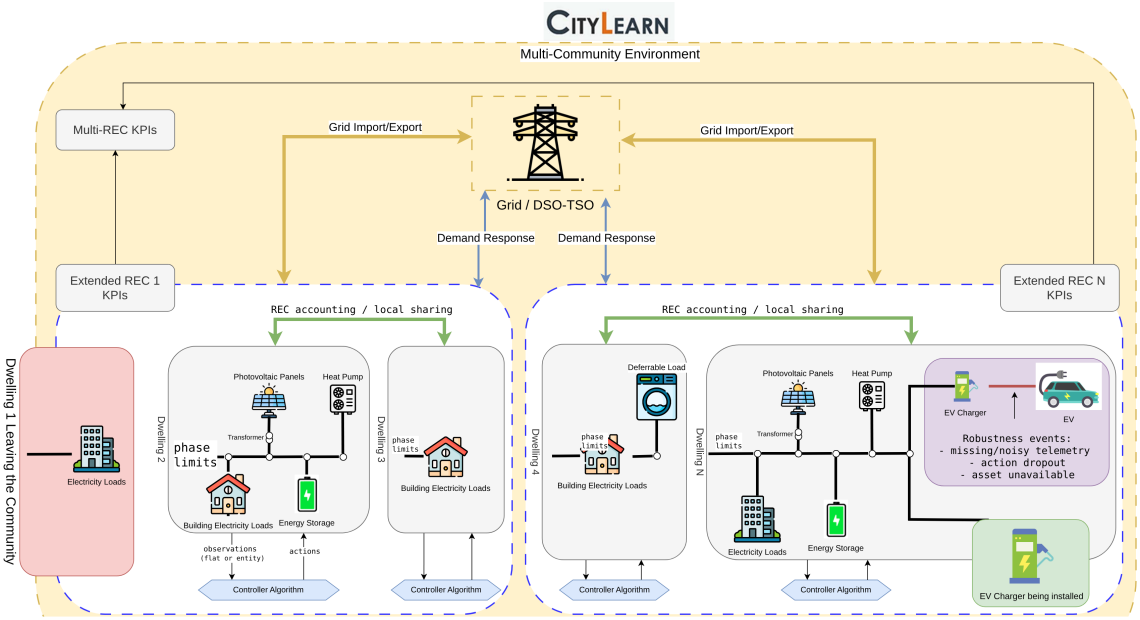


Figure 5.1: Representation of one or more RECs in CityLearn v3. Reproduced from Fonseca, Ferreira, Sousa, et al. (2026).

5.2.1 Scenarios, Time, and the Simulation Step

A scenario combines time series with the information needed to interpret them. It identifies the buildings and equipment, their capacities and connections, the services to deliver, the observations exposed to a controller, and the events active during the episode. This configuration distinguishes two experiments that use identical household demand but differ in charging availability, an electrical limit, or a missing measurement.

At timestep t, the simulator first resolves scheduled changes and constructs the available observations. The controller then proposes an action, which passes through the applicable feasibility and availability rules before the resources advance:

$$\begin{aligned} \tilde{o}_t &= \mathcal{O}(x_t, w_t, e_t), & a_t^{\text{req}} &= \pi(\tilde{o}_t), \\ a_t^{\text{app}} &= \Gamma(a_t^{\text{req}}, x_t, e_t), & (x_{t+1}, y_t) &= F(x_t, a_t^{\text{app}}, w_t, e_t). \end{aligned} \tag{5.1}$$

Here, x_t is the represented physical and service state, w_t contains external inputs such as demand and weather, and e_t contains active events. The observation function $\mathcal{O}$ produces the controller-visible information $\tilde{o}_t$, and policy π returns requested action a_t^{req}. Operator Γ accounts for the configured limits, availability, and command handling to obtain applied action a_t^{app}. The transition function F produces the next state x_{t+1} and the physical, service, and accounting outcomes y_t.

For the charging example, the SoC after an interval depends on the power actually applied. Retaining the original request makes it possible to distinguish a controller that respects the connec-

tion limit from one whose proposals repeatedly require correction, even when both produce the same physical trajectory. This separation gives the safety experiments access to the decision and its execution. It corresponds to the controller, supervisor, and equipment roles described in Section 4.2.

Time and units are part of this interpretation. Power and instantaneous limits use kW, interval energy uses kWh, prices use currency/kWh, and carbon intensity uses $kgCO_2$/kWh. Average power converts to interval energy through

$$E_t = P_t \frac{\Delta t}{3600}, \tag{5.2}$$

where E_t is energy in kWh during interval t, P_t is its average power in kW, and Δt is the interval duration in seconds. The factor 3600 converts seconds to hours. A 7.2 kW charger consequently supplies 1.8 kWh in 15 minutes or 0.03 kWh in 15 seconds.

The same care applies to PV inputs. A normalised production profile is scaled by installed capacity, whereas an absolute production series already describes the site's output. The loader checks the declared clock, timestamp spacing, and episode bounds. Columnar storage and windowed loading make short timesteps practical for repeated learning experiments without requiring each run to process an entire high-frequency dataset.

Figure 5.2 illustrates why temporal resolution matters to safety-sensitive studies. It represents the same synthetic annual trace at one-hour, 15-minute, and 15-second resolution. The annual energy is preserved, while the visible peak rises from 96.8 to 188.2 and 281.5 kW as shorter intervals retain events hidden by averaging. A simulator intended to study electrical headroom therefore needs an explicit clock and consistent energy accounting; the experiment can then choose the resolution appropriate to its control question (Fonseca, Ferreira, Sousa, et al. 2026).

5.2.2 Resources, Electrical Limits, and Participant Services

Buildings retain background demand, PV generation, grid exchange, and their available flexibility resources. Inherited thermal and storage components preserve compatibility with established CityLearn studies. The extensions developed here connect stationary storage, EV charging, and complete appliance cycles to the services and electrical conditions of each installation.

Stationary storage and electrical headroom. A stationary battery has an energy capacity, charging and discharging power, efficiency, and state-of-charge limits. Its admissible exchange changes with stored energy and with the headroom available at the building. Charging a battery and a vehicle at the same time may therefore require less power than either device could accept alone.

The electrical model represents total import/export limits and, where configured, separate limits for each phase. With import taken as positive, the limits are

$$-X_b^{\max} \le P_{b,t} \le I_b^{\max}, \qquad -X_{b,\phi}^{\max} \le P_{b,\phi,t} \le I_{b,\phi}^{\max} \quad \forall \phi \in \Phi_b. \tag{5.3}$$

Here, b is a building, t the timestep, and Φ_b its electrical phases. Total and phase power are $P_{b,t}$ and $P_{b,\phi,t}$; $I^{\max}$ and $X^{\max}$ denote the positive import and export limits. All powers are in kW.

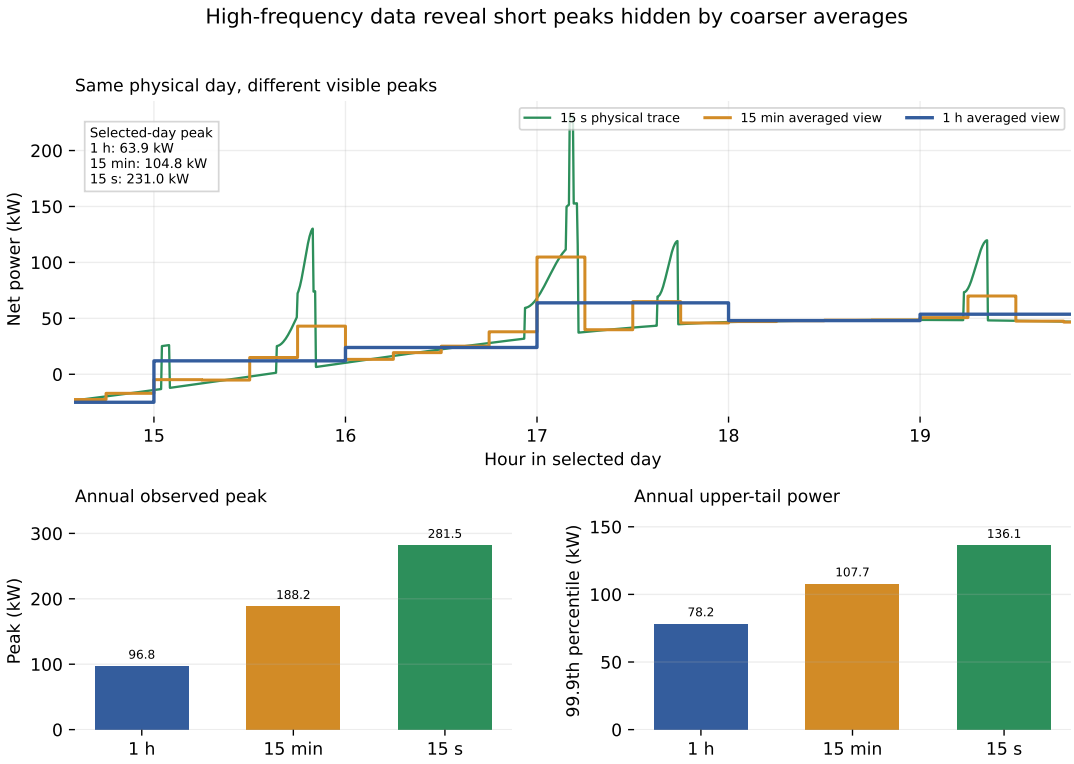


Figure 5.2: Peak visibility at three temporal resolutions of the same synthetic annual trace.

If the house already imports 4 kW on a phase limited to 7 kW, a 7.2 kW charging request has only 3 kW of headroom. Representing the connection as well as the device rating exposes this competition between simultaneous uses. Requested pressure measures the excess before constraint handling, while residual violation measures exceedance in the applied trajectory, including cases where background demand alone exceeds a limit. Section 5.4.2 defines how these quantities enter the safety comparison.

Vehicles and charging sessions. A charging session specifies the connected vehicle, arrival SoC, departure time, requested departure state, and applicable power bounds. These quantities determine the remaining energy requirement and the time in which it can be met. As 08:00 approaches, the same postponement that was harmless at 18:00 may make the target unreachable. The simulator follows battery response through the session and evaluates service at departure.

The model gives the vehicle, physical charging port, and session separate identities. When the example vehicle leaves at 08:00, another vehicle may use the same port later that day: the service requirement changes while the installed equipment remains the same. Adding a second port changes the resources the controller can address. This separation allows experiments to vary service demand and resource composition independently, and to associate each departure outcome with the correct request.

The vehicle catalogue can link visits across sites. In the dataset runtime used here, each session supplies its arrival SoC; battery evolution then follows the applied charging power until departure. A window beginning during an occupied session uses the corresponding boundary state. The departure record compares achieved SoC with the requirement and gives the remaining energy deficit. Lower electricity consumption can therefore be read alongside the service actually delivered.

Deferrable appliances. A deferrable appliance offers a different decision: when to begin a complete programme. A cycle catalogue supplies the interval-energy profile and duration, while a request supplies an admissible start window, completion deadline, and service priority. Choosing a start commits the simulation to that programme. This preserves its energy and duration when a controller shifts it towards a favourable price or a period of solar production.

$$t_c^{\text{earliest}} \le s_c \le t_c^{\text{latest}}, \qquad s_c + d_c - 1 \le t_c^{\text{deadline}}. \tag{5.4}$$

Here, c identifies a requested cycle, s_c is its selected start timestep, and d_c is its duration in timesteps. The quantities t_c^{earliest}, t_c^{latest}, and t_c^{deadline} are the earliest start, latest start, and final admissible completion timestep. For a discrete profile $q_{c,\tau}$, the index τ denotes the interval within the cycle and $q_{c,\tau}$ gives its energy demand.

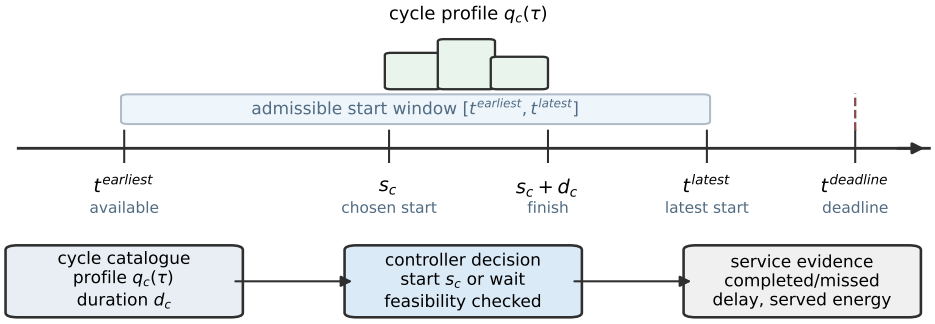


Figure 5.3: A complete appliance programme scheduled within its admissible start window.

Figure 5.3 relates the scheduling choice to the complete service. Once a cycle starts, its progress is followed alongside electrical limits. Completed and missed cycles, delay, delivered energy, and unserved energy make the consequence visible. This permits the evaluation to distinguish demand shifted successfully from demand simply left unserved.

5.2.3 Local Sharing and Demand-Response Services

An aggregate REC balance can hide simultaneous import and export by different participants. The simulator calculates local sharing in each interval, preserving the coincidence of demand and surplus, and settles that exchange at member level. A neighbour can then benefit from solar

production when it is available, while storage and scheduling determine how much demand can be moved into that interval. Member demand, local exchange, and residual grid exchange remain separate quantities.

For the active member set $\mathscr{C}_t$, local sharing is bounded by simultaneous demand and surplus:

$$\begin{aligned} D_{\mathscr{C},t} &= \sum_{b \in \mathscr{C}_t} \max(E^{\text{net}}_{b,t}, 0), \\ S_{\mathscr{C},t} &= \sum_{b \in \mathscr{C}_t} \max(-E^{\text{net}}_{b,t}, 0), \\ L_{\mathscr{C},t} &= \min(D_{\mathscr{C},t}, S_{\mathscr{C},t}). \end{aligned} \tag{5.5}$$

Here, $E^{\text{net}}_{b,t}$ is member b's net interval energy at time t, positive for demand and negative for surplus. For active members $\mathscr{C}_t$, $D_{\mathscr{C},t}$, $S_{\mathscr{C},t}$, and $L_{\mathscr{C},t}$ denote total demand, surplus, and locally shared energy, all in kWh.

Participant weights allocate the available exchange within demand and supply, and remaining energy passes through the grid boundary. The scenario defines the price of local exchange and the remuneration of residual export.

The settlement example in Figure 5.4 applies two accounting rules to the same historical 17-member trajectory. Under the configured sharing rule, 17.0 MWh is exchanged locally and total cost changes from 24.8 to 22.0 kEUR. Individual savings range from 68.8 to 525.7 EUR. The implementation retains this distribution because the REC total alone cannot show which participants benefit from the available flexibility (Fonseca, Ferreira, Sousa, et al. 2026).

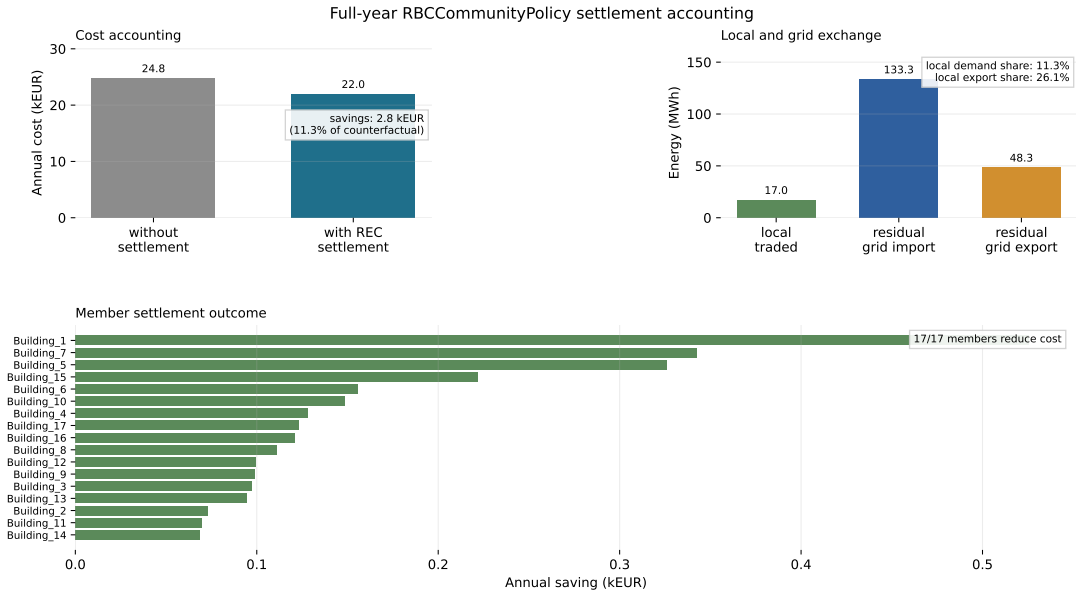


Figure 5.4: REC and participant outcomes under two accounting rules for the same historical trajectory.

DR introduces a service requested from outside the REC. Each request declares its issuer, direction, interval, target power, tolerance, remuneration, penalty, and reference method. Keeping the request explicit distinguishes service delivery from an ordinary response to price. For a reduction request, charging the example vehicle later can release power now, provided its departure requirement remains achievable.

$$d_{r,t} = \begin{cases} P_t - B_{r,t}, & \text{up request,} \\ B_{r,t} - P_t, & \text{down request.} \end{cases} \tag{5.6}$$

Here, r identifies a demand-response request, P_t is REC import power, $B_{r,t}$ is its reference power, and $d_{r,t}$ is delivered response, in kW. An up request asks for increased import, and a down request asks for reduced import; positive $d_{r,t}$ therefore denotes movement in the requested direction. Credited response is capped by the target, shortfall accounts for tolerance, and Equation 5.2 converts power into interval energy for settlement.

The Portugal scenarios use the native preceding-hour mean as their reference, recorded with the result. A paired run with the same controller and no activation measures the change caused by the request. This separates contractual accounting from additional response; Section 5.4 also retains rebound and terminal stored energy in the comparison.

5.2.4 Changing Participation and Operating Conditions

A REC can acquire a charger, lose a participant, or encounter a temporary equipment failure. Scenario events specify when these changes occur and which identities they affect. Membership and asset events are resolved before the controller acts, so state progression, observations, actions, settlement, and evaluation use the same active collection.

A removed member retains its earlier records and leaves subsequent REC accounting. A newly installed charger contributes observations, actions, and service obligations from its activation. Stateful assets use their declared boundary state when associated with the runtime, and historical records preserve earlier instances. The configuration determines whether an event represents temporary availability or a permanent change in the REC. Section 5.3.3 explains the permanent histories selected for the Portugal collection.

Figure 5.5 follows one such sequence through the implementation. A member joins at step 1000, a charger is added at 1300, another charger is removed at 1500, and PV equipment is removed at 1600. The structured observations and action rows follow the resulting resource collection. This is the setting required to study scalability under changing composition: the later control methods must encounter the updated resources, and their results must remain associated with the services active at each time (Fonseca, Ferreira, Sousa, et al. 2026).

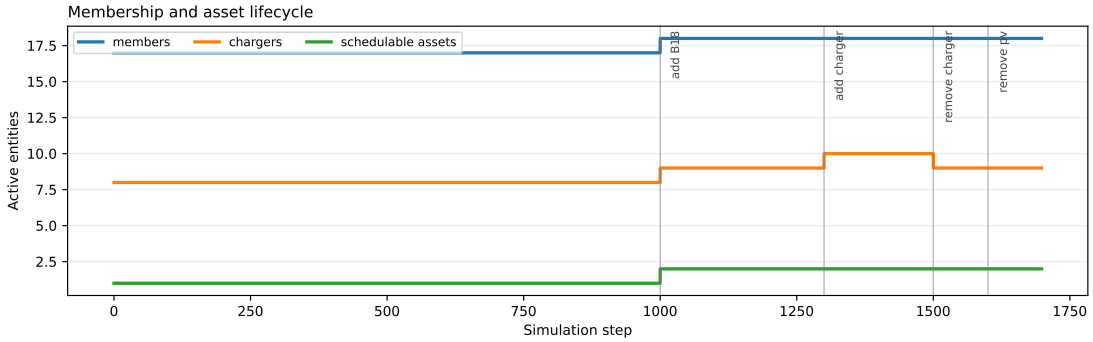


Figure 5.5: Propagation of membership and equipment events through the simulated resource collection.

Robustness requires a further experimental capability. The same REC must be able to encounter a specified failure while retaining the underlying demand, resources, and service requests of its nominal comparison. For the example vehicle, a missing state-of-charge reading and a lost charging command are different problems: one changes the information available to decide, while the other changes the effect of the decision.

Failures act at different points in the same decision loop. A missing measurement changes the information supplied to a policy; a biased forecast changes an expectation; a delayed or lost command changes actuation; and unavailable equipment restricts the feasible resource set. Their effect can be expressed through separate transformations:

$$\begin{aligned} \tilde{o}_t &= \rho_o(o_t, \mathscr{E}_t^o, \mathscr{E}_t^u), \qquad \tilde{f}_t = \rho_f(f_t, \mathscr{E}_t^f), \\ a_t^{\mathrm{cmd}} &= \rho_a(a_t^{\mathrm{req}}, \mathscr{E}_t^a), \qquad a_t^{\mathrm{app}} = \Gamma(a_t^{\mathrm{cmd}}, x_t, e_t). \end{aligned} \tag{5.7}$$

Here, o_t and f_t are nominal observations and forecasts, and their tilded forms are the values exposed after perturbation. The event sets $\mathscr{E}_t^o$, $\mathscr{E}_t^f$, $\mathscr{E}_t^a$, and $\mathscr{E}_t^u$ describe observation, forecast, command, and availability failures. Functions ρ_o, ρ_f, and ρ_a apply their configured transformations. The intermediate command a_t^{cmd} is the request after command disturbance, while Γ denotes the remaining action handling, including resource availability, used to obtain a_t^{app}. These expressions identify the failure channels; their configured order is retained with the scenario.

Separate event streams make each failure channel an experimental variable. A missing observation can first be studied with working equipment, then combined with a lost command to examine their interaction. Noise, bias, stuck values, clipping, delay, dropout, and equipment unavailability specify different disturbances; seeds make stochastic perturbations repeatable. The nominal demand and service requests remain common to the paired runs, so changes in their outcomes can be related to the imposed disturbance.

Figure 5.6 illustrates the resulting separation in a diagnostic run. Missing observations, forecast bias, command dropout, and equipment unavailability occupy different windows. The exported trajectory retains the nominal information, the information exposed to the controller, and the

requested and applied actions. Later robustness comparisons can consequently measure service loss and recovery under a reproducible disturbance and examine which part of the control loop was affected (Fonseca, Ferreira, Sousa, et al. 2026).

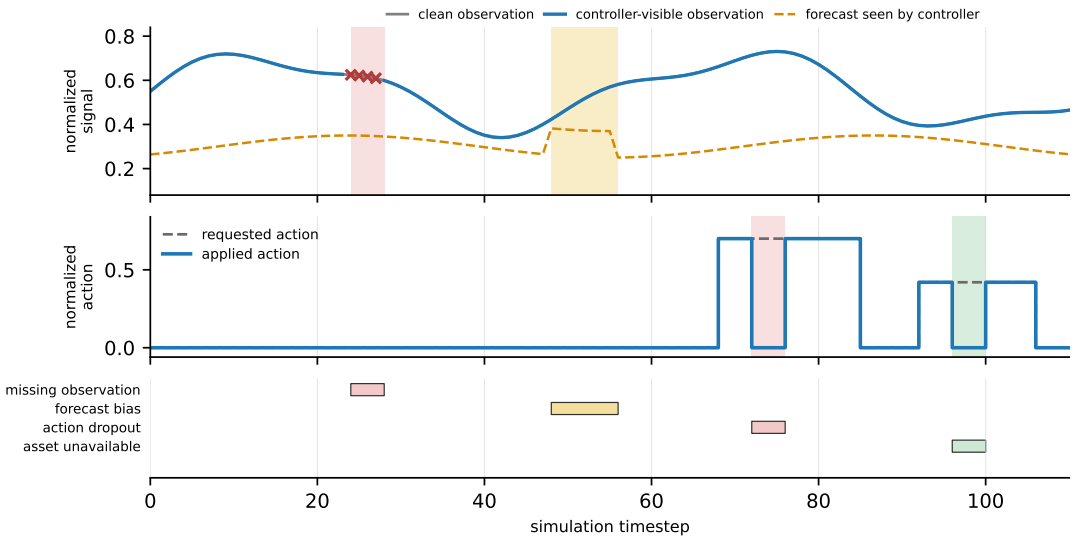


Figure 5.6: Separate observation, forecast, command, and equipment-availability disturbances in the diagnostic run.

5.2.5 Controller Interfaces and Multiple Communities

The interaction follows the familiar reset-and-step sequence: initialise an episode, receive an observation, submit an action, and obtain the next observation, reward, and diagnostic information. A flat interface supports methods expecting fixed arrays. A structured interface organises data by entity type, identifier, relationship, availability, and action bounds. When a charger is installed, its row identifies the resource and its building instead of leaving its meaning dependent only on a vector position.

Both views use the same resource models and scenario events. This allows experiments to examine the effect of controller representation while keeping the physical setting common. Existing methods can use a compatible fixed view, while set- and graph-based methods can process active resource collections. Chapter 7 develops the typed representation used for changing local interfaces. Resource identities, observation fields, and action bounds also supply the input/output description exported with the model in Chapter 4, connecting simulated evaluation to deployment.

Several RECs can be simulated together through `MultiCommunityEnv`. Each has its own scenario, buildings, controller, rewards, and outcomes, with a common clock advancing them in lockstep. Synchronised runs share the timestep, episode length, and compatible interface settings. This permits population and portfolio studies to measure local performance alongside the combined computational workload. Exchange between RECs is a separate scenario choice; the aggregation rules in Section 5.4.3 combine their results without altering their physical trajectories.

The implementation was examined through regression tests, physical and time-unit checks, controller-interface checks, and consistency checks on indicators and exports. The simulator study documents these procedures, their outcomes, and computational measurements (Fonseca, Ferreira, Sousa, et al. 2026); Appendix B.5 retains the supporting records. The figures above draw on its archived examples and show the implemented mechanisms that the datasets and controller comparisons below use.

5.3 Community Datasets and Experimental Scenarios

The datasets turn the capabilities described above into a progression of experimental settings. Small RECs make individual decisions easy to inspect; larger RECs introduce more resources, different daily routines, and concurrent service demands. Controlled scenario variants then change selected operating conditions. The Portugal collection follows this progression through four scenario bases and nine configurations, giving the later controller studies a common set of inputs whose purpose and assumptions are explicit.

The current edition covers 2023 and 2024 continuously at 15-minute resolution. It contains 70,176 records: 35,040 for 2023 and 35,136 for leap-year 2024. Two consecutive years provide different weather, prices, and routine realisations while preserving the REC history. The collection is identified as `portugal-native-2023-2024-v3.1`; Appendix B.3 gives its descriptive analysis, audit results, and reproducibility record. Earlier annual experiments remain identified with their original datasets in Appendix B.4.

5.3.1 Construction and Common Assumptions

Public energy and weather sources provide the external chronology, while synthetic members and routines supply heterogeneous REC activity. Portuguese demand statistics and E-REDES profiles inform consumption levels and shapes; household presence, opening hours, and simplified end uses introduce variation between members (Direção-Geral de Energia e Geologia 2023; Pordata 2023). PV production follows the site's geometry and installed equipment using the archived solar resource. Operador del Mercado Ibérico de Energía (OMIE) supplies the wholesale price chronology, and the reference tariff adds the declared variable components. Weather and carbon inputs retain their own sources and derivations, documented in Appendix B.3.

The Coordinated Universal Time (UTC) timeline preserves elapsed time through daylight-saving changes. Civil-time fields in `Europe/Lisbon` preserve the meaning of arrivals, opening hours, holidays, and school calendars. This matters for the running example: an 08:00 departure is a local service requirement, while the energy delivered depends on the number and duration of physical intervals before it.

The second year preserves the members and equipment and continues the composition reached at the end of 2023. Permanent departures and retirements remain effective. New weather, market data, and deterministic realisations of routines supply another year of operating conditions. Sessions

crossing New Year retain their continuity, so midnight does not create an artificial vehicle arrival. A simulation beginning partway through this history must initialise the corresponding topology and resource state.

Prices use one common Baixa Tensão Normal (BTN) bi-hourly indexed reference across all members. The stored series contains the full variable purchase price; export to the grid is unremunerated. Local exchange is disabled in the distributed data schemas and can be enabled in the resolved experiment configuration at 80% of the common grid-purchase price. This keeps the price convention explicit while preserving differences in the members' electrical connections and equipment.

Forecast inputs distinguish information already available from later realisations. Price forecasts use published day-ahead information when available and persistence otherwise; weather uses daily persistence. The corresponding forecast errors and publication boundary are analysed in the appendix. These simple references provide a reproducible information setting for the controller comparisons.

5.3.2 Community Bases and Their Experimental Roles

The four bases offer different ways of examining controller behaviour. Table 5.1 gives their complete inventories; the differences in ownership and activity explain the experimental role of each one.

Core-4: inspectable local decisions. Four residential members form a compact REC with one PV site, one stationary battery, and two charging sites. This makes it possible to follow a decision from one resource to the REC balance. The evening charging example can be inspected alongside a neighbour's solar production or a battery schedule, and a missed departure can be traced back through the preceding actions. Core-4 is consequently useful when establishing that a controller receives the intended information and delivers the expected services before undertaking more expensive studies.

Core-15: residential diversity and method development. Fifteen households introduce more combinations of generation, charging, appliance use, and consumption-only participation. Different routines make flexibility available at different times even though the setting remains residential. This supports method development, reward and observation studies, and repeated comparisons of candidate designs at a moderate computational cost. Overlapping arrivals and appliance windows make several households compete for favourable periods. The controller must use differences in their available time while maintaining each service.

Core-30: mixed activity and controlled comparisons. Twenty-four households and six service premises extend demand and charging activity across the day. Ten PV sites, three stationary batteries, fifteen charging connectors, and nineteen controllable appliances create concurrent decisions under finite electrical headroom. Core-30 Normal supplies the reference REC for the Topology, Safety,

Robustness, and Combined configurations. Daytime service activity changes the demand available to absorb solar production, while residential charging extends obligations into the evening. Keeping this physical base common across variants isolates the operating condition being studied.

Premium-100: larger and more heterogeneous communities. Eighty households and twenty service premises include offices, retail, clinics, cafes, workshops, and facilities with longer operating hours. The complete base contains twenty batteries and sixty-five charging connectors; one office has twelve ports. Premium Normal supports studies of larger populations and richer local interfaces, while Premium All-in combines changing participation with disturbances and demand-response services. The concentration of chargers at individual sites creates a further scheduling difficulty: several departures can compete for one building's electrical headroom. The scalability study therefore concerns both REC size and the number of decisions within a member.

Table 5.1: Complete physical inventories in the Portugal 2023–2024 collection.

Quantity	**Core-4**	**Core-15**	**Core-30**	**Premium-100**
Members	4	15	30	100
PV sites	1	4	10	36
Stationary batteries	1	1	3	20
Charging sites	2	5	10	41
Charging connectors	2	6	15	65
Vehicle identities	2	6	19	85
Controllable appliances	2	10	19	64
Consumption-only members	1	5	10	32

Ownership overlaps across all four bases. A household may produce PV energy without owning a battery, and a consumption-only member may benefit from energy supplied by another participant. The inventories therefore describe an uneven distribution of control opportunities. Population and composition both change between bases; experiments intended to isolate the effect of population size select matched compositions in addition to reporting the base used.

A charging site, a physical connector, a vehicle identity, and a visit remain separate quantities. The two-year static catalogues contain 9,691 sessions in Core-30 and 41,288 in Premium-100, including visits censored at the record boundaries. Residential visits include commuting, night work, and daytime presence; services introduce shorter and repeated visits. Requested departure SoC ranges from 50% to 100%. The catalogue therefore extends the evening-arrival example into a wider range of service windows.

Controllable appliance counts refer to physical devices. Programme profiles and requests are counted separately, while fixed appliance demand is already included in the non-shiftable load. The appendix retains both quantities, together with programme energy and the time available to schedule each request.

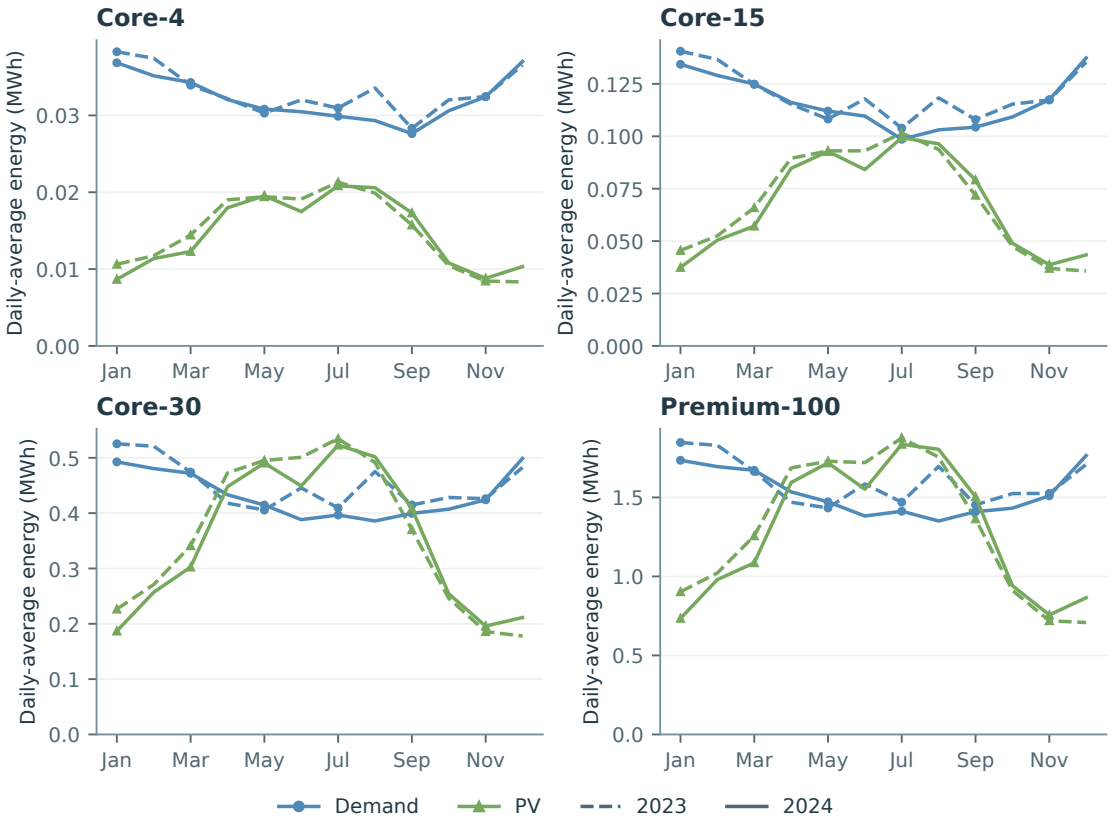


Figure 5.7: Monthly daily-average input demand and PV production in the four static REC bases.

Figure 5.7 shows the seasonal setting for these decisions. In Premium-100, non-shiftable input demand changes from 583.81 to 560.19 MWh between 2023 and 2024, and PV production changes from 476.60 to 469.16 MWh. Within a year, the balance also changes with the season. A controller consequently encounters different opportunities for solar use even when the installed equipment is unchanged.

The daily profiles in Figure 5.8 show the timing more directly for Core-30. The winter demand peak occurs in the evening, after PV production has fallen, while the summer mean profile has a pronounced daytime solar surplus over non-shiftable demand. Residential vehicle presence is greatest overnight; service vehicles provide charging opportunities during the day. The controller must fit charging into these different windows while meeting departure requirements. The current connectors permit charging only, so stationary batteries provide the stored-energy flexibility between periods of production and later demand.

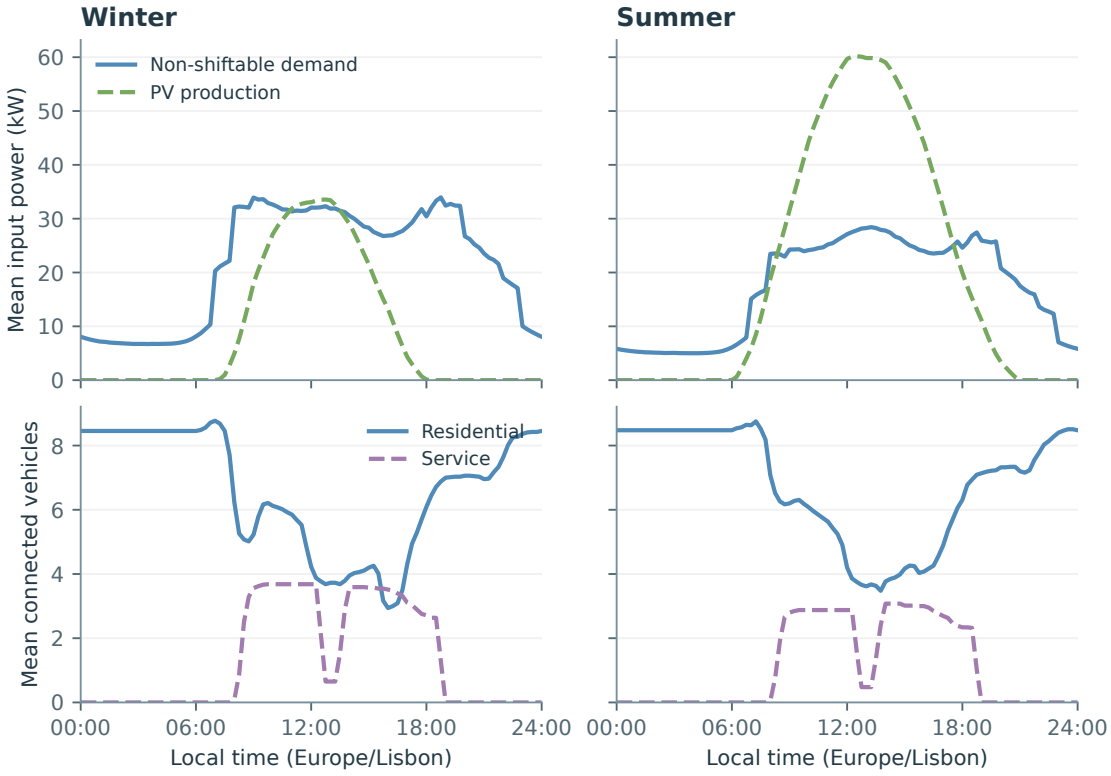


Figure 5.8: Core-30 daily input and vehicle-presence profiles: Monday–Friday means in winter (December–February) and summer (June–August) of 2024, in Lisbon local time.

5.3.3 Scenario Variants and the Questions They Address

The configurations distinguish the REC being studied from the condition applied to it. The Normal cases retain the complete membership and equipment catalogue without the additional event campaigns. They establish how a controller operates with the resources, services, and electrical limits of that base. The remaining configurations make selected difficulties explicit, as summarised in Table 5.2.

Safety changes the headroom against which actions are evaluated, making service delivery and reliance on limiting central to the comparison. Robustness retains a reproducible failure schedule, allowing nominal and disturbed trajectories to be compared through the affected services and recovery period. Topology changes which members and assets are active. These configurations give the corresponding RQs concrete experimental contrasts. Combined and All-in then examine their interaction within an integrated workload; paired diagnostic replays can distinguish individual effects when needed.

The topology histories describe permanent participation changes. Core-30 Topology and Combined begin with 26 active members and finish with 27; Premium All-in begins with 80 and finishes with 88. The full catalogues contain the resources available to those histories. Events also install,

Table 5.2: Purpose of the scenario variants and their reference comparisons.

Configuration	**Change relative to its Normal base**	**Question supported**
Core-30 Topology	Permanent member and equipment additions, removals, and replacements.	Can control continue as the active resource collection changes?
Core-30 Safety	Tighter selected building and phase limits.	Can the controller deliver services under reduced headroom, and how much action correction is needed?
Core-30 Robustness	Data and equipment events, observation noise, and grid interruptions.	How do disturbances affect decisions, service, and recovery?
Core-30 Combined	Membership changes, electrical stress, health events, and interruptions.	What happens when these conditions occur in the same REC history?
Premium All-in	Participation changes, failures, interruptions, seasonal noise, and demand-response requests.	How does control perform when a larger, heterogeneous REC faces several operating demands?

replace, or retire equipment. A member that leaves exits subsequent REC demand and service accounting, and the same departure is not replayed in 2024.

Failure events and observation noise remain separate scenario settings. The former specify intervals of missing data, forecast bias, delayed or lost commands, and equipment unavailability. Core-30 Robustness additionally includes its continuous measurement-noise setting, while Premium All-in uses seasonal noise campaigns. Backup during grid interruptions follows the aggregate convention supported by the selected runtime. The timing and intensity of these conditions remain explicit, allowing service loss and recovery to be evaluated against matched nominal operation.

DR appears only in Premium All-in. Each year includes two one-hour reduction requests of 80 kW and one one-hour absorption request of 40 kW, together with a separately identified four-hour reduction request of 1500 kW. The last is a deliberately excessive request used to examine behaviour under unattainable service demand. Its outcome belongs to a separate stress category from the operational candidate requests.

Development and reserved confirmation files select different event schedules within these configurations. A paired comparison retains the physical inputs and declares the event or operating condition changed. The two-year chronology also permits temporal separation, with training, selection, and confirmation windows specified by each controller study. Initialisation preserves the REC history at the selected start time, so a member that departed earlier remains absent.

5.3.4 Data Characterisation and Quality

The dataset analysis follows the same distinctions used in construction: installed and active resources, input energy and controlled energy, scheduled visits and delivered charging, and requested services and completed services. Appendix B.3 develops that analysis through distributions, temporal profiles, and event summaries. It also records the checks supporting the current edition.

The input checks cover the time axis, calendar, inventories, resource identities, connection schedules, appliance windows, forecasts, and event histories. Individual charging requests are also examined against their connection duration, accepted power, losses, and battery taper. The resulting audit accompanies the descriptive analysis in the appendix. Concurrent demand, electrical limits, disturbances, and the controller's decisions then determine which services are delivered during simulation.

The archived examples in Section 5.2 illustrate the implemented mechanisms. The Portugal collection supplies the inputs for the controller comparisons that follow, with its descriptive analysis and audit retained together in the appendix.

5.4 Performance Indicators and Evaluation Protocol

The value of a control decision depends on the service it delivers and the resources it uses. Returning to the example vehicle, a lower electricity bill is useful when the vehicle is ready at 08:00 and the charging schedule respects the installation limits. A REC assessment must also show whether the gain is shared among its members and how the controller behaves when conditions change. The evaluation therefore reads energy and economic outcomes together with service, safety, robustness, and computational effort.

The simulator extends the inherited energy indicators with participant services, local settlement, electrical constraints, and operating events. The definitions below establish the common scorecard and comparison rules used in the following chapters. Scenario choice determines the members, resources, prices, and active services; Chapters 6 and 7 specify the learning methods, training budgets, data windows, seeds, and ablations. Each result is thus read against the same definitions while retaining the conditions of its experiment.

5.4.1 Energy, Cost, and Community Outcomes

Grid import and export record how much energy crosses the selected building or REC boundary. Their difference is net exchange, but the two directions remain available separately: identical net totals can arise from very different patterns of consumption and surplus. Local sharing records which part of member demand is met within the REC and how the associated payments are allocated. The accounting boundary and settlement settings are consequently part of every comparison.

For a matched reference controller, cost saving is

$$\Delta C = C_{\text{ref}} - C_{\pi}. \tag{5.8}$$

Here, C_{π} is the cost of the evaluated policy π, and C_{ref} is reference cost over the same members, intervals, prices, and settlement rules. The saving ΔC is expressed in currency and is positive when the evaluated policy reduces cost. Demand-response payments and penalties remain identifiable within the reported cost after settlement. Absolute savings also remain interpretable when the reference cost is zero or negative.

Participant outcomes extend this reading beyond the REC sum. The savings count includes members active in the scored period with finite costs in both runs; a member saves when its cost difference is positive. The scorecard gives this count, its percentage of eligible members, and the distribution of absolute savings. Percentage cost reductions additionally require a positive reference cost. In a changing REC, each member is compared over its active period. Figure 5.4 shows why this matters: the same sharing mechanism produces different benefits for members with different generation and demand.

Solar self-consumption relates generation to the amount retained within the stated accounting boundary:

$$S_{\text{PV}} = 100\frac{G - X_{\text{PV}}}{G}, \qquad G > 0. \tag{5.9}$$

Here, G is total PV generation and X_{PV} is the energy attributed to PV export at that boundary, both in kWh. The indicator S_{PV} is a percentage; it is undefined when there is no generation. Building and REC values refer to different boundaries, since energy exported by one member may be consumed by another. The source totals accompany the ratio so that the level of aggregation remains clear.

Battery throughput sums the charging and discharging energy of stationary batteries at the declared metering boundary, in kWh. Vehicle-to-Grid (V2G) export separately totals the energy supplied by vehicle batteries, also in kWh. Their interpretation depends on the cost, service, and grid outcomes achieved, together with any degradation model included in the experiment. A larger throughput can accompany useful solar shifting or unnecessary cycling; its magnitude explains resource use without supplying an independent ranking.

Operational emissions accumulate imported electricity multiplied by the corresponding interval carbon intensity, giving kg CO_2. The report identifies whether imports are measured at individual member meters or after REC balancing, and uses the same boundary for all methods. Any export credit follows an explicitly declared accounting rule. These quantities allow two equally inexpensive schedules to be distinguished by renewable utilisation, storage use, or emissions.

The grid-shape indicators use REC net power, positive for import. The all-time peak is its maximum over the scored period. Daily peak is the mean of the maxima in successive 24-hour windows from the episode start; at 15-minute resolution, each complete window contains 96 intervals. A final partial window is included when present. The daily load-factor penalty averages one minus the ratio of mean to maximum power in those windows. Raw peaks are in kW, while

the load-factor penalty is dimensionless. The scorecard reports both peaks in kW; the load-factor penalty remains available in the wider catalogue.

Ramping uses the sum of positive changes in REC net power between consecutive intervals, including intervals of net export. It therefore measures upward changes, such as the rise when evening charging begins as solar production falls. The scorecard reports this accumulated upward change in kW over the complete evaluation window. It is a sum of power increments, rather than a rate in kW per hour. These conventions follow the simulator's power-based comparison outputs. Legacy indicators calculated from interval energy retain their separate units; comparisons use the same resolution and convention throughout.

5.4.2 Service Delivery, Safety, and Robustness

Vehicle service is evaluated at departure with a tolerance of five percentage points of SoC. Minimum acceptable service is reached at or above the requested target minus this tolerance. Target accuracy requires the final state to lie within five percentage points on either side of the target. For an 80% request, minimum service begins at 75%, while the accuracy band is 75–85%. The corresponding rates are

$$S_{\min} = 100\frac{N_{\min}^{\text{met}}}{N_{\min}^{\text{eligible}}}, \qquad S_{\text{tol}} = 100\frac{N_{\text{tol}}^{\text{met}}}{N_{\text{tol}}^{\text{eligible}}}. \tag{5.10}$$

Here, $S_{\min}$ and S_{tol} are the minimum-service and within-tolerance percentages. For each criterion, N^{eligible} counts eligible departures and N^{met} counts those meeting the criterion. A rate is undefined when its eligible count is zero. The counts and the number of excluded departures accompany both percentages.

An eligible departure has an observed session end, a valid target and achieved SoC, and a threshold reachable under the simulator's maximum-charge feasibility screen. This screen starts from the session boundary state and follows the available connection intervals, charger and battery power limits, efficiencies, and battery taper. It applies the configured electrical headroom and outage handling. Eligibility is calculated for each threshold, independently of whether the controller meets it. The common five-point tolerances give both criteria the same lower threshold; their success counts differ when charging exceeds the upper accuracy limit. Individually unattainable requests remain visible alongside the eligible results.

The service record gives failed departures as a count over eligible departures and reports both the total and largest individual energy deficit. For each eligible departure, the deficit is the positive difference between the battery energy required at the minimum-service threshold and the energy present at departure, in kWh. This distinguishes many small shortfalls from a few severely underserved vehicles. Energy missing from the original requested target remains a separate diagnostic, since the minimum-service criterion includes the stated tolerance. Unattainable requests and exclusions caused by missing or incomplete records retain separate counts. A paired disturbance comparison also identifies departures that lose eligibility and reports service on the common eligible cohort, so that an outage cannot improve the apparent rate simply by removing difficult requests.

Appliance service is read through completed and missed cycles, completion deadlines, start delay, delivered energy, and unserved energy. A shifted washing programme can then be distinguished from a programme that never ran. Demand-response records likewise retain requested and credited response, shortfall, compliance, and settlement for each activation. These service quantities remain visible beside energy savings, including in Combined and All-in scenarios where several obligations compete for the same flexibility.

Electrical safety is measured on the applied trajectory, while requested pressure describes the actions proposed before limiting. Accumulated exceedance can be written as

$$V = \frac{\Delta t}{3600} \sum_{t \in \mathscr{T}} \sum_{\ell \in \mathscr{L}_t} \left([P_{\ell,t} - I_\ell^{\max}]_+ + [-P_{\ell,t} - X_\ell^{\max}]_+ \right). \tag{5.11}$$

Here, $\mathscr{T}$ contains the scored timesteps and $\mathscr{L}_t$ the configured total-building and phase boundaries active at timestep t. Applied power $P_{\ell,t}$ is positive for import; $I_\ell^{\max}$ and $X_\ell^{\max}$ are positive import and export limits, all in kW. The notation $[z]_+ = \max(z, 0)$ retains positive excess, and Δt is the timestep in seconds. Thus, V measures accumulated constraint exceedance in kWh, counting each violated total or phase limit. The residual calculation uses a numerical tolerance of 10^{-5} kW.

The frequency of actual action corrections accompanies this applied safety result:

$$A_{\text{corr}} = 100 \frac{N_{\text{corr}}}{N_{\text{decision}}}. \tag{5.12}$$

Here, N_{decision} counts eligible building-level decision opportunities with an active controllable resource and a supplied proposal. The numerator N_{corr} counts those opportunities in which at least one command is actually changed by the declared feasibility or supervisory stages. The rate A_{corr} is a percentage, with a stated numerical tolerance for detecting changes. Each building decision is counted once even if several commands or stages change it.

A call to a validator with an unchanged output adds no correction. Missing, invalid, or late decisions are counted once per building decision opportunity when a usable action is unavailable by the declared control deadline. Their denominator includes all opportunities requiring a decision, including those with no proposal. This differs from the correction denominator, which requires a supplied proposal. The report names the stages included and preserves proposed, submitted, and applied actions, following Section 5.2.1.

Fallback records describe the supervisor's response. An episode is a consecutive period of fallback within one building; separate buildings contribute separate episodes. Total duration is summed in building-minutes, with overlapping periods therefore retained for each affected building. Per-resource corrections and their magnitude remain available for diagnosis. A corrected command or a fallback decision can preserve the requested service, so intervention and missed service keep separate counts. Injected fault episodes and their duration describe exposure to the disturbance and are recorded independently of these outcomes.

Robustness uses the same outcomes in paired nominal and disturbed runs. The comparison

fixes the underlying scenario, controller, and random seeds and changes the selected disturbance. It reports energy and cost differences, missed or delayed services, and action correction during the event and afterwards. Execution recovery is measured from clearance of the declared external fault to the first usable controller decision applied within its deadline. Median and maximum recovery time accompany the number of recovered and unrecovered episodes. Episodes still unresolved at the observation boundary retain their elapsed duration as a lower bound. Service recovery is reported separately where an obligation can still be fulfilled; a missed departure remains a missed service even after timely execution resumes.

5.4.3 Scorecard, References, and Aggregation

The simulator provides a much wider set of performance indicators than can be usefully displayed in every comparison. Member savings use matched Business as Usual (BAU) operation as a reference, with its storage variants defined in Chapter 6. The common scorecard in Table 5.3 selects fifteen indicators that keep the results aligned across methods, datasets, and chapters. It gives an economic outcome its energy, service, and operational context. The wider catalogue in Appendix B.6 remains available for questions about appliance service, local settlement, DR, topology, failures, comfort, degradation, and multi-REC operation.

Table 5.3: Common scorecard. Physical and monetary quantities retain their absolute units; percentages describe service, utilisation, or intervention directly.

Category	Indicators and units	Reading
Economy	REC cost [EUR].	Lower, with service retained.
Emissions	Operational emissions [kg CO_2].	Lower under the same accounting rule.
Energy	External REC import [kWh]; external REC export [kWh]; solar self-consumption [%].	Read the two exchange directions with local solar use.
Grid shape	Mean daily peak [kW]; maximum peak [kW]; accumulated upward ramping [kW].	Lower under matched service and timestep.
EV service	Minimum service [%]; target accuracy [%, $\pm$5 SoC percentage points].	Retain the eligible and excluded departure counts.
Safety and supervision	Electrical violations [kWh]; actual action corrections [% of eligible building decisions].	Zero residual violation; interpret correction with delivered service.
Resource use	Stationary-battery throughput [kWh]; V2G export [kWh].	Explain how flexibility was used.
Member benefit	Members saving against matched BAU [n/N].	Show the distribution of REC benefit.

Results use the same names and units throughout the thesis. Cost is the REC total under the declared tariff and settlement convention; savings in EUR may accompany it. Grid import and export are measured at the REC boundary, while the sum of member-meter imports is retained separately where useful. Service and self-consumption percentages keep their direct meaning: they express fulfilled departures or retained generation, rather than normalisation against another controller. The

principal tables report cost, energy, emissions, and power in absolute units. Reference differences support interpretation without replacing those quantities.

The common operating reference uses ordinary vehicle charging, execution of mandatory appliance requests, and simple PV self-consumption storage. A no-storage variant isolates the contribution of enabling that resource. Progressively informed RBCs use prices, generation, peaks, and REC information. Chapter 6 develops these references alongside optimisation and learning. Member benefit uses the matched storage-enabled BAU unless another reference is explicitly named; an incremental hierarchy comparison can additionally report savings against the same local controller without coordination.

The scorecard can be presented in two blocks with the same method order: economy, emissions, energy, and grid shape first; service, safety, resource use, and member benefit second. A focused comparison can select the methods and columns relevant to its question. A charging study, for example, may place cost beside the two EV-service rates and intervention, while a coordination study also needs the REC power profile and member outcomes. Safety and disturbance studies add a compact service-and-event table: failed obligations, their energy deficit, unavailable decisions, fallback, and recovery. Each table retains its dataset, complete evaluation window, and indicator definitions, allowing the comparisons to accumulate as the methods are introduced.

Additional indicators follow the services present in the experiment. A dataset containing deferrable appliances requires completion and missed-deadline outcomes beside the energy comparison; demand-response experiments retain delivered response, shortfall, and settlement. A disabled service is identified as such. Unavailable measurements remain marked as unavailable, rather than filled with a favourable zero. Export and battery throughput are diagnostic quantities, and a high value alone does not establish better control.

These observations connect directly to the four research dimensions established in Chapter 1. *Scalability* follows control quality as the number of members and resources grows, together with training time [h], peak memory [GB], and inference latency [ms per building decision]. It also records the preparation or retraining needed after a configuration change. *Deployability* concerns the work needed to place the controller in service: configuration and integration effort [person-hours], elapsed deployment time, runtime dependencies, and the successful reproduction of the evaluated decisions through the deployed inference path. Those measurements accompany the scorecard in the corresponding studies.

Safety is read through service fulfilment, physical violations, and the interventions supporting the applied decisions. *Robustness* uses the same outcomes in paired nominal and disturbed conditions, supplemented by fallback and service-specific recovery times. The common scorecard thus supplies a stable account of useful operation, while each investigation adds the observations needed to examine its RQ. Keeping these meanings stable allows the architecture, simulation, and learning studies to contribute to the same argument.

Aggregation follows the measured quantity. Cost sums the member bills under the selected settlement rule; internal payments cancel in the REC total when transfers between members are

represented. Energy and emissions accumulate over the scored intervals at their declared boundary. Solar self-consumption is calculated from total generation and PV-attributed export, rather than the unweighted mean of building percentages. Maximum peak is taken from the aggregate REC trajectory, mean daily peak averages its daily maxima, and ramping accumulates its successive positive power changes. None of these power quantities is obtained by summing independently computed building peaks.

Service rates pool the corresponding met and eligible event counts, and correction rates pool corrected and eligible building decisions. Their denominators and exclusions remain visible. Member benefit counts matched active members with finite bills and retains the range of absolute savings in the detailed results. Each dataset is reported separately. Where repeated training runs are available, each run first yields a scorecard for the full test window; the report then gives the mean and dispersion across independent training seeds, retaining paired simulator conditions for each comparison.

5.4.4 Temporal Protocol and Reproducible Comparisons

The two-year Portugal collection separates controller development from operation on a later annual sequence. The studies in Chapter 6 use 2023 for training and validation and reserve the complete available 2024 sequence for final evaluation. A full year exposes a controller to changing demand, solar production, prices, and service requirements across the seasons. Short event plots explain particular decisions within that annual account.

Training allows the agent to interact with the simulator and improve its policy. Validation evaluates selected policies on reserved periods to choose hyperparameters, reward settings, and checkpoints. Four fortnightly blocks—the first fourteen days of February, May, August, and November 2023—provide validation conditions in different seasons. The remaining 2023 intervals supply training episodes. Table 5.4 identifies the role of each partition.

Table 5.4: Temporal separation for the principal controller comparisons.

Stage	Data window	Use
Training	2023, excluding the four reserved validation blocks.	Generate interaction experience, update the policy, and fit preprocessing statistics.
Validation	Days 1–14 of February, May, August, and November 2023.	Compare configurations and select checkpoints with learning disabled.
Final test	Complete available 2024 sequence.	Evaluate the selected policy with its parameters, preprocessing, and action-selection rule fixed.

The split follows the dataset's UTC clock. Training episodes retain contiguous time intervals; replay records and learning targets stop at the partition boundaries. Validation observations and rewards do not enter policy updates or the fitting of preprocessing statistics. Initialisation reconstructs the active resources and relevant preceding history at each episode's start, with any warm-up

kept outside the score. The continuous test year then carries stored energy and unfinished services forward without daily or monthly resets.

BAU and fixed rule-based references use the same annual test window as the learned controllers. BAU can also be evaluated over the full development year to characterise conventional operation, with 2023 and 2024 reported separately. The primary comparison pairs each controller with its reference within the same scenario and year. The seven Core-30 and Premium-100 scenarios provide this principal account; Core-4 and Core-15 support development and diagnostic inspection. Comparisons across REC sizes additionally retain their different resource compositions.

Candidate and reference share the dataset edition, active members and resources, physical inputs, service requests, prices, settlement, electrical limits, events, timestep, and scored interval list. Initial stored energy, preceding REC history, and the handling of services crossing either boundary are also matched. Changes of membership follow the same event schedule for all methods. The record gives the first and last scored action timestamps and the executed interval count; warm-up observations and an unexecuted terminal observation contribute no artificial energy or cost. Near-boundary forecast handling follows the same convention across the comparison.

The selected checkpoint and preprocessing statistics are frozen for evaluation. A study of online adaptation declares its update rule separately and compares it under the same test conditions. Repeated training seeds reveal variation in the learned solution, while paired simulator seeds retain common disturbances for candidate and reference. Episode segmentation preserves resource state and unfinished services across the full test sequence unless resets are themselves part of the declared experiment. This prevents a series of freshly initialised short episodes from silently replacing continuous operation.

Information access and protective settings accompany each result. Local and REC-aware methods retain their declared observation boundaries, and forecasts use the same availability convention. A perfect-information schedule is reported separately as an opportunity reference. An optimisation method evaluated through successive horizons still covers the complete test sequence, carrying state and unfinished obligations between horizons. A short mathematical solve remains a diagnostic with its own scope; simulator replay supplies the operating outcomes needed for the common scorecard.

Disturbance studies retain complete-window outcomes as well as the intervals around each event. They fix the underlying scenario, controller, and paired random conditions, then change the declared disturbance. Additional energy, missed service, intervention, and recovery can consequently be attributed to that change. Demand-response assessment likewise retains the reference estimator, credited response, penalties, and cost after settlement. A no-activation replay with the same policy measures additional response, while subsequent intervals reveal rebound and differences in stored energy. Operational candidate requests and deliberately excessive stress requests retain separate categories.

Computational comparisons identify the hardware, software, population, resource counts, observation/action sizes, interaction budget, and update budget. Training time and memory describe

development; inference measurements report the declared timing boundary, batch size, warm-up, and latency distribution. The 95th-percentile latency describes the slower part of that distribution, while late decisions are counted against the actual control deadline over the complete test window. The timing boundary states whether observation preparation, inference, validation, and dispatch are included. A change of resources additionally records model reuse, rebuilding, retuning or retraining, and the associated interruption and service outcomes.

Deployment records separate person-hours of configuration and integration from elapsed time between the start of installation and the first valid applied decision. They retain the supporting services, resource mapping, preprocessing, and controller composition placed in operation. Reproduction of the evaluated behaviour is checked by replaying the same recorded inputs through the packaged controller and counting matching requested actions within a declared tolerance. Timing compliance and successful dispatch are then assessed on the complete execution path. These measurements accompany the operating scorecard and explain how readily a useful controller can be placed in service.

The experiment record preserves the data, software, controller version, evaluation conditions, and resulting trajectories through the lifecycle described in Chapter 4.

5.5 Chapter Summary

The simulator development provides the experimental setting needed to investigate REC control under the conditions established in the opening chapters. CityLearn v3 connects resource models and participant services to electrical limits, changing membership, local exchange, external requests, and failures. The implemented examples show how these conditions enter the simulation and remain visible in observations, actions, and outcomes. Their incorporation into the official CityLearn distribution makes the capabilities available as a reusable research contribution.

The Portugal datasets organise those capabilities into an experimental progression, from inspectable residential decisions to larger RECs with mixed activities and interacting events. Their documented inputs and scenario purposes support matched comparisons, while the performance indicators connect energy and economic outcomes to services, constraints, participant benefits, and computational effort.

This foundation makes scalability, safety, and robustness accessible to controlled experiments through changes in resources, electrical headroom, and operating conditions. It supports deployability through the consistent resource and action descriptions carried into the architecture of Chapter 4. Chapter 6 now uses the simulator, datasets, and common evaluation rules to develop and compare the control methods.

Chapter 6

Development and Evaluation of Community Energy Controllers

The simulator and datasets developed in Chapter 5 allow control methods to be examined under the same operating conditions. The investigation begins with conventional operation and explicit scheduling, then considers local learning, multi-agent coordination, and control with changing resources. Each study examines a particular opportunity: improving a schedule, learning with less interaction, coordinating local decisions, or accommodating another resource. Together, these studies ask how the available flexibility can be used effectively and what is needed to sustain that control as the REC changes.

Working draft with invented results. The quantitative comparisons marked *illustrative* contain deliberately invented values for developing the chapter's argument and presentation. They are not experimental findings. MATD3's leading overall position in those comparisons is an author-selected assumption, informed by the author's account of earlier work. The separately identified Core-30 short-window BAU diagnostic is real. Illustrative values, rankings, and associated interpretations must be replaced by measured results before this chapter is used as scientific evidence.

6.1 Business-as-Usual References

Energy management begins with an installation that already serves its occupants. Household demand follows daily routines, a connected vehicle begins to charge, and an appliance runs when its user requests it. Where PV generation and a stationary battery are available, surplus solar energy can be stored during the day and used when consumption rises later. These decisions may be made independently by the equipment's existing settings, without a controller anticipating the needs of the whole building or REC.

BAU represents this conventional operation in the simulator. Its connection to the real-world problem lies in the equipment, operating practices, and services that precede the proposed control. The Portugal collection supplies the corresponding consumption, generation, tariffs, and user requirements through the data preparation described in Chapter 5. The resulting reference is a

simulation of a specified operating practice, against which the additional value of management can be assessed.

The vehicle introduced in Chapter 1 provides a useful example. It arrives in the evening and starts charging immediately, alongside the household's other demand. A stationary battery may already contain energy stored from the day's solar production and supply part of that demand. An improved controller inherits both the vehicle's departure obligation and the useful operation already provided by the battery. Its opportunity is to organise those resources more effectively over time.

6.1.1 BAU with and without Storage

Two references separate the contribution of stationary storage from the contribution of more deliberate scheduling. BAU without storage leaves the stationary battery inactive. It represents the same consumption and service needs being met without that resource participating in the energy balance. BAU with storage enables ordinary PV self-consumption: surplus production charges the battery, and sufficiently high local import triggers discharge within the configured power and state-of-charge limits. EV charging and appliance operation retain the same rules in both variants.

Table 6.1: Conventional operation represented by the two BAU variants.

Resource or service	**BAU without storage**	**BAU with storage**
Stationary battery	Inactive.	Stores local PV surplus and supplies local demand under the declared settings.
Connected EV	Immediate charging towards full charge, within the connection and equipment limits.	Same charging rule.
Deferrable appliance	Starts at its earliest feasible opportunity within the service window.	Same start rule.
Information used	Current local conditions and resource availability.	The same information, including the local energy balance for storage.

The paired comparison asks how much changes when the battery is used. Comparing the subsequent controllers with BAU with storage asks a further question: how much can better decisions add to an installation that already shifts solar energy into later consumption? This second reference is used throughout the controller studies. Savings can then be related to anticipation, allocation of available power, or coordination, with ordinary battery use already included in the starting point.

The storage rule responds to the local energy balance. It does not consult the electricity price. An evening discharge may coincide with an expensive interval because household demand is high then; deliberately retaining energy for that interval requires the price-aware reasoning introduced in Section 6.2.

Immediate EV charging also gives the reference a distinctive service outcome. The vehicle charges towards full charge even when its declared departure target is lower. It can therefore meet its minimum requirement while finishing outside the target band. The scorecard retains both minimum service and target accuracy so that a controller's savings can be read alongside the charge actually delivered to the user.

6.1.2 Annual Operation across Community Scenarios

The annual comparison places both BAU variants in each of the seven Core-30 and Premium-100 scenarios introduced in Chapter 5. The same members, input series, service requests, electrical limits, and event history are used within each pair; stationary-battery operation is the difference being examined. These seven scenarios also define the comparison scope for every controller introduced below.

An annual window follows the changing balance between household demand, solar production, and charging activity through all seasons. Each BAU variant is evaluated separately over the complete available 2023 and 2024 sequences. The 2024 reference is used for the held-out controller comparison; the 2023 account provides the corresponding development-year context. Tables 6.2 and 6.3 show the intended 2024 comparison, while Appendix B.12 retains the 2023 tables and the real Core-30 short-window diagnostic. The invented annual values are independent of that diagnostic execution.

Illustrative reading. Within Core-30 Normal, enabling storage changes the example purchase cost from EUR 27 000 to EUR 24 000 and raises solar self-consumption from 30.0% to 65.0%. The stationary battery supplies a useful part of conventional operation before any learning is introduced. The EUR 24 000 storage-enabled reference is carried unchanged into the later controller tables, giving each successive comparison the same starting point.

The other scenarios examine what happens to that operation under different demands. Safety restricts electrical headroom; Robustness introduces failures and interruptions; Topology changes the active members and equipment. Combined and Premium All-in place several of these conditions in the same REC history. A fall in annual cost can accompany a smaller active population or an unmet service, so the paired storage comparison remains within each scenario and year. The larger bill of a larger REC has a different meaning from an improvement in its control.

Illustrative reading. The service block makes those consequences visible. In the Core-30 Safety example, the two references miss 400 and 300 of the invented 10 000 eligible departures, respectively. Both have zero residual electrical-violation energy because the example retains protective action handling, yet the charge delivered before departure differs. The counts and rates in these tables are editorial fixtures, not statistics from the session catalogue. Measured eligibility, exclusions, and energy deficits accompany the eventual trajectories under the definitions in Section 5.4.2.

The common scorecard keeps cost, emissions, energy exchange, power, service, and intervention together. Appliance completion records whether the scheduled programme finishes within its service window. Premium All-in additionally reports the requested and delivered DR, shortfall, and

Table 6.2: **Illustrative values; invented.** Energy and economic outcomes for the two BAU variants over the complete 2024 window.

BAU variant	Cost [EUR]	CO_2 [kg]	Import [kWh]	Export [kWh]	Solar [%]	Daily [kW]	Max. [kW]	Ramp [kW]
Core-30 Normal								
Without storage	27 000	35 800	199 000	42 000	30.0	64.0	110.0	193 000
With storage	24 000	32 000	180 000	21 000	65.0	60.0	105.0	180 000
Core-30 Topology								
Without storage	25 400	34 000	190 000	39 000	30.4	61.0	108.0	185 000
With storage	22 800	30 700	172 800	23 500	58.0	57.0	102.0	174 000
Core-30 Safety								
Without storage	26 600	35 300	197 000	42 000	30.0	62.0	102.0	184 000
With storage	23 700	31 600	179 000	21 500	64.2	58.0	98.0	173 000
Core-30 Robustness								
Without storage	27 200	36 000	200 000	42 500	29.2	65.0	113.0	204 000
With storage	24 600	32 800	183 500	24 500	59.2	62.0	110.0	194 000
Core-30 Combined								
Without storage	25 500	33 900	189 000	39 200	30.0	60.0	100.0	186 000
With storage	23 200	31 200	174 000	25 800	53.9	57.0	97.0	178 000
Premium-100 Normal								
Without storage	111 000	146 000	780 000	290 000	38.3	255.0	490.0	750 000
With storage	96 000	128 000	682 000	210 000	55.3	230.0	445.0	690 000
Premium-100 All-in								
Without storage	105 000	140 000	730 000	264 000	37.1	246.0	460.0	766 000
With storage	93 000	125 000	651 000	198 000	52.9	225.0	430.0	720 000

Import/export use the external REC boundary; cost sums member purchase bills. All-in cost precedes the separate illustrative DR settlement in Table 6.31. Daily/Max. denote mean daily/maximum peak; Ramp is accumulated upward power change. Each pair uses the same scenario and year; different populations are not ranked by total cost.

settlement in Table 6.31. The energy-purchase bill and the response credit retain separate entries, so that a lower bill can be distinguished from remuneration for another service.

All subsequent annual comparisons use the complete available 2024 test sequence for the stated scenario, following the temporal protocol in Section 5.4.4. Initial conditions, scored intervals, information availability, and protective settings are matched. The current collection disables internal energy trading, gives external export zero remuneration, and permits charging-only EV operation. REC cost therefore sums member purchases, while import and export in the scorecard use the external REC boundary. BAU denotes the storage-enabled reference unless the variant is stated explicitly.

Each method is compared across Core-30 Normal, Topology, Safety, Robustness, and Combined, and Premium-100 Normal and All-in. The tables accompanying each method develop its results in manageable groups; Section 6.10.1 places all seven annual outcomes side by side. Where a learned controller cannot supply a usable action, the supervisor maintains operation. Its decisions remain in the annual account, with fallback reported separately.

Scalability is examined by placing this operating account beside computational demand and the

Table 6.3: **Illustrative values; invented.** Departure service, safety, and storage use for the two BAU variants over the complete 2024 window.

BAU variant	EV min. [%]	Missed EV [n/N]	EV band [%]	Viol. [kWh]	Corrected [%]	Battery [kWh]
Core-30 Normal						
Without storage	100.0	0/10 000	18.0	0	3.4	0
With storage	100.0	0/10 000	18.0	0	3.0	22 000
Core-30 Topology						
Without storage	99.0	90/9 000	18.0	0	4.0	0
With storage	99.2	72/9 000	18.2	0	3.5	19 000
Core-30 Safety						
Without storage	96.0	400/10 000	17.0	0	14.0	0
With storage	97.0	300/10 000	17.5	0	11.0	21 000
Core-30 Robustness						
Without storage	96.5	350/10 000	17.0	0	10.0	0
With storage	97.2	280/10 000	17.5	0	9.0	19 500
Core-30 Combined						
Without storage	93.0	630/9 000	16.0	0	19.0	0
With storage	94.5	495/9 000	16.5	0	16.0	17 500
Premium-100 Normal						
Without storage	99.0	300/30 000	18.0	0	6.0	0
With storage	99.3	210/30 000	18.5	0	4.5	190 000
Premium-100 All-in						
Without storage	91.0	2 250/25 000	16.0	0	23.0	0
With storage	93.0	1 750/25 000	16.5	0	19.0	150 000

Missed EV gives failed minimum-service departures over an invented common eligible cohort within each pair. These counts are invented cohorts, independent of the dataset audit. V2G is disabled. Battery denotes stationary-battery throughput. Appliance, DR, and runtime event outcomes are illustrated separately in Tables 6.30 and 6.31.

work needed to accommodate another configuration. Deployability adds the preparation and execution records needed to reproduce the controller in an installation. Service and intervention expose safety, while the disturbed scenarios and their event records expose robustness. The comparisons that follow develop these questions around the decisions each method can make.

6.2 Rule-Based Control and Optimisation

Ordinary operation establishes what an installation already does. A reference controller asks how much of the remaining opportunity can be captured by explicit decisions. Rules express practical knowledge about prices, deadlines, and available power. Optimisation considers those decisions together over a horizon. Both provide useful operating strategies, as well as demanding comparisons for learning.

The first EnergAIze study explored decentralised control with MADDPG and rule-guided exploration in an hourly, seventeen-member REC (Fonseca, Ferreira, Cabral, Severino, and Praça 2024). Subsequent development broadened the controller family and made service obligations and

electrical constraints more explicit. The references described here preserve that history while giving the current studies a common basis.

6.2.1 Rule-Based Control and Its Variants

The original RBC prioritised PV availability and the vehicle's departure requirement. Charging became urgent when the energy still required approached the amount that could be delivered before departure; otherwise, the controller could wait for a more favourable opportunity. This supplied an interpretable controller and a source of useful early experience for the first learning experiments. The later family extends the same practical reasoning across stationary batteries, EV charging, and deferrable appliances.

RBC Basic introduces price and deadline awareness. It favours charging in cheaper periods and preserves service as a deadline approaches. Stationary storage responds to cheap and expensive conditions within its bounds. Its attraction is the modest information and tuning needed to make decisions more purposeful than immediate service. Its thresholds also make its behaviour easy to inspect: a change in charging time can be related to a price condition or an approaching obligation.

The Smart family adds local PV production, demand, forecasts, and electrical headroom. Consider a vehicle needing 12 kWh during an eight-hour connection. A low current price alone does not determine whether it should charge immediately. The controller can also consider expected solar production, the power used by other equipment, and the time left after postponing the decision. When only two hours remain, the same energy deficit has a different operational meaning. The Smart rules follow this changing room for manoeuvre and coordinate service decisions with stationary storage.

RBC Smart Local fixes a strict information boundary within that family. It uses the building's measurements and requirements together with permitted public signals, such as the tariff and weather forecast. Another building's private state and raw REC aggregates are excluded. This distinction matters because the generic Smart implementation can use REC information when a corresponding local signal is absent. Smart Local supplies the matched reference and teacher for experiments that claim building-local execution.

RBC Community extends the decision context to REC import, surplus, and operating pressure. A local battery may respond differently when several buildings are drawing power together or when renewable production is available elsewhere. The extension changes the information supporting the decision; it also creates dependencies on the availability and interpretation of the aggregate signals. In the current dataset setting, internal trading is disabled, so REC awareness is assessed through scheduling and aggregate exchange rather than attributed to an unmodelled settlement benefit.

Signal-aware variants allow a higher-level coordinator to influence a rule through an effective price. A neutral signal recovers the unmodified condition; other values can encourage or discourage flexible consumption. These variants retain the underlying resource and service logic. Their role is developed with the hierarchy in Section 6.7, where the signal's authority can be assessed against the same local controller.

Table 6.4: Rule-based family: additional information, purpose, and operational dependence.

Controller	**Decision basis**	**Role in the investigation**
Original RBC	PV availability and departure urgency.	Historical reference and early exploration guidance.
RBC Basic	Price conditions and service deadlines.	Establish the value of simple anticipation.
RBC Smart	Solar, demand, forecasts, and headroom; REC fallback may be enabled.	Common foundation of the richer rule family.
RBC Smart Local	Strict local inputs and permitted public signals.	Strong local comparator and demonstration source.
RBC Community	Local conditions and REC aggregates.	Assess explicit coordination with wider information.
Signal-aware RBC	Underlying rule plus an effective-price signal.	Provide a responsive local controller for the hierarchy.

The variants require progressively more context and more decisions about priorities. Their cost remains largely the evaluation of explicit rules, although the supporting forecasts and REC services have their own computational and communication demands. The most informed variant need not dominate every installation: inaccurate forecasts or unavailable aggregate data can remove its expected advantage. A meaningful comparison therefore identifies the information actually supplied and examines service, cost, and action corrections together.

6.2.2 Optimisation over a Scheduling Horizon

Mixed-Integer Linear Programming (MILP) places the resource decisions inside a common scheduling problem. At each interval, continuous variables represent grid import and export, battery charging and discharging, and stored energy. EV variables exist over their connection sessions. Where required, binary variables prevent incompatible modes or select the start of a complete appliance programme. Once started, the programme contributes its specified power profile to the subsequent intervals.

The formulation connects those decisions through a power balance. For one building and interval, a compact expression is

$$p_{i,t}^{\mathrm{imp}} + p_{i,t}^{\mathrm{PV}} + p_{i,t}^{\mathrm{dis}} = p_{i,t}^{\mathrm{load}} + p_{i,t}^{\mathrm{EV}} + p_{i,t}^{\mathrm{app}} + p_{i,t}^{\mathrm{ch}} + p_{i,t}^{\mathrm{exp}}. \tag{6.1}$$

Here, i identifies the building and t the decision interval. The terms p^{imp} and p^{exp} are grid import and export, p^{PV} is PV production, p^{load} is inflexible demand, and p^{EV} and p^{app} are EV charging and scheduled appliance demand. Stationary-battery charging and discharging are p^{ch} and p^{dis}. All terms are non-negative powers in kW; the current comparison permits charging-only EV operation.

Battery decisions also couple consecutive intervals:

$$E_{i,t+1} = E_{i,t} + \eta_i^{\mathrm{ch}} p_{i,t}^{\mathrm{ch}} \Delta t - \frac{p_{i,t}^{\mathrm{dis}} \Delta t}{\eta_i^{\mathrm{dis}}}. \tag{6.2}$$

Here, $E_{i,t}$ is the stationary battery's stored energy in kWh, η_i^{ch} and η_i^{dis} are its dimensionless charging and discharging efficiencies, and Δt is the interval duration in hours. The charging and discharging powers have the meanings defined above. Energy and power bounds restrict the schedule, while the initial energy and declared terminal condition determine how the horizon begins and ends.

Connection windows, available charging power, departure requirements, and appliance deadlines give the remaining variables their service meaning. Electrical constraints limit the combinations that can operate concurrently, including the represented phase limits. For the running example, the optimiser must find enough charging opportunities before departure while accounting for household demand and the stationary battery. A low-cost interval is useful only to the extent that its available power can deliver the required energy.

The purchase objective is

$$\min \sum_{t \in \mathscr{T}} \sum_{i \in \mathscr{N}} \lambda_t p_{i,t}^{\mathrm{imp}} \Delta t. \tag{6.3}$$

Here, $\mathscr{T}$ is the scheduling horizon, $\mathscr{N}$ is the set of buildings included in the problem, λ_t is the grid-import tariff in EUR/kWh, $p_{i,t}^{\mathrm{imp}}$ is import power in kW, and Δt is the interval duration in hours. The current setting assigns zero export revenue. The implemented service-first formulation first minimises unavoidable EV departure shortfall and then minimises purchase cost subject to that service result.

Two choices determine the question answered by a solve. The first is its decision scope: local or REC-wide, and stationary storage alone or the full set of flexible resources. Fixing the EV and appliance trajectories isolates the remaining storage opportunity; optimising those services jointly gives a broader scheduling problem. The second is its information horizon. A perfect-information solve receives realised future demand, production, prices, and sessions. An operational rolling-horizon controller receives causal forecasts, applies an initial portion of its schedule, and solves again as new information arrives. It must also carry the energy state and unfinished services across successive solves.

The perfect-information formulation inspects the scheduling opportunity available within the stated model. Solver status and the optimality gap describe the mathematical solution; replay through the simulator connects that schedule to the operational indicators in Chapter 5. Forecast quality, solve time, and recovery from an unsuccessful solve become additional concerns when the formulation is used for rolling-horizon control.

An annual controller comparison carries stored energy and unfinished obligations across successive scheduling horizons. The resulting trajectory supplies the same scorecard used for BAU and learning. Each optimisation scope retains its represented resources and information access.

Table 6.5: Interpretation of the optimisation references. Information horizon is declared separately from decision scope.

Scope	**Scheduled decisions**	**Question answered**
Local, fixed service	One building's stationary battery; service trajectories held fixed.	How much can storage scheduling improve this operation?
REC, fixed service	Stationary batteries across buildings; service trajectories held fixed.	What coordination opportunity remains with those services?
Local, full flexibility	Battery, EV sessions, and appliance starts within one building.	What schedule is admissible and preferred locally?
REC, full flexibility	The represented flexible resources and shared constraints.	What joint schedule follows from the REC objective?

Table 6.6 separates the four scopes, allowing the value of coordinating storage to be distinguished from that of rescheduling the services themselves.

Table 6.6: **Illustrative values; invented.** Perfect-information scheduling over the complete Core-30 Normal test year.

Scope	Cost [EUR]	Daily [kW]	EV min. [%]	EV band [%]
local fixed	21 480	53.7	100.0	18.0
REC fixed	21 000	52.5	100.0	18.0
local flexible	19 560	48.9	100.0	98.0
REC flexible	18 720	46.8	100.0	98.0

Fixed-service scopes retain ordinary EV and appliance schedules. Flexible scopes schedule those services. Perfect information supplies future inputs within each scheduling horizon; energy state and obligations continue between horizons.

Illustrative reading. Local scheduling with fixed service costs EUR 21 480 in Core-30 Normal. Allowing EV and appliance schedules to change lowers that cost to EUR 19 560; coordinating the full flexible REC lowers it further to EUR 18 720. The fixed-service variants retain immediate charging and its low target-band accuracy. The flexible variants preserve minimum service while placing charge nearer the requested target. These perfect-information results describe the opportunity available with advance knowledge of the inputs.

Illustrative reading. Table 6.7 starts from an invented annual BAU cost of EUR 24 000. RBC Basic costs EUR 23 040, RBC Smart Local EUR 21 600, and RBC Community EUR 21 120. The richer rules also use more of the available solar production and reduce the mean daily peak in this example. Minimum service and target accuracy keep the economic comparison connected to what users receive. The remaining question is how much additional value learning can obtain from the same flexibility and service obligations.

Table 6.7: **Illustrative values; invented.** Full 2024 test-window illustration, Core-30 Normal. Ordinary operation and rule-based references.

Configuration	Cost [EUR]	CO_2 [kg]	Daily peak [kW]	Solar [%]	EV min. [%]	EV band [%]
Operating reference						
BAU	24 000	32 000	60.0	65.0	100.0	18.0
RBCs						
RBC Basic	23 040	30 800	57.0	66.0	99.2	75.0
RBC Smart Local	21 600	28 400	54.0	71.0	99.8	92.0
RBC Community	21 120	27 700	51.6	72.0	99.8	93.0

BAU uses stationary storage. Solar denotes solar self-consumption; EV band uses the target tolerance defined in Chapter 5.

6.3 Local Reinforcement Learning

Once a strong rule-based reference is available, learning can be examined against decisions that already make physical sense. The household controller must discover when flexibility is valuable, how much charging can be postponed, and when an attractive action now has an undesirable consequence later. The first learning formulation keeps that question within one building. The agent encounters different prices, demand, and charging opportunities during development, then applies the learned policy to the reserved test year. This establishes what local learning can contribute before joint training is introduced.

6.3.1 The Building as a Learning Agent

One agent controls the flexible resources assigned to one building. A battery and two chargers therefore produce several decisions for the same agent; they do not automatically become three independent learners. This organisation lets the building policy account for the headroom shared by its resources and the service requirements that compete for it. Other buildings remain part of the simulated environment, with their own demand and decisions.

An observation describes what the agent can use when making that decision. Current demand and PV production explain the immediate energy balance. Battery state and charging availability describe the resources that can respond. A vehicle's required charge and time to departure explain the service still to be delivered. Prices and causal forecasts supply context for comparing present and future use of flexibility. Table 6.8 groups these inputs by their role in the decision.

The observation encoder turns these quantities into the policy's input. Cyclic time features preserve the proximity of the end and beginning of a day; categorical indicators distinguish connection and operating states; continuous features use the declared scaling. A missing measurement also needs an explicit interpretation. A missing battery state and a battery with zero available energy are different conditions, even if an incautious preprocessing step would map both to zero.

Table 6.8: Local observations and their control purpose. The selected feature profile and normalisation are retained with the experiment.

Information	**Examples**	**Decision supported**
Present operation	Demand, PV, battery charge, current import.	Identify the immediate balance and usable energy.
Service requirements	Departure time, required charge, appliance window.	Determine what can be postponed and what is urgent.
Resource availability	Connection state, feasible power, electrical headroom.	Identify which commands can take effect.
External context	Current tariff, time of day, permitted forecasts.	Compare the value of acting now or later.
Recent execution	Applied action and relevant intervention feedback, when enabled.	Relate a requested decision to its actual consequence.

Figure 6.1 follows the local interaction. The observation is prepared at stage 1, passed to the actor at stage 2, and converted into resource commands at stage 3. The environment applies those commands subject to the resource and electrical models, then returns the next observation and reward. During learning, the resulting experience updates the policy and its value estimates. During evaluation, the selected policy acts with learning disabled.

Training uses the 2023 periods assigned to learning in Section 5.4.4. Demand, production, and user requirements supply the circumstances of each interaction; the agent's own decisions determine the resulting battery states, delivered services, and rewards. The experience used for learning is therefore generated by interaction with the simulator. Reserved seasonal blocks support validation, while the complete 2024 sequence tests the selected policy with its parameters and preprocessing fixed. Training returns describe how learning develops; the annual test scorecard describes the operation obtained after that development.

The fixed-layout actor can be expressed as

$$\widehat{\mathbf{a}}_{i,t} = \pi_{\theta_i}(f_i(o_{i,t})), \qquad \mathbf{a}_{i,t}^{\mathrm{app}} = \mathscr{E}_{i,t}(\widehat{\mathbf{a}}_{i,t}). \tag{6.4}$$

Here, i identifies the building, t the decision interval, $o_{i,t}$ its permitted observation, and f_i the observation encoder. The actor π_{θ_i} has learned parameters θ_i and produces the proposed action vector $\widehat{\mathbf{a}}_{i,t}$. The execution map $\mathscr{E}_{i,t}$ includes the configured local action handling and the environment's physical response, producing the applied vector $\mathbf{a}_{i,t}^{\mathrm{app}}$. For a stochastic actor, the first expression denotes an action drawn from its policy distribution; evaluation fixes the declared action-selection convention.

Each action entry has a resource-specific meaning. Stationary storage can charge or discharge, the current EV setting permits charging, and an appliance command requests a programme start when allowed. A bounded neural output must therefore be mapped to the corresponding range and resource. The stored transition retains the action delivered at the learner–environment boundary,

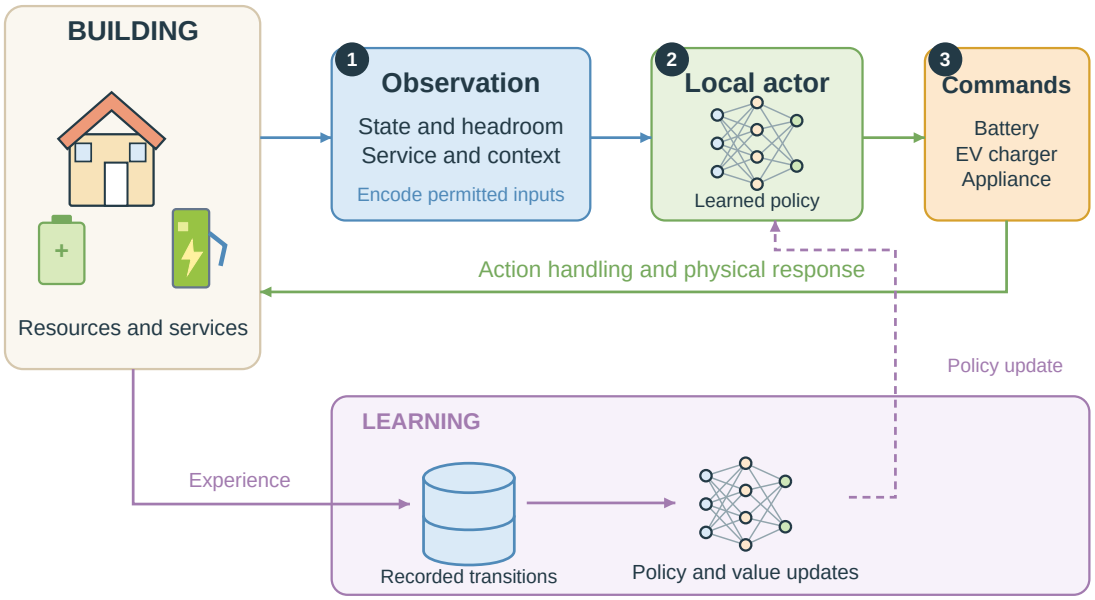


Figure 6.1: The building-level learning loop. Resource commands share one local decision process; learning uses the resulting experience.

while applied-action records reveal subsequent adjustment. Keeping both makes it possible to distinguish a policy's proposal from a feasible outcome supplied by the simulator.

The local reward follows the building's objective, with service and constraint terms defined by the selected profile. One local PPO or TD3 learner is instantiated for each controlled building, with separate value learning and experience. Private observations from other buildings do not enter the actor. This provides a direct operational interpretation: the policy can make its local decision from the same information boundary used during development.

6.3.2 PPO and TD3 as Local Alternatives

Two methods examine different ways of using the interaction. PPO learns a stochastic policy from recent trajectories and limits the size of its policy update (Schulman et al. 2017). Its local value network estimates the future return associated with an observation. For the household example, the collected sequence links an evening charging decision to the eventual departure, allowing a later consequence to inform the earlier action.

TD3 learns a deterministic actor from a replay buffer (Fujimoto, Hoof, et al. 2018). Two critics estimate action value; a conservative target, smoothed target actions, and less frequent actor updates moderate the influence of optimistic estimates. Replay allows the method to revisit a departure event while learning from later days. That reuse is attractive when unusual service events occupy only a small part of an annual trajectory.

The methods differ in their interaction and update patterns as well as their policy distributions. PPO refreshes its learning data as the policy changes. TD3 can make several updates from stored

experience, whose relevance depends on the states, actions, and operating conditions represented. The comparison therefore records environment interactions, update counts, and computational demand separately. Equal interaction counts do not imply equal training effort.

Both local methods retain a fixed observation and action layout in this first study. Their actor networks map the declared input vector through hidden layers to the resource commands; their local critics supply training information and remain outside ordinary inference. The comparison holds the information boundary, service criteria, and evaluation conditions constant. It asks whether adaptation improves on the strong local rule and whether the improvement survives a complete service-aware reading.

Table 6.9: **Illustrative values; invented.** Full 2024 test-window illustration, Core-30 Normal. Local learning against the strict local rule.

Configuration	**Cost** [EUR]	**CO_2** [kg]	**EV min.** [%]	**EV band** [%]	**Corrected** [%]
Local reference					
RBC Smart Local	21 600	28 400	99.8	92.0	4.0
Independent building learners					
Local PPO	22 080	29 200	98.2	81.0	9.5
Local TD3	21 600	28 800	98.8	85.0	8.0

Illustrative reading. Table 6.9 gives local PPO an annual cost of EUR 22 080 and local TD3 EUR 21 600, with minimum-service rates of 98.2% and 98.8%. RBC Smart Local also costs EUR 21 600 but delivers 99.8% minimum service and more departures within the target band. This example gives local learning a demanding reference: a similar bill can coexist with poorer service and more intervention. It motivates closer attention to experience and objectives, while raising another question—whether joint training can improve decisions whose consequences extend across buildings.

6.4 Multi-Agent Reinforcement Learning

Consider two households that both postpone charging until the same low-price interval. Each decision may reduce a local bill, yet their combined demand can create a sharper REC peak. Local adaptation alone gives each learner limited knowledge of the decisions that produced that peak. Multi-agent training makes those interactions part of the learning problem while retaining the building as the unit of operational control.

6.4.1 Local Actors and Centralised Training

CTDE separates the information used to improve a policy from the information needed to apply it. Each actor receives its permitted local observation and issues its building's resource commands. During training, the critics can additionally receive the declared joint observations and actions.

Figure 6.2 shows this distinction: the training process can examine a simultaneous REC decision, while each deployed actor retains a local inference path.

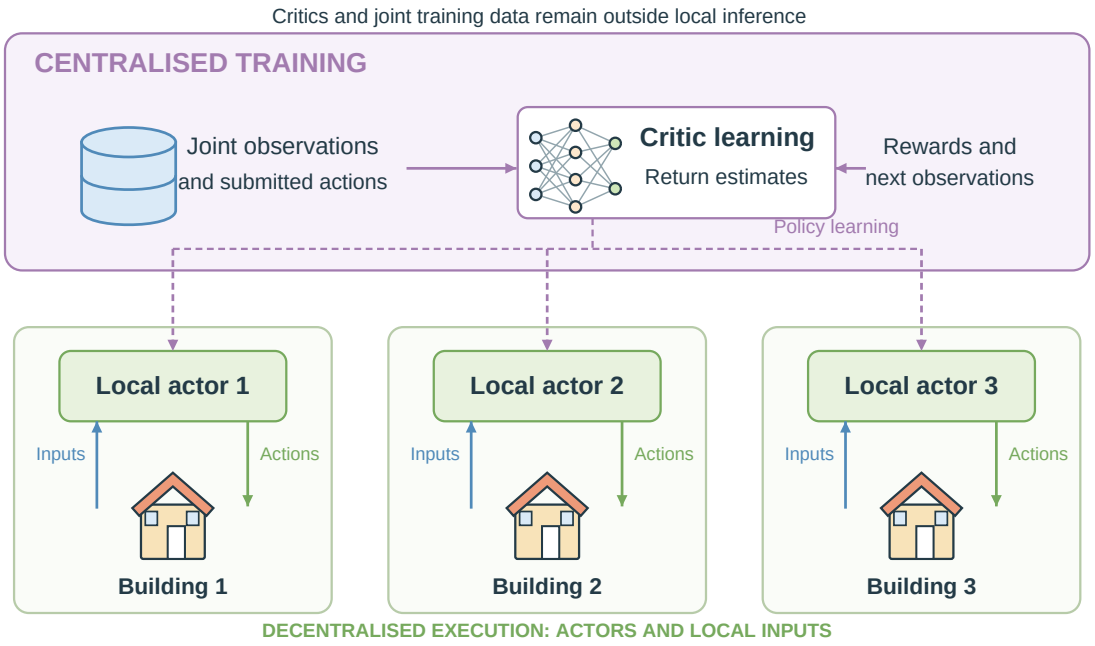


Figure 6.2: Joint information supports critic learning; the trained actors make building-level decisions from local inputs.

The joint context helps a critic distinguish outcomes that look similar from one building alone. A rise in REC import may follow that building's charging action, the simultaneous actions of its neighbours, or both. Including those actions in value estimation makes their influence available during training. The training context is the information explicitly assembled for the experiment; concatenating observations need not reveal every hidden physical state.

Coordinated training also requires an objective that values the intended interaction. If rewards contain only uncoupled local purchases and the environment imposes no shared consequence, a central critic alone does not create an incentive to reduce the REC peak. Local cost, REC exchange, service, and electrical pressure therefore retain explicit roles in the reward configuration. Comparisons between learning rules hold those roles constant, while reward changes are examined separately in Section 6.5.

This arrangement fits the architecture in Chapter 4. The local actor can be exported without the central critic, reducing its runtime dependencies. Observation encoding, action interpretation, retained rules, and local protection still accompany the evaluated controller. Decentralised execution consequently simplifies placement at the installation, while the training process can remain more demanding in memory, data access, and computation.

6.4.2 Multi-Agent Actor–Critic Methods

The learning rule determines how joint experience is used to improve each building's decisions. The methods considered here differ in the experience retained for training, the value estimates guiding their actors, and the way exploration enters the policy. These choices affect both the resulting control and the effort needed to obtain it.

MADDPG. The original EnergAIze formulation used MADDPG (Fonseca, Ferreira, Cabral, Severino, and Praça 2024; Lowe et al. 2017). Each building has a local actor, while its critic evaluates the joint decision during training. The critic can then relate a charging action to what the other buildings were doing. After training, each actor executes from its declared local inputs, fitting the runtime architecture described in Chapter 4.

An optimistic critic may assign excessive value to an action combination, encouraging the actor to repeat it. Reward scale and exploration influence which combinations enter replay, and the joint input grows with the represented REC. These sensitivities motivate examination of the critic estimates alongside the resulting control.

MATD3. MATD3 retains decentralised deterministic actors while introducing twin critics, target smoothing, and delayed actor updates (Ackermann et al. 2019). These changes address the influence of optimistic value estimates on policy learning.

The MATD3 training target uses the lower of the two target-critic estimates:

$$y_{i,t} = r_{i,t} + \gamma(1 - d_t) \min_{j \in \{1,2\}} Q_{i,j}^{\text{target}}\left(s_{t+1}, \widetilde{\mathbf{a}}_{t+1}\right). \tag{6.5}$$

Here, $y_{i,t}$ is the target value for agent i at interval t, $r_{i,t}$ is its reward, γ is the discount factor, and d_t identifies a terminal transition. The joint training state is s_{t+1}, $Q_{i,j}^{\text{target}}$ is target critic j, and $\widetilde{\mathbf{a}}_{t+1}$ contains the target actors' actions with bounded smoothing noise. Taking the lower estimate reduces the influence of optimistic value estimates; delayed actor updates give the critics additional updates between changes to the policy.

MAPPO. MAPPO uses local actors alongside a critic with central training information (Yu, Velu, et al. 2022). The shared training context can account for REC interactions, while PPO's trajectory-based updates offer an alternative to repeated replay. Independent Proximal Policy Optimisation (IPPO) retains local value estimation and supplies the corresponding comparison without that central information. The ensemble of independent PPO learners in Section 6.3 fulfils this role in the strict local setting.

HAPPO. HAPPO examines sequential policy updates (Kuba et al. 2022). Updating one actor changes the decision context of the others, so the method accounts for that progression during optimisation. This is relevant when buildings have different equipment and contribute different

forms of flexibility. It changes the update organisation while retaining the need for local actors, a declared training context, and an objective that reflects the REC task.

Entropy-Regularised Control. The entropy-regularised alternative uses Multi-Agent Soft Actor–Critic (MASAC), extending the actor–critic principle of Haarnoja et al. (2018) to the joint training setting. Its stochastic actors balance expected return with a preference for maintaining action diversity during learning. This provides a different exploration mechanism from adding noise to a deterministic actor. Entropy weights, reward scale, and action bounds consequently form part of its configuration. The underlying entropy-regularised learning principle is introduced in Chapter 2.

Table 6.10 summarises the mechanisms examined by these methods, with independent PPO retained as the local reference.

Table 6.10: Learning alternatives and the mechanism each comparison examines. All actors retain their declared local execution inputs.

Method	**Training organisation**	**Question examined**
Independent PPO	Local value learning from recent trajectories.	What can independent adaptation achieve?
MADDPG	Deterministic actors and joint-action critics with replay.	Does joint value estimation help interacting decisions?
MATD3	Twin critics, target smoothing, and delayed actor updates.	Does moderating optimistic estimates improve control?
MAPPO	Local stochastic actors and central value learning.	How does trajectory-based joint training compare?
HAPPO	Sequential actor updates with joint training information.	Does update order help heterogeneous cooperation?
MASAC	Entropy-regularised actors with joint value learning.	What does a different exploration objective contribute?

6.4.3 Comparative Evaluation

Comparing learning rules requires a common resource interface, reward, and observation access, with interaction and update budgets reported alongside computational demand. Cost, service, and intervention describe the resulting control. Training variability and value diagnostics help explain differences between the methods, while memory and update time reveal the effort required by their training arrangements.

The nominal comparison uses Core-30 Normal and the storage-enabled BAU established in Section 6.1.2. Each trained configuration is evaluated over the same complete 2024 sequence, and repeated training seeds give separate annual outcomes before their mean and dispersion are calculated. The Safety, Robustness, and reconfiguration studies then examine the conditions that change those outcomes.

The comparison below concerns selected complete controller configurations. In the result tables and figures, *MATD3 (residual)* identifies the service-aware composition developed in Section 6.5: the learned actor acts alongside a retained base rule and local feasibility handling. MATD3 continues to name the underlying learning method. This explicit result label keeps the complete controller identifiable when it reappears in the hierarchy and disturbance studies. Appendix B.7 retains the supporting configuration record.

Table 6.11: **Illustrative values; invented.** Full 2024 test-window illustration, Core-30 Normal. Common scorecard, energy and economic outcomes.

Configuration	**Cost** [EUR]	**CO_2** [kg]	**Import** [kWh]	**Export** [kWh]	**Solar** [%]	**Daily** [kW]	**Max.** [kW]	**Ramp** [kW]
References								
BAU	24 000	32 000	180 000	21 000	65.0	60.0	105.0	180 000
RBC Smart Local	21 600	28 400	166 000	17 400	71.0	54.0	96.0	167 400
RBC Community	21 120	27 700	163 000	16 800	72.0	51.6	91.0	158 400
Local learning								
Local PPO	22 080	29 200	169 000	19 200	68.0	56.4	99.0	172 800
Local TD3	21 600	28 800	167 000	18 600	69.0	55.2	97.0	169 200
Multi-agent learning								
MADDPG	20 880	27 800	163 500	17 400	71.0	52.8	94.0	162 000
MATD3 (residual)	19 680	25 200	155 000	14 400	76.0	49.2	86.0	153 000
MAPPO	20 640	26 900	161 000	16 200	73.0	51.6	92.0	160 200
HAPPO	20 400	26 500	159 500	15 600	74.0	51.0	90.0	158 400
MASAC	20 640	27 100	162 000	16 800	72.0	52.2	93.0	162 000

Daily and Max. denote mean daily and maximum REC peak. Ramp is accumulated upward power change. Imports and exports use the external REC boundary; cost sums member bills.

Illustrative reading. Tables 6.11 and 6.12 read the selected configurations through both parts of the scorecard. The residual MATD3 configuration occupies the leading overall position among the learned controllers as an editorial assumption informed by the author's account of earlier development. Its invented annual cost is EUR 19 680, accompanied by 25 200 kg CO_2, a 49.2 kW mean daily peak, and 99.8% minimum service. Target accuracy, throughput, and intervention help explain how that outcome is obtained. The examples assign intermediate economic outcomes to MADDPG and the other multi-agent alternatives; their service differences remain visible alongside cost.

Illustrative reading. The difference between RBC Community's EUR 21 120 and MATD3's EUR 19 680 is EUR 1 440 over the same annual test-window illustration. Figure 6.3 shows that economic margin, while the scorecard also gives MATD3 lower emissions and a smaller daily peak. Their information paths differ: the rule uses aggregates during execution, whereas MATD3 uses joint inputs for critic learning. That distinction explains the different routes through which the two configurations use REC information.

Table 6.12: **Illustrative values; invented.** Full 2024 test-window illustration, Core-30 Normal. Common scorecard, service, safety, and resource use.

Configuration	EV min. [%]	EV band [%]	Viol. [kWh]	Corrected [%]	Battery [kWh]	V2G [kWh]	Saving [n/N]
References							
BAU	100.0	18.0	0	3.0	22 000	0	–
RBC Smart Local	99.8	92.0	0	4.0	25 500	0	26/30
RBC Community	99.8	93.0	0	3.6	26 800	0	27/30
Local learning							
Local PPO	98.2	81.0	0	9.5	28 200	0	23/30
Local TD3	98.8	85.0	0	8.0	29 100	0	25/30
Multi-agent learning							
MADDPG	98.6	84.0	0	14.0	32 000	0	26/30
MATD3 (residual)	99.8	96.0	0	2.3	28 000	0	29/30
MAPPO	99.5	91.0	0	5.5	29 700	0	27/30
HAPPO	99.6	92.5	0	4.8	30 300	0	28/30
MASAC	99.3	90.0	0	6.5	31 000	0	27/30

Battery denotes stationary-battery throughput; Saving counts members with a lower bill than matched BAU. V2G is disabled in this setting. The invented departure cohort is recorded in Appendix B.12; the complete annual fixtures appear in Appendix B.12.4.

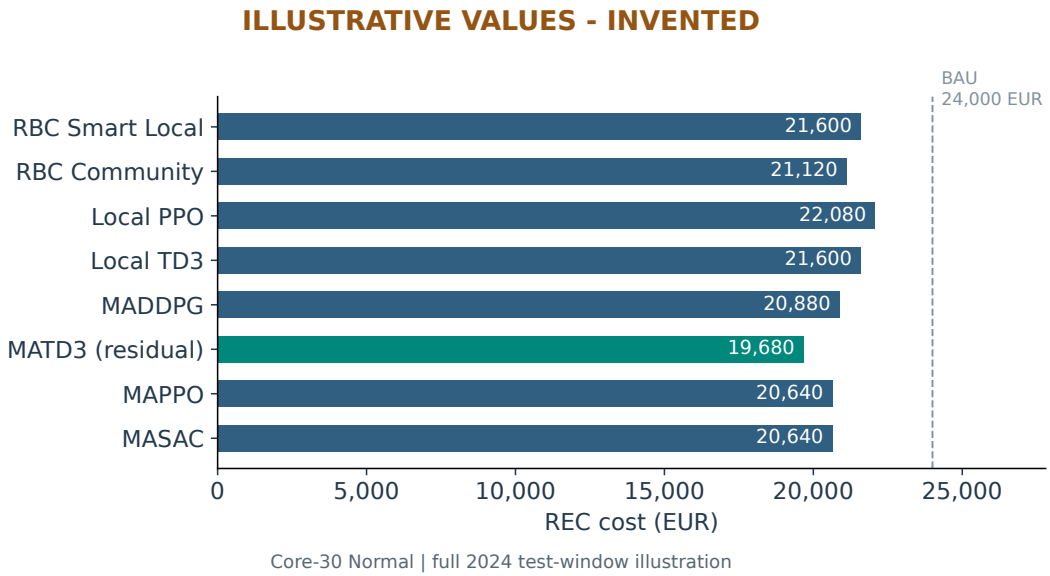


Figure 6.3: **Illustrative values; invented.** Cost comparison for selected controllers in the Core-30 Normal manuscript example.

The selected MATD3 controller combines a richer training process with a relatively simple local inference path. Its remaining assumptions concern the shape of the system for which that policy was prepared. The actor still expects a particular set of input and output positions; the critic still refers to a particular collection of agents. Better learning within that setting makes these assumptions more consequential when considering deployment into an evolving REC.

6.5 Controller Design and Training Strategies

The RBCs already express useful knowledge about charging deadlines, electrical headroom, and ordinary storage behaviour. Asking a learner to rediscover all of that knowledge increases the amount of interaction needed before it produces plausible decisions. The next part of the investigation therefore considers how rules and recorded trajectories can support learning, and how the learning objective can preserve the services that make an economic improvement useful.

6.5.1 Observation and Action Design

Learning quality depends on the problem presented to the network. An observation containing a departure requirement but no time to departure omits the urgency that gives the requirement its immediate meaning. A large vector containing every available signal can instead make useful structure difficult to identify and increases the preprocessing and training burden. The developed observation profiles move from broad compatibility views towards compact operational inputs: remaining service, connection state, feasible power, headroom, and selected causal forecasts.

Action design raises a related question. A single output layer can produce all resource commands, while separate output heads allow different resource groups to develop more specialised mappings from a shared representation. Battery dispatch, continuous EV charging, and an appliance-start request have different consequences even when each is represented by a bounded number. The comparison retains the same external command meanings while examining how the actor organises their prediction.

These choices are especially relevant to deployability. The feature order, scaling, missing-value convention, and mapping from output entries to equipment are needed whenever the model runs. A compact network reduces only part of the execution cost if observation preparation remains expensive or if the runtime cannot reconstruct the required information. The evaluation therefore follows the complete path from available data to requested actions.

6.5.2 Guided Exploration, Demonstrations, and Residual Control

Guided exploration uses a rule-based action during selected early interactions or blends it with the learned action while the policy develops. As the learner gains authority, it encounters a wider range of decisions. For the arriving vehicle, this can keep early experience close to a service-respecting

trajectory, allowing learning to focus on when charging is valuable rather than spending most of its initial interactions on disconnected chargers or missed departures.

Demonstrations use the rule in another way. Observation–action pairs from a teacher train the actor to reproduce a useful starting behaviour. Subsequent interaction then allows the policy to adapt beyond the examples it has seen. Optimisation schedules can also supply demonstrations, with their information assumptions and development-period boundaries retained. An expert with future knowledge supplies training examples whose operational value still has to be assessed with causal observations at evaluation time.

Residual control retains the rule during operation and learns a bounded correction:

$$a_{i,t} = \Pi_{i,t}(b_i(o_{i,t}) + g_i \odot \delta_{\theta_i}(o_{i,t})) . \tag{6.6}$$

Here, $a_{i,t}$ is the requested action of building i, $o_{i,t}$ is its observation, b_i is the base controller, and δ_{θ_i} is the learned correction with parameters θ_i. The vector g_i limits the authority of each correction, $\odot$ denotes element-wise multiplication, and $\Pi_{i,t}$ applies the configured local feasibility bounds at interval t. A zero correction recovers the base controller. The particular action dimensions assigned to the rule and the residual are part of the experiment.

This arrangement is useful when a transparent service policy already works well. The base can protect charging and appliance obligations while the learned component adjusts the use of flexibility. It also changes what must be deployed: the actor, base rule, action scaling, and local feasibility component collectively produce the evaluated behaviour. Packaging that composition preserves the relationship between development and runtime established in Chapter 4.

Table 6.13: **Illustrative values; invented.** Full 2024 test-window illustration, Core-30 Normal. MATD3 configurations using different forms of prior knowledge.

Configuration	Cost [EUR]	EV min. [%]	EV band [%]	Corrected [%]	Battery [kWh]
Unguided	21 840	96.0	73.0	12.4	33 700
Guided exploration	20 880	98.1	85.0	8.2	31 300
Demonstrations	20 400	99.0	90.0	6.1	29 400
Service-aware residual	19 680	99.8	96.0	2.3	28 000

Illustrative reading. Table 6.13 assigns annual costs of EUR 21 840, EUR 20 880, EUR 20 400, and EUR 19 680 to unguided learning, guided exploration, demonstration-assisted learning, and the service-aware residual configuration. Figure 6.4 relates those costs to actual action correction, which falls from 12.4% to 2.3% of eligible building decisions. The comparison holds the interaction budget and complete test window constant.

Illustrative reading. The leading residual MATD3 row in Table 6.11 represents the selected service-aware configuration in this example. Its performance would therefore belong to the complete method, including its retained base and protection, rather than to the neural actor alone. A measured ablation

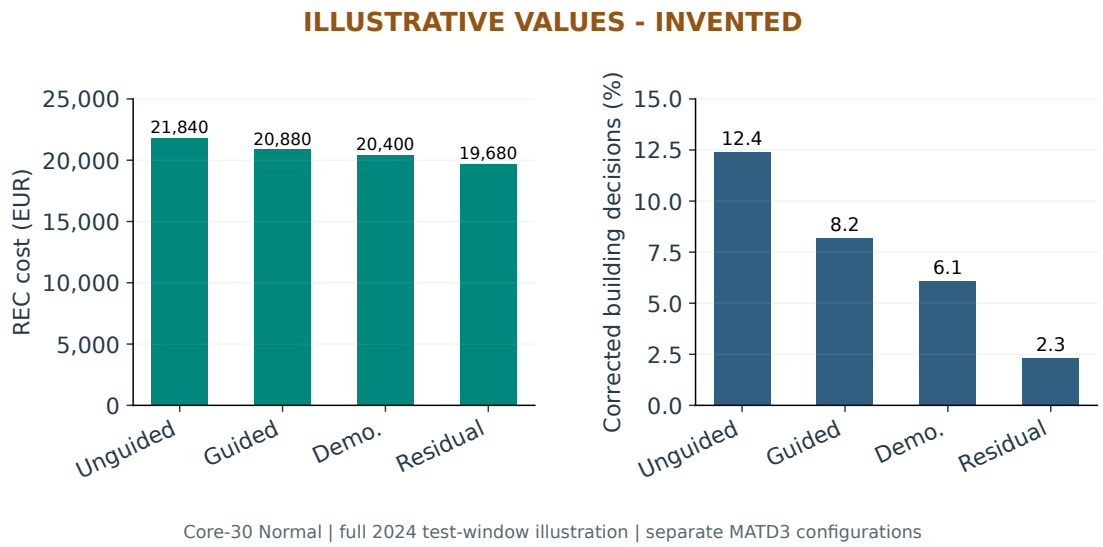


Figure 6.4: **Illustrative values; invented.** MATD3 configurations with different uses of prior knowledge. Correction denotes an eligible building decision with at least one actually adjusted command. All outcomes use the full Core-30 Normal 2024 test-window illustration.

can establish how much of the gain comes from the learning rule, the initial experience, or the operational composition by changing those elements separately.

6.5.3 Reward Design and Experience Sampling

A reward based too heavily on immediate energy cost can favour postponing charge until the departure requirement is missed. The electricity bill falls while the vehicle receives less service. Penalising every temporary charge deficit creates the opposite pressure: the learner charges as soon as the vehicle arrives, using little of the flexibility that motivated the control problem. The useful distinction is between a deficit that can still be recovered and an obligation that is becoming impossible to meet.

The controller development therefore examined the economic term alongside electrical pressure, service shortfall, and resource use. Departure consequences remain visible, while intermediate terms account for remaining connection time and deliverable charging power. If headroom contracts because another appliance is running, the amount of energy that can still be supplied before departure changes. The learner then faces a different decision even when the vehicle's current SoC is unchanged.

The scorecard keeps a stable meaning while these learning signals are investigated. A smoothed reward may provide a more useful training signal, but the final comparison still reports actual cost, minimum service, target accuracy, electrical violations, and grid shape. Battery throughput helps explain whether an improvement relies on more frequent cycling. This separation allows a reward change to be judged through the behaviour it produces.

Table 6.14: Learning difficulties and the mechanisms investigated to address them. Configuration versions and numerical parameters belong to Appendix B.7.

Difficulty	**Mechanism investigated**	**Behaviour to examine**
A departure consequence is sparse.	Service-aware reward and greater replay exposure to departure events.	Departure service and the preceding charging schedule.
Dense urgency encourages early charging.	Distinguish recoverable deficits from an approaching service failure.	Retained scheduling flexibility and required intervention.
A few active assets dominate experience.	Preserve per-resource actions and selected concurrent events in replay.	Service across resources, including infrequent ones.
A large throughput penalty suppresses storage.	Balance cycling cost against the value of flexibility.	Cost, throughput, and PV use together.
The exported actor loses its supporting rule.	Preserve the evaluated controller composition during packaging.	Reproduction of evaluated actions in inference.

Replay provides another way to make important consequences visible. Ordinary low-demand intervals can greatly outnumber departures, concurrent charging events, or restricted-headroom periods. Event-aware sampling increases exposure to selected transitions while retaining ordinary operation in the batch. The stored transition retains the action supplied at the learner–environment boundary, together with applied-action and intervention records where available. A teacher target is kept separately. The critic can then learn the consequence of the submitted decision, while a demonstration supplies a distinct supervised target for the actor.

The reward, exploration strategy, and replay distribution interact. A service penalty that is useful under uniform sampling may become dominant when service events are sampled more often. A careful comparison therefore changes one mechanism at a time and reads routine-period cost alongside the rare event of interest. The RQ is whether a more useful distribution of experience improves the resulting controller and reduces the interaction needed to obtain it.

These mechanisms connect safety and scalability within learning itself. Service-aware experience can reduce damaging proposals, and useful demonstrations can reduce the amount of simulation needed. Their dependence on a particular observation and action description also brings deployability back into view. Experience from one charger arrangement can be reused only when the receiving policy can interpret the resources and commands it describes.

6.5.4 Controller Variants and Ablation Studies

The development studies connect a control difficulty to a design choice and its operating consequences. For the example vehicle, persistent immediate charging motivates a closer examination of urgency in the reward; missed departures motivate attention to the experience retained around service events. Each change can then be read through charging time, delivered service, cost, and

required intervention. This gives the development sequence a practical meaning beyond the names of the configurations.

An ablation examines the contribution of one such choice under matched conditions. Demonstration pretraining can be enabled while the reward and later interaction budget remain fixed; a reward term can be changed while the observation and action meanings are retained. Table 6.15 identifies these comparisons. Their interpretation follows the mechanism being examined, including any trade-off between improved service and the effort needed to obtain it.

Table 6.15: Mechanism comparisons that separate controller design choices. These are comparison definitions, not completed numerical results.

Mechanism	**Matched change**	**Interpretation**
Observation design	Broad and compact causal inputs, with the same action and reward.	Value of information relative to representation cost.
Action heads	Shared and resource-specific heads, with matched command meaning.	Effect of separating unlike decisions inside the actor.
Exploration guidance	Guidance enabled or removed; same later training configuration.	Contribution of early experience to service and learning.
Demonstrations	Same archive and budget, with and without actor pretraining.	Value of an informed starting policy.
Reward and replay	One term or sampling mechanism changed at a time.	Effect on the targeted service event and routine operation.
Residual composition	Base alone, actor alone where admissible, and their composition.	Contribution of retained prior control and learned correction.

Author review. Confirm the configurations and ablations actually completed, their order where relevant, and the observations that motivated each retained change.

Local feasibility handling is examined through the proposed and adjusted actions. Their difference reveals when the actor requests infeasible operation, how the protection changes it, and whether the resulting schedule delivers the required service. These records help identify whether an improvement comes from the learned decision, the retained rule, or the protection supporting execution.

The guidance illustration compares complete configurations; the matched changes in Table 6.15 provide the corresponding mechanism studies. Demonstrations also introduce another source of experience: the records of earlier operation. The next section considers how far those records can support learning before a new policy interacts with the installation.

6.6 Offline Reinforcement Learning

A deployed controller accumulates a record of decisions and their consequences. The same is true of an RBC running through a simulated year. These records create another route to learning: prepare a policy from existing experience before allowing it to interact with the environment. For

an installation where exploratory actions can affect comfort, charging service, or equipment limits, that separation has practical value.

Editorial point to complete. The detailed account and attribution of the collaborative offline-learning study with Guilherme remain open until the author supplies the selected dissertation version. This section establishes its place in the argument and the relevant methods; no results from that study are included in the current comparison.

6.6.1 Operational Data for Policy Learning

An offline dataset contains transitions collected by a behaviour policy. For an energy installation, the record needs to associate an observation with the submitted action, its reward, and the next observation, while retaining the timing, resource configuration, and relevant execution information. A departure event then remains connected to the charging decisions that preceded it. The archive's usefulness depends on both the quality of those records and the range of conditions that produced them.

BC, already used for pretraining above, learns to reproduce the recorded actions. Offline RL uses recorded rewards and transitions to learn which behaviour is valuable (Levine et al. 2020). The distinction matters when the archive contains competent but imperfect operation: reproducing it supplies an initial policy, whereas value learning seeks opportunities to improve decisions within the support offered by the data.

A strong rule-based behaviour policy offers reliable services and repeatable trajectories, but it also shapes what the learner sees. If the rule almost always charges a vehicle immediately after arrival, the archive may contain little evidence about postponing that charge. The data can be extensive in time while narrow in action coverage. Combining compatible records from different policies, seasons, or operating conditions can broaden that coverage, provided their information and command meanings are retained.

This also explains the connection to the operational records in Chapter 4. When the Fail-Safe Supervisor limits an action or provides a fallback, the archive should retain that intervention together with the policy's proposal and the action submitted for execution. The resulting outcome belongs to the complete control process. Using those records for later training requires choosing a consistent learner–environment boundary so that a critic does not attribute the fallback's result to an action that was never submitted.

6.6.2 Offline Value Learning and Dataset Coverage

Two established approaches clarify the main methodological choice. IQL estimates values from actions observed in the dataset and extracts a policy through advantage-weighted imitation (Kostrikov et al. 2022). Its value-learning procedure avoids querying unseen actions to construct its policy-improvement target. This offers a way to favour valuable behaviour while reducing one source of unsupported extrapolation.

CQL instead regularises action-value estimates to discourage optimism about actions poorly supported by the dataset (Kumar, Zhou, et al. 2020). Its conservative term changes how value learning treats the difference between observed behaviour and candidate actions. The practical question is how that caution affects useful scheduling: strong conservatism can restrict improvement, while insufficient caution can favour actions whose consequences are poorly known.

These approaches provide context for the offline investigation; their precise role in the collaborative study remains to be confirmed in the editorial point above. Their evaluation has two useful references. The behaviour policy identifies whether the learned controller improves on the operation that generated the archive. The matched BAU reference gives the improvement its physical and economic context. The data-collection cost is retained separately from the offline training budget.

Generalisation also needs a concrete interpretation. A new year changes weather, demand, and mobility within an existing installation. A second building with compatible inputs and commands tests reuse across installations. Adding an unfamiliar resource changes the decision description itself. Those situations impose different demands on the learned model and on the archive from which it was prepared. Holding out time periods alone assesses the first of them.

Table 6.16: **Illustrative values; invented.** Offline-policy illustration against its behaviour-policy reference, Core-30 Normal.

Configuration	Cost [EUR]	EV min. [%]	EV band [%]	Corrected [%]
RBC Smart Local	21 600	99.8	92.0	4.0
Offline policy*	21 240	99.2	89.0	5.8
MATD3 (residual)	19 680	99.8	96.0	2.3

*This generic numerical example does not identify a tested offline algorithm or report the associated collaborative study.

Illustrative reading. The generic offline-policy example costs EUR 21 240 against its RBC Smart Local behaviour reference of EUR 21 600. Minimum service falls from 99.8% to 99.2%, while target-band accuracy falls from 92.0% to 89.0%. The modest economic improvement therefore accompanies weaker service. This provides a concrete reading of the archive question: useful scheduling experience must include the situations in which postponement leaves too little time to charge. The numerical row is a manuscript placeholder whose algorithm remains to be specified.

The resulting contribution to the chapter's story concerns access to experience. Learning can begin from earlier operation, reducing the need for exploratory data collection during that training stage. Its remaining dependence is the coverage and compatibility of the archive. This complements the next two investigations: hierarchy changes how decisions are divided, while control with changing resources changes how the decision problem is represented.

6.7 Hierarchical Community Coordination

A capable local controller resolves many of the decisions inside a building. REC coordination can then concentrate on how those decisions interact. This suggests a decomposition in which a

REC controller sends a compact signal and the building controllers retain responsibility for their resources and services. The investigation asks whether that smaller decision can improve a REC outcome without requiring one policy to command every charger, battery, and appliance directly.

6.7.1 Separating Community and Building Decisions

The hierarchy contains a REC coordinator and one local controller per building, as illustrated in Figure 6.5. The coordinator receives aggregate conditions and issues a signal at a declared interval. A local controller continues to act at every fifteen-minute simulator step, using the signal alongside its permitted local information. An hourly coordinator decision, for example, is held for four local steps, leaving the building controller to respond to arrivals and changes in headroom during that hour.

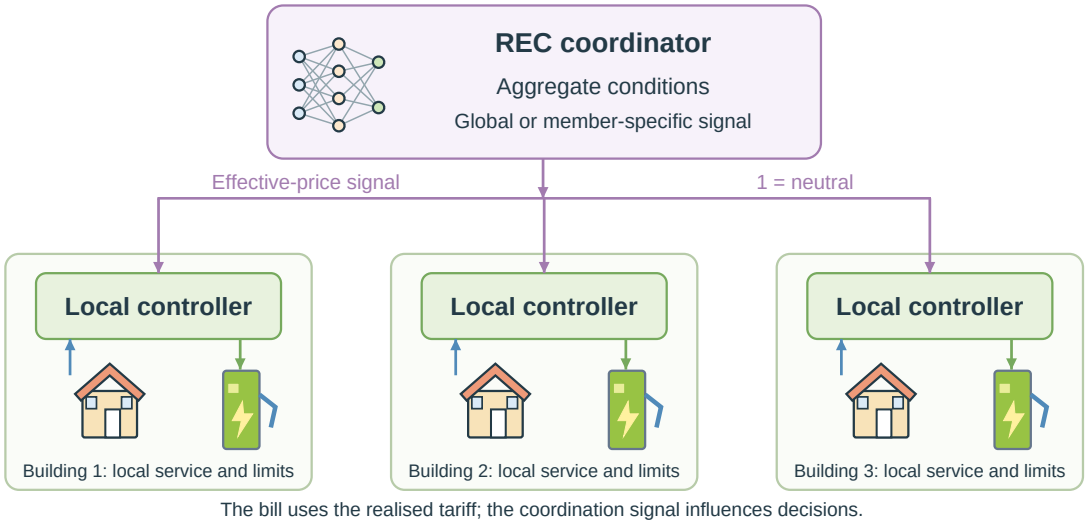


Figure 6.5: A REC signal influences local scheduling. Resource commands remain with the building controllers.

The experimental signal is an effective price multiplier. A value of one preserves the unmodified local price condition. A lower value can encourage flexible consumption, while a higher value can encourage postponement or stationary-battery support when the local controller has the corresponding freedom. The participant's actual bill continues to use the original tariff; the multiplier is an input to the decision process. This distinction preserves the economic comparison when the coordinator changes its signal.

For the running example, the coordinator might encourage demand during a period of REC PV surplus. The household can respond only if a vehicle is present, a battery has available capacity, or an appliance can start within its permitted window. A household close to a departure deadline may have little freedom to reduce charging. The response therefore depends on local circumstances that remain the responsibility of the building controller.

A single multiplier for the whole REC gives the coordinator a fixed-size output. Member-specific multipliers allow more selective use of flexibility, but the output then depends on the represented members. These two designs expose a direct trade-off between a compact REC decision and more precise allocation. They also make distributional outcomes relevant: aggregate savings can be concentrated in the buildings with the most usable flexibility.

The causal path of the signal is checked before learning the coordinator. The neutral signal should reproduce the standalone local controller. Fixed non-neutral signals then establish whether the intended local decisions actually change. In the price-aware residual composition, the signal conditions the local base controller while the learned residual retains its original observation convention. This gives the coordinator a defined route to influence actions while preserving the meaning of the residual's inputs.

These comparisons use frozen local controllers so that the effect of the coordinator can be interpreted separately. Joint adaptation of both levels is another RQ because changes in the local policy alter the response being learned at REC level. The fixed-local-policy study provides a clear starting point for understanding whether the signal has useful authority.

6.7.2 Community Benefits and Their Distribution

Table 6.17: **Illustrative values; invented.** Full 2024 test-window illustration, Core-30 Normal. Hierarchical coordination, energy and economic outcomes.

Configuration	Cost [EUR]	Daily peak [kW]	Ramping [kW]	Solar [%]
RBC Smart Local				
Standalone	21 600	54.0	167 400	71.0
Global signal	20 880	51.0	158 400	73.0
Per-member signal	20 640	49.8	156 600	74.0
MATD3 (residual)				
Standalone	19 680	49.2	153 000	76.0
Global signal	19 920	46.8	144 000	77.0
Per-member signal	20 160	45.6	140 400	78.0

Standalone is the same local controller with a neutral coordination signal. Cost uses the unchanged billing tariff.

Illustrative reading. The first block of Table 6.17 illustrates a setting in which coordination improves otherwise local rules. Annual cost falls from EUR 21 600 to EUR 20 880 with a global signal and to EUR 20 640 with member-specific signals. Mean daily peak and upward ramping also fall, while solar self-consumption rises. Table 6.18 keeps service beside the distribution of benefit: 22 and 24 of the 30 members have a lower bill than under their standalone local controller. The separate BAU column preserves the common reference used elsewhere in the chapter.

Illustrative reading. A different trade-off appears above the selected residual MATD3 controller. The invented hierarchical configurations reduce mean daily peak from 49.2 kW to 46.8 kW or 45.6 kW, while annual cost rises from EUR 19 680 to EUR 19 920 or EUR 20 160. These rows

Table 6.18: **Illustrative values; invented.** Full 2024 test-window illustration, Core-30 Normal. Service and distribution of coordination benefits.

Configuration	**EV min.** [%]	**EV band** [%]	**Corrected** [%]	**Save / BAU** [n/N]	**Save / local** [n/N]
RBC Smart Local					
Standalone	99.8	92.0	4.0	26/30	–
Global signal	99.8	93.0	3.8	28/30	22/30
Per-member signal	99.9	94.0	3.5	29/30	24/30
MATD3 (residual)					
Standalone	99.8	96.0	2.3	29/30	–
Global signal	99.8	96.0	2.5	28/30	17/30
Per-member signal	99.8	96.0	2.7	27/30	19/30

Save / BAU is the common member-benefit indicator. Save / local additionally compares each member with its own standalone controller. All illustrative rows have zero residual electrical violation.

represent a coordinator that values the REC power profile. Standalone residual MATD3 retains the lowest cost, while hierarchy accepts a modest economic trade-off for smoother exchange and higher solar use. Lower bills for some members can coexist with higher aggregate cost when other members bear a larger increase.

The comparison consequently needs the same local controller on both sides, a declared REC objective, and member-level interpretation. A reduction in peak can be valuable even when it does not produce the lowest bill. Service and local electrical constraints remain part of that reading because a REC signal cannot create flexibility where a participant has an immediate obligation.

Hierarchy also clarifies an architectural boundary. A compact signal reduces the number of REC-level decisions, while local inference remains close to the equipment. The interpretation of that signal must still be shared by the coordinator and its recipients. When the policy emits one value per member, membership and output order become part of the learned interface. The coordination study therefore leads naturally to a further question: what happens to these decisions when the resources or participating buildings change?

6.8 Control under Resource Reconfiguration

The preceding methods improve decisions for a described installation. The installation itself can also change. A household may add a second charger, remove a battery, or replace equipment; a REC may gain another member. The next investigation asks how a trained controller can continue to operate when the number of resources it observes and controls changes. That operational question motivates the representation choices that follow.

Editorial point to complete. The detailed attribution to Gustavo's study, its tested changes, and its experimental findings remain open until the author supplies the selected dissertation version. The focus is control with dynamic resources. The account below describes the corresponding representation and the current controller implementations without assigning unconfirmed results to that study.

6.8.1 Availability Changes and Structural Changes

An EV disconnecting from an existing charger changes availability. If the controller already has an input and output for that charger, it can represent the event through connection state and admissible action bounds. Installing an additional charger changes the number of independently addressable resources. A fixed output layer that previously produced one charger command now needs to produce two. The distinction separates a changing value in an existing description from a change to the description itself.

A fixed vector can reserve positions for a maximum number of resources and mask the unused entries. This is a useful solution when that maximum and the resource catalogue are known in advance. Its cost and coverage depend on the chosen capacity, and each reserved position still needs a stable interpretation. A variable-size policy instead processes the active resource records and produces a corresponding set of commands. Figure 6.6 illustrates the question with a second charger of a known type.

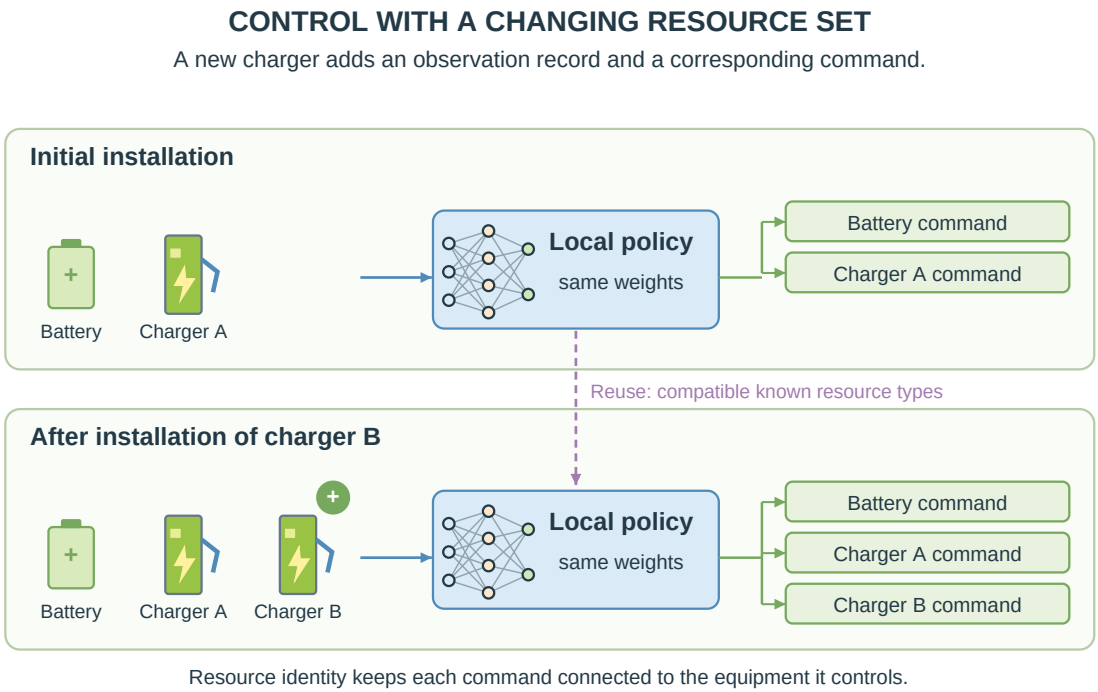


Figure 6.6: A change in resource count requires matching observations and commands. The same learned representation can be reused for compatible instances of known resource types.

The requirements extend beyond accepting a longer input. Each controllable resource needs its own output, and that output must return to the correct equipment. A decision also needs context: two chargers can have different departure obligations while sharing the same building headroom. The representation must therefore retain individual information and allow interactions among the resources relevant to a decision.

6.8.2 Variable-Size Policies and Their Operational Scope

Attention-based representations offer one way to meet those requirements (Lee, Lee, et al. 2019; Vaswani et al. 2017). Each resource contributes a feature record, which a type-specific projection maps to a common embedding size. Context records supply information such as building demand or price. The attention layers combine these representations, and output heads generate commands for the controllable resources. A second charger adds another compatible record and another associated command, while reusing the learned processing weights.

The PPO variant applies this representation within each building's policy and value-learning process. The MATD3 variant combines local resource-based actors with central twin critics. For critic learning, building-level summaries of resources and actions are aggregated to represent the joint decision. The learning rules retain their earlier purposes; the representation now addresses variability in the resources those rules process.

The current action catalogue covers stationary storage and EV chargers; appliance information supplies context. A matched comparison keeps appliance service under the same declared rule when that resource lies outside the learned action catalogue. This preserves the decision scope while examining the representation.

The comparison with a fixed-layout actor asks two questions. First, how much control quality and computation change when the original installation is retained. Second, what preparation is needed when compatible resources are added or removed. A compact fixed-layout actor can be effective for its original installation. The additional structure is useful when it reduces the rebuilding, retraining, or remapping needed for another supported configuration.

The implementation also has to preserve the meaning of earlier experience. An off-policy transition describes the arrangement of resources and actions present when it was collected. Layout signatures group compatible replay records so that historical charger commands remain associated with their corresponding inputs. In the current Transformer controller implementations, changes among known resource instances can be represented within existing buildings, while a change in the building population rebuilds the learned controller state. Variable local resource counts and uninterrupted reuse across a changing population are therefore separate capabilities.

Table 6.19: **Illustrative values; invented.** Full 2024 test-window illustration, Core-30 Normal. Nominal performance before a change of resource configuration.

Configuration	**Cost** [EUR]	**CO_2** [kg]	**EV min.** [%]	**EV band** [%]	**Corrected** [%]
Fixed-layout reference					
MATD3 (residual)	19 680	25 200	99.8	96.0	2.3
Variable-resource representations					
Transformer PPO	21 360	28 100	99.0	89.0	5.0
Transformer MATD3	20 160	25 700	99.7	95.5	2.0

Illustrative reading. Table 6.19 assigns Transformer PPO an annual cost of EUR 21 360 and a minimum-service rate of 99.0%, and Transformer MATD3 EUR 20 160 with 99.7% service. The selected residual MATD3 controller retains the lower nominal cost of EUR 19 680. Target accuracy and correction expose further operating differences. This comparison concerns the original installation over the full test window; the value of accommodating a changed resource count is examined through the separate event study described next.

A dynamic-resource evaluation follows the event as well as the annual aggregate. Table 6.20 retains the whole year, including the decisions supplied by the supervisor when the learned interface becomes unusable. The trained policies remain frozen throughout the reserved year. Preparation for another installation is assessed separately, before evaluation; fallback within the test sequence makes the resulting loss of learned control visible.

Table 6.20: **Illustrative values; invented.** Annual operation through Core-30 Topology changes.

Configuration	Cost [EUR]	Failed EVs [n]	Fallback [building-h]	Deficit [kWh]
RBC Smart Local	21 033	90	2 234.8	166.5
MATD3 (residual)	20 192	288	18 622.0	532.8
Transformer PPO	20 799	162	10 242.2	299.7
Transformer MATD3	19 631	99	10 242.2	183.1

Invented complete-year controller-plus-supervisor outcomes. Policies remain frozen on the held-out year. Fallback supplies commands outside the usable learned interface; it does not certify that the original implementation supports every topology event. The eligible departure cohort is 9 000.

Illustrative reading. The Topology comparison gives Transformer MATD3 a lower annual cost and fewer missed departures than the fixed-layout residual configuration. Its shorter fallback duration connects that outcome to a larger share of decisions supplied through the learned interface. Fallback is still substantial: known resource changes within a building and changes to the building population impose different demands on the current implementations. Annual service can remain high while supervision carries a considerable part of the operating year.

The resource-based policies extend the configurations that a trained controller can process. Their assessment also exposes the work required to keep observations, service obligations, commands, and protection consistent. The following section places this investigation alongside algorithmic responses to the wider problem of real-world control.

6.9 State-of-the-Art Methods for Real-World Control

Applying a learned controller to a REC involves several responsibilities at once. It must coordinate the resources with acceptable computation, be practical to install and maintain, respect physical and service constraints, and retain useful behaviour when information or equipment becomes unreliable. These are the scalability, deployability, safety, and robustness problems defined in Chapter 1. The

methods reviewed in Chapter 3 offer different responses to them, including responses developed outside energy systems.

The running household illustrates why their comparison belongs together. Reusing a policy for a second charger can reduce preparation and training, but both chargers must still receive the correct commands and share the available headroom. If one vehicle's measurement becomes stale, the controller must decide which services can continue. A method's practical value depends on this complete sequence, including the integration and supervision that support it. The following reference designs examine the mechanisms involved under the same REC-control requirements.

6.9.1 Task-Open Coordination and Compositional Learning

MOHITO represents coordination through relations between agents, tasks, and actions. Its evolving hypergraph allows the actor and critic to account for tasks that enter or leave and for the corresponding available actions (Anil et al. 2025). This supplies an algorithmic response to changing work, with the reported evidence coming from ridesharing and wildfire suppression. Its relevance to the thesis lies in coordinating decisions while service demands change, and in the reusable structure through which those decisions are represented.

For an energy adaptation, a building can be an agent and an EV session a task carrying an energy requirement and departure time. The charger identifies the equipment through which a decision is executed. Constructing the hypergraph requires defining which agent can serve a task and what each available charging action means. A power discretisation must retain its physical resolution in the comparison, while a continuous-action extension must be identified as an adaptation. The arrival of a vehicle then updates service demand without being confused with the installation of another charger or the admission of another building.

This mapping connects the method to all four assessment dimensions. Computation and policy reuse concern scalability; the work of binding tasks and actions to equipment concerns deployability. Applied commands and departure service expose the consequences of the accompanying constraint handling. Fault experiments examine what remains operational when an agent, measurement, or actuator becomes unavailable. Each result belongs to the evaluated combination of policy and supporting mechanisms, with the published task-open scope retained explicitly.

Compositional learning separates another part of the problem. COIN uses specialised perception and action modules around a shared controller, learning combinations of observation, action, and instruction spaces through BC (Luketina et al. 2025). The modular construction provides a reference for reusing learned processing when the way a system is observed or actuated changes.

In the household, perception modules can prepare measurements from meters, batteries, and chargers, while action modules translate the common decision representation into the corresponding commands. A second installation can reuse familiar modules in another combination. Its integration still needs resource identities, units, ranges, and equipment associations. A COIN-inspired PPO reference would test that modular principle under the thesis's learning protocol; it would be labelled as an adaptation because the published learner uses imitation.

The assessment follows both the reuse obtained and the operation delivered. Preparation and adaptation costs accompany energy performance; safety measurements follow the output through its protection and execution; disturbance tests retain service and recovery records. Familiar modules offer a useful basis for deployment, while their compatibility with the installation remains an explicit condition of the experiment.

6.9.2 Constraint Handling and Fault-Tolerant Learning

An algorithmic comparator can also act on the decisions produced by an otherwise unchanged policy. Continuous-action safety layers solve a state-dependent correction problem before a proposal is applied (Dalal et al. 2018). For the two chargers, the comparison follows how a requested power is modified when simultaneous charging exceeds the available headroom. The protection may preserve the electrical limit while postponing service, so correction magnitude and missed departures accompany the violation count.

Multi-agent shielding addresses the additional coupling between local decisions. Certified combinations of local obligations can support a shared safety specification when their joint assumptions hold (Adalat et al. 2026; Elsayed-Aly et al. 2021). This provides a reference for distinguishing a building's own limit from a transformer limit shared across the REC. In a matched energy study, the physical model, allocation assumptions, admissible actions, and fallback authority must be declared. They determine both the guarantee and the coordination freedom left to the learner.

Fault-tolerant learning addresses the behaviour learned before a failure occurs. MARTA augments a MARL learner with selective fault induction: a switching mechanism chooses the affected agent and timing, while an adversary supplies faulty behaviour (Mguni et al. 2026). The comparison with ordinary disturbance training concerns where adverse experience is concentrated and how the resulting policy behaves under faults. In the REC setting, an adaptation must state whether a fault affects an agent's decision, its observation, or the equipment response; these events have different physical consequences.

A robustness comparison therefore holds the fault exposure and information available to the controllers explicit. It retains service deficits, failed decisions, supervisory intervention, and recovery, together with nominal performance. Training cost also matters: improved fault tolerance may require additional interaction or adversarial optimisation. The same experiment thus connects robustness and safety to the computation required for learning and the mechanisms needed to maintain operation.

6.9.3 Common Control Task and Comparative Assessment

The reference set combines general algorithmic mechanisms with energy-specific developments. GNN-DT connects graph representations to offline charging decisions, while HetGPS combines graph-based coordination with physical corrective control (Orfanoudakis, Panda, et al. 2026; Wang,

Nimalsiri, et al. 2026). These studies complement the task-open, modular, safe-learning, and fault-tolerant methods above. Their published results retain their original datasets and assumptions; a common scorecard requires the corresponding energy adaptation to be run on the same task.

For mechanism attribution, a matched entity-based MAPPO reference can retain the same observations, central critic information, physical action scope, and training budget as the proposed method. Equal protection supplies a control for the contribution of representation, while a separate comparison changes the protective mechanism itself. Similar contrasts hold the learner fixed while adding fault training or a different packaging and compatibility procedure. This distinguishes the complete operating result from the contribution of a particular algorithmic choice.

Table 6.21: Algorithmic references for the real-world control problem and their comparative role.

Reference	**Mechanism examined**	**Principal measurements**
MOHITO-based adaptation	Coordination through changing agent–task–action relations.	Service and energy; computation, integration, and continued control under events.
COIN-inspired adaptation	Reuse of perception and action modules around a shared controller.	Adaptation and installation effort; control quality, physical admissibility, and disturbed operation.
Safety layers / shielding	Correction or restriction of policy proposals using declared constraints.	Violations, corrections, service, latency, and model/configuration requirements.
MARTA-based fault training	Selective adverse experience added to the learning process.	Retained service, failed decisions, recovery, nominal performance, and training cost.
GNN-DT / HetGPS	Energy-specific graph learning and, for HetGPS, corrective control.	Energy and service across configurations; data needs, computation, protection, and integration.
Matched entity MAPPO	Common learner and information with controlled representation and protection choices.	Contribution of each mechanism across the common scorecard and operational records.

The Transformer PPO and MATD3 controllers described earlier are implemented references. The additional adaptations in Table 6.21 define comparative studies whose implementations and numerical runs are not yet reported here. This status applies to the section as a whole; the existing illustrative tables remain separate from measured evidence.

Deployment assessment follows the complete controller package: preparation of inputs and commands, compatibility checks, loading on the target platform, replay agreement, and the effort needed to maintain or restore operation. The architecture in Chapter 4 supplies this common process. A modular policy, a graph policy, and a fault-trained policy are all assessed through it, using the same physical scope and execution authority.

The annual scenarios and absolute scorecard established in Chapter 5 remain the common basis. Nominal performance, resource changes, restricted headroom, and disturbances each expose part

of the problem; combined scenarios examine their interaction. Computation, preparation effort, intervention, and failed-service records accompany the energy indicators. The discussion can then assess which capabilities work together and what remains necessary to sustain useful real-world control.

6.10 Comparative Discussion

The studies examine several ways of making REC energy control useful. Explicit rules can supply dependable local decisions with modest computational effort; learning can use repeated experience to refine scheduling; hierarchy can change the REC power profile while retaining local responsibility. Resource-based representations extend the configurations that a trained policy can address. The discussion considers the value of those capabilities together with the conditions and effort required to retain them.

The method descriptions and the real Core-30 reference execution establish the present basis for that discussion. The invented comparisons illustrate its quantitative form; their rankings remain editorial assumptions pending the completed evaluation.

6.10.1 Comparison Scope and Interpretation

The annual comparison layout covers the implemented controller families across the same seven scenarios. The additional algorithmic reference designs in Section 6.9 remain separate until their adaptations and matched runs are available. Core-30 Normal establishes operation within a stable installation; Topology, Safety, and Robustness expose the consequences of resource changes, restricted headroom, and disturbances. Combined places those demands together. Premium-100 Normal and All-in extend the comparison to the larger REC. Table 6.22 gives the annual energy-purchase cost of each configuration; Table 6.23 gives the corresponding minimum EV service. The complete scorecards appear in Appendix B.12.4.

Illustrative reading. The two matrices expose different advantages. Residual MATD3 has the lowest nominal cost among the learned configurations in both RECs. The resource-based alternatives retain more of their service under changes and combined disturbances in this example. Perfect-information optimisation gives a demanding scheduling reference, with a different information allowance. Reading down one column compares methods under the same annual conditions; reading across a row shows how that controller responds to the different scenarios.

The vehicle example gives the annual account a practical meaning. A fixed-size controller may need its inputs and outputs rebuilt when another charger is installed. If the existing model cannot issue that command, the supervisor supplies it. The annual result then belongs to the complete controller and supervisor, while fallback duration identifies how much operation relied on the latter. The synthetic tables assume this full-year composition throughout; they are a completed-results mockup, including hypothetical fallback, rather than a record of tested implementation compatibility.

Table 6.22: **Illustrative values; invented.** Annual energy-purchase cost [EUR] across the seven scenarios.

Configuration	C30 N	C30 T	C30 S	C30 R	C30 C	P100 N	P100 A
BAU without storage	27 000	25 400	26 600	27 200	25 500	111 000	105 000
BAU with storage	24 000	22 800	23 700	24 600	23 200	96 000	93 000
RBC Basic	23 040	23 639	23 207	26 922	26 504	92 160	109 814
RBC Smart Local	21 600	21 033	21 757	23 520	22 968	86 400	94 581
RBC Community	21 120	20 566	21 273	22 947	22 458	84 480	92 479
MILP local fixed	21 480	20 610	21 318	24 219	22 633	85 920	94 056
MILP REC fixed	21 000	20 150	20 841	23 678	22 127	84 000	91 954
MILP local flexible	19 560	18 768	19 412	22 054	20 610	78 240	85 648
MILP REC flexible	18 720	17 962	18 578	21 107	19 725	74 880	81 970
Local PPO	22 080	22 654	22 240	25 800	25 399	88 320	105 239
Local TD3	21 600	22 162	21 757	25 240	24 847	86 400	102 951
MADDPG	20 880	21 423	21 031	24 720	24 019	83 520	99 519
MATD3 (residual)	19 680	20 192	19 823	22 080	22 639	77 400	93 800
MAPPO	20 640	21 177	20 790	23 520	23 743	82 560	98 375
HAPPO	20 400	20 930	20 548	23 837	23 467	81 600	97 232
MASAC	20 640	21 177	20 790	24 118	23 743	82 560	98 375
Offline policy*	21 240	21 792	21 394	24 819	24 433	84 960	101 235
Transformer PPO	21 360	20 799	21 515	23 208	22 713	85 440	93 530
Transformer MATD3	20 160	19 631	20 306	21 840	21 437	78 300	88 276

C30: Core-30; P100: Premium-100. N: Normal; T: Topology; S: Safety; R: Robustness; C: Combined; A: All-in. Each column covers its complete 2024 sequence, including supervisory fallback. *Generic offline-policy placeholder; the associated method remains to be specified.

Configurations are selected during 2023 validation, before the 2024 outcomes are examined. A configuration prepared for each REC measures performance after preparation. Continuing with the existing model through a change measures reuse and service continuity. The retained policies and preprocessing remain fixed over the held-out year. Appendix B.12.4 extends the same annual comparison to the guidance and hierarchy variants.

Within each supported setting, the storage-enabled BAU gives improvement its practical meaning: the installation already makes ordinary use of its battery. The informed rule family provides a further comparison with little training effort. A modest economic difference may favour that simpler controller when its service and maintenance requirements suit the installation. Learned configurations can offer a different balance of cost, coordination, service, and preparation effort.

The outcome belongs to the configuration actually evaluated. The residual MATD3 result includes its retained base rule; the matched ablations examine what the learned component and its supporting mechanisms each add. Information access also remains part of the comparison. Central critics use joint training information, REC-aware rules require runtime aggregates, and a perfect-information optimiser receives future inputs. Those differences help explain an outcome alongside the measured operating and computational quantities.

Table 6.23: **Illustrative values; invented.** Annual minimum EV service [%] across the seven scenarios.

Configuration	C30 N	C30 T	C30 S	C30 R	C30 C	P100 N	P100 A
BAU without storage	100.0	99.0	96.0	96.5	93.0	99.0	91.0
BAU with storage	100.0	99.2	97.0	97.2	94.5	99.3	93.0
RBC Basic	99.2	96.2	97.2	95.2	92.2	98.7	90.2
RBC Smart Local	99.8	99.0	97.4	97.7	96.3	99.3	94.8
RBC Community	99.8	99.0	97.8	97.5	96.3	99.3	94.8
MILP local fixed	100.0	99.8	99.5	98.6	98.2	99.9	97.2
MILP REC fixed	100.0	99.8	99.5	98.6	98.2	99.9	97.2
MILP local flexible	100.0	99.8	99.5	98.6	98.2	99.9	97.2
MILP REC flexible	100.0	99.8	99.5	98.6	98.2	99.9	97.2
Local PPO	98.2	95.2	96.2	94.2	91.2	97.7	89.2
Local TD3	98.8	95.8	96.8	94.8	91.8	98.3	89.8
MADDPG	98.6	95.6	92.0	93.6	91.6	98.1	89.6
MATD3 (residual)	99.8	96.8	98.6	97.3	92.8	99.3	90.8
MAPPO	99.5	96.5	97.5	96.3	92.5	99.0	90.5
HAPPO	99.6	96.6	97.6	95.6	92.6	99.1	90.6
MASAC	99.3	96.3	97.3	95.3	92.3	98.8	90.3
Offline policy*	99.2	96.2	97.2	95.2	92.2	98.7	90.2
Transformer PPO	99.0	98.2	97.0	96.7	95.5	98.5	94.0
Transformer MATD3	99.7	98.9	98.8	97.9	96.2	99.2	94.7

C30: Core-30; P100: Premium-100. N: Normal; T: Topology; S: Safety; R: Robustness; C: Combined; A: All-in. Each column covers its complete 2024 sequence, including supervisory fallback. *Generic offline-policy placeholder; the associated method remains to be specified.

6.10.2 Economic Performance, Service Delivery, and Safety

The running example makes the relation between cost and service particularly clear. A controller can lower the bill by shifting the required charge into a cheaper feasible interval. It can also lower the bill by failing to deliver that charge. Aggregate cost alone cannot distinguish those outcomes. Minimum service, target accuracy, appliance completion, and energy served establish whether the economic change comes from scheduling or from a different service being delivered.

Electrical feasibility adds another layer. A controller may repeatedly request more charging power than the connection permits, while the simulator clips its commands and produces an entirely feasible trajectory. The executed outcome describes successful protection, yet the proposals expose a poor allocation of headroom. Keeping the requested and applied actions separate reveals the source of feasible behaviour and the amount of intervention on which the controller depends.

Safety can be examined before adding a disturbance by tightening the available headroom. The same vehicle arrivals and service requirements then leave less freedom for concurrent charging. Core-30 Safety supplies this comparison. Proposed-action corrections reveal how often the controller requests an inadmissible combination, while the service rate shows whether the remaining flexibility is scheduled effectively.

Illustrative reading. Table 6.24 assigns MATD3 a correction rate that rises from 2.3% under nominal headroom to 6.8% under the tighter limits, with 98.6% minimum service. The Transformer

Table 6.24: **Illustrative values; invented.** Full 2024 test-window illustration, Core-30 Safety. Nominal correction rates retain the matched Normal reference.

Configuration	Corr. N [%]	Corr. S [%]	EV min. S [%]	EV band S [%]	Viol. S [kWh]
RBC Smart Local	4.0	9.0	97.4	88.0	0.0
MADDPG	14.0	25.0	92.0	74.0	0.0
MATD3 (residual)	2.3	6.8	98.6	93.0	0.0
Transformer MATD3	2.0	5.1	98.8	94.0	0.0

N: Normal; S: Safety. Corrections count eligible building decisions with an actual command change.

variant has a smaller correction rate in this invented example.

Target accuracy and residual electrical violation complete that comparison. Constraint handling can keep the applied trajectory within limits while the policies make different use of the available charging window. Table 6.25 follows the resulting service loss to departure: it gives the number of eligible requests missed, the energy still needed to reach the minimum-service threshold, and the largest individual shortfall.

Table 6.25: **Illustrative values; invented.** Missed EV service over the full 2024 test-window illustration. Paired Core-30 Normal and Safety settings.

Scenario	Failed [n]	Eligible [N]	Deficit [kWh]	Largest [kWh]
RBC Smart Local				
Normal	20	10 000	40	6
Safety	260	10 000	780	12
MADDPG				
Normal	140	10 000	420	14
Safety	800	10 000	3 040	19
MATD3 (residual)				
Normal	20	10 000	34	5
Safety	140	10 000	350	9
Transformer MATD3				
Normal	30	10 000	54	6
Safety	120	10 000	264	8

The invented cohort has 10 000 eligible departures and 200 additional unattainable requests in each setting; it is not a measured dataset count. Failed departures fall below the minimum service threshold. Deficit sums their battery-energy shortfall; Largest is the maximum individual shortfall. Appliance completion is illustrated in Table 6.30; DR is inactive.

Illustrative reading. In the common invented cohort, MATD3's 98.6% minimum-service rate under tighter headroom corresponds to 140 failed departures, compared with 20 in Normal. Their combined deficit is 350 kWh, with a largest individual shortfall of 9 kWh. The 200 additional unattainable requests remain visible beside that cohort. The counts and energy deficits give the annual rate a practical meaning: how often the vehicle leaves without the minimum charge and how much service is missing when it does.

Service-aware rewards, explicit action bounds, and residual rules address different parts of this behaviour. A reward changes what the policy learns to prefer. An action adjustment changes what is submitted or executed. A retained rule can supply decisions that the neural component leaves unchanged. Their roles can be combined productively, as long as the evaluated composition is the one subsequently deployed. The Fail-Safe Supervisor then provides validation, limiting, logging, and fallback when the learned controller cannot supply a usable inference.

REC objectives introduce a further trade-off. Reducing a shared peak can shift consumption into a more expensive period for some members. The hierarchy example makes this visible by comparing each coordinated configuration with the same local controller under a neutral signal. Member-level bills explain who bears the change, while the aggregate indicators explain its REC value. A peak reduction, a lower total bill, and a more even distribution of benefits are distinct outcomes that may require different priorities.

6.10.3 Computational Demand, Policy Reuse, and Deployability

Scalability concerns the complete learning and control process. One actor per building keeps the local inference problem modest, but maintaining many independent learners can multiply training and model-management effort. A central critic can learn from joint decisions while its input and update cost grow with the REC. Attention-based aggregation changes that organisation and brings its own cost, particularly as the number of resource records increases. These mechanisms motivate measurements at both training and inference time.

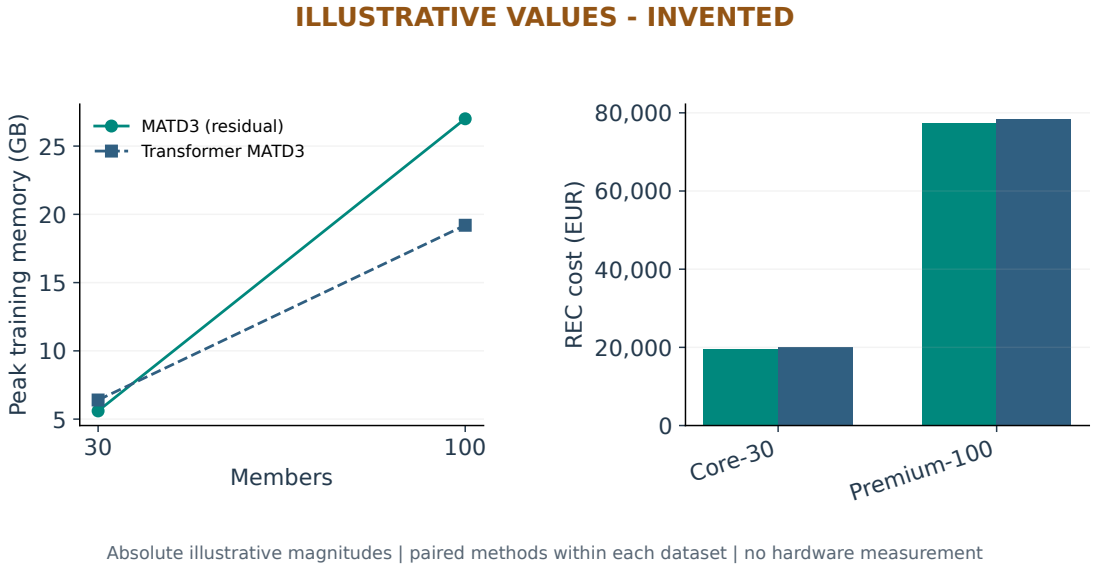


Figure 6.7: **Illustrative values; invented.** Peak training memory in GB and full 2024 test-window cost in EUR. Methods are paired within Core-30 Normal and Premium-100 Normal; the absolute magnitudes are invented.

Illustrative reading. Figure 6.7 assigns the residual MATD3 configuration peak training-memory demands of 5.6 and 27.0 GB for 30 and 100 members. The Transformer variant uses 6.4 and 19.2 GB in the same invented example. The relative memory demand changes between the two RECs, whose size and resource composition both differ. Annual costs compare the methods within each REC. The common hypothetical execution target and timing boundary are stated with Table 6.26.

The figure varies population across separately configured studies. Continued operation through a population change is a different experiment. A representation with favourable training-memory growth can still require a rebuild when a member joins, just as a low-latency actor can remain tied to one equipment arrangement. Computational growth, supported configuration changes, and service continuity describe complementary aspects of scalability and dynamic reconfiguration that annual cost alone leaves hidden.

Table 6.26: **Illustrative values; invented.** Computational demand in the two Normal RECs.

Configuration	Memory [GB]	Training [h]	p95 [ms]	Late [n/N]
MATD3 residual / C30	5.6	18.4	2.8	12 / 1 054 080
Transformer MATD3 / C30	6.4	21.7	4.1	18 / 1 054 080
MATD3 residual / P100	27.0	86.2	3.5	84 / 3 513 600
Transformer MATD3 / P100	19.2	69.8	5.8	106 / 3 513 600

Illustrative common-target assumption: 32-GB training accelerator, eight-core edge CPU for serial building inference, batch size 256, equal update budget, and a 1-s inference deadline. Timings and deadline counts are invented; no hardware benchmark was executed. Model inference excludes upstream data acquisition.

Illustrative reading. Table 6.26 places memory beside training time, inference latency, and late decisions. In the larger REC, the Transformer example reduces training time from 86.2 to 69.8 hours while increasing the 95th-percentile building inference time from 3.5 to 5.8 milliseconds. A more compact joint learning representation can therefore coexist with a more expensive local decision. The late-decision count retains the tail of execution that an average or percentile can hide.

Guidance and offline learning offer another form of reuse: earlier behaviour becomes a source of experience. Demonstrations can reduce unproductive exploration, and an archive can support training without new interaction during that stage. The benefit depends on the event coverage and action support that the records contain. Repeated examples of ordinary evenings may contribute less to departure reliability than a smaller collection containing restricted headroom and concurrent obligations. The amount of data and its decision-relevant diversity therefore deserve separate attention.

Hierarchy reduces the coordinator's action dimension when one signal influences many buildings. Its usefulness then depends on the response available locally. A household with an imminent departure may be unable to postpone charging whatever signal it receives. More selective member-level signals increase the coordinator's authority, but they also reintroduce membership-dependent outputs and a larger allocation problem. The comparison is a trade-off between compact coordination, local autonomy, and the precision with which shared flexibility can be directed.

Preparing the controller for operation. Deployability follows the selected controller into the execution path described in Chapter 4. The rule-based reference needs its resources, limits, and contextual inputs configured. A learned controller also needs the observation preparation and action mapping used during evaluation; a residual controller retains its base rule. Table 6.27 makes these dependencies visible alongside the installation and execution measurements.

Table 6.27: **Illustrative values; invented.** Preparation and execution of the three controller packages in the same Core-30 installation.

Record	RBC Smart Local	MATD3 residual	Transformer MATD3
Configuration and integration [person-h]	2.0	7.5	5.0
First valid applied decision [h]	2.4	8.1	5.7
Replay agreement $[n/N]$	10 000/10 000	10 000/10 000	10 000/10 000
Successful dispatch $[n/N]$	9 998/10 000	9 997/10 000	9 998/10 000
Additional known charger [person-h]	0.5	3.0	1.0

Invented installation exercise with the same inputs, services and equipment interface. Replay uses a normalised-action tolerance of 10^{-6} before supervision. Dispatch counts confirmed interface hand-offs; it does not measure physical completion. Preparation includes the residual base rule where applicable.

Illustrative reading. Preparing the residual MATD3 package takes 7.5 person-hours in Table 6.27, against 2.0 for the local rule and 5.0 for the Transformer package. The retained rule, ordered inputs, action mapping, and preprocessing contribute to that preparation. Adding a known charger requires a further 3.0 person-hours for the fixed-layout package and 1.0 for the resource-based package. Equal replay agreement shows that the preparation reproduces the evaluated decisions; dispatch counts then follow those decisions to the equipment interface. These are separate stages in making the controller usable.

6.10.4 Robustness and Resource Reconfiguration

Robustness asks how useful behaviour is retained when the expected operating conditions deteriorate. A missing measurement changes the evidence available for a decision; an unavailable charger changes the action that can be realised. Both can affect the same departure service through different paths. Paired scenarios identify that degradation against the corresponding nominal controller, retaining the same underlying demand and service requirements wherever possible.

Illustrative reading. Table 6.28 illustrates why nominal performance and robustness need separate discussion. MATD3's invented annual cost rises from EUR 19 680 to EUR 22 080, while minimum service falls from 99.8% to 97.3%. Transformer MATD3 has a slightly lower disturbed cost of EUR 21 840 and retains 97.9% service, alongside fewer corrected decisions. The paired rows also expose the change in target accuracy. Their interpretation concerns an editorial hypothesis: preserving resource identity and availability information may help under selected disturbances. The service and intervention records below separate the operational outcomes underlying that response.

Table 6.28: **Illustrative values; invented.** Full 2024 test-window illustration. Paired Core-30 Normal and Robustness settings.

Scenario	Cost [EUR]	EV min. [%]	EV band [%]	Corrected [%]	Viol. [kWh]
RBC Smart Local					
Normal	21 600	99.8	92.0	4.0	0.0
Robustness	23 520	97.7	87.0	12.0	0.0
MADDPG					
Normal	20 880	98.6	84.0	14.0	0.0
Robustness	24 720	93.6	71.0	29.0	0.0
MATD3 (residual)					
Normal	19 680	99.8	96.0	2.3	0.0
Robustness	22 080	97.3	90.0	8.5	0.0
MAPPO					
Normal	20 640	99.5	91.0	5.5	0.0
Robustness	23 520	96.3	83.0	15.0	0.0
Transformer MATD3					
Normal	20 160	99.7	95.5	2.0	0.0
Robustness	21 840	97.9	92.0	6.5	0.0

Each pair retains the controller and underlying REC; the Robustness setting adds the declared disturbances. Rates cover the complete window.

Table 6.29 reads the same comparison through missed service. The paired rows retain the complete test window and the common invented departure cohort. This makes the change in the service rate visible as a change in the number and severity of missed obligations.

Illustrative reading. MATD3 moves from 20 to 270 failed departures in this example, and its accumulated minimum-service deficit rises from 34 to 702 kWh. Transformer MATD3 moves from 30 to 210, with a disturbed deficit of 504 kWh. These additional magnitudes describe the consequences of the service rates already shown, while the largest shortfall identifies a severe individual outcome that the annual percentage can obscure.

Intervention records explain how the system preserved operation. A smaller service loss may arise from the learned policy, local protection, or the supervisor's fallback. Table 6.30 keeps the corresponding observations beside the service results. Fault exposure, unavailable decisions, and fallback episodes are counted separately, because the same injected disturbance can be handled without a missed decision or can lead to a longer period of supervisory control.

Illustrative reading. Table 6.30 connects interruptions to their operational consequences. Residual MATD3 requires fallback for 9 487 building decisions, compared with 5 218 for Transformer MATD3, under the same 120 injected fault episodes. The corresponding appliance records retain both missed programmes and unserved energy. Recovery begins when the external fault clears and ends at the first usable decision; a departure or appliance deadline already missed remains a service failure.

Illustrative reading. DR adds another obligation in Premium-100 All-in. Residual MATD3 delivers

Table 6.29: **Illustrative values; invented.** Missed EV service over the full 2024 test-window illustration. Paired Core-30 Normal and Robustness settings.

Scenario	**Failed** [n]	**Eligible** [N]	**Deficit** [kWh]	**Largest** [kWh]
RBC Smart Local				
Normal	20	10 000	40	6
Robustness	230	10 000	690	14
MADDPG				
Normal	140	10 000	420	14
Robustness	640	10 000	2 304	22
MATD3 (residual)				
Normal	20	10 000	34	5
Robustness	270	10 000	702	13
MAPPO				
Normal	50	10 000	125	10
Robustness	370	10 000	1 147	18
Transformer MATD3				
Normal	30	10 000	54	6
Robustness	210	10 000	504	11

The invented cohort has 10 000 eligible departures and 200 additional unattainable requests in each setting; it is not a measured dataset count. Failed departures fall below the minimum service threshold. Deficit sums their battery-energy shortfall; Largest is the maximum individual shortfall. Appliance completion is illustrated in Table 6.30; DR is inactive.

Table 6.30: **Illustrative values; invented.** Runtime and service outcomes over the complete Core-30 Robustness year.

Record	RBC Smart Local	MATD3 residual	Transformer MATD3
Injected fault episodes	120	120	120
Eligible decisions	1 054 080	1 054 080	1 054 080
Unavailable decisions	1 138	9 487	5 218
Fallback episodes	95	791	435
Fallback [building-min]	17 070	142 305	78 270
Recovery median [min]	30	30	30
Recovery maximum [min]	120	120	120
Unrecovered episodes	0	0	0
Appliance requests	6 000	6 000	6 000
Appliance deadlines missed	72	50	50
Appliance energy unserved [kWh]	100.8	70.0	70.0

All counts, exposures and durations are invented aggregates. Unavailable decisions require fallback; duration sums the affected fifteen-minute building intervals. Recovery is measured after the external fault clears. Missed departures remain failures after recovery.

Table 6.31: **Illustrative values; invented.** Operational DR in Premium-100 All-in, complete 2024 illustration.

Configuration	Requested [kWh]	Delivered [kWh]	Shortfall [kWh]	Credit [EUR]
BAU without storage	12 000	0	12 000	0.00
BAU with storage	12 000	0	12 000	0.00
RBC Smart Local	12 000	8 376	3 624	1 005.12
MATD3 (residual)	12 000	10 104	1 896	1 212.48
Transformer MATD3	12 000	11 256	744	1 350.72

Invented operational cohort and EUR 0.12/kWh credit; BAU makes no requested response adjustment. Energy-purchase cost is reported before this separate settlement. Deliberately excessive stress requests are excluded from this operational cohort and its compliance denominator.

10 104 kWh of the invented 12 000 kWh operational request, leaving 1 896 kWh unmet and earning EUR 1 212.48. This settlement is kept outside the energy-purchase totals. An annual energy improvement and successful delivery of a requested response are therefore visible as separate outcomes.

Structural change tests a related but different assumption. A newly installed charger adds a command that the controller must generate and deliver. A missing charger measurement changes the information supporting an existing command. A new household changes the participating population. A flexible representation can accommodate some of these changes while the surrounding learning and execution components remain tied to the earlier arrangement. The current Transformer implementations make this boundary visible through their support for known local resource instances and their rebuild behaviour when the building population changes.

Attention and resource-wise outputs already support useful reuse across compatible resource counts. Their value remains part of the progression developed here. Continued operation also depends on the relationships around the network: the observation must describe the intended resource, the proposed action must use its current bounds, and execution must address the correct equipment. Maintaining those relationships together becomes the next question once variable-size processing is available.

6.10.5 The Remaining Research Problem

The four research dimensions established in Chapters 1 and 3 are connected by the same operational setting. A controller must remain computationally manageable, be practical to place in an installation, respect its resources and services, and retain useful behaviour under disturbance. The methods investigated here offer useful capabilities at each of those points. Sustaining them as the installation changes requires the policy, resource description, feasibility handling, and execution path to continue referring to the same control problem.

Return to the household with a stationary battery and two chargers. Adding a third charger of a familiar type requires an observation record, a command, and a place within the shared headroom. Reusing a trained resource encoder addresses part of that change. The action mapping and feasibility

calculation also need to recognise the new charger, while replay and model packaging need to retain a compatible interpretation. If its state measurement later fails, the affected decisions need an appropriate response without unnecessarily stopping the rest of the building.

The unresolved question is how learned coordination and its supporting mechanisms can jointly maintain performance with manageable computation and integration effort, admissible actions, and useful service under disturbance. Table 6.32 states that question through the four research dimensions. Its argument follows from the controller structures and operating assumptions; it does not depend on the invented numerical ranking.

Table 6.32: Method assumptions that motivate the next stage of the investigation. These are qualitative design questions, independent of the illustrative numerical rankings.

Dimension	**Progress developed here**	**Question that remains**
Scalability	Local execution, variable-size resource representations, and compact REC signals.	How can control quality be retained across sizes and compositions with proportionate computation, adaptation, and retraining?
Deployability	Exported actors with explicit inputs, configurations, and supporting components.	How can the evaluated controller be installed, maintained, and restored with reliable behaviour and manageable operator effort?
Safety	Service-aware objectives, constrained references, and local action protection.	How can learning, protection, and execution jointly preserve electrical limits and service, with their interventions made explicit?
Robustness	Disturbance scenarios, availability information, intervention records, and fallback.	How can the policy and its supporting mechanisms retain service under data, communication, and equipment disturbances and recover afterwards?

This focus leaves other important questions visible. The archive may omit uncommon services or faults; REC benefits may be distributed unevenly; communication and privacy conditions can restrict the training information available; users may revise their requirements or override an action. The broader synthesis in Chapter 3 places these concerns within the field. They continue to shape the circumstances in which the methods developed here can be useful.

The next chapter develops TI-MARL as an integrated response to these real-world control requirements. Its representation, learning, local protection, fault response, and deployment contract are assessed together against the algorithmic references examined here. The comparison follows energy and service outcomes through the four research dimensions, with matched studies attributing the contribution of individual mechanisms.

6.10.6 Chapter Summary

The investigation progresses from two BAU references and explicit scheduling to local and multi-agent learning. Controller improvements refine the information, experience, and service priorities guiding a decision. Offline learning uses recorded operation, hierarchy separates REC and building responsibilities, and variable-size policies accommodate compatible resource changes.

The annual comparison begins with conventional operation in each REC scenario. Storage-enabled BAU then provides a common reference for the controllers, whose development uses 2023 and whose final comparison covers the reserved 2024 sequence. The illustrative tables develop the discussion around service, MATD3's nominal performance, hierarchical grid shaping, and the cost and flexibility of resource-based representations. The real Core-30 short-window BAU diagnostic retains its separate record in the appendix. The seven-scenario comparison retains annual cost, service, intervention, and supervisory continuity for each controller; the appendix supplies the complete numerical scorecards. The resulting research problem concerns how scalability, deployability, safety, and robustness can be supported together as the installation evolves. Chapter 7 develops that problem through a consistent description of the resources, information, and actions involved in control.

Chapter 7

Typed-Interface Multi-Agent Reinforcement Learning: TI-MARL

Chapter 6 examined how control and learning methods contribute to the practical use of MARL in RECs. The remaining problem concerns their combined performance: sustaining useful coordination with manageable computation and integration effort, respecting physical and service constraints, and retaining service under disturbance. This chapter develops TI-MARL as an integrated response to the scalability, deployability, safety, and robustness requirements defined in Chapter 1.

The explanation follows a household controller into operation and through changes affecting its resources and measurements. Another charger is installed, a required measurement becomes unavailable, and another building joins the REC. These situations connect representation and learning to constraint handling, fault response, and deployment. The chapter develops each mechanism before examining their relationship to the algorithmic approaches reviewed in Chapter 3 and the common comparison in Chapter 6.

7.1 Problem Formulation and Method Overview

A useful controller must preserve the relationship between the policy evaluated in development and the decisions applied in an installation. Its inputs must describe the intended resources; its commands must respect their electrical and service conditions; and the loss of information or equipment must leave an explicit path for continued operation or fallback. These responsibilities exist in the original installation and persist as its composition and operating conditions change.

TI-MARL connects them through a common typed description used by the learner, local constraint handling, health-dependent action availability, compatibility checks, and execution records. Shared processing and population-aware value estimation support reuse across compatible resources. The method's scientific question is whether these mechanisms together improve continued control across the four research dimensions. An interface whose size can change is one representational capability within that method; the assessment concerns the complete control process.

The neighbouring methods supply substantive algorithmic references. Task-open and compositional learning address coordination and reuse; graph-based energy policies relate resource structure to decisions; safety layers and shielding constrain actions; fault-tolerant training prepares policies for adverse conditions (Anil et al. 2025; Dalal et al. 2018; Luketina et al. 2025; Mguni et al. 2026; Orfanoudakis, Panda, et al. 2026). Their mechanisms and supporting processes enter the comparison through the same energy, service, computation, and operational requirements.

7.1.1 Typed-Interface Decision Processes

The one-agent reduction isolates interface reconfiguration from cross-agent learning. A Typed-Interface Markov Decision Process (TI-MDP) is defined by

$$\mathscr{M}^{\mathrm{TI}} = \left\langle \mathscr{S}, \Sigma, \mathscr{H}, \mathscr{C}, \Gamma^{\mathrm{loc}}, P, P_{\Sigma}, R, \gamma \right\rangle, \tag{7.1}$$

where $\mathscr{S}$ is the physical and operational state space; Σ and $\mathscr{H}$ are the families of capability schemas and health states; $\mathscr{C}$ is the Typed Interface Compiler (TIC); and Γ^{loc} describes local resource relations and constraints. The kernel P governs physical transitions, while P_{Σ} governs changes to the interface state, including topology and health. The reward function is R, and $\gamma \in [0,1)$ discounts future rewards. Their time-indexed instances, such as Σ_t and $\mathscr{H}_t$, describe the current interface. Learning in this one-agent model is denoted Typed-Interface Reinforcement Learning (TI-RL).

A Typed-Interface Markov Game (TIMG) adds an active agent population, local information boundaries, coupled outcomes, and shared resources:

$$\mathscr{G}^{\mathrm{TI}} = \left\langle \mathscr{S}, \mathscr{N}, \Sigma, \mathscr{H}, \mathscr{C}, \Gamma, P, P_{\Sigma}, \{R_i\}_{i \in \mathscr{N}}, \gamma \right\rangle. \tag{7.2}$$

Here $\mathscr{N}$ is the population of possible agent identities, Γ includes local and shared relations, and R_i is the reward function of agent i; the remaining objects retain their preceding roles. At time t, $\mathscr{N}_t \subseteq \mathscr{N}$ denotes the active population. Each agent i has a schema $\Sigma_{i,t}$ and health state $\mathscr{H}_{i,t}$. The physical state is augmented with the active population and interface state:

$$\tilde{s}_t = \left(s_t, \mathscr{N}_t, \{\Sigma_{i,t}, \mathscr{H}_{i,t}\}_{i \in \mathscr{N}_t}, \Gamma_t\right). \tag{7.3}$$

Here s_t is the physical state, and $\tilde{s}_t$ retains the active members, their schemas and health states, and the current relations Γ_t. This is an augmented modelling state; each actor observes only its authorised view. The compiler produces the policy-facing interface of agent i:

$$\mathscr{I}_{i,t} = \mathscr{C}\left(\Sigma_{i,t}, \mathscr{H}_{i,t}, \mathrm{view}_i(\tilde{s}_t)\right) = \left(\mathscr{V}_{i,t}, \mathscr{E}_{i,t}, \mathscr{G}^{a}_{i,t}, \mathscr{P}_{i,t}, \mathscr{B}_{i,t}\right), \tag{7.4}$$

where $\mathscr{V}_{i,t}$ and $\mathscr{E}_{i,t}$ are the typed interface nodes and relations, $\mathscr{G}^{a}_{i,t}$ is the variable set of active action-group instances, $\mathscr{P}_{i,t}$ is the set of valid action-port instances, and $\mathscr{B}_{i,t}$ contains the current parameter domains and local hard constraints. The view function enforces the information boundary:

local actors receive their local interface and explicitly authorised aggregate context, not the private observations of other agents.

The policy produces one decision for each active action-group instance:

$$a_{i,t} = \{a_{i,g,t}\}_{g \in \mathscr{G}^a_{i,t}}, \qquad a_{i,g,t} = (p_{i,g,t}, \eta_{i,g,t}), \tag{7.5}$$

where $p_{i,g,t}$ is a selected valid port and $\eta_{i,g,t}$ is its parameter vector. The local action is a variable structured bundle whose contents follow the active interface. A constraint-aware construction Φ_i maps raw decoded group decisions into a locally feasible bundle:

$$\bar{a}_{i,t} = \Phi_i\left(a_{i,t}, \Gamma^{\mathrm{loc}}_{i,t}\right), \qquad \bar{a}_{i,t} \in \mathscr{A}^{\mathrm{loc}}_{i,t}. \tag{7.6}$$

Here Φ_i is the local construction rule, $\bar{a}_{i,t}$ is its output, and $\mathscr{A}^{\mathrm{loc}}_{i,t}$ is the feasible set defined by the current local constraints. The multi-agent joint action is

$$\bar{\mathbf{a}}_t = (\bar{a}_{i,t})_{i \in \mathscr{N}_t}. \tag{7.7}$$

The bold symbol $\bar{\mathbf{a}}_t$ collects the constructed bundles of the active agents. Each bundle is encoded into resource commands, with execution feedback retaining the action actually applied. The physical transition is

$$s_{t+1} \sim P(\cdot \mid s_t, \bar{\mathbf{a}}_t). \tag{7.8}$$

The transition kernel P gives the distribution of the next physical state s_{t+1} conditional on s_t and the joint constructed action. Local and REC outcomes are coupled through this transition. A representative reward decomposition is

$$r_{i,t} = r^{\mathrm{loc}}_{i,t} + \alpha_i r^{\mathrm{com}}_t - \beta_i c^{\mathrm{loc}}_{i,t} - \eta_i c^{\mathrm{shared}}_t, \tag{7.9}$$

Here $r_{i,t}$ is the reward of agent i, $r^{\mathrm{loc}}_{i,t}$ measures its local outcome, and r^{com}_t measures the REC outcome. The non-negative costs $c^{\mathrm{loc}}_{i,t}$ and c^{shared}_t represent declared local and shared constraint penalties. The coefficients α_i, β_i, and η_i set their relative weights. Local terms can include energy cost and service; REC terms can include peak, ramping, renewable utilisation, or a demand-response objective. When $|\mathscr{N}_t| = 1$ and Γ_t contains only local relations, the model reduces to TI-RL.

The energy formulation uses one building controller per agent. Chargers, batteries, meters, and service sessions are entities within that boundary. Admission concerns new instances or compositions of recognised semantic types. Introducing an unfamiliar physical capability requires its type, interpretation, and control support to be defined before it can participate.

The reduction also clarifies the experimental sequence. Typed-Interface Proximal Policy Optimisation (TI-PPO) tests whether the typed local interface and its reconfiguration can be learned without central training. Typed-Interface Multi-Agent Proximal Policy Optimisation (TI-MAPPO) keeps the same actor and execution path but uses the active population during value estimation.

Differences between the two can therefore be attributed to centralised training rather than to a different deployed controller.

7.1.2 Method Overview

Figure 7.1 separates the runtime path from the training path. Each actor evaluation receives one compiled local interface and authorised aggregate signals. It does not receive the raw private state of another member. TI-MAPPO's critic pools the active agent set during training; TI-PPO uses only the local interface. The actor, compiler, projector, and command codec are otherwise the same.

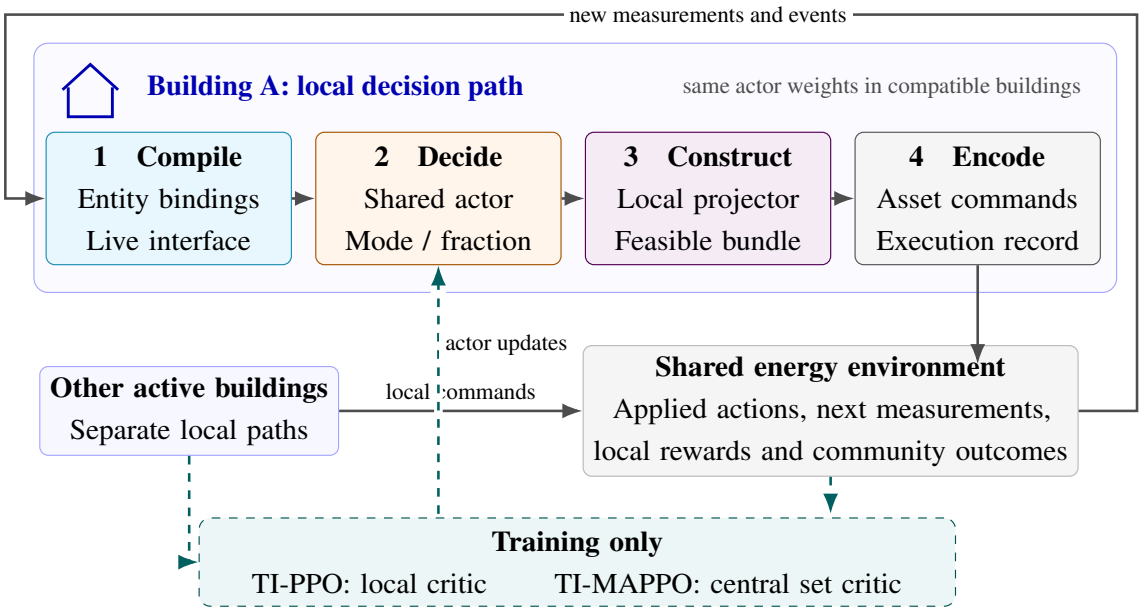


Figure 7.1: TI-MARL decision path within each building. Local construction precedes execution; the REC critic participates only in training.

Table 7.1 identifies the role of each component. The same interface snapshot accompanies the decision through these stages, so a charger retains its identity when its observations become actor inputs and its selected power becomes an equipment command.

The architecture is deliberately asymmetric between training and operation. The actor remains local in both phases. Centralisation is introduced only in TI-MAPPO's value function and the experiment infrastructure, where it can improve credit assignment without creating a runtime dependency on unrestricted global state. REC observations remain typed observation parts with explicit scope, provenance, time, and health.

Table 7.1: Main TI-MARL components and their interpretation in the running energy example.

Component	Generic role	Energy example
TIC	Binds deployment-neutral member contracts to current entities, observations, events, health, groups, bounds, and constraints	Adds a known charger group when it becomes active; scopes failure to the affected group; contracts V2G when its evidence is stale
Hierarchical typed encoder	Aggregates observations by channel and sensor before forming a local latent state	Keeps two charger sessions distinct through semantic and structural identities without learning an embedding for a concrete charger identifier
Shared hybrid actor	Produces one categorical mode and one bounded continuous fraction for each active group	Controls two chargers, one battery, and one deferrable service with shared group-type heads rather than an asset-specific output head
Analytic local projector	Applies validity, dynamic bounds, service floors, and joint total/per-phase headroom	Preserves an urgent EV or mandatory deferrable start while scaling competing charging decisions to the available connection capacity
Local or central set critic	Supplies TI-PPO local values or TI-MAPPO values conditioned on the active member set	Retains the same decentralised actor while comparing independent and REC-aware value estimation

7.2 Typed Interfaces and Runtime Compilation

The interface provides a stable vocabulary for a changing installation. A schema declares what a resource means and which capabilities it can expose; compilation binds that declaration to the resources and evidence present at the current step. This separates a new instance of a familiar charger from a new kind of equipment requiring additional control semantics.

7.2.1 Schemas, Entities, and Dynamic Module Instances

A capability schema describes the potential interface of an agent. It identifies the agent role, module types, observation templates, action-group templates, constraints, compatibility requirements, and fallback semantics. Runtime state identifies which module instances and controlled entities are active now. Separating the two prevents fast-changing values such as SoC or connection state from being treated as schema revisions.

A building can contain several instances of the same module type. In the running example, agent A contains two charger instances, one stationary battery, one deferrable-service module, and one main meter:

$$\mathscr{M}_{A,t} = \{m_1^{\text{meter}}, m_1^{\text{charger}}, m_2^{\text{charger}}, m_1^{\text{battery}}, m_1^{\text{def}}\}. \tag{7.10}$$

Here $\mathscr{M}_{A,t}$ is the module set of building A at time t; the superscripts identify module types and the subscripts distinguish their instances. Each charger can be related to a currently connected EV session. The following observation identifies the SoC source for one such session:

```
semantic type: EV_SOC
entity: ev_session_17
source module: charger_1
```

The source and session identities distinguish this measurement from the SoC of the vehicle connected to `charger_2`. The corresponding action group is similarly bound:

```
group type: EV_CHARGER_MODE
owner module: charger_1
target entity: ev_session_17
```

The compiler represents these relationships as typed edges such as `owns`, `connected_to`, `observes`, `controls`, and `requires`. Entity binding is encoded in the interface semantics and does not depend on position in a flat vector.

When `charger_3` is installed, a module-addition event creates a new module node, its observation parts, one EV-charger action-group instance, valid port instances, and the relevant constraint edges. The neural network is not extended with a new permanent head. The same group and port decoder used for the existing chargers is applied to the new instance. Removing the charger removes that subgraph. A semantically unknown module is admitted only when a compatible parent type and decoder are available; otherwise its learned control path remains inactive and fallback semantics apply.

The implementation deliberately separates four identifiers that flat action layouts often conflate. A member owns the local controller boundary; a charger identifies installed infrastructure; an EV identifies a vehicle; and a session records the temporary occupancy of one charger by one EV. A domestic charger can retain the same vehicle over many sessions, whereas one office charger can serve several vehicles in one day. The action group belongs to the physical charger, while service observations belong to the currently bound session. Session replacement therefore rebinds runtime values without rebuilding an unchanged charger structure.

7.2.2 Typed Observation Parts and Community Context

An observation part is a typed object with an identity and declared semantics. A representative definition is

$$o_k = (\tau_k, \mathrm{id}_k, \mathrm{entity}_k, \mathrm{source}_k, x_k, u_k, q_k, h_k, t_k, s_k), \tag{7.11}$$

where k indexes an observation part, id_k identifies that part, entity_k identifies the entity described, and source_k identifies its source. The remaining entries are τ_k, the semantic type, x_k the value, u_k

the unit, q_k quality or uncertainty metadata, h_k the health state, t_k the timestamp, and s_k the scope. The scope can be self, local, neighbourhood, REC, or global. The source can be a physical sensor, derived signal, forecast, external platform, or coordinator message.

REC information is represented through the same object model. A building may receive numerical aggregates such as REC net demand, transformer loading, remaining headroom, PV surplus, a price multiplier, or a demand-response target. These values do not reveal the raw private state of another building. Their provenance and health remain explicit:

```
semantic type: COMMUNITY_TRANSFORMER_LOADING
scope: community; source: platform_service
unit: percent; health: HEALTHY
```

In the energy instantiation, an EV connected from 08:00 until 18:00 with a current SoC, target SoC, maximum charging power, and departure requirement already exposes the information from which a policy can learn temporal flexibility. Useful derived quantities such as time to departure, remaining required energy, or required average charging power can be supplied as typed observation parts when they improve learning or interpretability. Their inclusion is fixed by the experimental configuration.

Table 7.2 lists representative observation parts for the running example.

Table 7.2: Representative typed observation parts for a building with two chargers. Values are illustrative.

Semantic type	Entity/source	Value	Scope	Decision meaning
House net power	main meter	2.1 kW	self	Current aggregate import used to evaluate local electrical limits
EV SoC	session 17 / charger 1	0.35	local	Energy state of the EV controlled through charger 1
EV target SoC	session 17	0.80	local	Minimum service target at departure
Departure time	session 17	10 h	local	Temporal slack available to the charging policy
EV SoC	session 28 / charger 2	0.62	local	Distinct SoC bound to charger 2 and its connected session
Battery SoC	battery 1	0.70	local	Available storage state
REC headroom	platform service	5.5 kW	REC	Aggregate remaining shared capacity made visible to all authorised agents
REC peak state	platform service	high	REC	REC context used by local actors and the reward

7.2.3 Dynamic Action Groups and Typed Parameterised Ports

The output interface is organised around action-group instances. Each controllable module creates one or more groups whose ports are mutually exclusive within the group but can be combined with ports from other groups. For example,

$$\mathscr{P}_{\text{charger}_1,t} = \{\text{IDLE-EV}, \text{CHARGE-EV}(p), \text{DISCHARGE-EV}(p)\}, \tag{7.12}$$

$$\mathscr{P}_{\text{battery}_1,t} = \{\text{IDLE-BATTERY}, \text{CHARGE-BATTERY}(p), \text{DISCHARGE-BATTERY}(p)\}, \tag{7.13}$$

$$\mathscr{P}_{\text{def}_1,t} = \{\text{IDLE}, \text{START}\}. \tag{7.14}$$

Each set $\mathscr{P}$ lists the ports of the named resource at time t. In these examples, the argument p denotes a requested physical power; the deferrable service has a discrete start decision. An action port itself is a typed callable capability:

$$p = \left(\tau_p, \text{id}_p, \text{group}_p, \text{owner}_p, \Theta_p, U_p, B_{p,t}, V_{p,t}, D_p, K_p\right), \tag{7.15}$$

where τ_p is the port type, id_p its identity, group_p the action group to which it belongs, and owner_p its owning module. The remaining entries are Θ_p, the parameter schema, U_p the units, $B_{p,t}$ the current parameter domain, $V_{p,t}$ the validity state, D_p the observation and capability dependencies, and K_p the resource effects used by local constraints, critics, and optional safety checks.

A new charger creates another instance of the charger action group. It does not create another set of network parameters. The implemented actor emits a categorical mode and a Beta-distributed fraction in $[0,1]$ for every active group. The fraction is relative to the currently compiled physical upper bound and is converted to a signed simulator command only once by the codec. This convention avoids applying a health-dependent or topology-dependent bound twice. A deferrable start is indivisible: after START is selected, its typed cycle profile determines the subsequent demand rather than a continuous power fraction.

Table 7.3: Representative action ports for two charger instances and one stationary battery.

Port	**Owner/target**	**Parameter**	**Current validity and bounds**	**Dependencies and local effects**
Charge EV	charger 1 / session 17	power [kW]	valid, $[0, 7.2]$	connected session; increases house import
Discharge EV	charger 1 / session 17	power [kW]	valid, $[0, 4.0]$	V2G, trustworthy SoC, reserve; reduces house import
Charge EV	charger 2 / session 28	power [kW]	valid, $[0, 11.0]$	distinct connection and EV state; increases house import
Discharge EV	charger 2 / session 28	power [kW]	absent	charger 2 has no V2G capability
Discharge battery	battery 1	power [kW]	valid, $[0, 3.0]$	battery SoC and reserve; reduces house import

For example, suppose `charger_1` is rated at 7.4 kW but the current connection and EV state reduce its compiled charging bound to 4.0 kW. A sampled fraction of 0.75 requests 3.0 kW. The network output remains in the same numerical domain when the available bound later changes; the interface gives the output its current physical meaning. The trace stores the selected mode, fraction, compiled bound, requested command, projected command, and applied simulator value separately.

7.3 Policy Architecture and Multi-Agent Learning

Once the interface has been compiled, the actor must interpret its current observations and produce a decision for each controllable group. Shared processing allows another known charger to reuse the existing policy. Centralised training can then account for the interaction between buildings while preserving the local information available during operation.

7.3.1 Local Information and Hierarchical Encoding

Each agent executes a logically local Typed-Interface Policy (TIP). A building actor receives its compiled interface, optional memory, and authorised aggregate REC observations. It does not receive the raw loads, EV states, or private service requirements of other buildings. This boundary is identical during evaluation and deployment; privileged information used by a central critic during training is not silently passed to the actor.

The current implementation shares one actor across all compatible members:

$$\pi_i = \pi_\theta \left(\mathscr{I}_{i,t}, m_{i,t} \right), \tag{7.16}$$

Here π_i is the policy instance of agent i, π_θ is the shared actor with parameters θ, $\mathscr{I}_{i,t}$ is its local snapshot, and $m_{i,t}$ is optional local memory. Parameter sharing preserves separate observations. House A and House B execute the same weights on different local snapshots, and member role remains an encoded feature. The compiler rejects unknown semantic or action-group types rather than silently allocating a random representation. This keeps zero-shot admission limited to new instances and compositions of known contracts.

This structure supports population changes without a permanent actor slot per agent. A new building instantiates the appropriate local policy family after schema compatibility has been checked. Removing a building removes its actor instance from the active population; the shared parameter set and the actors of the remaining agents are unchanged.

The compiled local interface is encoded as variable-cardinality typed sets. The hierarchy follows the simulator's measurement structure: observation parts are grouped into channels, channels into sensors, and sensors into the local member representation. Structural relations remain available when an action group asks for its relevant observations. This hierarchy follows the implemented measurement structure and preserves the binding needed by the decoder.

Each observation token combines its normalised value with quality and validity indicators, health, semantic type, sensor type, channel type, unit, scope, declared use, a deterministic semantic fingerprint, and temporal-horizon features. The first mapping is

$$z_k^o = E_\theta^o \left(x_k, q_k, h_k, e_{\tau_k}, e_{\text{sensor}}, e_{\text{channel}}, e_{u_k}, e_{s_k}, e_{\text{use}}, f_k^{\text{id}}, f_k^{\text{time}} \right). \tag{7.17}$$

Here z_k^o is the embedding of observation part k, and E_θ^o is its learned encoder. The values x_k, q_k, and h_k retain the meanings in Equation 7.11; each e is an embedding of the named metadata field. The features f_k^{id} and f_k^{time} encode semantic identity and temporal information. Permutation-equivariant relational layers aggregate observation tokens within their channels and sensors, followed by learned pooling over the local sensor set:

$$z_{i,t} = F_\theta \left(\left\{ \left\{ \left\{ z_k^o \right\}_{k \in c} \right\}_{c \in s} \right\}_{s \in i} \right). \tag{7.18}$$

The mapping F_θ aggregates the nested sets: k indexes observation parts, c channels, and s sensors belonging to member i. The common latent $z_{i,t}$ summarises the complete local decision context. Concrete asset identifiers are not learned as permanent embeddings. Instance distinction is carried by the compiled structural binding and deterministic observation identity, while semantic families determine reusable parameters. Reordering observations, chargers, or members therefore does not change the intended meaning.

Each active action group starts from its type embedding and the local latent. In the action-conditioned variant, the group also queries observation tokens related to its owner, target entity, and dependencies. The two chargers can use the same mode and fraction heads while attending to different sessions. Attention across the active group set then exposes local competition before decoding. Adding a charger adds one group evaluation; it does not change the actor or critic parameter count.

During replay and BC updates, immutable type indices and set membership are prepared once and reused across optimisation epochs. Runtime values and health are never cached as static input. Packed evaluation removes repeated per-agent device transfers while preserving the same set semantics. These changes affect throughput, not the mathematical information surface.

7.3.2 Dynamic Grouped Action Decoding

The actor does not contain a permanent neural head for each asset instance. A group-type head is applied to every active group of that type. For group g and candidate mode m,

$$\ell_{i,g,m,t} = f_\theta^{\text{select}} \left(z_{i,t}, z_{i,g,t}, e_{\tau_g} \right), \tag{7.19}$$

Here $\ell_{i,g,m,t}$ is the logit for candidate mode m of group g, $z_{i,g,t}$ is its contextual embedding, e_{τ_g} represents its type, and f_θ^{select} is the shared selection head. The categorical mode is selected after

invalid modes have been masked:

$$\Pr\left(m_{i,g,t} = m\right) = \mathrm{Softmax}_{m \in \mathscr{M}_{i,g,t}^{\mathrm{valid}}}\left(\ell_{i,g,m,t}\right). \tag{7.20}$$

Here $m_{i,g,t}$ is the selected mode, $\mathscr{M}_{i,g,t}^{\mathrm{valid}}$ is its admissible candidate set, and Softmax normalises the corresponding logits into probabilities. A second head generates positive Beta parameters. Sampling $u_{i,g,t} \sim \mathrm{Beta}(\alpha_{i,g,t}, \beta_{i,g,t})$ gives the bounded fraction

$$\eta_{i,g,t} = u_{i,g,t}, \qquad (\alpha_{i,g,t}, \beta_{i,g,t}) = d_{\tau_g}(z_{i,g,t}). \tag{7.21}$$

The decoder d_{τ_g} produces positive shape parameters $\alpha_{i,g,t}$ and $\beta_{i,g,t}$ for the Beta distribution; its sample $u_{i,g,t}$ supplies the fraction $\eta_{i,g,t}$. The log probability used by PPO is the sum of the categorical and Beta terms. At training time the two parts are sampled. Deterministic replay can use categorical argmax or an `expected_signed` rule that combines charge and discharge probabilities, the Beta mean, a configurable gain, a deadband, and a non-idle logit margin. These controls were added because an apparently small stochastic preference can become an implausibly persistent deterministic charge or discharge command after export.

All groups are decoded as one bundle from the same local context. A representative raw bundle is

$$a_{A,t}^{\mathrm{raw}} = \left\{ \begin{array}{l} \texttt{charger_1} : \textsc{charge-ev}(4.0\ \mathrm{kW}), \\ \texttt{charger_2} : \textsc{charge-ev}(3.5\ \mathrm{kW}), \\ \texttt{battery_1} : \textsc{discharge-battery}(1.0\ \mathrm{kW}), \\ \texttt{deferrable_1} : \textsc{start} \end{array} \right\}. \tag{7.22}$$

Here $a_{A,t}^{\mathrm{raw}}$ collects the four proposed decisions for building A at time t; their powers are illustrative. The bundle-level representation allows the actor to learn priorities among the two EV sessions, battery use, a pending service, and REC conditions. It does not by itself guarantee that the combined power remains within local electrical limits or that an urgent service remains feasible. Those requirements are handled by the final local construction.

7.3.3 Centralised Training and Local Execution

TI-MARL does not require a REC controller agent. Each building acts independently from its own compiled interface and authorised REC observations. The method becomes multi-agent because the local bundles form one joint action, the physical transition depends on that joint action, local actions compete for shared resources, and rewards include local and REC consequences.

At each training step, the constructed local bundles form the joint action in Equation 7.7. Its transition produces the next interfaces, local rewards, and REC outcomes. Simultaneous charging can reduce shared headroom even when both buildings satisfy their own connection limits. Repeated interaction supplies the experience from which local actors learn those collective consequences.

Joint training means that actor updates are derived from the same joint transition. It does not mean that actors share private runtime inputs or that one global actor replaces them. Stable member identities govern the rollout when population changes. A joining member has no predecessor in Generalised Advantage Estimation; a departing member receives a terminal boundary; members that remain can bootstrap across the topology event. The rollout is not cleared and the networks are not resized.

A conventional central critic often concatenates all agent observations. That representation is tied to a fixed order and population. TI-PPO avoids centralisation and estimates V_i from the local typed snapshot. TI-MAPPO retains the same actor but estimates values from the variable active set.

The implemented central critic encodes each member with the same typed snapshot encoder, mean-pools the active local latents into a REC summary, and evaluates each stable member identity from its local and REC representations:

$$V_{\phi,i,t} = g_\phi \left(z_{i,t}, \frac{1}{|\mathcal{N}_t|} \sum_{j \in \mathcal{N}_t} z_{j,t} \right). \tag{7.23}$$

Here $V_{\phi,i,t}$ is the value estimate for member i, g_ϕ is the learned value mapping, and $z_{j,t}$ is the encoded snapshot of active member j. The mean contains all $|\mathcal{N}_t|$ active members. The mapping is permutation equivariant: reordering members reorders their values but does not alter their association. Cardinality can change without reconstructing a joint input layer. Mean pooling supplies a compact REC context. Distinct feeder neighbourhoods with identical aggregate summaries receive the same pooled representation. Representing those neighbourhoods explicitly would require a richer critic input or aggregation rule.

7.3.4 Proximal Policy Optimisation with Typed Interfaces

TI-MARL defines the interface, actor structure, local bundle semantics, and population handling. The current learner supports two controlled PPO variants.

TI-PPO. The shared actor and a parameter-shared local critic are updated from local member trajectories. This is the typed one-agent learner applied independently across the active population. It isolates the effect of the interface and actor from centralised value estimation.

TI-MAPPO. The actor is paired with the central set critic in Equation (7.23). The critic may use the active joint training context; the actor retains its local information boundary. A checkpoint pins the backbone and critic kind, so a TI-PPO state cannot be loaded into TI-MAPPO by accident.

For a transition generated by old parameters θ_{old}, the hybrid group probability ratio is

$$r_{i,g,t}(\theta) = \exp\left[\log \pi_\theta(m_{i,g,t}, \eta_{i,g,t} \mid \mathscr{I}_{i,t}) - \log \pi_{\theta_{\text{old}}}(m_{i,g,t}, \eta_{i,g,t} \mid \mathscr{I}_{i,t})\right]. \tag{7.24}$$

Here $r_{i,g,t}(\theta)$ compares the current and collection-policy probabilities for the same recorded mode, fraction, and interface. The parameters θ_{old} identify the collection policy. With joint-agent credit, the clipped PPO objective uses the sum of group log probabilities for that member. With typed-group credit, the ratio and advantage remain attached to group g:

$$L^{\text{group}}(\theta) = \mathbb{E}_{i,g,t}\left[\min\left(r_{i,g,t}(\theta)\hat{A}_{i,g,t}, \text{clip}(r_{i,g,t}(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_{i,g,t}\right)\right]. \tag{7.25}$$

Here $\hat{A}_{i,g,t}$ is the advantage estimate assigned to group g, ε is the clipping tolerance, and the expectation averages recorded agent–group transitions. The operator clip bounds the ratio to $[1-\varepsilon, 1+\varepsilon]$. The group critic supplies a matching baseline. EV service and charger-constraint consequences are routed to the responsible charger family; stationary-storage consequences reach the storage group; deferrable completion reaches its start group. Economic settlement and genuinely shared grid effects remain common. This avoids teaching the second charger that it caused a service failure belonging to the first, while preserving REC coupling.

The comparison with MATD3 in Chapter 6 retains a practical reference from a different learning family. Attribution to the interface is assessed more directly through PPO-based controls with matched actor inputs, training budgets, and protection.

7.3.5 Training Data and Time Resolution

Moving the controller to 15-minute data increased the effective learning horizon and exposed numerical failures that were less visible in the earlier hourly experiments. TI-MARL therefore treats the physical duration of a transition as part of the learner contract. When γ_{ref} and λ_{ref} are declared for reference duration Δt_{ref}, the factors used at runtime are

$$\gamma_{\Delta t} = \gamma_{\text{ref}}^{\Delta t/\Delta t_{\text{ref}}}, \qquad \lambda_{\Delta t} = \lambda_{\text{ref}}^{\Delta t/\Delta t_{\text{ref}}}. \tag{7.26}$$

Here Δt is the transition duration and Δt_{ref} the reference duration. The factors γ and λ control reward discounting and Generalised Advantage Estimation respectively, preserving their declared time basis as the step changes. The implementation normalises critic targets, uses a Huber value loss by default, clips policy ratios and gradient norms, and records approximate Kullback–Leibler (KL) divergence, clipping fraction, explained variance, and finite-gradient checks. Advantages can be normalised globally or per stable member identity. Entropy coefficients can differ by action-group family; a charger and a deferrable start do not need the same exploration pressure. A policy anchor can constrain movement from a selected actor and can be reset deliberately when a checkpoint is resumed.

The rollout is on-policy and retains the raw typed decision, projected decision, intervention scope, stable agent and group identities, reward decomposition, termination semantics, current and

next snapshot hashes, and simulator execution feedback. Teacher actions and auxiliary-planning examples never enter that PPO rollout. This boundary matters: an update remains attributable to experience generated by the current policy even when demonstrations are used elsewhere in training.

The use of prior knowledge in Section 6.5.4 motivates an optional BC warm start with RBC Smart Local (the implementation class `RBCSmartPolicy`). The teacher acts on the physical simulator interface; its requested commands are decoded back into the active typed groups and stored with the exact interface snapshot on which they were valid. Pretraining updates the actor only. The critic is not trained from teacher returns, and the complete demonstration snapshots are discarded after pretraining.

Idle actions dominate many annual trajectories. A plain average cross-entropy loss can therefore learn an impressive cloning accuracy while missing the charge, discharge, V2G, or start events that make the teacher useful. The implementation supports bounded inverse-frequency weighting and a hierarchical mode mean. It can run a final unweighted calibration phase so that balancing rare actions does not permanently distort the deployed prior.

Two optional causal auxiliaries were added after the warm-start experiments exposed sparse service decisions. The EV planner labels typed states using only current service variables and declared forecasts: urgent charging, price-aware delay, authorised V2G with departure reserve, or idle. The storage planner identifies PV capture, low-price charging, high-price discharge, and idle from the same deployment-visible surface. Each auxiliary predicts both mode and fraction, trains the actor only, and keeps a bounded reservoir stratified by decision reason. Rare urgent or arbitrage examples therefore survive without contaminating the PPO rollout.

These planners supply training labels; runtime decisions remain with the actor and analytic projector. Pure TI-MAPPO, BC warm start, EV auxiliary, storage auxiliary, and their combinations remain separate ablations. This separation makes the contribution of prior knowledge testable: prior knowledge is attached to a typed and causal state, while the learned actor still owns execution.

7.4 Constraint Handling and Degraded Operation

The compiled interface serves two connected purposes in protection. It determines which decisions remain meaningful under the current evidence, then supplies the physical and service constraints used to construct their joint local effect. A charger fault and a building power limit therefore enter different stages of the same decision path.

7.4.1 Health-Aware Causal Compilation

Health is part of the interface semantics and is also available to the neural network. The implemented states are HEALTHY, DEGRADED, STALE, MISSING, FAILED, and UNKNOWN. Fault evidence remains a separate record. A simulator event such as `dropout` or `stuck` is not mapped to

one universal health label; the compiler considers event duration, freshness, criticality, declared availability, channel semantics, and recovery hysteresis.

A charger failure disables only the associated charger group and its ports. Other modules controlled by the member remain active. A stale EV SoC can preserve charging while blocking V2G because the discharge reserve can no longer be established. Failure of the main house meter has a wider consequence: the complete member is isolated when aggregate import or phase headroom cannot be verified. Loss of a REC signal removes that coordination input but leaves the local controller available. During a grid outage, charging, V2G, and a new deferrable start are blocked; safe stationary-storage discharge may remain available when its local dependencies still hold.

The compiler computes the transitive effect of a health event through a dependency closure:

$$\mathscr{H}^{\star}_{i,t} = \mathrm{Closure}\left(\mathscr{H}_{i,t}, D_{i,t}\right), \tag{7.27}$$

where $D_{i,t}$ is the declared dependency relation, Closure propagates the observed health state $\mathscr{H}_{i,t}$ through it, and $\mathscr{H}^{\star}_{i,t}$ records the resulting consequences. These consequences concern observations, ports, groups, bounds, local constraints, and operational modes. For the EV SoC example, the closure can be summarised as

$$\mathscr{P}^{\mathrm{valid}}_{i,t} \leftarrow \mathscr{P}^{\mathrm{valid}}_{i,t} \setminus \{p : \tau_p = \textsc{discharge-ev},\ \mathrm{owner}_p = \texttt{charger_1}\}. \tag{7.28}$$

Here $\mathscr{P}^{\mathrm{valid}}_{i,t}$ is the valid port set of member i, τ_p is a port's type, and owner_p identifies its equipment. Set subtraction removes only the discharge capability of the affected charger. Charging remains available only if its own dependencies and service constraints can still be established.

The dependency rule changes the decision interface itself, giving the policy an explicit account of the capabilities still available. Recovery is explicit: a group or port returns only after its declared fresh-data and hysteresis conditions hold. The closure log records the subject, state, rule, consequence, and duration used in that decision.

The compiler rebuilds structural plans only when topology changes. Values, health evidence, session identity, validity, and bounds are refreshed at every step without treating routine runtime churn as a new neural architecture. This separation is important in the 15-second stress case, where value and session updates occur far more frequently than asset installation or removal.

7.4.2 Constraint-Aware Local Bundle Construction

Local hard constraints should not depend exclusively on a negative reward. An actor can learn to reduce violations, but safety-critical electrical and service limits require explicit semantics. TI-MARL keeps the learned preference in the raw bundle and applies a constraint-aware parameterisation to obtain the final local bundle.

Let $a^{\mathrm{raw}}_{i,t}$ denote the decoded bundle and $\mathscr{A}^{\mathrm{loc}}_{i,t}$ the feasible set induced by the local typed graph. The construction is

$$\bar{a}_{i,t} = \Phi_i\left(a_{i,t}^{\mathrm{raw}}, \mathscr{A}_{i,t}^{\mathrm{loc}}\right), \qquad \bar{a}_{i,t} \in \mathscr{A}_{i,t}^{\mathrm{loc}}. \tag{7.29}$$

Here Φ_i constructs the final bundle $\bar{a}_{i,t}$ from the raw decisions and the current local feasible set. The implementation uses an analytic joint projector. It first rejects a removed or invalid group, applies a forced typed fallback when one exists, clips the learned fraction to $[0,1]$, and respects the minimum operating power declared by the selected port. It then reserves mandatory deferrable starts and the EV charging power needed to remain within the configured service schedule. Remaining charge and discharge requests are jointly scaled against total and phase-specific import or export headroom. A command that falls below its typed operating floor after scaling returns to IDLE.

For the running example, one local constraint is

$$P_t^{\mathrm{ch1}} + P_t^{\mathrm{ch2}} + P_t^{\mathrm{def}} - P_t^{\mathrm{bess,dis}} \le P_{A,t}^{\max}. \tag{7.30}$$

Here P_t^{ch1} and P_t^{ch2} are the two charging powers, P_t^{def} is the deferrable load, and $P_t^{\mathrm{bess,dis}}$ is stationary-battery discharge. The quantity $P_{A,t}^{\max}$ is the remaining import headroom after accounting for the other loads and generation of building A. This total limit is evaluated together with phase incidence. If both chargers lie on L1, unused headroom on L3 cannot make their combination feasible. A three-phase service contributes to all three phase rows according to its declared incidence. The projector can retain a configurable headroom reserve and a completion margin for deferrables. This construction resolves competition within the member; REC coordination remains with the learned policies.

Two diagnostics distinguish a useful feasibility mechanism from one that hides poor policy learning:

$$I_t^{\mathrm{loc}} = \mathbb{1}\left[a_{i,t}^{\mathrm{raw}} \notin \mathscr{A}_{i,t}^{\mathrm{loc}}\right], \tag{7.31}$$

$$D_t^{\mathrm{loc}} = \left\|\bar{a}_{i,t} - a_{i,t}^{\mathrm{raw}}\right\|. \tag{7.32}$$

Here I_t^{loc} uses the indicator $\mathbb{1}$ to record an infeasible raw bundle, and D_t^{loc} measures its adjustment. The norm is evaluated on the common physical-command vector after decoding; discrete mode changes are counted separately. Frequent or large adjustments indicate reliance on the projector and weak learning of coherent bundles. Requested, projected, and applied actions remain separate in the transition record, so a physically safe trajectory cannot conceal an infeasible raw policy. The optional shield-aware update removes an intervened group from the PPO policy loss and can distil the final feasible decision into that group; unrelated groups remain eligible for learning.

7.4.3 Shared Limits and Operational Supervision

The Typed Constraint Graph (TCG) records ownership, connection, dependency, and constraint relations. A charger contributes power to its building and to the phases on which it operates. These

relations supply local construction and identify the shared resources affected by the resulting commands. The central set critic uses member summaries; the TCG is the physical description supporting the decision process.

For buildings sharing a transformer, the REC limit can be written as

$$\sum_{i \in \mathcal{N}_t} K_i^{\text{import}}(\bar{a}_{i,t}) \leq P^{\max}_{\text{transformer},t}. \tag{7.33}$$

Here K_i^{import} maps member i's constructed action to its resulting import under the current physical state, and $P^{\max}_{\text{transformer},t}$ is the permitted aggregate import. The sum spans the active members $\mathcal{N}_t$. Satisfaction of each building's local limit alone does not establish this shared inequality. In the current method, REC observations, coupled rewards, and centralised value estimation support learned coordination; residual shared violations remain part of the evaluation.

Operational supervision sits outside this learned coordination. The Fail-Safe Supervisor described in Chapter 4 validates equipment requests, limits them when necessary, and supplies a rapid rule-based fallback when inference is unavailable. Within TI-MARL, dependency closure can already disable an affected group or isolate a member whose essential measurements are missing. The execution record distinguishes those interface decisions, local projection, supervisory intervention, and the final equipment response. Later retraining can therefore retain the circumstances of an intervention without attributing the fallback's outcome to an action the actor never executed.

This division preserves a precise scope of authority. The local construction enforces the encoded member constraints when the feasible set is non-empty, the physical model is correct, and the procedure terminates normally. An infeasible service obligation is recorded as unmet; electrical protection cannot create missing capacity or charging time. REC-wide hard enforcement would require a declared allocation or jointly certified constraint mechanism in addition to the local projector.

7.5 Compatibility and Deployment

7.5.1 Interface Snapshots and Admission

Every decision is associated with an immutable interface snapshot containing active entities, observation parts, action groups, ports, validity, bounds, dependencies, health, and schema versions. The snapshot allows an experiment transition to be reconstructed even when the runtime topology subsequently changes.

Before a checkpoint is restored, its compatibility signature is checked against the current contract, schema, type registry, health rules, learning backbone, critic, actor context, and credit-assignment mode. A new instance of a known charger type can be admitted without changing network architecture. A semantically unknown type is rejected. Compiler migration requires explicit opt-in and matching contract hashes; tensor compatibility alone is insufficient.

For a second installation, these checks identify whether the trained controller can be loaded as it stands or requires a revised interface and evaluation. It prevents a model from being resumed or moved merely because tensor dimensions happen to match. The packaging and deployment process in Chapter 4 uses this distinction at the model handoff.

7.5.2 Checkpoint, Export, and Actor-Only Handoff

The `ti_marl_checkpoint_v1` format stores the actor, critic, optional typed-group critic, optimisers, rollout, random-number states, compiler state, BC state, resolved configuration, software versions, and compatibility signature. Checkpoint replacement is atomic. Restoring to a different compatible member composition is allowed because stable semantic types, rather than the active population, determine network shape.

Two exports have different purposes. The research artefact retains actor and critic state for evaluation and is marked `deployable: false`. A separate `ti_marl_deployment_bundle_v1` omits the critic and optimisation state and carries the actor, resolved typed interfaces, compiler and health rules, projector configuration, normalisation declaration, versions, and compatibility signature. This actor-only bundle defines the intended handoff to the inference lifecycle, but the current generic serving path does not yet certify it as a production TI-MARL deployment. Section 4.4.4 describes the general model contract and its relationship to method-specific runtime requirements.

7.5.3 Execution-Level Decision Traces

Typed interfaces support explanations grounded in the actual decision path. A trace can identify

- the active entities, modules, and schema versions;
- the local and REC observations available to the actor;
- observations that were degraded, stale, missing, or failed;
- the action groups and ports that existed at that step;
- invalid ports and the dependency or health rule that invalidated them;
- the port and parameter selected for each active group;
- any local feasibility adjustment and the active local constraint;
- simulator execution feedback, including requested and applied values; and
- the final action executed by each asset.

For example, a trace can state that `charger_1` selected CHARGE-EV while DISCHARGE-EV was unavailable because the bound EV SoC source was stale. It can separately record that `charger_2` was scaled to preserve the L1 headroom and whether the simulator applied the

resulting request. These statements follow compiler and constraint semantics; they are not inferred from attention weights.

Transitions are written to buffered gzip-compressed JavaScript Object Notation Lines (JSONL) chunks. Each transition references the current and next content-addressed snapshots. Complete snapshots are deduplicated and emitted periodically or when topology and health events require them. This keeps the audit trail useful on long high-frequency runs without serialising the complete interface graph at every step.

7.5.4 Formal and Operational Properties

The method supports several properties whose validity can be checked independently of control performance.

Entity-binding integrity. Every observation and action instance identifies its source, owner, and target entity. The compiler rejects ambiguous or orphaned bindings.

Interface soundness. A compiled valid port has an active owner module, satisfied hard dependencies, and a recognised parameter schema. Health-dependent contraction is deterministic under a fixed snapshot.

Local bundle feasibility. The final local bundle belongs to the encoded local feasible set when that set is non-empty, the construction terminates normally, and the encoded constraints describe the physical system correctly.

Information-boundary preservation. The deployed actor consumes only local interface objects and authorised aggregate observations. Privileged critic information is excluded from the actor package.

Safe admission. Unknown or incompatible schema elements do not become executable action ports merely because a numerical embedding can be produced.

Trace consistency. The explanation record references the same interface snapshot, selected ports, constraint rules, and final action used by execution.

These properties define testable invariants for compilation, admission, execution, and persistence. Energy and service performance are assessed through the experimental comparisons that follow.

7.6 Experimental Methodology

The evaluation assesses the complete TI-MARL controller and its individual mechanisms through the four dimensions defined in Section 1.2.2. A controller must first provide useful energy control in the original installation. Resource changes and disturbances then test whether that performance can be retained, how much intervention is needed, and what work is required to continue operation. Chapters 5 and 6 supply the common scenarios, annual comparison window, and scorecard.

7.6.1 Reference Methods and Attribution

BAU with storage represents conventional operation, while RBC Smart Local supplies a causal, inspectable control reference. The selected residual MATD3 configuration supplies the practical learned reference developed in Chapter 6. Transformer PPO and MATD3 examine reuse across known local resource instances. The MILP reference has a separate role: perfect-information scheduling indicates the opportunity within its declared optimisation model and horizon. Only a solver output replayed over the same scored window can enter the annual physical comparison.

The algorithmic comparison follows Section 6.9. MOHITO-based and COIN-inspired adaptations examine coordination and modular reuse. Safety-layer and shielding references examine constraint enforcement, while MARTA-based fault training examines robustness learned through adverse experience. Graph Neural Network (GNN)-DT and HetGPS retain the relevant energy-specific comparisons. Each adaptation keeps its original mechanism explicit and declares the additional modelling, protection, and deployment support used in the common task.

The complete-controller comparison measures the annual operating outcome together with computation, integration effort, interventions, and recovery. Matched ablations then examine particular mechanisms. A protected entity-based MAPPO reference holds the learner, actor observations, critic information, action scope, and training budget constant while changing representation and compilation. Further contrasts change protection, fault handling, or compatibility checks with the remaining configuration held fixed. These studies attribute the observed effects without narrowing the overall research problem to one mechanism.

Ablations of entity binding, semantic metadata, dynamic bounds, group credit, REC observations, and compatibility checks extend this design. Each changes one declared mechanism. The simulator retains its physical protections when an algorithmic protection is removed, and both proposed and applied actions are recorded. This exposes the cost of an ablation without treating a rejected command as successful control.

7.6.2 Annual Scenarios and Event Comparisons

The principal suite contains Core-30 Normal, Topology, Safety, Robustness, and Combined, together with Premium-100 Normal and All-in. These are the seven annual scenarios used in Chapter 6. Normal establishes the original installation; the focused variants separate resource changes, electrical pressure, and disturbances; the combined scenarios examine their interaction. Premium-100

Table 7.4: Matched comparisons for attributing TI-MARL behaviour.

Comparison	Held constant	Question
Entity MAPPO / TI-MAPPO	PPO, information, population-aware critic, action scope, protection, budget.	What does the compiled interface add to a capable set representation?
TI-PPO / TI-MAPPO	Compiler, actor, local construction, observations, reward.	What is gained through centralised value estimation?
Health metadata / dependency closure	Detected events, measurement quality, thresholds, fallback.	Does changing dependent capabilities preserve more useful control?
Protected / unprotected actor	Actor and held-out simulation trajectory.	How much feasibility is supplied by local construction?
Local / action-conditioned decoding	Type vocabulary, observations, backbone, budget.	Does explicit group-to-observation binding improve repeated-asset decisions?
Pure PPO / prior knowledge	Evaluation inputs, action scope, later learning budget.	Which gains depend on demonstrations or causal auxiliaries?

contains 80 residential, 17 small-service, and 3 larger-service members. DR is evaluated only where the scenario enables it, with its financial credit and physical additionality reported separately.

The first event comparison adds a known charger to an existing building. The assessment follows its admission, the binding of the new session observations, the additional action group, and the allocation of local headroom. Actor parameter count, compiler cost, required configuration work, and subsequent service are all recorded. A controller can accept the new group structurally yet schedule it poorly, so interface acceptance and control quality remain separate measurements.

The second comparison removes a measurement needed for one decision. A stale SoC tests whether dependent discharge is suspended while unaffected resources retain service. A failed main meter tests a broader loss of trustworthy headroom; loss of an aggregate REC signal tests the continuation of local control. Candidate and reference receive the same actor-visible fault evidence. Recovery is timed against common freshness and hysteresis rules, distinguishing detection delay from the controller's response after detection.

The third comparison introduces another building. It follows admission of the local actor, population handling in the learner, continuity of the remaining members' trajectories, and the change in REC outcomes. New members and new local assets are varied separately before being combined. Held-out compositions contain recognised types; an unknown type exercises rejection and fallback, with that event recorded as an admission decision.

Each annual scorecard includes the complete controller–supervisor trajectory. Event-centred records explain what happened within it: learned decisions retained, groups suspended, fallback duration, recovery, and missed service. Preparation or retraining for another installation is measured outside the frozen test year. This keeps the full operating consequence visible while preventing a change of policy during evaluation from being presented as immediate reuse.

7.6.3 Measurements across the Four Research Dimensions

The absolute scorecard retains energy-purchase cost, emissions, REC import and export, PV self-consumption, daily and maximum peak, upward ramping, EV minimum service and target accuracy, electrical-violation energy, action correction, battery throughput, V2G energy, and the number of members benefiting. Definitions, units, denominators, and aggregation follow Chapter 5. A single reward value cannot express all these outcomes, particularly when a low cost accompanies missed departures or frequent intervention.

Table 7.5: Mechanisms and complementary evidence for the four research dimensions.

Dimension	Mechanisms examined	Evidence retained
Scalability and dynamic reconfiguration	Shared actor, variable observation/group sets, central set critic, structural caching.	Energy and service across counts; training/adaptation time, memory, throughput, compiler and inference latency.
Deployability	Versioned interfaces, compatibility checks, actor bundle, stable resource bindings.	Configuration and validation effort; accepted/rejected changes; replay agreement, loading and recovery time.
Safety	Dependency validity, physical bounds, local construction, external supervision.	Requested/projected/applied commands; violated energy, correction frequency and magnitude, missed service.
Robustness	Scoped capability loss, surviving local decisions, fallback, recovery rules.	Fault exposure, unavailable groups, retained decisions, failed events, fallback and recovery duration, service deficits.

The same mechanism can affect several dimensions. Reusing an actor after charger installation can support dynamic reconfiguration while reducing training, adaptation, and integration effort. Computational growth is assessed under scalability; the effort needed to prepare and maintain that actor belongs to deployability. Health-dependent port removal contributes to safety by excluding an unsupported request and to robustness by retaining unaffected decisions. The measurements in Table 7.5 give those related claims distinct empirical meanings.

Computational results identify hardware, batch size, member count, observation count, active groups, time resolution, and deterministic decoding settings. Constant parameter count is reported alongside the increasing work of processing additional resources. Operational measurements similarly identify their scope: validating an actor bundle in software and commissioning a real-world installation involve different tasks and durations.

7.6.4 Training, Selection, and Paired Evaluation

The temporal split follows the annual protocol of Chapter 6. Training uses 2023 excluding the first fourteen UTC days of February, May, August, and November, which are reserved for validation. Episode boundaries prevent a learning target from crossing into a held-out block. Preprocessing statistics are fitted on the training inputs and frozen. The complete 2024 sequence supplies the common test window, with no policy updates or tuning against its outcomes.

The repetition design uses development seeds 123, 456, and 789 and five independently trained confirmation replicas with seeds 1009, 2027, 3037, 4051, and 5059. Candidate and reference share the same scenario chronology, initial physical conditions, exogenous inputs, and event schedule. Neural seeds vary the learned controller rather than the physical task. The principal estimate is the paired annual difference for each seed, with individual seed outcomes and the uncertainty interval retained. Daily or seasonal breakdowns interpret that complete window without replacing it.

Nominal selection first checks electrical and service requirements. Applied electrical-violation energy has a numerical tolerance of 10^{-6} kWh; at least 99% of feasible EV departures must satisfy the minimum service requirement; appliance deadlines and unserved energy are checked separately. EV target accuracy and appliance service must not deteriorate relative to the matched reference. Daily peak and ramping may increase by at most 5%, while PV self-consumption may decrease by at most two percentage points. These thresholds are declared selection rules, applied before the test year is opened.

Among candidates passing those conditions, the selected policy must reduce cost by at least the larger of EUR 0.01 and 0.1% of the absolute reference cost. Daily peak, ramping, and solar utilisation resolve ties. In disturbed or physically infeasible periods, the record instead retains exposure, requested service, delivered service, and recovery. A power outage without backup cannot be judged by an uninterrupted-supply requirement. The appropriate comparison uses the same event for both controllers and a matched undisturbed replay to explain the attributable loss.

7.7 Results and Comparative Analysis

The available results establish the operation of the implemented interface, learner, and persistence mechanisms. The annual experimental comparisons specified above remain to be completed. This section reports the software evidence directly and retains the quantitative control questions separately, without assigning performance values to unexecuted comparisons.

7.7.1 Interface and Learning Implementation

The focused regression suite passed all 139 tests in the repository snapshot based on Algorithms commit `a3c8194e`, using `softcpsrecsimulator==1.8.0`. The retained test log and source hashes identify the tested state. The suite covers compilation, policy runtime, checkpoints, traces, the public typed-interface path, a dynamic simulator integration case, and a high-frequency topology fixture. Table 7.6 summarises the exercised behaviour.

The integration tests establish a concrete connection between the formal objects and the simulator. A resource change reaches the compiler, creates or removes the corresponding groups, and passes through actor evaluation and action encoding. The topology fixture also distinguishes structural rebuilding from ordinary measurement updates. Its latency ceiling is a regression condition for that fixture; deployment timing requires the hardware-specific measurements in Section 7.6.3.

Table 7.6: TI-MARL behaviour established by the focused 139-test regression suite.

Mechanism	Exercised behaviour	Interpretation
Interface compilation	Determinism, ordering, semantic distinction, unknown-type rejection, dynamic bounds, repeated groups.	The tested resource associations survive changes in representation order and known instances.
Local construction	Total/per-phase limits, EV service floors, discharge reserves, operating minima, mandatory starts.	The tested bundles follow their encoded local contracts, with proposal and adjustment retained.
Local and central training	TI-PPO and TI-MAPPO execute training in the simulator integration case.	Both critic arrangements operate with the common typed actor.
Changing population	Join/leave trajectory boundaries, compatible restoration, unchanged parameter dimensions.	Population changes preserve the tested member identities and surviving trajectories.
Health and execution records	Fault evidence reaches health, validity, closure records, commands, and execution feedback.	The tested intervention can be traced to its affected capability and rule.
High-frequency topology	Six structural events yield seven structural versions; the compiler p95 meets a 500 ms test ceiling.	Routine value updates avoid unnecessary structural rebuilding in the 15-second fixture.

7.7.2 Comparative Questions and Evidence Scope

The tests support the implemented treatment of identity, validity, local construction, and restoration. The remaining empirical question is the benefit obtained when those mechanisms are used throughout an operating year. For the added charger, that benefit would appear in retained service and reduced rebuilding or retraining effort. For the missing measurement, it would appear in fewer unnecessary suspensions while respecting the affected physical constraints. For the joining building, it would appear in useful coordination at an acceptable computational and integration cost.

Those outcomes require the matched comparisons in Section 7.6. A protected entity-based controller may achieve comparable energy performance; an explicit contract can still reduce integration ambiguity, but that becomes a deployability result requiring its own measurement. Conversely, accepting another charger without rebuilding the network establishes structural reuse while leaving its learned schedule to be assessed. The illustrative tables in Chapter 6 supply a reporting layout, not evidence of an advantage for TI-MARL.

7.8 Discussion

The scientific question is whether TI-MARL sustains useful REC control across scalability, deployability, safety, and robustness. The method connects learned coordination to resource interpretation, local protection, fault response, and model handoff. Its comparison with the state of the art therefore follows both the complete operating result and the contribution of each mechanism.

7.8.1 Relationship to State-of-the-Art Methods

MOHITO's task-open formulation is especially relevant when service demands appear and disappear. In the running example, a vehicle arrival creates a session with an energy requirement and a deadline. TI-MARL distinguishes that session from the charger that serves it and from the building that owns the local controller. Its compiled action group remains attached to the charger while the target session changes. The distinction becomes consequential when a charger is installed or fails: that event changes a capability and its dependencies, rather than merely adding or removing a task. The energy adaptation in Section 6.9 makes these objects explicit before comparing the methods.

COIN provide a complementary form of modularity. Their separation of perception, a shared controller, and action modules concerns which learned components can be reused across interface families. TI-MARL places explicit runtime bindings and constraints around repeated instances of recognised types. Two identical charger types share learned processing while retaining separate sessions, bounds, and phase incidence. This identifies the additional relationship under study; a modular reference equipped with the same bindings and protection is a stronger comparator than an unprotected fixed vector.

The same reasoning applies to graph-based energy methods. GNN-DT associates actions with charging resources across changing fleets, and HetGPS combines a reusable graph policy with physical correction. These are substantial precedents for the representation and safety components of the problem. TI-MARL's distinguishing design is the common executable interface used by the actor, dependency rules, local construction, compatibility checks, and traces. Its empirical contribution depends on whether this connection improves the outcomes or effort measured under matched conditions.

Fault-tolerant learning supplies a further algorithmic comparison. MARTA changes the adverse experience used in training, whereas TI-MARL's dependency rules determine which capabilities remain available after the recorded evidence changes. The two mechanisms can complement one another. A matched experiment compares ordinary and fault-aware training under the same detection, protection, and recovery conditions, so that retained service can be attributed to the learner or the operational response.

7.8.2 Scalability and Deployability

A shared decoder allows the third charger to use the same learned parameters as the first two. The set critic similarly accepts another compatible building without extending a concatenated input layer. These choices address structural growth. Their practical value depends on processing time, memory, retained service, and the training or adaptation needed after the change. More resources still produce more observations, groups, and interactions to evaluate.

Deployability asks what must be done to use that controller in the changed installation. The new charger must be identified, associated with the right measurements, assigned to its electrical

Table 7.7: Relationships to neighbouring mechanisms and the specific TI-MARL comparison.

Neighbouring mechanism	Capability already established	TI-MARL comparison
Task-open hypergraphs	Relations between changing tasks, agents, and actions.	Persistent equipment and temporary sessions, with capability-dependent validity and bounds.
COIN	Recombination of perception and action modules around a shared controller.	Runtime bindings of repeated resource instances and their constraints through one contract.
Energy graph policies	Resource-level decisions, fleet/topology transfer, and, in HetGPS, corrective control.	Equal physical information and protection, with compilation and operational effort measured separately.
Safety layers and compositional shields	Explicit correction or certified local obligations supporting declared safety properties.	Local feasible construction linked to current resources; global guarantees require shared assumptions and enforcement.
Fault-tolerant learning	Selective adverse experience improves the policy response to the studied faults.	Equal fault exposure, detection and fallback; retained service, recovery, and training cost.
Model lifecycle tooling	Versioning, packaging, validation, and deployment management.	Compatibility of the complete resource-to-command interpretation at the model handoff.

constraints, and checked against the model's supported types. The compiled interface and compatibility signature preserve these associations in the actor bundle. They make reuse explicit enough to validate, and provide a common basis for the services described in Chapter 4.

The two dimensions overlap without becoming interchangeable. Avoiding network reconstruction can reduce computational work and installation effort at the same time. Yet a small model may still require extensive manual mapping, and a well-packaged model may be too expensive to evaluate on the target hardware. Separate timing, resource, and integration measurements reveal which benefit the method provides. The current actor bundle establishes the algorithm-side handoff; complete production serving and commissioning remain additional evidence requirements.

7.8.3 Safety and Robustness

Safety concerns the action that reaches the resource and its consequences. A stale vehicle SoC can remove the discharge port because its reserve cannot be established. A valid charging request can still be reduced when two chargers compete for the same phase headroom. These are distinct interventions: dependency closure determines available capabilities, while local construction reconciles their combined physical effect. Requested, projected, and applied records make both visible alongside missed departures and other service outcomes.

Formal shielding provides an important comparison. Joint safety can follow from local obligations when their conjunction has been certified against the global specification (Adalat et al. 2026). TI-MARL's member-level projector has a narrower authority: it enforces the encoded local set. Shared transformer safety remains an outcome of the learned joint operation and the declared external protections. The four-dimension assessment therefore cannot convert local feasibility into a claim of complete REC safety.

Robustness concerns how much useful control remains during the same disturbance and how operation recovers. Disabling one charger's discharge while retaining another charger's service is more selective than suspending the entire building. That selectivity depends on trustworthy fault evidence and correct dependencies. Training against corrupted inputs can complement it by improving decisions over the remaining information. Equal detection thresholds and fallback rules are essential to determine whether an observed improvement comes from the policy, the compiled response, or earlier knowledge of the fault.

7.8.4 Scope of the Contribution

TI-MARL integrates reusable multi-agent learning with local protection, health-dependent operation, and a compatible model handoff. The common typed description gives these mechanisms a consistent account of resources, observations, available actions, and constraints. Their combined value is assessed through retained energy and service performance, computation, integration effort, intervention, and recovery. Matched comparisons determine which mechanisms account for that value.

The present implementation supports recognised resource and action families, declared health rules, and encoded local constraints. Its mean-pooled critic summarises the active REC, while its actor bundle defines the algorithm-side deployment requirements. The focused tests establish operation under the exercised conditions. Comparative control and deployment evidence determines the practical benefit, including cases where another method achieves similar performance with different preparation or computational costs.

7.9 Chapter Summary

TI-MARL connects multi-agent learning, local protection, health-dependent operation, and model handoff through a common typed description. Shared actors and local or central critics support REC control, while resource bindings and constraints preserve the meaning of decisions. The assessment follows scalability, deployability, safety, and robustness through the same energy and service task, with matched comparisons examining the contribution of individual mechanisms.

Chapter 8 returns to the installations and operational experience behind these requirements. Within the architecture of Chapter 4, data services, learned decisions, supervision, and equipment interfaces must sustain that control together.

Chapter 8

Real-World Deployment and Validation

The preceding chapters developed the representation, simulation, control, and lifecycle mechanisms required to move learned energy control beyond an isolated experiment. This chapter examines that movement through two distinct demonstrators. The OPEVA demonstrator connects REC operation, residential and corporate charging installations, external platforms, vehicle information, simulation, and distributed control. The DemFlex demonstrator provides a second and independently attributable setting. Their sites, assets, control authority, evaluation periods, and results are kept separate even where both instantiate the EnergAIze architecture.

Real-world evidence spans several levels. Historical measurements can drive a simulation; a target device can execute a policy without actuating an asset; a controller can operate in shadow beside the active system; and a bounded field test can apply physical commands. None of these results establishes sustained closed-loop operation by itself. Each demonstrator is reported with its physical configuration, integration path, controller authority, duration, interventions, and evidence level.

8.1 Evidence Levels and Comparative Reading

The validation uses the following hierarchy:

1. **real-data simulation**: historical operational data drive the simulator;
2. **target-hardware inference**: the controller runs on the intended class of device;
3. **hardware- or software-in-the-loop**: the policy interacts with physical or emulated interfaces in a closed loop;
4. **shadow operation**: decisions are produced and compared but not applied;
5. **limited physical or field control**: actions affect assets during a bounded pilot; and
6. **sustained operational control**: repeated closed-loop use forms part of normal site operation.

The evidence level is part of each claim. Real-data simulation supports long controller comparisons over identical periods, without showing whether a device accepted the requested action. Integration tests establish protocol and interface compatibility, while long-term energy benefit requires a different study. Real-world control exposes latency, rejection, fallback, and user constraints; a short pilot still provides no seasonal evidence. The two demonstrators are assessed against the same questions: which assets were represented, which information was available, which decisions were produced, which commands were applied, which protection remained external, and what outcome was measured.

Figure 8.1 maps each thesis demonstrator to the exact evidence level and action authority it supports.

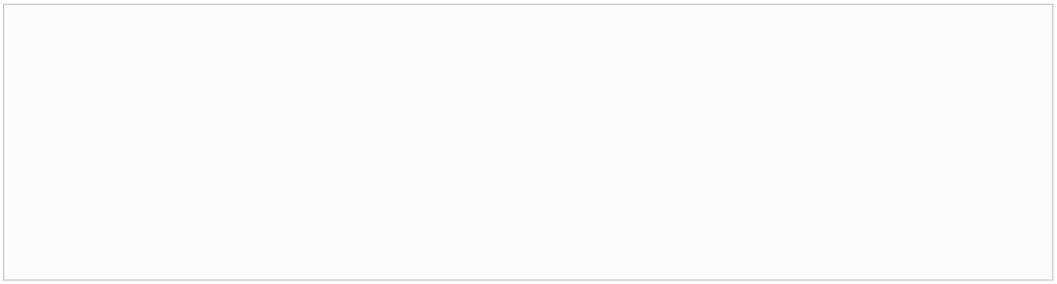


Figure 8.1: Validation evidence levels and the claims supported by each level.

Controller deployment uses the model packaging and target-selection process described in Section 4.4. A method is included in a demonstrator only when its control performance, stability, interface compatibility, runtime resources, and protection mechanisms are adequate for the intended evidence level. A transparent RBC remains available as a reference or fallback. The EnergAIze MADDPG controller preserves continuity with the OPEVA work; later MATD3, hierarchical, or TI-MARL policies require their own target-scoped validation before receiving physical authority.

8.2 OPEVA Demonstrator

8.2.1 Control Problem and Physical Configuration

OPEVA Demo 9 addresses the coordination problem created when EV charging is added to buildings that already have non-controllable demand, PV generation, storage, and site-power limits. Charging immediately at connection can satisfy the driver while increasing a building or REC peak. Delaying or modulating the session can reduce cost and align demand with local PV production, but only within the vehicle's connection window and required departure energy. Corporate installations add a further allocation problem because several simultaneous charging requests can exceed the power available to the charging switchboard or site (Salvan and OPEVA Consortium 2026).

The demonstrator connects three complementary contexts. The Cleanwatts Living Lab represents a heterogeneous REC of real residential participants, including consumers without controllable assets, prosumers with PV production, and flexsumers with batteries or domestic EV charging. The

i-charging S. Mamede factory represents an industrial site with one 50 kW DC charger, two AC charging units with two outlets each, PV generation, meters, and a local controller. The i-charging headquarters represents a larger office installation with one 50 kW DC charger, eighteen 22 kW AC chargers, PV generation, building loads, meters, and a common charging switchboard. Simultaneous charging can request more than 400 kW; site-level power allocation is a physical requirement with direct consequences for the optimisation problem (Salvan and OPEVA Consortium 2026).

Across the three sites, the reported installation comprises four domestic 7.4 kV A AC chargers, two corporate 7.4 kV A AC units with two outputs, eighteen corporate 22 kV A AC chargers, two 50 kW DC chargers, two local charging controllers, and three AC meters. These devices are distributed across installations with different demand, generation, storage, communication, and control capabilities. The resulting heterogeneity is central to the demonstrator: the controller cannot assume that every participant owns the same assets or exposes the same telemetry.

The later real-data study associated with the demonstrator represents 17 installations. The first 15 form a heterogeneous residential and commercial REC; the remaining two represent an office and an industrial facility with PV generation and multiple EV chargers. The dataset contains 18 months of operational measurements aligned to 15-minute intervals. This simulation configuration and the physical Demo 9 installation are related research objects, but they are not interchangeable: the dataset supports long-horizon comparison, while the physical sites provide interface, device, and bounded-actuation evidence.

8.2.2 Integration with the Energy-Control Framework

The demonstrator instantiates EnergAIze as a path from heterogeneous physical interfaces to a controlled action. Cleanwatts provides the REC-facing platform for residential measurements, PV and consumption forecasts, batteries, energy prices, and selected actuation. My.i-charging monitors charging sessions and connects the residential and corporate chargers to the wider system. At the corporate sites, omni-e local controllers acquire charger and meter data and enforce local power-management functions. Open Charge Point Protocol (OCPP) connects chargers with the charging platform, Modbus Transmission Control Protocol/Internet Protocol (TCP/IP) connects selected meters and chargers to local controllers, OCPI exchanges charging information and profiles between platform boundaries, and Representational State Transfer (REST) or RabbitMQ interfaces connect the data and algorithm services (Salvan and OPEVA Consortium 2026).

Percepta and the associated Data Provider middleware isolate the controller from those provider-specific details. Receivers acquire measurements from each platform; translators normalise identifiers, units, timestamps, and message formats; per-installation managers align the required decision interval, identify asset relationships, and handle data gaps; and predictors invoke the policy associated with the installation. The reverse path sends the validated result to the provider responsible for the corresponding battery or charger. Training and inference use the same physical meanings while following different data paths: historical records are assembled into reproducible datasets for

simulation and learning, whereas runtime messages are collected and aggregated periodically for the active controller (Fonseca, Sousa, et al. 2025; Salvan and OPEVA Consortium 2026).

Vehicle SoC illustrates why the integration layer is part of the control problem. Many chargers report session status and energy but do not expose the vehicle's current SoC. PulseCharge obtains available vehicle telemetry through manufacturer-facing services and combines it with user-declared or predicted flexibility. Recurrent user schedules provide a low-interaction baseline, predictions use historical routines when available, and explicit user input has priority when a journey departs from the routine. Consumption, PV-production, and flexibility prediction services provide additional derived observations and can temporarily compensate for unavailable direct measurements. Missing or low-confidence information remains distinguishable from a healthy observation because it changes which charging or V2G decisions can be justified.

The Flexibility Scheduling Algorithm receives the resulting observations and produces building-level actions for controllable assets. CityLearn and EVLearn reproduce the relevant building, EV, charger, battery, and REC conditions so that rule-based, optimisation, single-agent, and multi-agent controllers can be trained or compared without trial and error on the real-world installations (Fonseca, Ferreira, Cabral, Severino, Nweye, et al. 2025; Fonseca, Ferreira, Cabral, Severino, and Praça 2024). At runtime, the policy can execute centrally as one model instance per installation in a cloud environment or locally beside an installation. In both arrangements, provider and device protections retain the final authority to accept, limit, reject, or override a command.

Figure 8.2 presents the three OPEVA sites, the assets available at each, and the data and command path through the partner platforms and EnergAIze components.

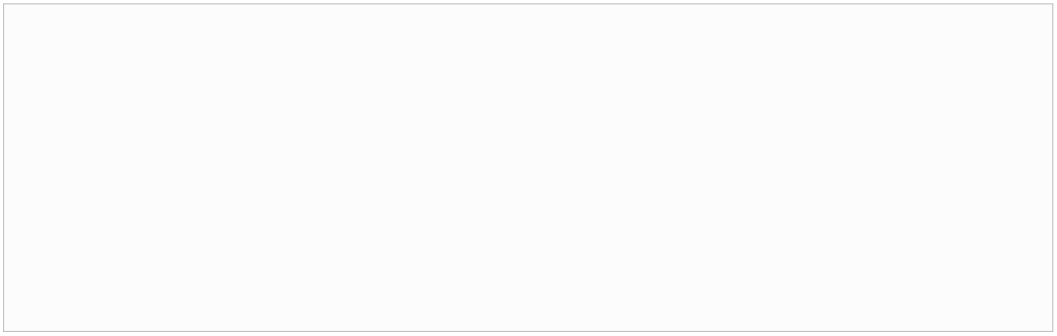


Figure 8.2: OPEVA Demo 9 sites, assets, platforms, and EnergAIze integration path.

8.2.3 Scenarios and Validation Strategy

Four use cases organise the demonstrator. Their value lies in the progression from a common residential charging problem to V2G, in-transit charging, and constrained fleet charging. Their implementation levels must remain explicit.

The real-data simulation study compares unmanaged operation, passive REC energy sharing, rule-based scheduling, and EnergAIze coordination under common data and pricing assumptions. The 18-month records are aligned to 15-minute intervals, with accumulated and instantaneous

Table 8.1: OPEVA Demo 9 scenarios and the evidence boundary reported in deliverable D6.3.

Scenario	Control question	Reported implementation	Boundary
1	Schedule residential charging within the connection window and departure requirement	Simulation and real setting	Physical action authority and duration require final log confirmation
2	Use a connected EV for local storage and V2G	Simulation and partial real integration	Bidirectional charger operation remained simulated
3	Select and reserve an in-route charging option using trip, tariff, range, and REC information	Simulation and partial integration	Public charging and REC/charger communication were not available for complete real-world execution
4	Prioritise several corporate charging requests under building loads, PV production, user needs, and site-power limits	Simulation and partial real integration	Testing was bounded to avoid disturbing office and industrial operation

quantities treated differently during resampling. Short gaps, longer missing periods, timestamp inconsistencies, physically invalid values, and post-gap accumulation spikes are handled according to declared data-type rules. This processing enables a common experiment but does not erase the fact that some controller inputs were interpolated, repeated, estimated, or unavailable in the original system.

The final evidence must keep this campaign separate from the earlier nine-building benchmark reported in D6.3. That benchmark combined demonstrator measurements with CityLearn data and simulated EV flexibility. It supported the initial controller and simulator evaluation, whereas the later 17-installation campaign addresses a different population, time resolution, data horizon, and performance-and-equity question. Numerical results from the two campaigns are not pooled.

8.2.4 Results and Demonstrated Boundary

The four-building operational study provides preliminary evidence that the complete data and coordination path can work with real participants and imperfect measurements. It reports approximately 9% lower daily peak demand and 5% lower energy cost for the evaluated coordinated operation (Fonseca, Sousa, et al. 2025). These values are interpreted with the reported site, baseline, duration, weather, participant behaviour, and degree of direct control; they are not extrapolated to all 17 installations or to sustained autonomous operation.

The later real-data simulation campaign provides the long-horizon comparison. Relative to the unmanaged reference, EnergAIze reports a 10.2% reduction in grid energy demand, a 13.6% reduction in energy cost, a 14.2% improvement in renewable self-consumption, a 20.7% reduction in peak demand, and a 30.1% reduction in ramping under the stated scenario and pricing assumptions. Participant-level results remain heterogeneous, which is scientifically important: aggregate improvement does not establish that participants with different assets and flexibility obtain equal benefits.

The systems evidence is similarly layered. The Cleanwatts and my.i-charging integration exchanged charging-session data; OCPI charging profiles provided a path for control; local controllers operated at the corporate sites; the middleware produced periodic real-data messages for inference; and selected Cleanwatts-device actuation tests had begun at the reporting point (Salvan and OPEVA Consortium 2026). The same report described containerised model and middleware deployment as an early-stage activity. OPEVA established substantial evidence from real platforms, real data, local controllers, and bounded actuation. V2G, in-route charging, fleet-wide policy deployment, and sustained closed-loop operation retained scenario-specific limitations.

Figure 8.3 aligns one audited OPEVA period with observation quality, policy state, requested and applied charging actions, site demand, limits, and interventions.

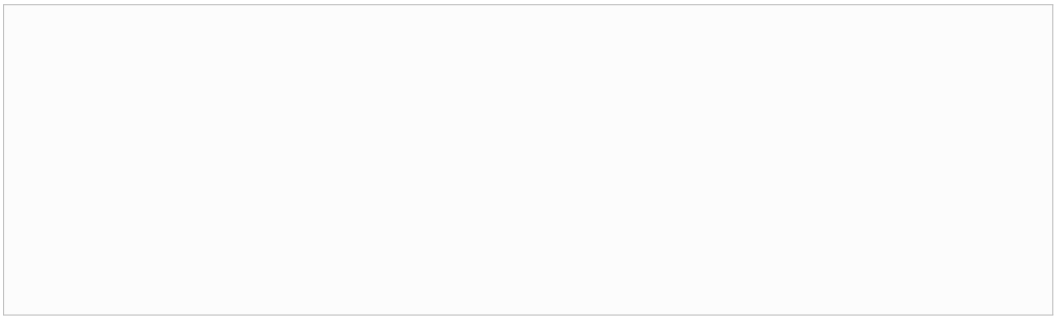


Figure 8.3: Timeline of one audited OPEVA control or integration period.

8.3 DemFlex Demonstrator

DemFlex constitutes the second demonstrator context of this chapter. It is kept separate from OPEVA because its physical setting, available assets, data lineage, control authority, experimental periods, and deployment history define a different body of evidence. The same EnergAIze abstractions can be reused, but compatibility of an interface or component does not permit a result, asset count, or evidence level to be transferred between demonstrators.

8.4 Cross-Demonstrator Lessons and Simulation-to-Operation Differences

The demonstrators expose differences that are easily hidden by a nominal simulator. Real timestamps and APIs are not perfectly aligned. SoC can be unavailable or estimated. User-declared departure times can be absent or incorrect. Physical platforms can reject, quantise, delay, or override commands. Assets can be unavailable for reasons not represented in the training scenario. Edge devices can restart and communication can be interrupted.

These differences are classified as data, interface, dynamics, execution, constraint, or lifecycle shifts. The appropriate response depends on the source of the mismatch. A unit or timestamp

error requires a data-contract correction; a provider-specific action requires an adapter; a known observation failure requires an explicit degraded mode; a persistent distribution shift can justify retraining; and an incompatible capability must be rejected when numerical imputation would erase its semantics. The learning algorithm is changed only when the evidence identifies it as the relevant failure source.

Fail-safe behaviour is evaluated through proposed and executed actions separately. Missing or stale critical data can contract the current interface, activate conservative bounds, invoke a rule-based fallback, hold a previously safe value, or suspend learned control. The number, duration, cause, and effect of these interventions are part of the result. A stable physical trajectory produced almost entirely by external correction is evidence for a useful assurance layer, but weak evidence for the policy itself.

Figure 8.4 turns the observed differences between simulation assumptions and demonstrator conditions into explicit findings, mitigations, and remaining limits.

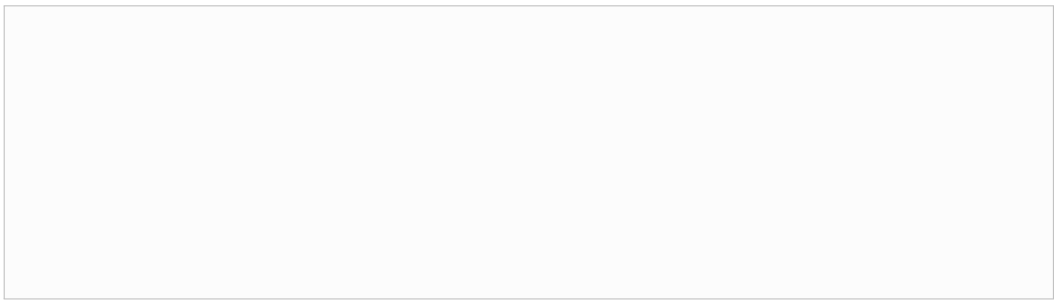


Figure 8.4: Observed differences between simulation assumptions and demonstrator conditions.

The evidence remains bounded by the number of sites, deployment duration, seasonal coverage, partner access, manual steps, and degree of direct actuation. Weather, occupancy, prices, and participant behaviour can confound energy outcomes. Where randomised field periods are unavailable, matched baselines, replay, and counterfactual simulation can support interpretation without extending the causal claim beyond the study design.

Figure 8.5 reports the final demonstrator evidence without collapsing simulated, integration, and physical outcomes into one score.

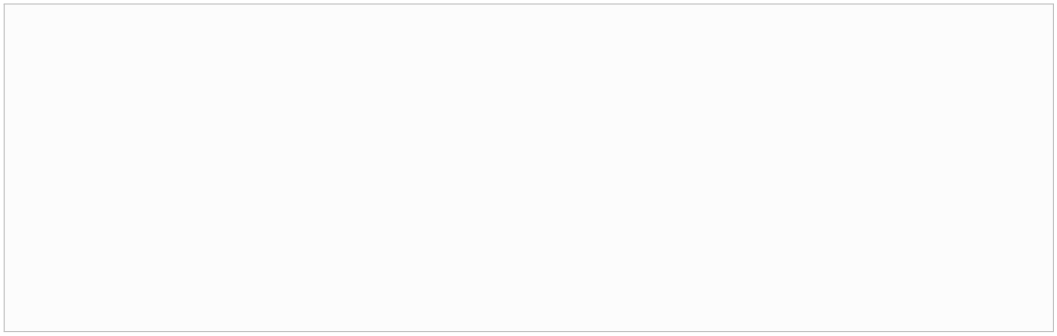


Figure 8.5: Verified OPEVA and DemFlex results by evidence level.

8.5 Contribution to the Research Questions and Chapter Summary

The two demonstrators contribute to all four RQs by testing interactions that the preceding chapters isolate experimentally. For RQ1, they establish the heterogeneous participant, asset, interface, installation, and runtime scales actually reached, together with declared availability changes outside a fixed benchmark; larger controlled population claims remain in simulation. For RQ2, they test the path from historical data and training to packaging, external-platform integration, edge or cloud inference, monitoring, updates, and recovery. For RQ3, they expose physical limits, service obligations, requested and applied actions, and platform or device interventions. For RQ4, they reveal data-quality failures, missing vehicle state, communication and equipment problems, degraded operation, fallback, and the available recovery evidence.

Chapter synthesis. OPEVA demonstrates how EnergAIze joins heterogeneous REC and charging assets, external platforms, data normalisation, vehicle and flexibility information, simulation, learned control, and bounded actuation. Its strongest current evidence combines long-horizon real-data simulation, real-platform integration, local charging control, and preliminary field outcomes, while several scenarios remain partially simulated or bounded by platform access and site-operation constraints. DemFlex is treated as a separate demonstrator so that its contribution can be stated from its own configuration and evidence.

Together, the demonstrators show what changes when the architectural and algorithmic contributions meet physical assets, provider interfaces, imperfect data, runtime resources, and human authority. The next chapter synthesises that evidence with the architecture in Chapter 4 and the controlled results from Chapters 5–7 and answers the RQs explicitly.

Chapter 9

General Discussion and Answers to the Research Questions

This thesis started from a practical difficulty. A controller that performs well in a simulator is still several steps away from controlling a distributed physical system. Its observations must retain the same meaning outside the experiment, its actions must remain valid for the assets that are currently available, physical and service constraints must be enforced, and the complete decision path must survive software, hardware, and organisational change. These conditions become harder when participants, chargers, vehicles, sessions, measurements, and computing targets do not remain fixed.

The work addressed that difficulty through one cumulative argument. EnergAIze established the system boundaries and common contracts. The simulator made heterogeneous assets, reconfiguration, failures, and physical limits reproducible. The controller progression identified what rule-based, optimisation-based, single-agent, and multi-agent methods solve under increasingly demanding conditions. TI-MARL then changed the policy-facing object from a fixed vector into a compiled, entity-bound interface. The lifecycle infrastructure connected experiments to heterogeneous compute, policy export, target-specific inference, monitoring, and recovery. OPEVA and DemFlex provide the real-world contexts in which the assumptions and limits of those contributions are examined.

The discussion does not select one algorithm as a universal winner. It asks what the combined evidence establishes, which mechanism produced each result, and how far that result can be carried beyond the evaluated setting. This distinction matters in a thesis that contains software verification, controlled simulation, real-data simulation, target-runtime evidence, integration with external platforms, and bounded real-world operation. These evidence levels answer different questions.

9.1 What the Combined Evidence Establishes

Five bodies of evidence support the thesis argument.

The simulator establishes that the conditions motivating the research can be represented together and audited. The annual dataset suite contains heterogeneous residential and service members, several asset combinations, multiple chargers per member, EV sessions, deferrable loads, local settlement, total and per-phase limits, health events, and changes in membership and asset availability. The simulator work is important because a policy cannot be credited for robustness or reconfiguration when those conditions are absent from the environment. Its tests and audits support claims about representation, execution, and reproducibility; they do not by themselves support a claim about controller quality.

The controller progression establishes that complexity must earn its place. RBCs provide transparent feasibility and strong references. Optimisation exposes the objective, constraints, and value of information. Single-agent RL tests adaptation without the coordination burden. MADDPG and MATD3 introduce decentralised continuous multi-agent control and address part of the instability observed during learning. BC, guided exploration, replay strategies, offline learning, distillation, and hierarchy tackle data use, training effort, reuse, and the separation of local and REC decisions. Their contribution is both methodological and diagnostic: each stage identifies a problem that can be solved without assuming that every later mechanism is already necessary.

TI-MARL establishes a different form of controller contract. The audited implementation compiles a versioned member schema and current runtime evidence into typed observations, dynamic action-group instances, health-dependent validity, bounds, dependencies, and local constraints. One shared actor processes these compiled local interfaces and produces a hybrid action bundle for the groups that exist at that step. TI-PPO uses a local typed critic; TI-MAPPO retains the same actor and runtime path while estimating value from the active member set. The analytic projector applies local service and electrical constraints before command encoding. The focused regression suite provides implementation evidence for these semantics, dynamic known-type composition, topology-aware trajectories, checkpoint restoration, health closure, and requested–projected–applied traces. It does not replace the multi-seed control comparison.

The lifecycle work establishes that experiment execution and policy serving can share a governed hand-off without forcing every computing target to adopt the same infrastructure. The Job Orchestrator retains persistent job identity and state. Worker adapters execute the same scientific payload through local Docker, the personal workstation, Deucalion, Union, and Jetson-compatible paths. Algorithms exports validated ONNX or rule-based bundles for the general inference path. The deployment backend catalogues bundles and activates a selected version on a target-specific inference service, while the UI exposes state, logs, health, and operator-controlled reversal. The evidence supports heterogeneous execution and per-target bundle management. It does not yet support atomic TI-MARL policy-set rollout, automatic KPI-triggered rollback, or production certification of the typed actor bundle.

The demonstrators establish why the separation between these layers is necessary. OPEVA combines residential and corporate charging, PV generation, storage, site limits, external platforms, local controllers, vehicle information, prediction, simulation, and learned control. The reported four-building study found approximately 9% lower daily peak and 5% lower energy cost in its evaluated setting (Fonseca, Sousa, et al. 2025). The later 17-installation real-data simulation reported reductions of 10.2% in grid energy demand, 13.6% in energy cost, 20.7% in peak demand, and 30.1% in ramping, together with a 14.2% improvement in renewable self-consumption. Those results retain the baseline, data-processing, action-authority, and evidence-level boundaries stated in Chapter 8. They are not evidence of sustained autonomous control across every OPEVA site. DemFlex is analysed as a separate body of evidence and is not populated with OPEVA assumptions.

Table 9.1 condenses the role of each evidence layer. The claim boundary is as important as the result because it prevents a unit test, a long simulation, or a successful API exchange from being presented as real-world controller validation.

Table 9.1: Role of the principal evidence layers in the thesis argument.

Evidence layer	What it establishes	What it does not establish alone
Simulator tests and audits	Correct representation and execution of the declared scenario, assets, events, constraints, and interfaces	Controller superiority or field benefit
Controlled algorithm experiments	Relative control quality, learning behaviour, ablation effects, and sensitivity under matched scenarios	Transfer to an unseen site or reliable physical actuation
TI-MARL regression and stress tests	Executability and traceability of typed compilation, variable groups, local projection, topology change, and persistence	Better cost or coordination than the strongest baseline
Lifecycle and runtime tests	Reconstructable jobs, heterogeneous execution, bundle validation, inference loading, observation, and recovery behaviour	Automatic coordinated rollout or physical safety
Real-data and demonstrator evidence	Behaviour under real measurements, platforms, devices, users, and bounded action authority	Universal, seasonal, or cross-domain effectiveness

9.2 Answer to RQ1: Scalability of Heterogeneous Multi-Agent Control

RQ1 asks under which conditions MARL-based control can maintain acceptable performance and resource use as the number and diversity of members and controlled assets increase and their declared availability changes. The results support a conditional answer. The current control interface must be part of the decision process, and scalability must be expressed as a measured operating envelope rather than inferred from agent count alone.

The formulation separates the member, the physical asset, the EV, and the charging session. These identities have different lifetimes and cannot safely be collapsed into one index. A member

can remain active while a charger becomes unavailable. One charger can serve several EV sessions over time. A service building can expose several independently controlled chargers. The typed interface records these relations and associates every observation, action group, bound, health rule, and constraint with the entity to which it applies.

At runtime, the TIC constructs one deterministic local snapshot from the versioned schema and current system evidence. The shared actor encodes the observations that exist and decodes a mode and bounded fraction for each active action group. A newly active instance of a known charger type reuses the same semantic encoder and group head; the network does not gain a new output coordinate. Stable member identities preserve survivor trajectories when members join or leave, while EV-session rebinding changes the relevant entity relation without forcing an unnecessary structural rebuild.

The multi-agent component is retained through the joint physical transition, shared resources, local and REC reward terms, authorised aggregate observations, and, in TI-MAPPO, a central set critic used only during training. Execution remains decentralised. A member does not need the raw private state of every other participant. This boundary makes the information assumption visible and allows TI-PPO and TI-MAPPO to be compared without changing the deployed actor.

Local and collective objectives cannot be reduced to aggregate cost. The evaluation therefore includes EV service, deferrable completion, individual cost outcomes, the number of members that improve, peak, ramping, renewable self-consumption, import, export, and residual constraint pressure. A REC improvement that depends on a small group receiving worse service fails the intended interpretation even when total cost decreases.

The answer is bounded to known semantic and action-group families. TI-MARL can admit new instances and tested combinations of those families; it does not infer the meaning or safety requirements of an arbitrary unknown device. An unsupported type is rejected or routed to an explicit fallback. This is a controlled form of plug-and-control, not unrestricted zero-shot control.

The controller mechanisms address complementary bottlenecks. Decentralised execution limits runtime dependence on a central joint action. Parameter sharing prevents the number of actor parameters from growing with every known asset instance. The hierarchical typed encoder aggregates observations by channel and sensor before constructing group context. Dynamic group heads avoid a preallocated universal output vector. TI-MAPPO's set critic accepts a variable active population without rebuilding a fixed concatenated input. Packed evaluation and cached structural preparation reduce repeated runtime work.

None of these mechanisms makes cost constant. More observations and action groups still require more encoding and decoding work. A larger active population increases critic and simulator work. Topology events add compilation and trace cost. Training sample requirements can grow even when parameter count does not. The appropriate evidence reports control quality together with compiler, projector, actor, and critic latency; training throughput; memory; trace volume; communication; and checkpoint size.

The answer to RQ1 is therefore the largest tested combination that satisfies declared control and

resource gates, together with the bottleneck encountered next. Core-15 supports rapid development, Core-30 isolates nominal, safety, health, and dynamic conditions, and Premium-100 combines population scale, heterogeneous members, several chargers per member, reconfiguration, faults, and physical pressure. Several experiment workers can shorten the campaign, but they do not alter the algorithmic envelope. Results obtained in one profile do not imply the same envelope under another control interval, information structure, or target resource budget.

9.3 Answer to RQ2: Deployability and Governed MARL Lifecycle

RQ2 asks how evaluated MARL controllers can be brought into use in real-world installations with manageable preparation, training, adaptation, and integration effort, and maintained there with reproducible behaviour. The implemented answer begins with a resolved experiment contract and ends with an observed target state.

An experiment is identified by its resolved configuration, source and image revisions, simulator and dataset, scenario, interface, reward, seed, target, logs, checkpoints, results, and KPI analysis. The Job Orchestrator maintains persistent job state. Workers translate a common payload into the execution contract of local Docker, the personal workstation, Deucalion, Union, or Jetson-class infrastructure. Target-specific authentication, scheduling, staging, images, and result recovery do not alter the scientific identity of the job. This makes training and retraining portable while preserving comparison and lineage.

The general inference path packages the policy with the information needed to reconstruct its input and output semantics. The manifest records topology, ordered observations, encoders, actions, bounds, algorithm format, and artefact paths. The inference service validates the bundle, rebuilds preprocessing, selects the declared agent, performs ONNX or rule-based execution, maps outputs to named bounded actions, and exposes health and active-bundle identity. The backend catalogues bundles and loads a selected bundle into a configured target. Previous versions remain available for operator-controlled reversal.

TI-MARL contributes an algorithm-side deployment contract through resolved interfaces, compiler state and rules, compatibility signatures, stable identities, actor-only bundles, and execution traces. The current generic inference service does not yet certify that typed actor bundle as a production serving path. The distinction prevents a research checkpoint from being described as deployed merely because its weights can be serialised.

The demonstrated lifecycle is strongest for reproducible experiment execution, heterogeneous worker adaptation, explicit remote recovery state, validated general bundles, per-target loading, and operator-visible reversal. Deployability is evidenced through reconstruction and parity, worker portability, target compatibility, automation, traceability, time to a valid artefact, manual intervention, observable activation, and recoverable state. Atomic activation of a coordinated policy set across disconnected targets, automatic compatibility decisions, KPI-triggered rollback, and complete TI-MARL serving remain outside the implemented guarantee.

9.4 Answer to RQ3: Physical and Operational Safety

RQ3 asks how physical, electrical, and service constraints can be represented, enforced, and assessed throughout training and real-world control. The evidence supports a layered answer in which policy learning, deterministic feasibility, external supervision, and device protection retain separate responsibilities.

The simulator represents total and per-phase limits, asset bounds, EV charging and departure requirements, V2G reserve, mandatory deferrable deadlines, outages, and the distinction between requested and applied control. TI-MARL uses the same typed semantics in action validity, dynamic bounds, and local feasibility. The actor proposes a raw hybrid bundle. The analytic local projector enforces validity, service floors, mandatory starts, local total headroom, and local per-phase headroom. The command and trace retain the raw, projected, and applied values, so intervention is measured instead of hidden inside the environment.

The guarantee remains local and conditional on the correctness of the encoded model. The current TI-MARL implementation contains no cross-member hard corrector for a shared feeder or REC limit. Shared constraints are exposed, learned, and evaluated from the applied trajectory; a residual violation is a failed outcome. Device protection and the external Fail-Safe Supervisor retain authority in real-world operation. A low violation count after frequent external correction is evidence that the assurance layer worked, not that the policy learned safe behaviour.

9.5 Answer to RQ4: Robustness to Real-World Disturbances

RQ4 asks whether MARL-based control can retain acceptable service, degrade in a controlled manner, and recover when data, communication, equipment, or operating conditions depart from nominal assumptions. Planned growth and declared known-type changes are assessed under RQ1. RQ4 begins when a required assumption is missing, stale, erroneous, failed, or outside the trained operating conditions.

The simulator makes disturbance type, duration, severity, and affected scope part of the experiment record. It represents missing or delayed data, forecast error, communication failure, device unavailability, outages, and unexpected operating changes. Health handling is scoped through declared dependencies. Stale vehicle information can contract the affected EV actions without disabling an unrelated stationary battery. A failed charger can remove its group while leaving the member active. Loss of a critical main meter can force a wider degraded mode. Recovery uses explicit hysteresis so that a fluctuating signal does not repeatedly enable and disable control.

Robustness is reported through degradation and recovery rather than one nominal score. Relevant outcomes include service loss, violation energy, invalid requests, fallback duration, unnecessary shutdown, recovery time, and the difference between requested and executed actions. A method is robust within the tested disturbance envelope when it maintains the declared service and safety gates or degrades in the controlled manner specified by the experiment. The conclusion remains

conditional on the declared dependency and health models and does not cover arbitrary unknown faults.

Figure 9.1 links each answer to the corresponding contribution, evidence level, and boundary.

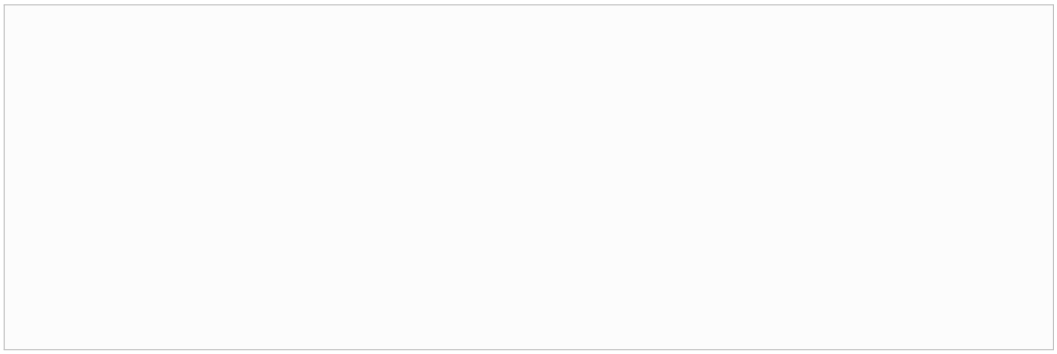


Figure 9.1: Research-question, contribution, finding, and evidence map.

9.6 Interactions among the Four Requirements

The four requirements affect one another, often in ways that a single reward or benchmark conceals.

Scalability and control quality. Parameter sharing and aggregation reduce model growth, but aggressive aggregation can remove distinctions required for local service or phase-specific limits. More expressive typed interfaces preserve those distinctions at the cost of additional tokens, compilation, and trace data. Scale must therefore be reported together with the control information retained.

Safety and performance. The local projector can reject a nominally profitable action because it violates service or electrical limits. That reduction in reward is not a loss of controller quality under the physical problem. Frequent projection, however, indicates that the actor has not internalised the feasible region. Raw and final bundles are needed to distinguish valid conservatism from weak learning.

Robustness and specialisation. A policy tuned to one REC, season, or stable asset set may outperform a more transferable policy in nominal operation. Typed representations, health-aware compilation, historical data, and broader scenario coverage can improve transfer while increasing training and validation cost. The preferred balance depends on how often the site changes and how expensive retraining or a failed decision is.

Deployability and model complexity. A controller can pass simulation gates and still exceed a target's memory or latency budget. A model can fit each target separately while its preprocessing, compiler, communication, or update protocol remains incompatible. Resource and compatibility

gates belong to promotion because they influence which controller can be used, not merely how it is installed.

Reconfiguration and assurance drift. An interface change can invalidate a trained action head, an observation alias, a safety bound, a fallback dependency, or a previously validated policy combination. The typed interface and lifecycle meet at this point. The controller must expose the current contract, and the operational system must decide whether the resulting composition remains admissible.

Figure 9.2 summarises these interactions without treating the four terms as independent scores.

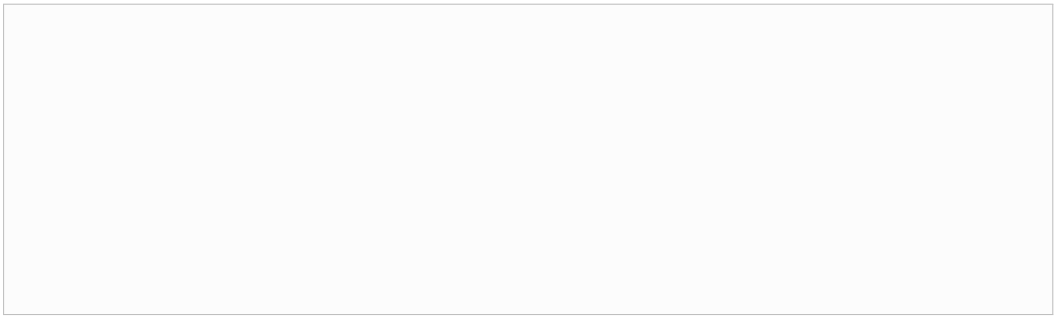


Figure 9.2: Interactions among scalability, deployability, safety, and robustness.

9.7 Scientific Contributions, Generality, and Responsible Use

The central scientific contribution is TI-MARL's executable connection between a versioned control contract and multi-agent learning. Its novelty does not lie in PPO, attention, parameter sharing, health rules, or projection separately. The method compiles an entity-bound interface; evaluates it through one shared typed actor; associates decisions with current groups and physical meanings; applies health, validity, bounds, and local feasibility through the same semantics; and carries stable identities and compatibility evidence across topology change. TI-PPO and TI-MAPPO isolate the effect of local and central-set value estimation while retaining that common actor and runtime path.

This principal contribution rests on six supporting outcomes: the evidence-based formulation of the integrated real-world MARL problem; the EnergAIze research architecture; the audited simulator and annual scenario suite; the progression of control and learning solutions; the experiment and inference lifecycle; and the demonstrator evidence. They should not be presented as seven unrelated novelties. Together they explain why the algorithm has the interface it has and how its claims can be tested and carried towards use.

The empirical conclusions remain energy-domain conclusions. A typed charger action does not become a robotic manipulation action by renaming it. The transferable part is the problem structure: agents can gain or lose known capabilities; observations and actions belong to identifiable entities; availability changes validity and bounds; local decisions interact through shared resources; and

policies must retain their meaning across training and runtime. Application to robotics, industrial equipment, traffic control, or distributed edge services requires new schemas, physical constraints, baselines, and evidence.

Responsible REC control also requires participant-level interpretation. Lower aggregate cost can coexist with worse outcomes for a minority, and average EV service can hide one missed departure. The scorecard therefore includes benefit coverage, service, constraints, and intervention alongside energy and cost. Local inference reduces some data transfer but does not remove privacy risks from logs, forecasts, identities, or model updates. Cybersecurity, certification, regulation, and long-term organisational governance constrain deployment but are not claimed as primary contributions of this thesis.

9.8 Limitations and Threats to Validity

Construct validity. Dynamic population, safety, robustness, scalability, and real-world operation have several meanings. The thesis separates session changes from member admission, local projection from shared physical safety, nominal noise from declared faults, population size from resource scale, and real-data simulation from real-world control. Some ambiguity remains when source projects use broader terminology.

Internal validity. RL comparisons depend on implementation, reward, information access, tuning effort, checkpoint selection, seeds, and stochastic environments. Common manifests, paired windows, hard promotion gates, strong baselines, ablations, and independent seeds reduce this risk. They cannot compensate for an unequal information or tuning budget, which must remain visible in each comparison.

External validity. The RECs, assets, tariffs, weather, participants, devices, and event distributions bound the results. Core and Premium scenarios broaden controlled coverage, while OPEVA and DemFlex expose conditions absent from nominal simulation. Neither large simulation nor one demonstrator establishes universal performance.

Physical and causal validity. The simulator and local projector depend on the correctness of their electrical, asset, service, and settlement models. Field outcomes can be confounded by occupancy, weather, price, maintenance, participant behaviour, platform changes, and manual intervention. Matched baselines and replay improve interpretation but do not create randomised field evidence.

Implementation and attribution. The work builds on CityLearn, established algorithms, open-source packages, student work, partner platforms, and collaborative European projects. Releases, repository history, contribution statements, data ownership, and demonstrator records are required to distinguish original design, adaptation, implementation, integration, data provision, and experiment execution.

9.9 Implications and Open Research Problems

The results point to a smaller set of open problems than the broad real-world MARL label suggests. Unknown asset families still require admission semantics, policy selection, and safety configuration. Shared hard constraints need allocation mechanisms whose effect on fairness and service can be evaluated. Assurance rules must detect when their own observations or physical models are no longer trustworthy. Continual and offline improvement must preserve promotion evidence and previously validated behaviour. Coordinated updates over intermittent networks need explicit mixed-version and local recovery semantics. Longer replicated deployments are required to study seasonal drift, maintenance, interventions, and benefit distribution.

The thesis therefore answers the four RQs at different evidence levels. RQ1 receives a concrete typed-interface formulation, scaling mechanisms, and an explicit control–resource envelope. RQ2 receives an implemented experiment-to-inference lifecycle with stated limits around automated promotion, coordinated rollout, and TI-MARL serving. RQ3 receives layered local feasibility and a clear boundary around shared physical assurance. RQ4 receives health-aware operation, controlled disturbance protocols, and degradation and recovery evidence. The quantitative strength of each answer remains tied to the final evidence reported in its contribution chapter.

The main conclusion is that the difficult part of real-world MARL lies in preserving meaning across layers. The physical problem, simulator, policy interface, feasibility logic, experiment, package, target, and applied action cannot be evaluated as unrelated objects. EnergAIze provides the system context for those relations, and TI-MARL makes the changing local control contract an explicit part of learning. Chapter 10 condenses this argument and states what the thesis ultimately establishes.

Chapter 10

Conclusions and Future Research

This thesis investigated how MARL can move from controlled experiments towards use in distributed, heterogeneous, and dynamically reconfigurable Cyber-Physical Systems (CPSs). RECs and EV charging provided a demanding empirical setting: local assets interact through shared infrastructure; participants expose different information and flexibility; chargers, vehicles, and sessions change independently; physical and service limits cannot be ignored; and the controller must eventually run through real software, devices, and organisations.

The research did not produce one isolated algorithm followed by an application. It produced a connected path. EnergAIze defined the architecture and responsibilities. The simulator made the relevant conditions reproducible. A progression of controllers established which problems could be solved with transparent rules, explicit optimisation, single-agent learning, conventional MARL, data-efficient learning, and hierarchy. TI-MARL addressed the remaining fixed-interface assumption. Experiment orchestration and distributed inference carried controller artefacts across heterogeneous computing environments. OPEVA and DemFlex then provided distinct real-world contexts for testing the resulting claims.

10.1 What the Research Established

The state-of-the-art review showed that scalable MARL, dynamic teams, safe learning, physical testbeds, deployment tooling, and long-running intelligent systems already contain strong individual solutions. The residual problem was their composition. The assumptions that make one mechanism effective often remain implicit and can fail when the system interface, physical constraints, population, information quality, or runtime changes.

The first conclusion is that the experimental world is part of the method. The simulator is not a neutral source of reward. Its identities, timebase, units, assets, events, constraints, observations, actions, settlement, and data-quality rules determine what a controller can learn and what a result means. The annual dataset suite and the CityLearn v3 extensions make heterogeneous members, multiple chargers, EV sessions, deferrable services, phase limits, health problems, and topology

changes available under repeatable conditions. This foundation permits the later algorithmic claims to be tested instead of assumed.

The second conclusion is that controller complexity must be justified by the control problem. Rule-based control remains valuable for transparency, feasibility, development, and fallback. Optimisation remains valuable when the model and horizon are available and an explicit bound or operational schedule is needed. Single-agent methods isolate adaptation from coordination. Conventional MARL becomes useful when distributed information and repeated interaction matter. MATD3, guided exploration, BC, replay strategies, offline learning, distillation, and hierarchy address specific problems of stability, data use, runtime cost, and local–REC decomposition. Their value does not disappear when TI-MARL is introduced.

The third conclusion concerns the object presented to the policy. Fixed observation and action arrays are appropriate while all relevant entities and meanings are known in advance. They become fragile when one building gains a second charger, an EV session moves, a device loses a capability, or a measurement failure changes which action remains defensible. TI-MARL represents the active local interface as a versioned, entity-bound object. A deterministic compiler derives typed observations, dynamic groups, health-dependent validity, bounds, dependencies, and local constraints. A shared actor produces one coherent hybrid bundle for the groups that currently exist. TI-PPO and TI-MAPPO use the same actor and execution path, differing in whether value estimation is local or conditioned on the active population.

The fourth conclusion is that learned actions and feasible actions must remain distinguishable. The actor produces a raw bundle; the analytic projector applies local service and electrical requirements; the device or simulator returns the applied outcome. Keeping these stages in the trace reveals whether good physical behaviour came from the policy, the feasibility layer, external protection, or a combination. The current guarantee is local and model-dependent. Shared limits remain an empirical multi-agent outcome because the evaluated TI-MARL version contains no cross-member hard corrector.

The fifth conclusion is that scale has several independent costs. Parameter sharing can keep model size stable while token processing, critic work, communication, simulation, or sample demand grows. Distributed workers can shorten a campaign without making the algorithm itself more scalable. A policy can train on a GPU and still fail an edge latency or compatibility requirement. Population, heterogeneity, topology events, memory, throughput, communication, training time, inference time, and control quality must be reported together.

The sixth conclusion is that deployment starts before a model file is copied and continues after it is loaded. The experiment, source, image, simulator, dataset, interface, reward, seed, checkpoint, package, target, active version, and observed actions form one evidence chain. The implemented lifecycle reconstructs jobs across heterogeneous workers, validates general inference bundles, loads selected versions into target-specific services, exposes health and logs, and permits operator-controlled reversal. Automatic coordinated rollout, KPI-triggered rollback, and production TI-MARL serving remain separate engineering and research steps.

Finally, the real-world work shows why these distinctions matter. OPEVA connects real participants, corporate charging, local controllers, external platforms, vehicle and flexibility information, historical data, simulation, and bounded actuation. Its reported studies show improvements in cost, peak, grid demand, ramping, and renewable use within their declared settings. They also expose missing SoC, asynchronous data, platform boundaries, site limits, partial scenario implementation, and heterogeneous participant outcomes. DemFlex supplies a second demonstrator whose claims must be derived from its own site, controller, data, authority, and duration.

10.2 Answers to the Research Questions

RQ1: scalability of heterogeneous multi-agent control. Changing participants and assets are handled by separating stable agent identity from module, entity, and session identity and by compiling the active control interface at runtime. Known observation and action semantics are reused across new instances, while validity determines which groups can act. A shared local actor preserves decentralised execution; aggregate context, coupled outcomes, and the TI-MAPPO set critic support REC coordination. Parameter sharing, hierarchical typed encoding, dynamic group heads, a variable-population critic, and cached preparation remove several fixed-size assumptions without making computation independent of population or interface size. The answer is the measured envelope that satisfies both control and resource gates. Unknown semantic families still require explicit admission or fallback.

RQ2: deployability and governed MARL lifecycle. Reproducible job identity, heterogeneous worker adapters, persistent remote state, validated policy bundles, target profiles, inference health, active-version records, logs, and operator-controlled reversal connect training and retraining to serving. TI-MARL adds resolved interface, compiler, compatibility, persistence, and trace contracts to that hand-off. The resulting evidence concerns reconstruction, portability, compatibility, automation, traceability, time and manual intervention, and recoverable target state. Coordinated atomic rollout, automated promotion and rollback, and production TI-MARL serving remain beyond the current implementation evidence.

RQ3: physical and operational safety. Safety is incorporated through explicit scenario constraints, dynamic validity and bounds, local analytic projection, service floors, external supervision, device protection, and requested–projected–applied traces. The same physical, electrical, and service semantics connect training, evaluation, correction, and execution. The thesis supports conditional local feasibility and empirical shared-constraint assessment, not universal formal certification.

RQ4: robustness to real-world disturbances. Robustness is addressed through explicit fault scenarios, causal health and dependency rules, forced degraded modes, scoped fallback, recovery hysteresis, and traces that connect disturbances to retained service and applied action. Data,

communication, equipment, and operating deviations are evaluated by type, severity, degradation, unnecessary service loss, intervention, and recovery. The answer remains conditional on the tested disturbance envelope and the correctness of the declared health and dependency models.

10.3 Principal Contributions

TI-MARL is the principal algorithmic contribution. Its novelty lies in the executable connection between a versioned, entity-bound control contract and multi-agent learning. The same typed semantics organise what the actor observes and invokes, how health changes validity, how local feasibility is applied, how topology-aware state is restored, and how decisions are traced. This contribution is narrower and more defensible than claiming novelty for PPO, attention, parameter sharing, or action projection separately.

Six supporting contributions make that method meaningful and testable:

1. an integrated formulation of real-world MARL applicability that connects control semantics, the four cross-cutting requirements, and explicit levels of evidence;
2. the EnergAIze control and experimentation architecture, which assigns responsibilities and contracts across data, simulation, control, experimentation, inference, deployment, and operation;
3. CityLearn v3 and the canonical annual REC dataset suite for heterogeneous, event-driven, constrained, reconfigurable, and disturbance-aware experiments;
4. a progressive family of rule-based, optimisation, single-agent, multi-agent, data-reuse, and hierarchical controllers evaluated under a common protocol;
5. a governed experiment-to-inference lifecycle spanning heterogeneous compute, reproducible jobs and artefacts, bundle validation, target activation, monitoring, and operator-controlled reversal; and
6. cross-level empirical evidence from simulation, real measurements, external platforms, edge execution, OPEVA, and DemFlex, kept separate by demonstrator and evidence level.

The software produced during the research includes the simulator, datasets and validation tools, controller implementations, TI-MARL compiler and runtime, orchestration services, worker adapters, inference service, backend and UI support, experiment configurations, and demonstrator integrations. Software volume is evidence of engineering effort, not scientific validity. Tests, comparisons, ablations, systems measurements, releases, and demonstrator records provide the corresponding evidence.

Figure 10.1 summarises how the contributions combine without presenting them as independent inventions.

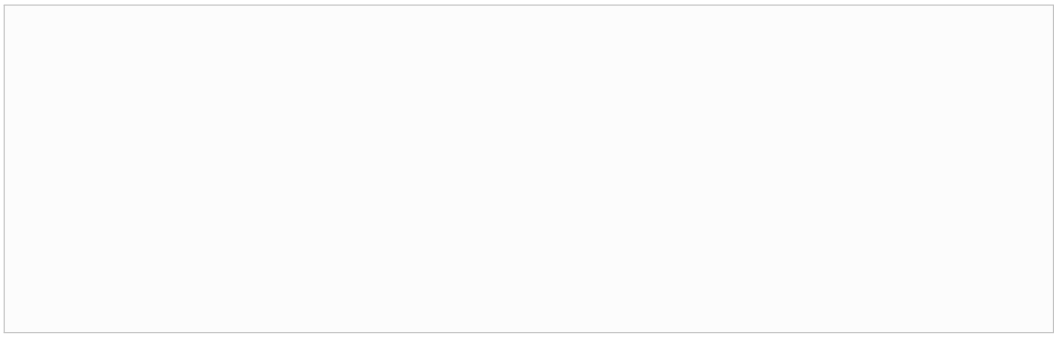


Figure 10.1: Contribution stack and cumulative thesis argument.

10.4 Assessment of the Central Statement

The central statement evaluated by this dissertation was

> *MARL can improve energy performance in real-world RECs while meeting participants' service requirements, with manageable computational and integration effort. Integrating learning, constraint handling, and supervised execution can preserve useful control as resources and operating conditions change.*

The body of work supports that statement within the evaluated energy-domain scope. No individual layer carries the complete claim. A learning result depends on the world and interface against which it was produced. A feasible action depends on the current entities, measurements, bounds, and physical model. A checkpoint remains meaningful only while its preprocessing, compatibility, and execution contracts are preserved. A healthy process does not prove acceptable control, and a successful physical command does not prove seasonal benefit. The integrated design is necessary because a change at one layer can invalidate evidence obtained at another.

The statement is supported most strongly as a systems and method argument, with different empirical depths across its components. The simulator, progressive controllers, TI-MARL implementation, lifecycle, and OPEVA evidence establish concrete parts of the path. Final multi-seed comparisons, complete cross-target systems measurements, production TI-MARL serving, DemFlex evidence, and long-duration field operation determine the remaining quantitative reach. This boundary narrows the conclusion without weakening its central lesson.

10.5 Limits of the Conclusions

The empirical evidence is concentrated in RECs, buildings, batteries, PV systems, EVs, and charging infrastructure. Mechanism-level transfer to another CPS is plausible where agents have changing known capabilities, entity-bound observations and actions, local and shared constraints, and heterogeneous runtime targets. Performance and safety do not transfer without new domain models and evidence.

The evaluated scenarios cover a wider range of topology, faults, physical pressure, and REC size than the original benchmarks, but they remain designed scenarios. Field evidence is bounded by sites, duration, authority, participants, weather, platform access, and manual intervention. The local projector depends on the correctness of the encoded constraints. The central set critic uses a deliberately modest REC summary. Unknown asset families, formal shared safety, adversarial security, certification, and fully autonomous fleet-wide updates remain outside the demonstrated contribution.

10.6 Future Research Directions

Admission of new asset families. A stronger plug-and-control process should identify a new asset, verify its schema and physical model, select a compatible policy or fallback, test local and shared constraints, and monitor its effect after admission. This requires governance for schema evolution as well as more asset types.

Shared physical assurance. Local projection should be complemented by allocation methods for shared feeder, transformer, or REC constraints. The allocation rule must expose its consequences for service and fairness; a central corrector is not neutral merely because it restores feasibility.

Larger and richer TI-MARL structures. Future work should compare TI-PPO and TI-MAPPO with graph and attention-based critics, alternative learning backbones, larger populations, multiple RECs, and richer electrical topology. These experiments must separate gains from the typed interface from gains produced by the optimiser or critic.

Assurance under model and data error. The compiler and projector should represent confidence in measurements and physical relations, detect inconsistent assumptions, and report when a guarantee no longer applies. Formal barrier, reachability, or shield methods are relevant where suitable models exist, but must also adapt to interface change.

Continual and distributed learning. Historical, offline, federated, and continual learning can reduce direct exploration and support adaptation across RECs. Promotion gates, replayable evidence, privacy constraints, and protection against forgetting must remain part of the update process.

Coordinated deployment over intermittent networks. Mixed versions, partial activation, disconnection, local rollback, and recovery need explicit policy-set semantics. The next lifecycle step is to determine which combinations remain admissible while independent targets change at different times.

Long-duration and replicated real-world studies. Seasonal and multi-site studies should report energy and cost together with service, participant benefit, data incidents, projection and fallback interventions, rejected commands, resource use, policy changes, and operator actions. Such records are needed to distinguish an effective policy from an effective surrounding assurance system.

Fairness, explanation, security, and governance. Participant-level benefit, explanation of binding and projection decisions, human override, privacy, cybersecurity, certification, and regulatory accountability deserve dedicated research beyond their role as constraints in this thesis.

10.7 Closing Remarks

The work began with learning-based energy coordination and expanded as real systems exposed assumptions that a nominal benchmark could leave invisible. Feasible control led to explicit optimisation. Adaptation led to decentralised coordination. Training instability and data scarcity led to stronger learning and reuse mechanisms. Local and REC objectives led to hierarchy. Changes in assets, sessions, health, and action meaning led to TI-MARL. Reproducibility and real-world use led to orchestration, packaging, inference, monitoring, and recovery.

The final lesson is simple but demanding. A credible intelligent controller must expose what it controls, what it can currently observe and invoke, how an action becomes feasible, which experiment produced the policy, which runtime is executing it, what was actually applied, and what evidence supports its continued use. EnergAIze supplies the surrounding system, TI-MARL supplies a policy formulation for changing typed interfaces, and the simulator, controller progression, lifecycle, and demonstrators supply the evidence needed to test both. Together, they move the discussion from a successful learning curve towards an auditable path for intelligent control in changing physical systems.

References

Achiam, J., D. Held, A. Tamar, and P. Abbeel (2017). "Constrained Policy Optimization". In: *Proceedings of the 34th International Conference on Machine Learning*. Vol. 70. Proceedings of Machine Learning Research, pp. 22–31. URL: https://proceedings.mlr.press/v70/achiam17a.html.

Ackermann, J., V. Gabler, T. Osa, and M. Sugiyama (2019). *Reducing Overestimation Bias in Multi-Agent Domains Using Double Centralized Critics*. DOI: 10.48550/arXiv.1910.01465. arXiv: 1910.01465 [cs.LG].

Adalat, O., E. Hamel-De le Court, and F. Belardinelli (2026). "Contract-Based Compositional Shielding for Safe Multi-Agent Reinforcement Learning". In: *arXiv preprint arXiv:2606.14130*. Accepted to EUMAS 2026; version 2, July 2026. DOI: 10.48550/arXiv.2606.14130. arXiv: 2606.14130.

Agrawal, A., B. Amos, S. Barratt, S. Boyd, S. Diamond, and J. Z. Kolter (2019). "Differentiable Convex Optimization Layers". In: *Advances in Neural Information Processing Systems*. Vol. 32. URL: https://papers.nips.cc/paper_files/paper/2019/hash/9ce3c52fc54362e22053399d3181c638-Abstract.html.

Ahilan, S. and P. Dayan (2019). *Feudal Multi-Agent Hierarchies for Cooperative Reinforcement Learning*. DOI: 10.48550/arXiv.1901.08492. arXiv: 1901.08492.

Albrecht, S. V., F. Christianos, and L. Schäfer (2024). *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. Cambridge, MA: MIT Press. URL: https://www.marl-book.com.

Almilaify, Y., K. Nweye, and Z. Nagy (2023). "SCALEX: Scalability Exploration of Multi-Agent Reinforcement Learning Agents in Grid-Interactive Efficient Buildings". In: *Proceedings of the 10th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation*, pp. 261–264. DOI: 10.1145/3600100.3623749.

Ames, A. D., X. Xu, J. W. Grizzle, and P. Tabuada (2017). "Control Barrier Function Based Quadratic Programs for Safety Critical Systems". In: *IEEE Transactions on Automatic Control* 62.8, pp. 3861–3876. DOI: 10.1109/TAC.2016.2638961. URL: https://arxiv.org/abs/1609.06408.

Anil, G., P. Doshi, D. A. Redder, A. Eck, and L.-K. Soh (2025). "MOHITO: Multi-Agent Reinforcement Learning using Hypergraphs for Task-Open Systems". In: *Proceedings of the Forty-first Conference on Uncertainty in Artificial Intelligence*. Vol. 286. Proceedings of Machine Learning Research. PMLR, pp. 149–171. URL: https://proceedings.mlr.press/v286/anil25a.html.

Arshad, S. B., Y. Che, A. Ahmad, and M. A. Alaqil (2026). "Robust multi-agent reinforcement learning framework for intelligent PV-integrated smart energy systems under uncertainty". In: *Scientific Reports* 16.1, p. 21027. DOI: 10.1038/s41598-026-51702-9. URL: https://doi.org/10.1038/s41598-026-51702-9.

Balaji, B., A. Bhattacharya, G. Fierro, J. Gao, J. Gluck, D. Hong, A. Johansen, J. Koh, J. Ploennigs, Y. Agarwal, M. Bergés, D. Culler, R. K. Gupta, M. B. Kjærgaard, M. Srivastava, and K. Whitehouse (2018). "Brick: Metadata schema for portable smart building applications". In: *Applied Energy* 226, pp. 1273–1292. DOI: 10.1016/j.apenergy.2018.02.091.

Bekkemoen, Y. (2024). "Explainable Reinforcement Learning (XRL): A Systematic Literature Review and Taxonomy". In: *Machine Learning* 113, pp. 355–441. DOI: 10.1007/s10994-023-06479-7.

Blad, C., S. Bøgh, C. Kallesøe, and P. Raftery (2023). "A Laboratory Test of an Offline-Trained Multi-Agent Reinforcement Learning Algorithm for Heating Systems". In: *Applied Energy* 337, p. 120807. DOI: 10.1016/j.apenergy.2023.120807.

Blum, D., J. Arroyo, S. Huang, J. Drgoňa, F. Jorissen, H. T. Walnum, Y. Chen, K. Benne, D. Vrabie, M. Wetter, and L. Helsen (2021). "Building Optimization Testing Framework (BOPTEST) for Simulation-Based Benchmarking of Control Strategies in Buildings". In: *Journal of Building Performance Simulation* 14.5, pp. 586–610. DOI: 10.1080/19401493.2021.1986574.

Böhmer, W., V. Kurin, and S. Whiteson (2020). "Deep Coordination Graphs". In: *ICML; PMLR 119*. URL: https://proceedings.mlr.press/v119/boehmer20a.html (visited on 10/17/2026).

Bondi, A. B. (2000). "Characteristics of Scalability and Their Impact on Performance". In: *Proceedings of the 2nd International Workshop on Software and Performance*. ACM, pp. 195–203. DOI: 10.1145/350391.350432.

Booth, A., C. Carroll, I. Ilott, L. L. Low, and K. Cooper (2013). "Desperately Seeking Dissonance: Identifying the Disconfirming Case in Qualitative Evidence Synthesis". In: *Qualitative Health Research* 23.1, pp. 126–141. DOI: 10.1177/1049732312466295.

Cai, Y., X. He, H. Guo, W.-Y. Yau, and C. Lv (2024). "Transformer-based Multi-Agent Reinforcement Learning for Generalization of Heterogeneous Multi-Robot Cooperation". In: *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 13695–13702. DOI: 10.1109/IROS58592.2024.10802580.

Caputo, S., E. Iacobelli, M. De Lucia, S. Jayousi, and L. Mucchi (2026). "Reinforcement Learning-Based Management in IoT-Enabled Renewable Energy Communities: An Approach to Optimization for Comfort, Economy, and Sustainable Performance". In: *Sensors* 26.5, p. 1682. DOI: 10.3390/s26051682. URL: https://doi.org/10.3390/s26051682.

Charbonnier, F., T. Morstyn, and M. D. McCulloch (2022a). "Coordination of resources at the edge of the electricity grid: Systematic review and taxonomy". In: *Applied Energy* 318, p. 119188. DOI: 10.1016/j.apenergy.2022.119188. URL: https://doi.org/10.1016/j.apenergy.2022.119188.

— (2022b). "Scalable Multi-Agent Reinforcement Learning for Distributed Control of Residential Energy Flexibility". In: *Applied Energy* 314, p. 118825. DOI: 10.1016/j.apenergy.2022.118825.

Charbonnier, F., B. Peng, J. Vienne, E. Stai, T. Morstyn, and M. McCulloch (2025). "Centralised rehearsal of decentralised cooperation: Multi-agent reinforcement learning for the scalable coordination of residential energy flexibility". In: *Applied Energy* 377, 124406. DOI: 10.1016/j.apenergy.2024.124406.

Chen, B., Z. Cai, and M. Bergés (2020). "Gnu-RL: A Practical and Scalable Reinforcement Learning Solution for Building HVAC Control Using a Differentiable MPC Policy". In: *Frontiers in Built Environment* 6, 562239. DOI: 10.3389/fbuil.2020.562239. URL: https://doi.org/10.3389/fbuil.2020.562239.

Chen, S., Y. Yan, J. Hu, and C. Feng (2026). "A Spatio-Temporal Attention-Based Multi-Agent Deep Reinforcement Learning Approach for Collaborative Community Energy Trading". In: *Energies* 19.7, p. 1730. DOI: 10.3390/en19071730. URL: https://doi.org/10.3390/en19071730.

Chen, X., G. Qu, Y. Tang, S. H. Low, and N. Li (2022). "Reinforcement Learning for Selective Key Applications in Power Systems: Recent Advances and Future Challenges". In: *IEEE Transactions on Smart Grid* 13.4, pp. 2935–2958. DOI: 10.1109/TSG.2022.3154718.

Cheng, H., H. Luo, Z. Liu, W. Sun, W. Li, and Q. Li (2024). *Reinforcement Learning Based Robust Volt/Var Control in Active Distribution Networks With Imprecisely Known Delay*. Preprint; version 1. arXiv: 2402.17268v1. URL: https://arxiv.org/abs/2402.17268v1 (visited on 10/17/2026).

Chow, Y., M. Ghavamzadeh, L. Janson, and M. Pavone (2018). "Risk-Constrained Reinforcement Learning with Percentile Risk Criteria". In: *Journal of Machine Learning Research* 18.167, pp. 1–51. URL: https://jmlr.org/papers/v18/15-636.html.

CityLearn Project (Sept. 9, 2026a). *CityLearn Version 3 Release History*. URL: https://www.citylearn.net/guides/releases.html (visited on 10/01/2026).

— (2026b). *CityLearn: Official Reinforcement Learning Environment for Demand Response and Load Shaping*. URL: https://github.com/citylearn-project/CityLearn (visited on 10/01/2026).

— (2026c). *CityLearn: Python Package Distribution*. URL: https://pypi.org/project/citylearn/ (visited on 10/20/2026).

Claus, C. and C. Boutilier (1998). "The Dynamics of Reinforcement Learning in Cooperative Multiagent Systems". In: *Proceedings of the Fifteenth National Conference on Artificial Intelligence*. AAAI Press, pp. 746–752. URL: https://cdn.aaai.org/AAAI/1998/AAAI98-106.pdf.

Coraci, D., S. Brandi, T. Hong, and A. Capozzoli (2024). "An innovative heterogeneous transfer learning framework to enhance the scalability of deep reinforcement learning controllers in buildings with integrated energy systems". In: *Building Simulation* 17, pp. 739–770. DOI: 10.1007/s12273-024-1109-6.

Coraci, D., A. Silvestri, G. Razzano, D. Fop, S. Brandi, E. Borkowski, T. Hong, A. Schlueter, and A. Capozzoli (2025). "A scalable approach for real-world implementation of deep reinforcement learning controllers in buildings based on online transfer learning: The HiLo case study". In: *Energy and Buildings* 329, 115254. DOI: 10.1016/j.enbuild.2024.115254.

Cui, J., W. Y. Yap, C. Prosper, B. Balaji, and J. Chen (2023). "Deploying Reinforcement Learning based Economizer Optimization at Scale". In: *ACM BuildSys Workshop on Reinforcement Learning for Energy Management (RLEM)*. URL: https://www.amazon.science/publications/deploying-reinforcement-learning-based-economizer-optimization-at-scale.

Dalal, G., K. Dvijotham, M. Vecerik, T. Hester, C. Paduraru, and Y. Tassa (2018). *Safe Exploration in Continuous Action Spaces*. arXiv: 1801.08757.

Darbali-Zamora, R., M. Jimenez Aparicio, J. C. Berg, and N. S. Gurule (2026). "Reinforcement-Learning Control for Coordinated Voltage Regulation in Distribution Systems Using a Power Hardware-in-the-Loop Platform". In: *IEEE Access* 14, pp. 18298–18315. DOI: 10.1109/access.2026.3658416. URL: https://www.researchgate.net/publication/400200666%5C_Reinforcement-Learning%5C_Control%5C_for%5C_Coordinated%5C_Voltage%5C_Regulation%5C_in%5C_Distribution%5C_Systems%5C_Using%5C_a%5C_Power%5C_Hardware-in-the-Loop%5C_Platform.

Deshpande, K., P. Möhl, A. Hämmerle, G. Weichhart, H. Zörrer, and A. Pichler (2022). "Energy Management Simulation with Multi-Agent Reinforcement Learning: An Approach to Achieve Reliability and Resilience". In: *Energies* 15.19, p. 7381. DOI: 10.3390/en15197381. URL: https://doi.org/10.3390/en15197381.

Diddigi, R. B., D. S. K. Reddy, and S. Bhatnagar (2017). *Multi-Agent Q-Learning for Minimizing Demand-Supply Power Deficit in Microgrids*. DOI: 10.48550/arxiv.1708.07732. URL: https://doi.org/10.48550/arxiv.1708.07732.

Direção-Geral de Energia e Geologia (2023). *Consumo por Município e Tipo de Consumidor*. Official Portuguese electricity-consumption statistics. URL: https://www.dgeg.gov.pt/pt/estatistica/energia/eletricidade/consumo-por-municipio-e-tipo-de-consumidor/ (visited on 08/21/2026).

Dulac-Arnold, G., N. Levine, D. J. Mankowitz, J. Li, C. Paduraru, S. Gowal, and T. Hester (2021). "Challenges of Real-World Reinforcement Learning: Definitions, Benchmarks and Analysis". In: *Machine Learning* 110.9, pp. 2419–2468. DOI: 10.1007/s10994-021-05961-4.

Dusparic, I., C. Harris, A. Marinescu, V. Cahill, and S. Clarke (2013). "Multi-agent residential demand response based on load forecasting". In: *2013 1st IEEE Conference on Technologies for Sustainability (SusTech)*, pp. 90–96. DOI: 10.1109/sustech.2013.6617303. URL: https://doi.org/10.1109/sustech.2013.6617303.

Dusparic, I., A. Taylor, A. Marinescu, V. Cahill, and S. Clarke (2015). "Maximizing renewable energy use with decentralized residential demand response". In: *2015 IEEE First International Smart Cities Conference (ISC2)*, pp. 1–6. DOI: 10.1109/isc2.2015.7366212. URL: https://doi.org/10.1109/isc2.2015.7366212.

Dusparic, I., A. Taylor, A. Marinescu, F. Golpayegani, and S. Clarke (2017). "Residential demand response: Experimental evaluation and comparison of self-organizing techniques". In: *Renewable and Sustainable Energy Reviews* 80, pp. 1528–1536. DOI: 10.1016/j.rser.2017.07.033. URL: https://doi.org/10.1016/j.rser.2017.07.033.

E-REDES (2018). *Manual de Ligações à Rede Elétrica de Serviço Público*. 9th edition. Official connection manual. URL: https://www.e-redes.pt/sites/eredes/files/2019-02/EDPDistribui%C3%A7%C3%A3o_Manual_Ligacoes_Rede_2018_9%C2%AA_edi%C3%A7%C3%A3o.pdf (visited on 08/21/2026).

— (2025). *Manual de Ligações à Rede Elétrica de Serviço Público*. 11th edition, Section 3.2. URL: https://provedordocliente.e-redes.pt/Files/PDF/Manual_Ligacoes_a_Rede_Revisto_11_%20Edicao_2025_26Dez.pdf (visited on 10/12/2026).

Eichelbeck, M., H. Markgraf, and M. Althoff (2026). "CommonPower: A Framework for Safe Data-Driven Smart Grid Control". In: *IEEE Transactions on Smart Grid* 17.1, pp. 71–82. DOI: 10.1109/TSG.2025.3616402.

Electricity Maps (2026a). *Carbon Intensity: API Reference*. URL: https://app.electricitymaps.com/docs/reference/carbon-intensity (visited on 10/13/2026).

— (2026b). *Methodology*. URL: https://www.electricitymaps.com/data/methodology (visited on 10/13/2026).

Elsayed-Aly, I., S. Bharadwaj, C. Amato, R. Ehlers, U. Topcu, and L. Feng (2021). "Safe Multi-Agent Reinforcement Learning via Shielding". In: *Proceedings of the 20th International Conference on Autonomous Agents and Multiagent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, pp. 483–491. arXiv: 2101.11196. URL: https://www.ifaamas.org/Proceedings/aamas2021/pdfs/p483.pdf.

Entidade Reguladora dos Serviços Energéticos (Jan. 4, 2024). *ERSE aprova projeto-piloto de flexibilidade FIRMe*. URL: https://www.erse.pt/comunicacao/destaques/erse-aprova-projeto-piloto-de-flexibilidade-firme/ (visited on 09/25/2026).

— (2026a). *Manual do Utilizador: Simulador de Potência Contratada*. Official guidance on Portuguese low-voltage contracted-power levels. URL: https://www.erse.pt/media/02ibhrwu/manualutilizador_pt.pdf (visited on 08/21/2026).

— (2026b). *Mobilidade Elétrica: Como Funciona*. URL: https://www.erse.pt/consumidores-de-energia/mobilidade-electrica/como-funciona/ (visited on 08/21/2026).

— (2026c). *Tarifas e preços: eletricidade*. URL: https://www.erse.pt/atividade/regulacao/tarifas-e-precos-eletricidade/ (visited on 10/13/2026).

European Parliament and Council of the European Union (2018). *Directive (EU) 2018/2001 of the European Parliament and of the Council of 11 December 2018 on the Promotion of the Use of Energy from Renewable Sources*. L 328, pp. 82–209. Official Journal of the European Union. URL: https://eur-lex.europa.eu/eli/dir/2018/2001/oj.

— (2019). *Directive (EU) 2019/944 of the European Parliament and of the Council of 5 June 2019 on Common Rules for the Internal Market for Electricity and Amending Directive 2012/27/EU*. L 158, pp. 125–199, Articles 2(20) and 32. Official Journal of the European Union. URL: https://eur-lex.europa.eu/eli/dir/2019/944/oj (visited on 09/25/2026).

Faia, R., B. Ribeiro, C. Goncalves, L. Gomes, and Z. Vale (2023). "Multi-Agent Based Energy Community Cost Optimization Considering High Electric Vehicles Penetration". In: *Sustainable Energy Technologies and Assessments* 59, p. 103402. DOI: 10.1016/j.seta.2023.103402.

Fang, X., J. Wang, G. Song, Y. Han, Q. Zhao, and Z. Cao (2019). "Multi-Agent Reinforcement Learning Approach for Residential Microgrid Energy Scheduling". In: *Energies* 13.1, p. 123. DOI: 10.3390/en13010123. URL: https://doi.org/10.3390/en13010123.

Farokhi, F. (2020). "Review of Results on Smart-Meter Privacy by Data Manipulation, Demand Shaping, and Load Scheduling". In: *IET Smart Grid* 3.5, pp. 605–613. DOI: 10.1049/iet-stg.2020.0129.

Feng, C. and A. L. Liu (2025). "Peer-to-peer energy trading of solar and energy storage: A networked multiagent reinforcement learning approach". In: *Applied Energy* 383, p. 125283. DOI: 10.1016/j.apenergy.2025.125283. URL: https://arxiv.org/abs/2401.13947.

Fierro, G., M. Pritoni, M. AbdelBaky, P. Raftery, T. Peffer, G. Thomson, and D. E. Culler (2018). "Mortar: An Open Testbed for Portable Building Analytics". In: *Proceedings of the 5th Conference on Systems for Built Environments*. ACM. DOI: 10.1145/3276774.3276796.

Fierro, G., A. Saha, T. Shapinsky, M. Steen, and H. Eslinger (2022). "Application-driven creation of building metadata models with semantic sufficiency". In: *ACM International Conference on Systems for Energy-Efficient Built Environments (BuildSys)*. DOI: 10.1145/3563357.3564083. URL: https://doi.org/10.1145/3563357.3564083.

Foerster, J., N. Nardelli, G. Farquhar, T. Afouras, P. H. S. Torr, P. Kohli, and S. Whiteson (2017). "Stabilising Experience Replay for Deep Multi-Agent Reinforcement Learning". In: *Proceedings of the 34th International Conference on Machine Learning*. Vol. 70. Proceedings of Machine Learning Research. PMLR, pp. 1146–1155. URL: https://proceedings.mlr.press/v70/foerster17b.html.

Foerster, J. N., G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson (2018). "Counterfactual Multi-Agent Policy Gradients". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1, pp. 2974–2982. DOI: 10.1609/aaai.v32i1.11794.

Fonseca, T., L. L. Ferreira, B. Cabral, R. Severino, K. Nweye, D. Ghose, and Z. Nagy (2025). "EVLearn: Extending the CityLearn Framework with Electric Vehicle Simulation". In: *Energy Informatics* 8.1, p. 16. DOI: 10.1186/s42162-024-00445-w.

Fonseca, T., L. L. Ferreira, B. Cabral, R. Severino, and I. Praça (2024). "EnergAIze: Multi Agent Deep Deterministic Policy Gradient for Vehicle-to-Grid Energy Management". In: *2024 IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm)*. IEEE, pp. 232–237. DOI: 10.1109/SmartGridComm60555.2024.10738095. arXiv: 2404.02361.

Fonseca, T., L. L. Ferreira, J. Landeck, L. Klein, P. Sousa, and F. Ahmed (2022). "Flexible Loads Scheduling Algorithms for Renewable Energy Communities". In: *Energies* 15.23, p. 8875. DOI: 10.3390/en15238875.

Fonseca, T., L. L. Ferreira, A. Sousa, A. Mohammadi, and Z. Nagy (Sept. 18, 2026). *CityLearn v3: A Configurable Simulation and Evaluation Framework for Realistic Control Studies of Renewable Energy Communities*. Preprint. DOI: 10.48550/arXiv.2609.21570. arXiv: 2609.21570.

Fonseca, T., C. Sousa, R. Venâncio, P. Pires, R. Severino, P. Rodrigues, P. Paiva, and L. L. Ferreira (2025). "Control of Renewable Energy Communities using AI and Real-World Data". In: *2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA)*. IEEE, pp. 1–8. DOI: 10.1109/ETFA65518.2025.11205562. arXiv: 2505.17321.

Fraija, A., N. Henao, K. Agbossou, S. Kelouwani, and M. Fournier (2024). "Cooperative price-based demand response program for multiple aggregators based on multi-agent reinforcement learning and Shapley-value". In: *Sustainable Energy, Grids and Networks* 40, p. 101560. DOI: 10.1016/j.segan.2024.101560. URL: https://doi.org/10.1016/j.segan.2024.101560.

Fu, X., G. Wu, Y. Shi, X. Jiang, and W. Yang (2026). "Research on Intelligent Network Formation and Flexible Interconnection Mechanisms for Low-Voltage Distribution Networks in Cyber-Physical Systems". In: *Energy Engineering* 123.11, p. 23. DOI: 10.32604/ee.2026.074212. URL: https://www.techscience.com/energy/v123n11/68914/html.

Fujimoto, S. and S. S. Gu (2021). "A Minimalist Approach to Offline Reinforcement Learning". In: *Advances in Neural Information Processing Systems*. Vol. 34, pp. 20132–20145. URL: https://arxiv.org/abs/2106.06860.

Fujimoto, S., H. van Hoof, and D. Meger (2018). "Addressing Function Approximation Error in Actor-Critic Methods". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. PMLR, pp. 1587–1596. URL: https://proceedings.mlr.press/v80/fujimoto18a.html.

Fujimoto, S., D. Meger, and D. Precup (2019). "Off-Policy Deep Reinforcement Learning without Exploration". In: *Proceedings of the 36th International Conference on Machine Learning*. Vol. 97. Proceedings of Machine Learning Research, pp. 2052–2062. URL: https://proceedings.mlr.press/v97/fujimoto19a.html.

Gao, G., Y. Wen, and D. Tao (2023). "Distributed Energy Trading and Scheduling Among Microgrids via Multiagent Reinforcement Learning". In: *IEEE Transactions on Neural Networks and Learning Systems* 34.12, pp. 10638–10652. DOI: 10.1109/tnnls.2022.3170070. URL: https://doi.org/10.1109/tnnls.2022.3170070.

Gilmer, J., S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl (2017). "Neural Message Passing for Quantum Chemistry". In: *Proceedings of the 34th International Conference on Machine Learning*. Vol. 70. Proceedings of Machine Learning Research, pp. 1263–1272. URL: https://proceedings.mlr.press/v70/gilmer17a.html.

González-Briones, A., S. López Flórez, C. Álvarez-López, C. Ramos, and S. Rodríguez González (2026). "Enhancing Energy Efficiency and Economic Benefits with Battery Energy Storage Systems: An Agent-Based Optimization Approach". In: *Electronics* 15.11, p. 2269. DOI: 10.3390/electronics15112269. URL: https://doi.org/10.3390/electronics15112269.

Goodfellow, I., Y. Bengio, and A. Courville (2016). *Deep Learning*. MIT Press. URL: https://www.deeplearningbook.org/.

Gopstein, A., C. Nguyen, C. O'Fallon, N. Hastings, and D. A. Wollman (2021). *NIST Framework and Roadmap for Smart Grid Interoperability Standards, Release 4.0*. Tech. rep. NIST SP 1108r4. National Institute of Standards and Technology. DOI: 10.6028/NIST.SP.1108r4.

Grant, M. J. and A. Booth (2009). "A Typology of Reviews: An Analysis of 14 Review Types and Associated Methodologies". In: *Health Information and Libraries Journal* 26.2, pp. 91–108. DOI: 10.1111/j.1471-1842.2009.00848.x.

Gronauer, S. and K. Diepold (2022). "Multi-Agent Deep Reinforcement Learning: A Survey". In: *Artificial Intelligence Review* 55.2, pp. 895–943. DOI: 10.1007/s10462-021-09996-w.

Gu, S., J. G. Kuba, Y. Chen, Y. Du, L. Yang, A. Knoll, and Y. Yang (2023). "Safe Multi-Agent Reinforcement Learning for Multi-Robot Control". In: *Artificial Intelligence* 319, p. 103905. DOI: 10.1016/j.artint.2023.103905.

Haarnoja, T., A. Zhou, P. Abbeel, and S. Levine (2018). "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. PMLR, pp. 1861–1870. URL: https://proceedings.mlr.press/v80/haarnoja18b.html.

Hagström, F., V. Garg, and F. Oliveira (2025). "Employing federated learning for training autonomous HVAC systems". In: *Energy and Buildings* 340, 115761. DOI: 10.1016/j.enbuild.2025.115761. arXiv: 2405.00389. URL: https://doi.org/10.1016/j.enbuild.2025.115761.

Harrold, D. J. B., J. Cao, and Z. Fan (2022). "Renewable energy integration and microgrid energy trading using multi-agent deep reinforcement learning". In: *Applied Energy* 318, p. 119151. DOI: 10.1016/j.apenergy.2022.119151. URL: https://arxiv.org/abs/2111.10898.

Hausknecht, M. and P. Stone (2015). "Deep Recurrent Q-Learning for Partially Observable MDPs". In: *AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*. Arlington, Virginia, USA. URL: https://www.cs.utexas.edu/~pstone/Papers/bib2html/b2hd-SDMIA15-Hausknecht.html.

Hermans, B., S. S. W. Walker, J. H. A. Ludlage, and L. Özkan (2024). "Model Predictive Control of Vehicle Charging Stations in Grid-Connected Microgrids: An Implementation Study". In: *Applied Energy* 368, p. 123210. DOI: 10.1016/j.apenergy.2024.123210.

Hodge, C. (2017). *Aligning PEV Charging Times with Electricity Supply and Demand*. NREL/TP-5400-68623. National Renewable Energy Laboratory. URL: https://docs.nlr.gov/docs/fy17osti/68623.pdf.

Hosseini, M., S. Erba, A. Mazaheri, A. Moazami, and V. M. Nik (2025). "From flexible building to resilient energy communities: A scalable decentralized energy management scheme based on

collaborative agents". In: *Energy and Buildings* 337, p. 115651. DOI: 10.1016/j.enbuild.2025.115651. URL: https://doi.org/10.1016/j.enbuild.2025.115651.

Hosseini, M., P. Papadopoulos, M. S. Shahid, K. Javanroodi, S. Erba, E. Grover-Silva, P. Riederer, A. Moazami, F. Wurtz, B. Delinchant, S. Carlucci, M. Aghaei, and V. Nik (2026). "Experimental Insights on CIRLEM: Enhancing Energy Efficiency and Flexibility in Buildings". In: *Energy and Buildings* 351, p. 116655. DOI: 10.1016/j.enbuild.2025.116655.

Hribar, J., M. Mohorčič, and A. Čampa (2025). "Improving energy autonomy of positive energy districts using multi-agent deep reinforcement learning". In: *Scientific Reports* 15.1, p. 27798. DOI: 10.1038/s41598-025-12554-x. URL: https://www.nature.com/articles/s41598-025-12554-x.

Hu, S., F. Zhu, X. Chang, and X. Liang (2021). "UPDeT: Universal Multi-agent Reinforcement Learning via Policy Decoupling with Transformers". In: *ICLR*. DOI: 10.48550/arXiv.2101.08001. URL: https://arxiv.org/abs/2101.08001 (visited on 10/17/2026).

Huang, Q. and K. Doya (2023). *Distributed Reinforcement Learning for DC Open Energy Systems*. URL: https://www.climatechange.ai/papers/iclr2023/48.

Huang, W., I. Mordatch, and D. Pathak (2020). "One Policy to Control Them All: Shared Modular Policies for Agent-Agnostic Control". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research. PMLR, pp. 4455–4464. URL: https://proceedings.mlr.press/v119/huang20d.html.

Hussain, S., C. Lai, and U. Eicker (2023). "Flexibility: Literature Review on Concepts, Modeling, and Provision Method in Smart Grid". In: *Sustainable Energy, Grids and Networks* 35, p. 101113. DOI: 10.1016/j.segan.2023.101113.

Jaegle, A., S. Borgeaud, J.-B. Alayrac, C. Doersch, C. Ionescu, D. Ding, S. Koppula, D. Zoran, A. Brock, E. Shelhamer, O. Hénaff, M. M. Botvinick, A. Zisserman, O. Vinyals, and J. Carreira (2022). "Perceiver IO: A General Architecture for Structured Inputs & Outputs". In: *International Conference on Learning Representations*. URL: https://arxiv.org/abs/2107.14795.

Jain, A., A. Szot, and J. J. Lim (2020). "Generalization to New Actions in Reinforcement Learning". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research. PMLR, pp. 4661–4672. URL: https://proceedings.mlr.press/v119/jain20b.html.

Jelić, M., D. P. Corrêa, D. Jelić, L. Berbakov, D. Werner, M. N. I. Maruf, I. Lázaro, I. Fernández, M. Keane, and N. Tomašević (2025). "Integrated cloud platform for energy management of self-sustainable island communities". In: *Energy Reports* 13, pp. 6233–6250. DOI: 10.1016/j.egyr.2025.05.001.

Jin, B., S. Sun, Y. Zhang, S. Zheng, and H. Yu (2026). "Telemetry-Robust Safe Zero-Shot Graph Multi-Agent Reinforcement Learning for Active Voltage Control Across Distribution Feeders". In: *Sensors* 26.18, 5752. DOI: 10.3390/s26185752.

Johannink, T., S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. Aparicio Ojea, E. Solowjow, and S. Levine (2019). "Residual Reinforcement Learning for Robot Control". In: *2019 International Conference on Robotics and Automation*, pp. 6023–6029. DOI: 10.1109/ICRA.2019.8794127.

Kaelbling, L. P., M. L. Littman, and A. R. Cassandra (1998). "Planning and Acting in Partially Observable Stochastic Domains". In: *Artificial Intelligence* 101.1–2, pp. 99–134. DOI: 10.1016/S0004-3702(98)00023-X.

Kaewdornhan, N., C. Srithapon, R. Liemthong, and R. Chatthaworn (2023). "Real-Time Multi-Home Energy Management with EV Charging Scheduling Using Multi-Agent Deep Reinforcement Learning Optimization". In: *Energies* 16.5, p. 2357. DOI: 10.3390/en16052357. URL: https://mdpi-res.com/d_attachment/energies/energies-16-02357/article_deploy/energies-16-02357.pdf.

Kamboj, S., W. Kempton, and K. S. Decker (2011). "Deploying Power Grid-Integrated Electric Vehicles as a Multi-Agent System". In: *Proceedings of the 10th International Conference on Autonomous Agents and Multiagent Systems*, pp. 13–20. URL: https://ifaamas.org/Proceedings/aamas2011/papers/BP1_G38.pdf.

Kannari, L., N. Wessberg, S. Hirvonen, J. Kantorovitch, and S. Paiho (2025). "Reinforcement Learning for Control and Optimization of Real Buildings: Identifying and Addressing Implementation Hurdles". In: *Journal of Building Engineering* 104, p. 112283. DOI: 10.1016/j.jobe.2025.112283.

Karimi Madahi, S. S., T. Van Puyvelde, G. Gokhale, B. Claessens, and C. Develder (2024). "Multi-source Transfer Learning in Reinforcement Learning-based Home Battery Controller". In: *Proceedings of the 11th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation*. ACM. DOI: 10.1145/3671127.3699534.

Kazmi, H., J. Suykens, A. Balint, and J. Driesen (2019). "Multi-agent reinforcement learning for modeling and control of thermostatically controlled loads". In: *Applied Energy* 238, pp. 1022–1035. DOI: 10.1016/j.apenergy.2019.01.140. URL: https://ftp.esat.kuleuven.be/pub/sista/suykens/reports/AppliedEnergy.pdf.

Kelly, J. and W. Knottenbelt (2015). "The UK-DALE dataset, domestic appliance-level electricity demand and whole-house demand from five UK homes". In: *Scientific Data* 2, 150007. DOI: 10.1038/sdata.2015.7.

Kempf, J. (2022). *kube-volttron: Rearchitecting the VOLTTRON Building Energy Management System for Cloud Native Deployment*. arXiv: 2210.14948. URL: https://arxiv.org/abs/2210.14948.

Keren, S., C. Essayeh, S. V. Albrecht, and T. Morstyn (2024). *Multi-Agent Reinforcement Learning for Energy Networks: Computational Challenges, Progress and Open Problems*. Preprint. DOI: 10.48550/arXiv.2404.15583. arXiv: 2404.15583 [cs.AI].

Khamphanchai, W., A. Saha, K. Rathinavel, M. Kuzlu, M. Pipattanasomporn, S. Rahman, B. Akyol, and J. Haack (2014). "Conceptual Architecture of Building Energy Management Open Source Software (BEMOSS)". In: *2014 IEEE PES Innovative Smart Grid Technologies Europe*. IEEE, pp. 1–6. DOI: 10.1109/ISGTEurope.2014.7028784.

Khan, M. M., S. Ahmad, M. Naeem, M. U. Ali, and S. H. Kim (2026). "A unified multi-agent optimization framework for intelligent PV-integrated smart energy systems". In: *Scientific Reports* 16.1, p. 20039. DOI: 10.1038/s41598-026-50395-4. URL: https://doi.org/10.1038/s41598-026-50395-4.

Kitchenham, B. and S. Charters (2007). *Guidelines for Performing Systematic Literature Reviews in Software Engineering*. EBSE Technical Report EBSE-2007-01. Version 2.3. Keele University and University of Durham. URL: https://www.cin.ufpe.br/~in1037/leitura/kitchenham2007guidelines.pdf.

Koh, W., W. Oh, S. Kim, S. Shin, H. Kim, J. Jang, J. Lee, and S.-Y. Yun (2025). "FlickerFusion: Intra-trajectory Domain Generalizing Multi-agent Reinforcement Learning". In: *ICLR*. DOI: 10.48550/arXiv.2410.15876. URL: https://arxiv.org/abs/2410.15876 (visited on 10/17/2026).

Kostrikov, I., A. Nair, and S. Levine (2022). "Offline Reinforcement Learning with Implicit Q-Learning". In: *International Conference on Learning Representations*. arXiv: 2110.06169. URL: https://openreview.net/forum?id=68n2s9ZJWF8.

Kreuzberger, D., N. Kühl, and S. Hirschl (2023). "Machine Learning Operations (MLOps): Overview, Definition, and Architecture". In: *IEEE Access* 11, pp. 31866–31879. DOI: 10.1109/ACCESS.2023.3262138.

Kuba, J. G., R. Chen, M. Wen, Y. Wen, F. Sun, J. Wang, and Y. Yang (2022). "Trust Region Policy Optimisation in Multi-Agent Reinforcement Learning". In: *International Conference on Learning Representations*. URL: https://openreview.net/forum?id=EcGGFkNTxdJ.

Kumar, A., Z. Fu, D. Pathak, and J. Malik (2021). "RMA: Rapid Motor Adaptation for Legged Robots". In: *Robotics: Science and Systems XVII*. DOI: 10.15607/RSS.2021.XVII.011. URL: https://doi.org/10.15607/RSS.2021.XVII.011 (visited on 10/17/2026).

Kumar, A., A. Zhou, G. Tucker, and S. Levine (2020). "Conservative Q-Learning for Offline Reinforcement Learning". In: *Advances in Neural Information Processing Systems*. Vol. 33. DOI: 10.48550/arXiv.2006.04779. arXiv: 2006.04779.

Lee, J., W. Wang, and D. Niyato (2020). *Demand-Side Scheduling Based on Multi-Agent Deep Actor-Critic Learning for Smart Grids*. Preprint. DOI: 10.48550/arXiv.2005.01979. arXiv: 2005.01979. URL: https://arxiv.org/pdf/2005.01979.

Lee, J., Y. Lee, J. Kim, A. Kosiorek, S. Choi, and Y. W. Teh (2019). "Set Transformer: A Framework for Attention-Based Permutation-Invariant Neural Networks". In: *Proceedings of the 36th International Conference on Machine Learning*. Vol. 97. Proceedings of Machine Learning Research, pp. 3744–3753. URL: https://proceedings.mlr.press/v97/lee19d.html.

Levine, S., A. Kumar, G. Tucker, and J. Fu (2020). *Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems*. DOI: 10.48550/arXiv.2005.01643. arXiv: 2005.01643 [cs.LG].

Li, B., H. Tang, Y. Zheng, J. Hao, P. Li, Z. Wang, Z. Meng, and L. Wang (2022). "HyAR: Addressing Discrete-Continuous Action Reinforcement Learning via Hybrid Action Representation". In: *International Conference on Learning Representations*. arXiv: 2109.05490. URL: https://openreview.net/forum?id=wQkaGq7Vz6q.

Li, P., J. D. Thomas, X. Wang, A. Khalil, A. K. Ahmad, R. Inacio, S. Kapoor, A. Parekh, A. Doufexi, A. Shojaeifard, and R. J. Piechocki (2022). "RLOps: Development Life-Cycle of Reinforcement Learning Aided Open RAN". In: *IEEE Access* 10, pp. 113808–113826. DOI: 10.1109/ACCESS.2022.3217511.

Littman, M. L. (1994). "Markov Games as a Framework for Multi-Agent Reinforcement Learning". In: *Proceedings of the Eleventh International Conference on Machine Learning*. Morgan Kaufmann, pp. 157–163. DOI: 10.1016/B978-1-55860-335-6.50027-1.

Liu, H.-Y., B. Balaji, S. Gao, R. Gupta, and D. Hong (2022). "Safe HVAC Control via Batch Reinforcement Learning". In: *ACM/IEEE International Conference on Cyber-Physical Systems (ICCPS)*, pp. 181–192. DOI: 10.1109/ICCPS54341.2022.00023. URL: https://doi.org/10.1109/ICCPS54341.2022.00023.

Liu, H.-Y., B. Balaji, R. Gupta, and D. Hong (2023). "Rule-based Policy Regularization for Reinforcement Learning-based Building Control". In: *ACM International Conference on Future Energy Systems (e-Energy)*, pp. 242–265. DOI: 10.1145/3575813.3595202. URL: https://doi.org/10.1145/3575813.3595202.

Liu, R. and Y. Chen (2023). *Learning a Multi-Agent Controller for Shared Energy Storage System*. Preprint. DOI: 10.48550/arXiv.2302.08328. arXiv: 2302.08328. URL: https://arxiv.org/abs/2302.08328.

Lou, C., Z. Jin, W. Tang, G. Geng, J. Yang, and L. Zhang (2026). "LLM-enhanced multi-agent reinforcement learning with expert workflow for real-time P2P energy trading". In: *IEEE Transactions on Smart Grid*. Early access. DOI: 10.1109/TSG.2026.3684885. URL: https://eprints.gla.ac.uk/384884/.

Lowe, R., Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch (2017). "Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments". In: *Advances in Neural Information Processing Systems*. Vol. 30, pp. 6379–6390. URL: https://proceedings.neurips.cc/paper/2017/hash/68a9750337a418a86fe06c1991a1d64c-Abstract.html.

Luketina, J., J. Lanchantin, S. Sukhbaatar, and A. Szlam (2025). "Compositional Interfaces for Compositional Generalization". In: *Proceedings of the 3rd Conference on Lifelong Learning Agents*. Vol. 274. Proceedings of Machine Learning Research. PMLR, pp. 692–709. URL: https://proceedings.mlr.press/v274/luketina25a.html.

Mai, V., P. Maisonneuve, T. Zhang, H. Nekoei, L. Paull, and A. Lesage-Landry (2024). "Multi-agent reinforcement learning for fast-timescale demand response of residential loads". In: *Machine Learning* 113.8, pp. 5203–5234. DOI: 10.1007/s10994-023-06460-4.

Marinescu, A., I. Dusparic, and S. Clarke (2017). "Prediction-Based Multi-Agent Reinforcement Learning in Inherently Non-Stationary Environments". In: *ACM Transactions on Autonomous and Adaptive Systems* 12.2, pp. 1–23. DOI: 10.1145/3070861. URL: https://doi.org/10.1145/3070861.

Marinescu, A., A. Taylor, S. Clarke, I. Serban, and C. Marinescu (2019). "Optimising residential electric vehicle charging under renewable energy: Multi-agent learning in software simulation and hardware-in-the-loop evaluation". In: *International Journal of Energy Research* 43.8, pp. 3853–3868. DOI: 10.1002/er.4559. URL: https://www.researchgate.net/publication/332939667%5C_Optimising%5C_residential%5C_electric%5C_vehicle%5C_charging%5C_under%5C_renewable%5C_energy%5C_Multi-agent%5C_learning%5C_in%5C_software%5C_simulation%5C_and%5C_hardware-in-the-loop%5C_evaluation.

Markgraf, H. and M. Althoff (2023). "Safe Multi-Agent Reinforcement Learning for Price-Based Demand Response". In: *ICLR 2023 Workshop on Tackling Climate Change with Machine Learning*. URL: https://www.climatechange.ai/papers/iclr2023/18.

Mehmood, U., S. D. Stoller, R. Grosu, S. Roy, A. Damare, and S. A. Smolka (2021). "A Distributed Simplex Architecture for Multi-agent Systems". In: *Dependable Software Engineering: Theories, Tools, and Applications*. Lecture Notes in Computer Science. Springer, pp. 239–257. DOI: 10.1007/978-3-030-91265-9_13. URL: https://doi.org/10.1007/978-3-030-91265-9_13 (visited on 10/17/2026).

Mguni, D., Y. Sun, H. Chen, W. Yang, A. Darabi, L. O. Orimoloye, and Y. Yang (2026). *Learning Robust Multi-Agent Policies via Selective Adversarial Fault Induction*. Preprint; version 2, 19 February 2026. arXiv: 2508.08800v2.

Michailidis, P., I. Michailidis, and E. Kosmatopoulos (2024). "Review and Evaluation of Multi-Agent Control Applications for Energy Management in Buildings". In: *Energies* 17.19, p. 4835. DOI: 10.3390/en17194835. URL: https://doi.org/10.3390/en17194835.

Mohammadi, P., R. Darshi, H. Gohari Darabkhani, and S. Shamaghdari (2025). "Multiagent Energy Management System Design Using Reinforcement Learning: The New Energy Lab Training Set

Case Study". In: *International Transactions on Electrical Energy Systems* 2025.1, p. 3574030. DOI: 10.1155/etep/3574030. URL: https://doi.org/10.1155/etep/3574030.

Moret, F. and P. Pinson (2019). "Energy Collectives: A Community and Fairness Based Approach to Future Electricity Markets". In: *IEEE Transactions on Power Systems* 34.5, pp. 3994–4004. DOI: 10.1109/TPWRS.2018.2808961.

Moshari, A., K. Javanroodi, and V. M. Nik (2026). "Real-world deployment of model-free reinforcement learning for energy control in district heating systems: Enhancing flexibility across neighboring buildings". In: *Applied Energy* 402, p. 126997. DOI: 10.1016/j.apenergy.2025.126997. URL: https://www.researchgate.net/publication/397527245%5C_Real-world%5C_deployment%5C_of%5C_model-free%5C_reinforcement%5C_learning%5C_for%5C_energy%5C_control%5C_in%5C_district%5C_heating%5C_systems%5C_Enhancing%5C_flexibility%5C_across%5C_neighboring%5C_buildings.

Muntwiler, S., K. P. Wabersich, A. Carron, and M. N. Zeilinger (2020). "Distributed Model Predictive Safety Certification for Learning-Based Control". In: *IFAC-PapersOnLine* 53.2, pp. 5258–5265. DOI: 10.1016/j.ifacol.2020.12.1205. arXiv: 1911.01832. URL: https://arxiv.org/abs/1911.01832.

Nachum, O., S. Gu, H. Lee, and S. Levine (2018). "Data-Efficient Hierarchical Reinforcement Learning". In: *Advances in Neural Information Processing Systems*. Vol. 31. URL: https://arxiv.org/abs/1805.08296.

Nagy, Z., J. R. Vázquez-Canteli, S. Dey, and G. Henze (2021). "The CityLearn Challenge 2021". In: *Proceedings of the 8th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation*. Association for Computing Machinery, pp. 218–219. DOI: 10.1145/3486611.3492226.

Nazloo, H. T. and P. Musilek (2026). "A Control-Centric Systematic Review of MARL for EV–Grid Coordination: From Predictive Input to Verifiable Feedback". In: *Electronics* 15.9, p. 1902. DOI: 10.3390/electronics15091902. URL: https://doi.org/10.3390/electronics15091902.

Negahdar, H., A. Karimi, Y. Khayat, and S. Golestan (2024). "Reinforcement learning-based event-triggered secondary control of DC microgrids". In: *Energy Reports* 11, pp. 2818–2831. DOI: 10.1016/j.egyr.2024.02.033. URL: https://doi.org/10.1016/j.egyr.2024.02.033 (visited on 10/17/2026).

Ng, A. Y., D. Harada, and S. Russell (1999). "Policy Invariance under Reward Transformations: Theory and Application to Reward Shaping". In: *Proceedings of the Sixteenth International Conference on Machine Learning*, pp. 278–287. URL: https://ai.stanford.edu/~ang/papers/shaping-icml99.pdf.

Nik, V. M. and M. Hosseini (2023). "CIRLEM: a synergic integration of Collective Intelligence and Reinforcement learning in Energy Management for enhanced climate resilience and lightweight computation". In: *Applied Energy* 350, p. 121785. DOI: 10.1016/j.apenergy.2023.121785. URL: https://doi.org/10.1016/j.apenergy.2023.121785.

Nweye, K., K. Kaspar, G. Buscemi, T. Fonseca, G. Pinto, D. Ghose, S. Duddukuru, P. Pratapa, H. Li, J. Mohammadi, L. L. Ferreira, T. Hong, M. Ouf, A. Capozzoli, and Z. Nagy (2025). "CityLearn v2: Energy-Flexible, Resilient, Occupant-Centric, and Carbon-Aware Management of Grid-Interactive Communities". In: *Journal of Building Performance Simulation* 18.1, pp. 17–38. DOI: 10.1080/19401493.2024.2418813.

Nweye, K., B. Liu, P. Stone, and Z. Nagy (2022). "Real-world challenges for multi-agent reinforcement learning in grid-interactive buildings". In: *Energy and AI* 10, 100202. DOI: 10.1016/j.egyai.2022.100202.

Nweye, K., S. Sankaranarayanan, and Z. Nagy (2023). "MERLIN: Multi-Agent Offline and Transfer Learning for Occupant-Centric Operation of Grid-Interactive Communities". In: *Applied Energy* 346, p. 121323. DOI: 10.1016/j.apenergy.2023.121323.

Oliehoek, F. A. and C. Amato (2016). *A Concise Introduction to Decentralized POMDPs*. Cham: Springer. DOI: 10.1007/978-3-319-28929-8.

Olorunfemi, B. O. and N. Nwulu (2023). "Multi-agent system implementation in demand response: A literature review and bibliometric evaluation". In: *AIMS Energy* 11.6, pp. 1179–1210. DOI: 10.3934/energy.2023054. URL: https://www.aimspress.com/aimspress-data/aimse/2023/6/PDF/energy-11-06-054.pdf.

Orfanoudakis, S., N. K. Panda, P. Palensky, and P. P. Vergara (2026). "A Graph Neural Network Enhanced Decision Transformer for Efficient Optimization in Dynamic Smart Charging Environments". In: *Energy and AI* 23, p. 100679. DOI: 10.1016/j.egyai.2026.100679.

Orfanoudakis, S., V. Robu, E. M. Salazar, P. Palensky, and P. P. Vergara (2025). "Scalable reinforcement learning for large-scale coordination of electric vehicles using graph neural networks". In: *Communications Engineering* 4, 118. DOI: 10.1038/s44172-025-00457-8. URL: https://www.nature.com/articles/s44172-025-00457-8 (visited on 10/17/2026).

Orr, J. and A. Dutta (2023). "Multi-Agent Deep Reinforcement Learning for Multi-Robot Applications: A Survey". In: *Sensors* 23.7, p. 3625. DOI: 10.3390/s23073625.

Pan, L., L. Huang, T. Ma, and H. Xu (2022). "Plan Better Amid Conservatism: Offline Multi-Agent Reinforcement Learning with Actor Rectification". In: *Proceedings of the 39th International Conference on Machine Learning*. Vol. 162. Proceedings of Machine Learning Research. PMLR, pp. 17221–17237. URL: https://proceedings.mlr.press/v162/pan22a.html.

Patrício, J. T., T. Lopes, A. Rocha, J. Martins, and R. A. Lopes (2026). "Development and Real-World Deployment of a Multi-Agent System to Support the Operation of Renewable Energy Communities". In: *Energy Reports* 15, p. 108967. DOI: 10.1016/j.egyr.2025.108967.

Peng, B., T. Rashid, C. Schroeder de Witt, P.-A. Kamienny, P. Torr, W. Böhmer, and S. Whiteson (2021). "FACMAC: Factored Multi-Agent Centralised Policy Gradients". In: *NeurIPS*. DOI: 10.48550/arXiv.2003.06709. URL: https://arxiv.org/abs/2003.06709 (visited on 10/17/2026).

Pereira, H., L. Gomes, and Z. Vale (2022). "Peer-to-peer energy trading optimization in energy communities using multi-agent deep reinforcement learning". In: *Energy Informatics* 5.S4, p. 44. DOI: 10.1186/s42162-022-00235-2. URL: https://doi.org/10.1186/s42162-022-00235-2.

Phan, D. T., R. Grosu, N. Jansen, N. Paoletti, S. A. Smolka, and S. D. Stoller (2020). "Neural Simplex Architecture". In: *NASA Formal Methods: 12th International Symposium, NFM 2020*. Vol. 12229. Lecture Notes in Computer Science. Springer, pp. 97–114. DOI: 10.1007/978-3-030-55754-6_6.

Pigott, A., C. Crozier, K. Baker, and Z. Nagy (2022). "GridLearn: Multiagent Reinforcement Learning for Grid-Aware Building Energy Management". In: *Electric Power Systems Research* 213, p. 108521. DOI: 10.1016/j.epsr.2022.108521.

Pinto, L., J. Davidson, R. Sukthankar, and A. Gupta (2017). "Robust Adversarial Reinforcement Learning". In: *Proceedings of the 34th International Conference on Machine Learning*. Vol. 70.

Proceedings of Machine Learning Research, pp. 2817–2826. URL: https://proceedings.mlr.press/v70/pinto17a.html (visited on 10/17/2026).

Pohl, E., K. Duwadi, T. Oddleifson, S. Ghosh, V. Ramasamy, and P. Gibbs (2024). *Distribution Grid Impact Study in Highland Park, Michigan: Understanding Rooftop Solar, Behind-the-Meter Energy Storage, Electric Vehicle Charging, and Building Electrification*. DOE/GO-102024-6110. National Renewable Energy Laboratory. URL: https://docs.nlr.gov/docs/fy24osti/87916.pdf.

Pokorn, M., A. Čampa, M. Smolnikar, M. Mohorčič, and J. Hribar (2026). "Cost Minimization in Energy Communities With Multi-Agent Deep Reinforcement Learning and Linear Programming". In: *IEEE Access* 14, pp. 43920–43937. DOI: 10.1109/ACCESS.2026.3676334. URL: https://dirros.openscience.si/IzpisGradiva.php?id=28672&lang=eng.

Pordata (2023). *Consumo de Energia Elétrica por Consumidor: Total e por Tipo de Consumo*. Series based on DGEG data; 2023 values provisional. URL: https://www.pordata.pt/portugal/consumo+de+energia+eletrica+por+consumidor+total+e+por+tipo+de+consumo-1231 (visited on 08/21/2026).

Prasad, A. and I. Dusparic (2019). "Multi-agent Deep Reinforcement Learning for Zero Energy Communities". In: *2019 IEEE PES Innovative Smart Grid Technologies Europe (ISGT-Europe)*, pp. 1–5. DOI: 10.1109/isgteurope.2019.8905628. URL: https://arxiv.org/abs/1810.03679.

Presidência do Conselho de Ministros (2022). *Decreto-Lei n.º 15/2022, de 14 de janeiro: Organização e funcionamento do Sistema Elétrico Nacional*. Consolidated version, Articles 189–190. Diário da República. URL: https://diariodarepublica.pt/dr/legislacao-consolidada/decreto-lei/2022-177634029 (visited on 09/25/2026).

Pu, L., S. Wang, X. Huang, X. Liu, Y. Shi, and H. Wang (2022). "Peer-to-Peer Trading for Energy-Saving Based on Reinforcement Learning". In: *Energies* 15.24, p. 9633. DOI: 10.3390/en15249633. URL: https://doi.org/10.3390/en15249633.

Qiu, D., J. Wang, J. Wang, and G. Strbac (2021). "Multi-Agent Reinforcement Learning for Automated Peer-to-Peer Energy Trading in Double-Side Auction Market". In: *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*, pp. 2913–2920. DOI: 10.24963/ijcai.2021/401. URL: https://www.ijcai.org/proceedings/2021/401.

Qiu, Q., F. Yang, and Y. Zhu (2021). *Set-Estimation based Networked Model Predictive Control for Energy Management of Faulty Microgrids*. Preprint; version 1. arXiv: 2106.03501v1. URL: https://arxiv.org/abs/2106.03501 (visited on 10/17/2026).

Qu, G., Y. Lin, A. Wierman, and N. Li (2020). "Scalable Multi-Agent Reinforcement Learning for Networked Systems with Average Reward". In: *NeurIPS*. URL: https://proceedings.neurips.cc/paper_files/paper/2020/hash/168efc366c449fab9c2843e9b54e2a18-Abstract.html (visited on 10/17/2026).

Rahman, A., N. Höpner, F. Christianos, and S. V. Albrecht (2021). "Towards Open Ad Hoc Teamwork Using Graph-based Policy Learning". In: *ICML; PMLR 139*. URL: https://proceedings.mlr.press/v139/rahman21a.html (visited on 10/17/2026).

Rajeswaran, A., S. Ghotra, B. Ravindran, and S. Levine (2017). "EPOpt: Learning Robust Neural Network Policies Using Model Ensembles". In: *International Conference on Learning Representations*. URL: https://arxiv.org/abs/1610.01283v4 (visited on 10/17/2026).

Rashid, T., M. Samvelyan, C. Schroeder de Witt, G. Farquhar, J. Foerster, and S. Whiteson (2018). "QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80.

Proceedings of Machine Learning Research, pp. 4295–4304. URL: https://proceedings.mlr.press/v80/rashid18a.html.

Reis, I. F. G., I. Gonçalves, M. A. R. Lopes, and C. H. Antunes (2021). "Business Models for Energy Communities: A Review of Key Issues and Trends". In: *Renewable and Sustainable Energy Reviews* 144, p. 111013. DOI: 10.1016/j.rser.2021.111013.

Ren, J. and H. Gao (2026). *Multi-agent Reinforcement Learning for Low-Carbon P2P Energy Trading among Self-Interested Microgrids*. Preprint. DOI: 10.48550/arXiv.2604.08973. arXiv: 2604.08973. URL: https://arxiv.org/abs/2604.08973.

Roijers, D. M., P. Vamplew, S. Whiteson, and R. Dazeley (2013). "A Survey of Multi-Objective Sequential Decision-Making". In: *Journal of Artificial Intelligence Research* 48, pp. 67–113. DOI: 10.1613/jair.3987.

Ross, S., G. J. Gordon, and J. A. Bagnell (2011). "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning". In: *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*. Ed. by G. Gordon, D. Dunson, and M. Dudík. Vol. 15. Proceedings of Machine Learning Research. PMLR, pp. 627–635. URL: https://proceedings.mlr.press/v15/ross11a.html.

Sajid, S., B. Li, B. Berehman, Q. Guo, Y. Kang, M. Athar, and A. Muqtadir (2025). "Decentralized Multi-Agent Reinforcement Learning Control of Residential Battery Storage for Demand Response". In: *Energies* 18.21, p. 5712. DOI: 10.3390/en18215712. URL: https://doi.org/10.3390/en18215712.

Sajid, S., B. Li, B. Qi, F. Liang, Y. Lei, and A. Muqtadir (2026). "Multi-Agent Reinforcement Learning for Demand Response in Grid-Responsive Buildings and Prosumer Communities: A PRISMA-Guided Systematic Review". In: *Energies* 19.9, p. 2170. DOI: 10.3390/en19092170.

Salvan, L. and OPEVA Consortium (Apr. 2026). *OPEVA D6.3: Presentation of All Final Demonstrators*. Tech. rep. D6.3. Version 1.0, approved 22 April 2026. Chips Joint Undertaking Project 101097267 — OPEVA.

Savino, S., T. Minella, Z. Nagy, and A. Capozzoli (2025). "A scalable demand-side energy management control strategy for large residential districts based on an attention-driven multi-agent DRL approach". In: *Applied Energy* 393, 125993. DOI: 10.1016/j.apenergy.2025.125993.

Schulman, J., F. Wolski, P. Dhariwal, A. Radford, and O. Klimov (2017). *Proximal Policy Optimization Algorithms*. DOI: 10.48550/arXiv.1707.06347. arXiv: 1707.06347 [cs.LG].

Shah, M. I. A., E. Barrett, and K. Mason (2025). "Uncertainty-Aware Knowledge Transformers for Peer-to-Peer Energy Trading with Multi-Agent Reinforcement Learning". In: *ECAI 2025*. IOS Press, pp. 3855–3861. DOI: 10.3233/FAIA251268. URL: https://arxiv.org/pdf/2507.16796.

Shah, M. I. A., M. E. Cruz Victorio, M. Duffy, E. Barrett, and K. Mason (2026). "Peer-to-peer energy trading in dairy farms using multi-agent reinforcement learning". In: *Applied Energy* 402, p. 127041. DOI: 10.1016/j.apenergy.2025.127041. URL: https://doi.org/10.1016/j.apenergy.2025.127041.

Shapley, L. S. (1953). "Stochastic Games". In: *Proceedings of the National Academy of Sciences* 39.10, pp. 1095–1100. DOI: 10.1073/pnas.39.10.1095.

Shi, Y., H. Pei, L. Feng, Y. Zhang, and D. Yao (2024). *Towards Fault Tolerance in Multi-Agent Reinforcement Learning*. Preprint. arXiv: 2412.00534 [cs.MA].

Šikšnys, L., T. B. Pedersen, M. Aftab, and B. Neupane (2019). "Flexibility Modeling, Management, and Trading in Bottom-up Cellular Energy Systems". In: *Proceedings of the Tenth ACM International Conference on Future Energy Systems*. Association for Computing Machinery,

pp. 170–180. DOI: 10.1145/3307772.3328296. URL: https://vbn.aau.dk/ws/files/320300442/3307772.3328296.pdf.

SoftCPS (Jan. 26, 2026a). *Demonstration of AI-Enhanced Energy Flexibility for Microgrids with Connected Smart Homes (DEMFLEX)*. Instituto Superior de Engenharia do Porto. URL: https://www2.isep.ipp.pt/softcps/?p=1235 (visited on 09/25/2026).

— (2026b). *SoftCPS Laboratory*. Instituto Superior de Engenharia do Porto. URL: https://www2.isep.ipp.pt/softcps/?page_id=577 (visited on 09/25/2026).

SoftCPS Research Group (2026a). *Algorithms for Energy Flexibility Optimization*. URL: https://github.com/Soft-CPS-Research-Group/Algorithms (visited on 10/08/2026).

— (2026b). *CityLearn UI: Simulation Configuration and Results Interface*. URL: https://github.com/Soft-CPS-Research-Group/citylearn-ui (visited on 10/20/2026).

— (2026c). *EnergAIze Inference: Source Repository*. URL: https://github.com/Soft-CPS-Research-Group/energAIze_inference (visited on 10/08/2026).

— (2026d). *EnergAIze UI: Source Repository*. URL: https://github.com/Soft-CPS-Research-Group/energaize_ui (visited on 10/20/2026).

— (2026e). *Job Orchestrator Agent: Source Repository*. URL: https://github.com/Soft-CPS-Research-Group/job_orchestrator_agent (visited on 10/08/2026).

— (2026f). *Job Worker Agent: Source Repository*. URL: https://github.com/Soft-CPS-Research-Group/job_worker_agent (visited on 10/20/2026).

— (2026g). *Percepta*. URL: https://github.com/Soft-CPS-Research-Group/Percepta (visited on 10/08/2026).

— (2026h). *Simulator: Research Fork of CityLearn for Renewable Energy Community Studies*. URL: https://github.com/Soft-CPS-Research-Group/Simulator (visited on 08/20/2026).

— (Aug. 16, 2026i). *SoftCPS REC Simulator*. Version 1.7.0. Python package. URL: https://pypi.org/project/softcpsrecsimulator/ (visited on 08/20/2026).

Sousa, C., T. Fonseca, L. L. Ferreira, R. Venâncio, and R. Severino (2025). "Percepta: High Performance Stream Processing at the Edge". In: *2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA)*. IEEE, pp. 1–4. DOI: 10.1109/ETFA65518.2025.11205806.

Srithapon, C. and D. Månsson (2023). "Predictive Control and Coordination for Energy Community Flexibility with Electric Vehicles, Heat Pumps and Thermal Energy Storage". In: *Applied Energy* 347, p. 121500. DOI: 10.1016/j.apenergy.2023.121500.

Stouffer, K., M. Pease, C. Tang, T. Zimmerman, V. Pillitteri, S. Lightman, A. Hahn, S. Saravia, A. Sherule, and M. Thompson (2023). *Guide to Operational Technology (OT) Security*. Tech. rep. NIST SP 800-82 Revision 3. National Institute of Standards and Technology. DOI: 10.6028/NIST.SP.800-82r3.

Sutton, R. S. and A. G. Barto (2018). *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press. URL: https://mitpress.mit.edu/9780262039246/reinforcement-learning/.

Sutton, R. S., D. McAllester, S. Singh, and Y. Mansour (1999). "Policy Gradient Methods for Reinforcement Learning with Function Approximation". In: *Advances in Neural Information Processing Systems*. Vol. 12. URL: https://papers.nips.cc/paper/1999/hash/464d828b85b0bed98e80ade0a5c43b0f-Abstract.html.

Sutton, R. S., D. Precup, and S. Singh (1999). "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning". In: *Artificial Intelligence* 112.1-2, pp. 181–211. DOI: 10.1016/S0004-3702(99)00052-1.

Symes Thompson, I., A. Caron, V. Mavroudis, and C. Hicks (Jan. 24, 2025). *Entity-based Reinforcement Learning for Autonomous Cyber Defence*. Version 3. DOI: 10.48550/arXiv.2410.17647. arXiv: 2410.17647 [cs.LG]. URL: https://arxiv.org/html/2410.17647v3.

Tabas, D. and B. Zhang (2022). "Computationally Efficient Safe Reinforcement Learning for Power Systems". In: *2022 American Control Conference (ACC)*, pp. 3303–3310. arXiv: 2110.10333. URL: https://arxiv.org/abs/2110.10333v2.

Taboga, V., J. Veilleux, D. Jang, A. Rankawat, and P.-L. Bacon (2026). *Building2Building: A Large Scale Benchmark for Generalizable Real-World Reinforcement Learning*. Preprint, version 1. DOI: 10.48550/arXiv.2607.16534. arXiv: 2607.16534.

Tan, M. (1993). "Multi-Agent Reinforcement Learning: Independent vs. Cooperative Agents". In: *Proceedings of the Tenth International Conference on Machine Learning*. Morgan Kaufmann, pp. 330–337. DOI: 10.1016/B978-1-55860-307-3.50049-6.

Tao, H., J. Wu, V. Casagrande, and F. Boem (2025). "A Distributed MPC-Guided Safe Reinforcement Learning for Load Frequency Control in Interconnected Microgrids". In: *IFAC-PapersOnLine* 59.27, pp. 150–155. DOI: 10.1016/j.ifacol.2025.12.094. URL: https://discovery.ucl.ac.uk/id/eprint/10211512/1/J3C2025_paper_final.pdf.

Terry, J. K., N. Grammel, S. Son, B. Black, and A. Agrawal (2023). *Revisiting Parameter Sharing in Multi-Agent Deep Reinforcement Learning*. arXiv. Preprint, version 8. DOI: 10.48550/arXiv.2005.13625. URL: https://arxiv.org/abs/2005.13625 (visited on 10/17/2026).

Thananjeyan, B., A. Balakrishna, S. Nair, M. Luo, K. Srinivasan, M. Hwang, J. E. Gonzalez, J. Ibarz, C. Finn, and K. Goldberg (2021). "Recovery RL: Safe Reinforcement Learning With Learned Recovery Zones". In: *IEEE Robotics and Automation Letters* 6.3, pp. 4915–4922. DOI: 10.1109/LRA.2021.3070252. URL: https://doi.org/10.1109/LRA.2021.3070252 (visited on 10/17/2026).

United Nations (2015). *Transforming Our World: The 2030 Agenda for Sustainable Development*. URL: https://sdgs.un.org/2030agenda (visited on 09/28/2026).

Vaswani, A., N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., pp. 5998–6008. URL: https://proceedings.neurips.cc/paper/7181-attention-is-all-you-need.

Vázquez-Canteli, J., J. Kämpf, G. Henze, and Z. Nagy (2019). "CityLearn v1.0: An OpenAI Gym Environment for Demand Response with Deep Reinforcement Learning". In: *Proceedings of the 6th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation*, pp. 356–357. DOI: 10.1145/3360322.3360998.

Vázquez-Canteli, J. R. and Z. Nagy (2019). "Reinforcement Learning for Demand Response: A Review of Algorithms and Modeling Techniques". In: *Applied Energy* 235, pp. 1072–1089. DOI: 10.1016/j.apenergy.2018.11.002.

Vezhnevets, A. S., S. Osindero, T. Schaul, N. Heess, M. Jaderberg, D. Silver, and K. Kavukcuoglu (2017). "FeUdal Networks for Hierarchical Reinforcement Learning". In: *Proceedings of the 34th International Conference on Machine Learning*. Vol. 70. Proceedings of Machine Learning Research, pp. 3540–3549. URL: https://proceedings.mlr.press/v70/vezhnevets17a.html.

Vu, L., T. Vu, T. L. Vu, and A. Srivastava (2024). "Multi-Agent Deep Reinforcement Learning for Distributed Load Restoration". In: *IEEE Transactions on Smart Grid* 15.2, pp. 1749–1760.

DOI: 10.1109/TSG.2023.3310893. URL: https://doi.org/10.1109/TSG.2023.3310893 (visited on 10/17/2026).

Wabersich, K. P. and M. N. Zeilinger (2021). "A Predictive Safety Filter for Learning-Based Control of Constrained Nonlinear Dynamical Systems". In: *Automatica* 129, p. 109597. DOI: 10.1016/j.automatica.2021.109597. arXiv: 1812.05506. URL: https://arxiv.org/abs/1812.05506.

Wang, J., Y. Hong, J. Wang, J. Xu, Y. Tang, Q.-L. Han, and J. Kurths (2022). "Cooperative and Competitive Multi-Agent Systems: From Optimization to Games". In: *IEEE/CAA Journal of Automatica Sinica* 9.5, pp. 763–783. DOI: 10.1109/JAS.2022.105506.

Wang, X., H. Xu, Y. Zheng, and X. Zhan (2023). "Offline Multi-Agent Reinforcement Learning with Implicit Global-to-Local Value Regularization". In: *Advances in Neural Information Processing Systems*. arXiv: 2307.11620. URL: https://arxiv.org/abs/2307.11620.

Wang, X., N. Nimalsiri, Y. Xia, P. Wang, and S. Halgamuge (2026). "HetGPS: Scalable Graph Multi-Agent Reinforcement Learning with Physics-Anchored Adaptive Safety for EV Charging". In: *arXiv preprint arXiv:2608.00679*. Preprint, released August 2026. DOI: 10.48550/arXiv.2608.00679. arXiv: 2608.00679.

Wang, X., Z. Ren, N. Li, and B. Dong (2026). "Scalable Physics-Informed Multi-Agent Reinforcement Learning for Building Energy System Control". In: *Advances in Applied Energy* 23, p. 100285. DOI: 10.1016/j.adapen.2026.100285.

Wang, Y., Y. Cui, Y. Li, and Y. Xu (2023). "Collaborative Optimization of Multi-microgrids System with Shared Energy Storage Based on Multi-agent Stochastic Game and Reinforcement Learning". In: *Energy* 280, p. 128182. DOI: 10.1016/j.energy.2023.128182. URL: https://arxiv.org/abs/2306.10754.

Watkins, C. J. C. H. and P. Dayan (1992). "Q-Learning". In: *Machine Learning* 8.3–4, pp. 279–292. DOI: 10.1007/BF00992698.

Webster, J. and R. T. Watson (2002). "Analyzing the Past to Prepare for the Future: Writing a Literature Review". In: *MIS Quarterly* 26.2, pp. xiii–xxiii. URL: https://www.jstor.org/stable/4132319.

Wilk, P., N. Wang, and J. Li (2024). "Multi-Agent Reinforcement Learning for Smart Community Energy Management". In: *Energies* 17.20, p. 5211. DOI: 10.3390/en17205211.

Wohlin, C. (2014). "Guidelines for Snowballing in Systematic Literature Studies and a Replication in Software Engineering". In: *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*. Association for Computing Machinery. DOI: 10.1145/2601248.2601268. URL: https://www.wohlin.eu/ease14.pdf.

Wu, H., D. Qiu, L. Zhang, and M. Sun (2024). "Adaptive Multi-Agent Reinforcement Learning for Flexible Resource Management in a Virtual Power Plant with Dynamic Participating Multi-Energy Buildings". In: *Applied Energy* 374, p. 123998. DOI: 10.1016/j.apenergy.2024.123998.

Wu, Y., W.-Y. Chiu, Y.-P. Tsai, S. Liu, and W. Hua (2026). "Multiagent reinforcement learning in enhancing resilience of microgrids under extreme weather events". In: *Expert Systems with Applications* 296, p. 129145. DOI: 10.1016/j.eswa.2025.129145. URL: https://doi.org/10.1016/j.eswa.2025.129145.

Xiao, L., X. Xiao, C. Dai, M. Peng, L. Wang, and H. V. Poor (2018). *Reinforcement Learning-based Energy Trading for Microgrids*. DOI: 10.48550/arxiv.1801.06285. URL: https://doi.org/10.48550/arxiv.1801.06285.

Xu, B., B. Zhao, W. Luan, and C. Long (2026). "Preference-aligned value stacking for household battery storage via a selective policy sharing multi-agent reinforcement learning algorithm".

In: *Applied Energy* 416, p. 127967. DOI: 10.1016/j.apenergy.2026.127967. URL: https://doi.org/10.1016/j.apenergy.2026.127967.

Xu, S., Y. Wang, Y. Wang, Z. O'Neill, and Q. Zhu (2020). "One for Many: Transfer Learning for Building HVAC Control". In: *ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys)*, pp. 230–239. DOI: 10.1145/3408308.3427617. arXiv: 2008.03625. URL: https://doi.org/10.1145/3408308.3427617.

yamoling (2026). *marlenv: A Unified Framework for Multi-Agent Reinforcement Learning*. Software documentation; revision c4e6b20849dc. URL: https://github.com/yamoling/multi-agent-rlenv/tree/c4e6b20849dc600aa0049db77a7fe4e6104d6a1c (visited on 10/19/2026).

Yang, Y., J. Hao, Y. Zheng, and C. Yu (2019). "Large-Scale Home Energy Management Using Entropy-Based Collective Multiagent Deep Reinforcement Learning Framework". In: *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*, pp. 630–636. DOI: 10.24963/ijcai.2019/89. URL: https://www.ijcai.org/proceedings/2019/89.

Yang, Y., R. Luo, M. Li, M. Zhou, W. Zhang, and J. Wang (2018). "Mean Field Multi-Agent Reinforcement Learning". In: *ICML; PMLR 80*. URL: https://proceedings.mlr.press/v80/yang18d.html (visited on 10/17/2026).

Ye, Z., D. Qiu, S. Li, Z. Fan, and G. Strbac (2025). "Federated Reinforcement Learning for decentralized peer-to-peer energy trading". In: *Energy and AI*, p. 100500. DOI: 10.1016/j.egyai.2025.100500. URL: https://doi.org/10.1016/j.egyai.2025.100500.

Yu, C., A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu (2022). "The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games". In: *Advances in Neural Information Processing Systems*. Vol. 35, pp. 24611–24624. URL: https://proceedings.neurips.cc/paper_files/paper/2022/hash/9c1535a02f0ce079433344e14d910597-Abstract-Datasets_and_Benchmarks.html.

Yu, L., Y. Sun, Z. Xu, C. Shen, D. Yue, T. Jiang, and X. Guan (2021). "Multi-Agent Deep Reinforcement Learning for HVAC Control in Commercial Buildings". In: *IEEE Transactions on Smart Grid* 12.1, pp. 407–419. DOI: 10.1109/tsg.2020.3011739.

Yu, X., Y. Lin, S. Wang, K. Lv, and S. Han (2024). *Improving Global Parameter-sharing in Physically Heterogeneous Multi-agent Reinforcement Learning with Unified Action Space*. DOI: 10.48550/arXiv.2408.07395. arXiv: 2408.07395 [cs.MA].

Zafra-Cabeza, A., J. Marquez, C. Bordons, and M. A. Ridao (2023). "An online stochastic MPC-based fault-tolerant optimization for microgrids". In: *Control Engineering Practice* 130, p. 105381. DOI: 10.1016/j.conengprac.2022.105381. URL: https://doi.org/10.1016/j.conengprac.2022.105381 (visited on 10/17/2026).

Zaharia, M., A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe, F. Xie, and C. Zumar (2018). "Accelerating the Machine Learning Lifecycle with MLflow". In: *IEEE Data Engineering Bulletin* 41.4, pp. 39–45. URL: https://people.eecs.berkeley.edu/~matei/papers/2018/ieee_mlflow.pdf.

Zaheer, M., S. Kottur, S. Ravanbakhsh, B. Póczos, R. Salakhutdinov, and A. Smola (2017). "Deep Sets". In: *Advances in Neural Information Processing Systems*. Vol. 30. arXiv: 1703.06114.

Zangato, T., A. Osmani, and P. Alizadeh (2025). "Data-Driven Policy Mapping for Safe RL-based Energy Management Systems". In: *Energy Reports* 13, pp. 1888–1909. arXiv: 2506.16352. URL: https://arxiv.org/abs/2506.16352.

Zhang, H., D. Qiu, K. Kok, and N. G. Paterakis (2025). “Reliability assessment of multi-agent reinforcement learning algorithms for hybrid local electricity market simulation”. In: *Applied Energy* 389, p. 125789. DOI: 10.1016/j.apenergy.2025.125789. URL: https://www.sciencedirect.com/science/article/pii/S0306261925005197.

Zhang, H., D. Wu, A. Zinflou, and B. Boulet (2025). “Meta-Reinforcement Learning for Building Energy Management System”. In: *IEEE Electrical Power and Energy Conference (EPEC)*. arXiv: 2210.12590. URL: https://arxiv.org/abs/2210.12590v3.

Zhang, Q., K. Dehghanpour, Z. Wang, F. Qiu, and D. Zhao (2021). “Multi-Agent Safe Policy Learning for Power Management of Networked Microgrids”. In: *IEEE Transactions on Smart Grid* 12.2, pp. 1048–1062. DOI: 10.1109/TSG.2020.3034827.

Zhang, Y., Z. Zhang, J. Ji, M. Quiñones-Grueiro, W. Barbour, D. Gloudemans, G. Zachár, C. Weston, G. Biswas, and D. B. Work (2026). “Real-World Deployment and Assessment of a Multiagent Reinforcement Learning-Based Variable-Speed-Limit Control System”. In: *IEEE Intelligent Transportation Systems Magazine* 18.2, pp. 68–85. DOI: 10.1109/MITS.2025.3592040. arXiv: 2503.01017.

Zhao, J., Y. Zhao, W. Wang, M. Yang, X. Hu, W. Zhou, J. Hao, and H. Li (2022). “Coach-Assisted Multi-Agent Reinforcement Learning Framework for Unexpected Crashed Agents”. In: *Frontiers of Information Technology & Electronic Engineering* 23, pp. 1032–1042. DOI: 10.1631/FITEE.2100594.

Zhou, H., A. Aral, I. Brandic, and M. Erol-Kantarci (2022). “Multiagent Bayesian Deep Reinforcement Learning for Microgrid Energy Management Under Communication Failures”. In: *IEEE Internet of Things Journal* 9.14, pp. 11685–11698. DOI: 10.1109/jiot.2021.3131719. URL: https://doi.org/10.1109/jiot.2021.3131719.

Appendix A

Simulation Studies of Community Energy Management

This appendix provides the full comparison supporting the simulation-based evidence discussed in Section 3.2.1. Table A.1 compares the 50 selected simulation applications. It groups studies by their principal application, although several combine trading, storage, and demand control. The assessment column identifies what is compared or measured, and the final column summarises the resulting observation. Experimental populations and evaluation periods refer to simulated systems, including cases supplied with measurements from real-world installations.

Table A.1: Simulation studies of community energy management

Study	Application	Scale and scenario	Assessment	Main observation
Local exchange and community markets				
Prasad and Dusparic (2019)	Renewable-energy sharing	3, 4 and 10 homes; three-day episodes	Sharing between household agents	Learned allocation of local renewable energy.
Pereira et al. (2022)	Peer-to-peer trading	50 participants; one market week	Trained and untrained traders	Outcomes differ between participants; learning changes the distribution of costs.
Qiu, Wang, et al. (2021)	Trading with batteries	Eight agents; Ausgrid profiles	Grid-only, random bidding and RL	More internal exchange and lower bills in the modelled auction.
Pu et al. (2022)	Buyer–seller trading	Modelled local market	Individual and community returns	Learning supports local exchange and energy-saving decisions.
Feng and Liu (2025)	PV and storage trading	Networked prosumers	Trading and storage decisions	Joint scheduling accounts for the network linking participants.

Continued on the next page.

Table A.1: Simulation studies of community energy management (continued).

Study	Application	Scale and scenario	Assessment	Main observation
Zhang, Qiu, et al. (2025)	Community and peer-to-peer markets	3–30 aggregators; one test week	Equilibrium, costs and computation	Reward convergence can accompany inferior market equilibria.
Ye et al. (2025)	Trading with heterogeneous batteries	250 household profiles; eight battery types	Independent, joint and federated learning	Improved trading returns and coordination in the simulated market.
Lou et al. (2026)	Network-constrained trading	20–100 prosumers; 141-node grid	Costs and voltage deviations	Trading performance is assessed together with network effects.
Shah, Barrett, et al. (2025)	Trading with uncertain forecasts	Ten farm/household prosumers; two EVs	Forecast-informed decisions	Forecast uncertainty affects trading and storage behaviour.
Chen, Yan, et al. (2026)	Community trading and storage	100 prosumers; 52 test days	Costs and peaks under a transformer limit	Coordinated decisions improve the evaluated economic and aggregate-load objectives.
Shah, Cruz Victorio, et al. (2026)	Renewable-energy trading between farms	Ten farms per country; one simulated year	Trading access and learned decisions	Separates benefits of local exchange from those of learning.
Xu, Zhao, et al. (2026)	Household storage and grid services	200 homes; year-long profiles	Trading, regulation and preferences	Batteries provide several services under assumed stable preferences.
Residential storage and flexible demand				
Fang et al. (2019)	Renewables, demand and EV/V2G	Residential microgrid	Agent benefits and energy scheduling	Scheduling balances the participants' interests in the modelled market.
Sajid, Li, Berehman, et al. (2025)	Residential batteries	17 homes; CityLearn	DR with independent learners	Battery schedules respond to the residential demand objective.
González-Briones et al. (2026)	PV and battery operation	Two homes; agent middleware	Energy use and economic benefit	Executable agents coordinate a simulated energy system.
Charbonnier, Morstyn, et al. (2022b)	EVs, heating and flexible loads	Up to 30 prosumers	Cost and computational scaling	Coordinated flexibility is evaluated as the household population grows.
Nweye, Sankaranarayanan, et al. (2023)	Household battery control	17 homes; CityLearn	Occupant-related objectives and reuse	Previously collected experience supports simulated community control.

Continued on the next page.

Table A.1: Simulation studies of community energy management (continued).

Study	Application	Scale and scenario	Assessment	Main observation
Savino et al. (2025)	Electrical and thermal storage	Up to 100 buildings	Demand management and scale	Coordinated control extends to a large modelled district.
Pokorn et al. (2026)	Community PV and batteries	Simulated energy community	Operating cost	Learned control and linear optimisation jointly reduce costs.
Khan et al. (2026)	PV, storage and flexible demand	5–30 homes; hourly, 24-hour horizon	Cost, comfort and asset degradation	Improves a composite objective under deterministic conditions.
Hribar et al. (2025)	EVs and shared storage	Positive-energy district; mixed input data	Energy autonomy	Coordinated flexibility increases local use of renewable energy.
Caputo et al. (2026)	Residential resources and comfort	11 homes, including five prosumers	Economy, comfort and sustainability	Evaluates multiple community objectives in an IoT-oriented model.
Wilk et al. (2024)	HVAC and EV charging	Three identical and nine heterogeneous buildings	Cooperative and competitive objectives	Collective performance and benefit distribution vary with coordination.
Fraija et al. (2024)	Price-responsive household demand	Multiple aggregators	Prices, peaks and allocation of benefits	Learned pricing and benefit allocation support DR.
Dusparic, Harris, et al. (2013)	Residential EV charging	Nine homes; 24-hour forecasts	Current versus forecast information	Forecasts inform coordinated charging decisions.
Dusparic, Taylor, Marinescu, Cahill, et al. (2015)	EV charging with renewable supply	Nine and 90 homes; 30 days	Demand–generation alignment	Decentralised decisions increase use of available renewable energy.
Dusparic, Taylor, Marinescu, Golpayegani, et al. (2017)	Residential DR	90 EVs in a household network	Learning, rules and other controllers	Compares self-organising strategies under a shared scenario.
Marinescu, Dusparic, et al. (2017)	Forecast-informed DR	Residential load scenarios	Adaptation to changing demand	Performance depends on predictions and changing consumption patterns.
Kaewdornhan et al. (2023)	PV and EV charging	Four prosumers and an aggregator	Dynamic versus fixed tariff structures	Coordinated schedules reduce costs under the tested prices.

Continued on the next page.

Table A.1: Simulation studies of community energy management (continued).

Study	Application	Scale and scenario	Assessment	Main observation
Lee, Wang, et al. (2020)	Household load scheduling	Ten homes	Cost, delay and demand peaks	Learned schedules balance expense and consumption timing.
Yang, Hao, et al. (2019)	PV, batteries and EV charging	200 homes; annual consumption profiles	Rules, Deep Q-Network (DQN) and collective learning	Coordination improves cost and peak-demand objectives.
Liu and Chen (2023)	HVAC with shared storage	Two buildings and one storage agent	Heuristics and centralised control	Shared flexibility brings cost benefits with temperature trade-offs.
Microgrids and coupled energy resources				
Deshpande et al. (2022)	Generation, storage and demand	Microgrid with historical profiles	Energy balance and linear-program reference	Learned decisions balance flows and generalise across tested conditions.
Huang and Doya (2023)	DC energy sharing	Four institutional houses represented in a power-interchange emulator	Distributed control in an emulator	Uses installation data to evaluate learned energy exchange.
Mohammadi et al. (2025)	Generation, storage and five loads	MATLAB model of an energy laboratory	Resource scheduling	Learning and auctions allocate energy within the laboratory model.
Arshad et al. (2026)	PV, batteries and flexible demand	Synthetic residential scenarios	Uncertainty, comfort and degradation	Evaluates coordinated operation under uncertain inputs.
Wu, Chiu, et al. (2026)	Storage and microgrid resilience	Institute of Electrical and Electronics Engineers (IEEE) 33-node cases; simulated disturbances	Outages, forecast errors and agent faults	Demonstrates resilience within the tested disturbance scenarios.
Gao et al. (2023)	Trading and microgrid scheduling	Connected microgrids; empirical inputs	Distributed scheduling and exchange	Joint decisions coordinate internal resources and external transactions.
Zhou et al. (2022)	PV, storage and demand	Microgrid with communication failures	Performance under lost communication	Explicitly tests learning-based control with impaired information exchange.

Continued on the next page.

Table A.1: Simulation studies of community energy management (continued).

Study	Application	Scale and scenario	Assessment	Main observation
Diddigi et al. (2017)	Storage and stochastic generation	Modelled microgrids	Demand–supply deficit	Learning reduces the mismatch between available power and demand.
Xiao et al. (2018)	Wind generation and trading	Microgrids; historical wind and prices	Energy transactions	Learned trading responds to variable renewable supply.
Harrold et al. (2022)	Multiple storage technologies and trading	Main microgrid and five trading partners	Rules, single-agent and multi-agent control	Benefits depend on resource coordination and trading opportunities.
Wang, Cui, et al. (2023)	Electricity, heat and shared storage	Three microgrids; 30 test days	Trading and storage arrangements	Shared resources improve the tested operating objectives.
Ren and Gao (2026)	Low-carbon energy trading	Four microgrids; normalised household profiles	Auction and learning strategies	Examines costs and emissions under simplified storage assumptions.
Building and network interactions				
Nweye, Liu, et al. (2022)	Grid-interactive building storage	Nine buildings; CityLearn	Prior experience and information sharing	Training experience affects performance; more shared information is not always better.
Pigott et al. (2022)	Building storage and grid operation	192 buildings; IEEE 33-node network	Costs, voltage and climate transfer	Aggregate voltage gains coexist with locally worse outcomes.
Hosseini, Erba, et al. (2025)	Collaborative building management	Six public-building models	Historical and future climate conditions	Examines flexibility and climate resilience across buildings.
Yu, Sun, et al. (2021)	Multi-zone commercial HVAC	Simulated building zones	Energy expense and indoor conditions	Coordinated thermal control balances cost and comfort.
Mai et al. (2024)	Fast residential DR	Up to 1000 thermostatic loads	Signal tracking, scale and heterogeneity	Coordination supports rapid response; resource differences affect performance.
Nik and Hosseini (2023)	Thermal energy management	Modelled building; climate scenarios	Energy, comfort and climate resilience	Collaborative agents adapt building operation within the simulation.

Appendix B

Reproducibility and Supplementary Material

This appendix organises the material required to reproduce and audit the principal findings without interrupting the main argument.

B.1 Claim and Evidence Inventory

A master inventory maps every retained claim to the relevant RQ, method, repository and commit, scenario, dataset, experiment identifier, seeds, metrics, result table or figure, evidence level, publication, author contribution, and known limitation. This mapping defines the evidence path used throughout Chapters 4–9.

B.2 Scenario and Environment Manifests

Reconstruction of the principal experiments requires schemas or manifests that record temporal resolution, units, buildings, assets, data sources, event streams, constraints, observation and action interfaces, reward definitions, KPI configuration, software versions, and referenced data checksums.

B.3 Portugal 2023–2024 Dataset Analysis and Audit

This section records the construction, descriptive analysis, and verification of the collection introduced in Section 5.3. The edition is `portugal-native-2023-2024-v3.1`, prepared for the simulator runtime identified by commit `978be68b`. The four scenario bases support nine configurations across 731 consecutive days. Each series has 70,176 quarter-hour records, with a regular UTC time axis and a corresponding Lisbon civil calendar. The descriptive quantities below are calculated from inventories, input series, service schedules, and configured events.

Three counting rules apply throughout. The complete inventory describes the resources present in a Normal configuration; the active inventory follows participation and equipment events; and a service request describes an obligation supplied to a controller. Energy delivered, self-consumption, grid exchange, and constraint outcomes are calculated from subsequent simulation. The member populations are constructed RECs with shared generation rules and external signals, so totals across bases are not treated as independent population samples.

B.3.1 Source Data, Preparation, and Reproduction

The preparation combines public Portuguese energy information with fictional households and service premises. Consumption profiles use activity and occupancy routines with simplified end-use components. E-REDES profiles inform their temporal shape, while Portuguese aggregate statistics anchor consumption levels (Direção-Geral de Energia e Geologia 2023; Pordata 2023). The household atlas gives readable descriptions of the same routines and equipment used to construct the data.

Solar inputs use the archived Photovoltaic Geographical Information System (PVGIS) resource for Lisbon, including the installed module area, tilt, orientation, shading, and inverter limit. Hourly resource inputs are interpolated to the quarter-hour clock. The 2024 weather preparation uses temperature, wind, and humidity from the fifth-generation European Centre for Medium-Range Weather Forecasts atmospheric reanalysis (ERA5) through Open-Meteo; nine missing solar instants, including seven daytime instants, were filled from the same-period ERA5 data and recorded individually. Humidity in 2023 follows a synthetic temperature-based approximation, whereas 2024 uses ERA5. The source ledger retains this difference when the two years are compared.

OMIE supplies the hourly Portuguese day-ahead price series. Its market periods are interpreted in Madrid civil time before UTC alignment. Every member uses the same BTN bi-hourly indexed purchase reference. The reconstruction applies the declared supplier, loss, access-tariff, and tax components for the appropriate year; fixed charges and Value Added Tax (VAT) are outside the variable-energy comparison. Realised prices retain their sign. The minimum final variable purchase price in the 2024 reference is 0.02644 EUR/kWh.

Operational carbon intensity is derived from the Redes Energéticas Nacionais (REN) generation and interchange chronology, with Red Eléctrica de España (REE) Spanish production intensity used for the imports proxy. The thermal coefficient is calibrated to the Agência Portuguesa do Ambiente (APA) mainland production factor for 2023 and retained for 2024. The resulting input represents an estimated average operational intensity. It supplies a consistent emissions accounting signal alongside price, while its source and calibration remain identifiable.

Raw source files, acquisition metadata, preparation scripts, and checksums accompany the collection. The current package contains 4,015 files. Its manifest identifies the data edition, runtime, archived 2023 predecessor, and descriptive report. Reproduction uses the archived sources and generation settings. The two-year scenario inputs preserve the 2023 demand, production, realised

prices, weather, and carbon values; forecast targets at the year boundary are updated to follow the continuous chronology.

B.3.2 Resource Diversity and Energy Profiles

Core-4 and Core-15 are residential. Core-30 contains 24 households and six services; Premium-100 contains 80 households and 20 services. Premium includes four offices, four retail shops, three clinics, three cafes, three workshops, a sports centre, a grocery store, and a care home. Residential floor areas range from 55 to 179 m^2 in Premium, whose households contain one to five residents. Opening calendars distinguish working days, weekends, holidays, and continuous services.

Table B.1: Generation, storage, and appliance inventories in the complete static bases.

Quantity	**Core-4**	**Core-15**	**Core-30**	**Premium**
PV capacity [kWp]	3.78	16.80	89.46	312.48
Battery capacity [kWh]	10.24	10.24	25.60	220.16
Battery rated power [kW]	5.00	5.00	12.68	108.40
Single-phase sites	4	14	23	72
Three-phase sites	0	1	7	28
Controllable appliances	2	10	19	64
Fixed appliances	5	18	27	84

Installed capacities vary independently of the price convention. Premium has nine batteries of 5.12 kWh, nine of 10.24 kWh, and two of 40.96 kWh. A household with a single-phase connection has different charging headroom from a service site with several ports and a three-phase connection. The electrical model maps the declared connection ratings to active-power ceilings under its unity-power-factor convention, using Portuguese connection classes as reference settings (E-REDES 2018; Entidade Reguladora dos Serviços Energéticos 2026a).

Table B.2 separates non-shiftable demand, PV input, and requested controllable-cycle energy. Fixed appliance energy is already part of the background demand and is counted once. EV charging and stationary-battery dispatch enter the energy balance when a controller is executed.

Table B.2: Annual input energy in the complete static bases, in MWh.

	Non-shiftable demand		**PV production**		**Cycles**
Base	**2023**	**2024**	**2023**	**2024**	**2024**
Core-4	12.10	11.79	5.44	5.37	0.29
Core-15	43.82	42.58	25.19	24.81	1.39
Core-30	164.94	158.39	131.38	129.07	3.62
Premium-100	583.81	560.19	476.60	469.16	11.78

The monthly profiles in Figure 5.7 expose seasonal differences that annual totals conceal. Figure B.1 additionally shows the spread across individual members. Consumption-only participants and larger service premises create an uneven relationship between demand and generation. This

diversity matters when interpreting a REC total: an improvement can arise in a small number of resource-rich members while others experience little change.

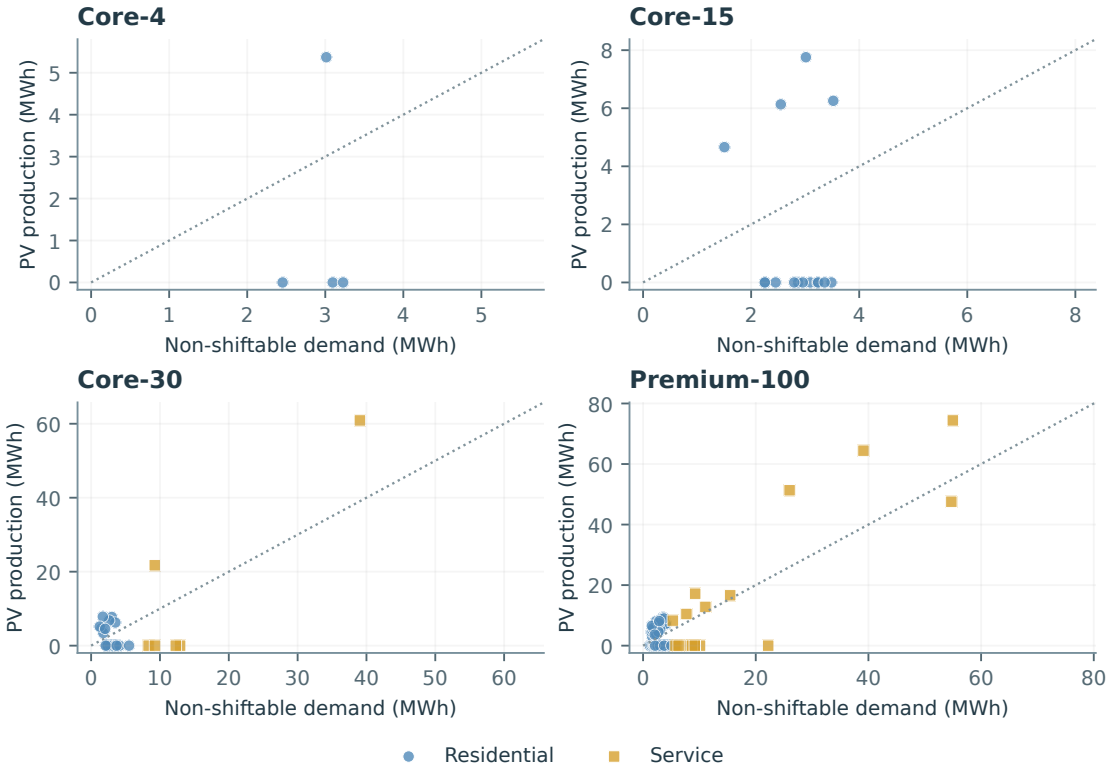


Figure B.1: Member-level non-shiftable demand and PV production in 2024. Each point represents one member of a complete static base; the dotted line denotes equal annual demand and production.

B.3.3 Charging and Appliance-Service Distributions

The complete catalogues contain 2, 6, 19, and 85 vehicle identities, associated with 2, 6, 15, and 65 connectors in Core-4, Core-15, Core-30, and Premium, respectively. Connector discharge power is zero in these datasets. The simulator's V2G capability therefore remains available for other configurations, while the current collection studies charging under its published equipment specifications.

Sessions are reconstructed from the Normal configuration's connection records. A visit spanning New Year remains one session attributed to its starting year. Complete visits exclude those censored at the beginning or end of the two-year record. Table B.3 reports the visit counts and complete-session service distributions.

The connection time distribution distinguishes residential parking from service visits. Long residence at a connector creates a scheduling window, although part of that interval may remain unused after the target is reached. Vehicle identity supports repeated visits and selected home–work

Table B.3: Charging-session counts and service distributions across the two-year catalogues.

Quantity	**Core-4**	**Core-15**	**Core-30**	**Premium**
Visits starting in 2023	620	1,925	4,826	20,524
Visits starting in 2024	617	1,919	4,865	20,764
Complete visits, both years	1,233	3,832	9,673	41,214
Median target SoC [%]	83.7	78.3	78.5	78.5
Targets at 100% [%]	2.84	1.28	0.95	0.77
Median dwell [h]	14.75	15.00	13.25	13.00
90th-percentile dwell [h]	40.75	43.25	29.75	37.00

links. Arrival SoC is supplied per session; the native runtime simulates charging from that boundary rather than propagating a complete transport-energy model between sites.

Controllable appliances are washing machines and dishwashers with integrated programme profiles. Across 2023 and 2024, the four bases contain 681, 3,443, 7,641, and 25,521 requests. Median start-window slack is six hours in Core-4 and Core-15 and three hours in Core-30 and Premium. Median programme duration ranges from 3.25 to 3.50 hours. Figure B.2 relates the distribution of vehicle targets to the scheduling freedom in appliance requests. These describe different forms of flexibility and retain different service measures in evaluation.

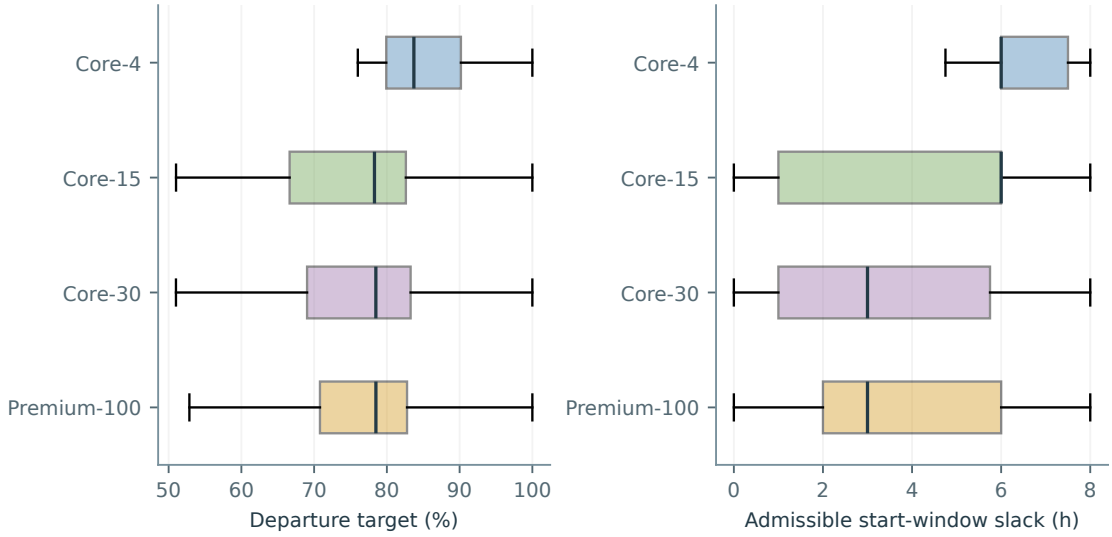


Figure B.2: Complete-session departure targets and appliance start-window slack in the two-year collection. Boxes show the interquartile range and median; whiskers reach the furthest values within 1.5 interquartile ranges. Outliers are omitted.

B.3.4 Events, Forecasts, and Experimental Boundaries

The event schemas determine the active REC. Core-30 Topology and Combined each admit four members and remove three, changing the active population from 26 to 27. Premium All-in changes from 80 to 88 active members. The 2023 participation and asset events remain effective throughout

2024. Separate health, outage, noise, and demand-response campaigns recur in the second year while preserving their declared duration and severity.

The union of scheduled outage intervals spans 32 hours over both years in each configuration with outages. Affected site-hours additionally depend on how many members are interrupted at once and whether they remain active. The native backup representation is aggregate; delivered energy during an outage depends on stored energy and the executed decisions. Permanent departure ends the member's simulated participation and its subsequent REC obligations.

Fault schedules distinguish development from confirmation. For example, the development price-bias event adds 0.025 EUR/kWh for six hours in each year; the reserved event uses a bias of −0.035 EUR/kWh for eight hours. Multiple channel rows can describe one event, so row counts and exposure duration are recorded separately. The native interface can expose health-event information. A controller experiment must consequently declare the selected observation fields and compare policies under the same information boundary.

Price forecasts use published OMIE values after the conservative 13:00 Lisbon publication boundary, with persistence before publication. Ninety-six support records for 1 January 2025 complete price forecast targets at the end of 2024 without extending the simulated period. Weather follows daily persistence. Table B.4 reports selected stored-input errors against the realised target value, excluding the first and last day and excluding inactive night periods from solar-error summaries.

Table B.4: Selected forecast mean absolute errors in the stored inputs.

Signal	**Year**	**1 h**	**6 h**	**24 h**	**Unit**
Temperature	2023	1.160	1.159	1.156	°C
Temperature	2024	1.252	1.251	1.250	°C
Direct irradiance, daytime	2023	179.512	179.512	179.440	W/m^2
Direct irradiance, daytime	2024	186.206	186.206	185.590	W/m^2
Purchase price	2023	0.031	0.039	1.391	cent/kWh
Purchase price	2024	0.028	0.028	1.282	cent/kWh

Persistence uses the same previous-day value for a given target, explaining the similar weather errors across horizons. Published day-ahead prices make the short-horizon price errors smaller. The derived consumption and PV forecast path in the original engine clips its target at the loaded episode boundary before applying persistence. Stored price and weather forecasts retain their intended targets, while experiments using the derived channels need a common treatment of the shortened terminal horizons.

Demand-response requests retain the native preceding-hour-mean reference. The operational candidates request 80 kW reduction or 40 kW absorption for one hour; the separate 1500 kW reduction request lasts four hours and is labelled as impossible stress. A comparison reports contractual response against the native reference and additional response against a paired no-activation replay. The earlier calibrated multi-day reference and private-service accounting belong to retired evaluation designs and are not part of this edition.

B.3.5 Audit Results and Evaluation Record

The current evidence consists of input checks and descriptive calculations. Table B.5 gives the checked object and result. The runtime is pinned in the manifest, so software verification and new controller execution can be associated with the same data edition when those experiments are conducted.

Table B.5: Verification records for the current Portugal collection.

Record	Checked quantities	Reported result
Two-year integrity	Time axis, leap day and daylight saving, common prices, preserved 2023 physical inputs, inventory, sessions, cycle windows, events, checksums, and unchanged runtime core.	2,828 checks; zero failures.
Forecasts and member histories	Publication-aware price targets, end-of-year support, forecast statistics, and correspondence between narratives and scenario events.	213 checks; zero failures.
Individual EV requests	Complete 2024 sessions checked against accepted charging power, losses, battery taper, and connection duration.	28,111 sessions checked.
Descriptive report	Inventory, energy aggregation, service counts, event summaries, and source identities.	34 consistency checks.

Individual session feasibility establishes that the declared request can fit its own charging opportunity under the checked assumptions. Concurrent household demand, other charging sessions, electrical limits, and disturbances enter the joint simulation. The two-year data release contains no new annual controller evaluation. Earlier BAU and rule-based results therefore remain attached to the historical records in the following section.

For each subsequent experiment, the evaluation record identifies the data and simulator versions, resolved schema, controller input fields, service denominators, initial states, training and scored intervals, seeds, reference policy, and market settings. Warm-up and recovery are retained separately from scored actions. A run reports the number of executed transitions and its terminal-state convention. This record implements the common protocol in Section 5.4, while the controller chapters define the method-specific selection and comparison settings.

B.4 Historical Annual Dataset Contract

This section archives the preceding annual suite and its original BAU/RBC Smart audit. Its inventory, service denominators, prices, and terminal-state convention belong to that historical dataset. The current Portugal 2023–2024 collection and its audit are described in Section 5.3 and Appendix B.3; the tables below are retained for reproducibility of earlier evidence.

The following record describes the preceding annual suite and the conditions of its archived experiments. The row-level record remains authoritative. Each family contains `dataset_manifest.json`, `settlement_contract.json`, variant schemas, Parquet catalogues for members, electrical services, chargers, EVs, sessions, and deferrables, event files, daily OMIE provenance,

validation output, and `file_checksums.sha256`. The generator seed is 2023 and the contract version is `2023-q15-v1.9`. All families span 35,040 consecutive 900-second UTC intervals in 2023 and retain `Europe/Lisbon` local-calendar and daylight-saving attributes. Routine instants are first constructed in Lisbon civil time and only then converted to UTC; the missing spring hour is shifted forward by one hour and the first occurrence of the repeated autumn hour is selected deterministically. A complete annual controller run appends an observation-only terminal boundary at 1 January 2024 00:00, yielding 35,041 state points and exactly 35,040 action transitions without adding a physical energy or settlement row.

B.4.1 Calendar, price forecasts, and settlement

The current-price column is the official Portuguese OMIE 2023 day-ahead series converted from EUR/MWh to EUR/kWh. Each physical hourly value is repeated over four 15-minute steps; the 23- and 25-period daylight-saving days are preserved by the continuous UTC calendar. Forecast columns target 1, 6, and 24 hours. For a delivery interval whose day-ahead result is available by the declared conservative 13:00 local cut-off, the forecast contains that published value; before the cut-off it contains the corresponding previous-day value. The entity bundle also exposes 15-minute and 3-hour horizons. These use the shortest stored horizon capable of targeting the same delivery step, read at the corresponding earlier issue time. At an episode cold start with no such issue in the retained window, the current price is used. This rule is tested independently against the controller-visible observation and prevents the interface from reconstructing an unpublished realised price from the full annual table.

Settlement matches member surplus and demand within the same step, uses equal importer weights and a local price equal to 80% of the grid-import price, and assigns zero remuneration to residual grid export. The retained grid-only counterfactual disables local matching but preserves the same member demand, generation, actions, and OMIE price. Taxes, network tariffs, supplier margins, and regulated export compensation are outside this experimental accounting contract.

B.4.2 Demand, PV, and stationary storage

Demand profiles combine class- and routine-specific intraday structure, seasonal variation, correlated daily multipliers, local peak-time shifts, reduced-activity days, and correlated member-level short-term variation. The raw trajectory is scaled to an annual target sampled by rejection from the bounded log-normal ranges in Table B.6; this avoids an artificial point mass at either bound. The class centres are 2,490, 16,580, and 175,900 kWh per consumer-year and are anchored to Portuguese 2023 residential, non-domestic, and industrial averages (Direção-Geral de Energia e Geologia 2023; Pordata 2023). The ranges and routine frequencies create controlled scenario heterogeneity; they are not empirical confidence intervals or inferred population proportions.

Table B.7 lists the behaviour strata. The assignment is deterministic and family-independent, so the first 15 members remain identical between Core-15-Stripped and Core-30. Within an archetype,

Table B.6: Static-asset generation contract by member class. PV and BESS entries apply only to members that own the corresponding asset.

Member class	Annual demand target	PV contract	Stationary BESS contract
Residential	1.1–6.5 MWh; centre 2.49 MWh	1.5–8.0 kW; target annual coverage 0.58–1.32	5, 7.5, 9.6, 10, 13.5, or 15 kWh; 2.5–6 kW product classes
Small service/commercial	7–45 MWh; centre 16.58 MWh	8–40 kW; target annual coverage 0.42–1.02	20, 30, 40, 50, 60, or 75 kWh; 10–30 kW product classes
Larger consumption	75–350 MWh; centre 175.9 MWh	30–150 kW; target annual coverage 0.32–0.82	60, 80, 100, 120, 160, or 200 kWh; 30–80 kW product classes

member-specific amplitudes, centres, activity reductions, annual targets, and short-term variation remain independently reproducible. Occupancy varies with the routine: commuter homes are largely unoccupied during the working day, home-working and home-day members retain daytime occupancy, and service occupancy follows its opening or process window.

Table B.7: Routine archetypes used to construct heterogeneous demand and flexibility.

Member class	Five stratified archetypes	Coupled effects
Residential	Early commuter; standard commuter; late commuter; home worker; predominantly home-day	Morning/evening/lunch balance, weekday absence, weekend profile, reduced-activity days, EV arrival/departure, and appliance request time
Small service/commercial	Office hours; six-day retail; split-day clinic; extended hospitality; early workshop	Opening days and hours, split or extended operation, occupancy, customer/employee charging windows, and type/timing of the deferrable service
Larger consumption	Day process; extended process; continuous process; morning-intensive process; evening-intensive process	Shift coverage, weekend reduction, maintenance days, shared-charger visits, and batch/process flexibility windows

PV capacity is obtained from the annual demand target, the common-profile specific yield, and the coverage range in Table B.6, subject to the stated class bounds. Orientation is sampled as east, south, or west with probabilities 0.25, 0.50, and 0.25; derating is sampled between 0.84 and 0.98 and combined with reproducible member-level shading. Generation is stored as absolute per-step energy. Stationary BESS capacity is selected from the product-like classes in the table by a stratified fraction of average daily demand, with a lower sizing floor tied to host PV. Nominal power is selected from the compatible product class and does not exceed 0.65 times energy capacity. Efficiency is sampled from 0.920–0.965, initial SoC from 0.25–0.60, depth of discharge from 0.80, 0.85, 0.90, or 0.95, standing-loss coefficient from 0.0005–0.0015, and capacity-loss coefficient from 0.5×10^{-5}–1.5×10^{-5}. A single-phase battery follows its member's assigned phase; a battery at a three-phase member is connected across all phases. These are benchmark parameter classes, not a market-share model.

B.4.3 Electrical connections and phase allocation

Table B.8 gives the rule used to construct member-level connection limits. The larger-consumption class uses a Baixa Tensão Especial (BTE) surrogate. The levels reproduce Portuguese connection classes at the resolution required by the simulator (E-REDES 2018; Entidade Reguladora dos

Serviços Energéticos 2026a). CityLearn advances active energy, so each kVA value is mapped to the same numerical kW ceiling under an explicit unity-power-factor assumption. For a balanced three-phase connection, each phase receives one third of the total ceiling; for a single-phase connection, the complete ceiling is assigned to one of L1–L3. Residential single-phase assignments rotate deterministically across members. The export ceiling is $\min(I_b^{\max}, P_b^{\mathrm{PV}} + P_b^{\mathrm{BESS}})$ and is divided using the same phase rule. This is a connection-headroom benchmark, not a voltage, reactive-power, protection, or feeder-power-flow model.

Table B.8: Electrical-service and charger contract. Charger powers are compatible with the charging classes described by Entidade Reguladora dos Serviços Energéticos (ERSE) (Entidade Reguladora dos Serviços Energéticos 2026b).

Configuration	**Contracted levels (kVA)**	**Installed charger power**	**Phase rule**
Residential, ordinary single-phase	3.45, 4.60, 5.75, 6.90, or 10.35	none, 3.7 kW, or 7.4 kW	Member and charger assigned wholly to a rotated L1, L2, or L3 REC phase
Residential, one three-phase charger	13.80 or 17.25	one 11 kW charger	Uncontrollable demand and charger divided over L1–L3
Residential, two chargers	17.25 or 20.70	two independently controlled 7.4 kW chargers	Chargers connected to two rotated individual phases; shared total and per-phase limits
Small service/commercial	20.70, 27.60, 34.50, or 41.40	two 11 kW chargers where equipped; 5.5 kW V2G limit	Balanced three-phase connection; each charger spans all phases
Larger consumption	69.0, 103.5, or 138.0	four or six 22 kW chargers where equipped; 11 kW V2G limit	Balanced three-phase BTE-level surrogate; each charger spans all phases

The member-level catalogue records connection type, contracted kVA, the originally sampled class, active-power assumption, native demand and export peaks, total import/export limits, L1–L3 limits, default split, and asset phase connection. The charger catalogue separately records each physical charger and its phase. This permits two chargers at one member to receive independent actions while remaining jointly constrained by one connection. A rejection check increases the connection class when the uncontrolled native trace would otherwise exceed the sampled level before any flexible action is applied. This affects only two Premium members: Member 094 is raised from 69 to 138 kVA for native peaks of 111.5233 kW import and 97.5324 kW export, and Member 100 is raised from 69 to 103.5 kVA for a 99.6939 kW import peak. The original level and upgrade flag remain in the schema; no other family member requires this correction.

Electrical reporting distinguishes the pre-projection pressure of the requested action from the post-projection residual of the applied trajectory. Both are integrated as energy and counted as events. The residual calculation is reconstructed from total and phase power histories with a 10^{-5} kW numerical tolerance and is the hard safety criterion. Requested pressure remains a controller diagnostic and can be non-zero while the residual is zero. Robustness reporting follows an analogous identity rule: observation, forecast, and action corruptions retain field- or port-level counts, whereas asset-unavailable duration counts each physical asset once per affected time step, irrespective of the number of internal fields that expose its status.

B.4.4 EVs, sessions, and deferrable services

EV capacities are sampled from 40, 50, 58, 64, 75, and 82 kWh, with 0.90 depth of discharge and initial SoC between 0.30 and 0.60. Residential chargers use one persistent vehicle identity; shared service chargers use pools of four vehicles. Residential visit probability, arrival centre, following-morning departure centre, temporal spread, and trip-energy scale depend on the host routine and differ between weekdays and weekends. Service and larger-consumption chargers use two to four routine-dependent probabilistic windows on their eligible operating days. Independent charger-level draws allow two chargers at the same member to receive different sessions. All times are jittered and rounded to the 15-minute contract; cross-midnight normalisation prevents two sessions from occupying one physical charger simultaneously.

Required session energy is sampled from class-conditional log-normal scenario distributions with a 6 kWh residential base median multiplied by a 0.66–1.12 routine factor, a 9 kWh small-service median, and a 15 kWh larger-consumption median. It is limited to 85% of $\Delta t\, P^{\max} \eta$ for residential sessions, 72% for shared service sessions, and 72% of EV capacity before arrival and target SoC are written. The catalogue also records the minimum full-power steps and residual temporal slack. Every ordinary session is individually deliverable at 95% charger efficiency. Joint service can nevertheless become infeasible after total/phase limits, concurrent sessions, topology changes, demand-response obligations, or injected failures are introduced. Table B.9 freezes the generated catalogue counts and the zero-infeasibility audit result.

Every occupied charger row additionally stores the explicit session identity and an optional current-SoC boundary reference. The reference follows the neutral constant-rate trajectory from the declared arrival SoC to the departure target and is consulted only when an episode, member, charger, or new session becomes active inside an occupied interval. It prevents an arbitrary catalogue initial SoC from being substituted for omitted history; it does not overwrite the controller-driven SoC during an uninterrupted session. This rule covers the 0, 3, 4, and 69 back-to-back session boundaries in Micro, Core-15, Core-30, and Premium, respectively, including the case in which the same residential EV begins the new session and charger occupancy remains continuous. The audit verifies that session identity agrees with the catalogue, the reference is finite and monotonic while connected, absent while disconnected, equal to arrival SoC at the session boundary, and leaves a deliverable remainder at every intermediate step. At runtime, a separate conservation check sums positive connected-session changes in stored EV energy and compares them with charger input. Any SoC gain not supported by metered input, beyond numerical tolerance, fails the controller-level audit.

All 47,258 distinct catalogue sessions across the four families are individually feasible before shared constraints and injected events are applied. For the dynamic variants, censoring is split by cause without double counting: Core-30 has 56 member-inactive and 28 charger-inactive sessions; Premium-100 has 13,048 and 17, respectively. The annual controller audit reconstructs these counts from the topology calendar and requires the recorded departure-event denominator to match. This separates an unavailable catalogue service from a failure to serve an obligation that the controller

Table B.9: Generated EV identities, annual session counts, and topology-observable departure obligations. A censored session is absent from the controller's service denominator because its member or physical charger does not exist at the relevant boundary.

Family/variant	**EVs**	**Sessions**	**Observable**	**Censored**
Micro-4-Q	2	554	554	0
Core-15-Stripped	7	1,757	1,757	0
Core-30 static	15	3,774	3,774	0
Core-30 Dynamic/Combined	15	3,774	3,690	84
Premium-100 Clean/AllIn	212	41,173	28,108	13,065

actually faced.

Each deferrable asset alternates between two complete cycle profiles and receives a member-specific energy multiplier from 0.82 to 1.22. Residential appliance types comprise washing machine, dishwasher, washer–dryer, domestic-hot-water cycle, and a mixed dishwasher/laundry class. Service types comprise cleaning or preconditioning, cold storage or cleaning, sterilisation or laundry, hospitality dishwashing or hot water, and compressor or pumping. Larger-member types comprise batch process, process auxiliaries, maintenance or pumping, thermal preparation, and late batch process. Requests occur probabilistically on routine-eligible days. Residential occurrence probabilities range from 0.52 to 0.70 and `must_run` probabilities from 0.66 to 0.76; the corresponding ranges are 0.82–0.90 and 0.88–0.94 for small services, and 0.90–0.94 and 0.94–0.97 for larger members. Earliest start, window length, completion margin, deadline, and priority vary reproducibly by asset and day. A new request is omitted if its earliest admissible start would overlap the preceding request's completion window for the same appliance. The schedule retains cycle identity, earliest/latest start, deadline, priority, and mandatory-service flag so completion, delay, and unserved energy can be evaluated.

The complete Micro, Core-15, Core-30, and Premium catalogues contain 214, 928, 1,892, and 7,360 deferrable cycles, respectively. All are observable in the static variants. Under the declared dynamic topology, Core-30 has 1,657 fully observable cycles, 234 whose start window occurs entirely while the member or appliance is absent, and one whose full start window is visible but whose completion interval crosses an asset-removal boundary. Premium has 5,950, 1,409, and one in the same categories. The final category is policy-dependent because the cycle can be completed before removal or interrupted by it; the annual audit accepts the corresponding completed-plus-missed denominator only within the explicit one-cycle interval. No-start cases are topology-censored, not unserved requests.

B.4.5 Topology and robustness calendars

Topology events change whether an entity exists in the active scenario; robustness events leave it in the topology but alter data, commands, or availability. Event logs and controller-facing masks preserve these semantics. Reinstallation creates a new runtime instance from the declared template and initializes a stateful asset at its boundary state, while the pre-removal instance remains in

historical KPI aggregation. It does not imply unobserved state continuity through the interval in which the asset is absent. Tables B.10 and B.11 summarise the UTC topology calendar; exact time steps, target and source identifiers, and restoration templates remain in each variant schema.

An `add_asset` event constructs a new runtime instance from the immutable catalogue template. Paired removal/restoration entries have replacement or recommissioning semantics: the restored capability starts from its declared initial state, and no hidden SoC, degradation, or unfinished-cycle continuity is inferred across the absence. Temporary loss of the same equipment is modelled through a robustness-unavailability event; the entity identity and accumulated state remain present while the selected telemetry or control channel is unavailable. These semantics avoid attributing an undocumented state-reset convention to a controller result.

Table B.10: Core-30 dynamic topology calendar. Paired dates denote removal and restoration.

UTC date/time	**Target**	**Change**
01 Feb 06:00	Member 016	First entry
04 Mar 12:00 / 04 Apr 18:00	Member 006, charger 02	Removal / restoration
26 May 20:00 / 06 Jun 06:00	Member 003, BESS	Removal / restoration
16 Jun 16:00	Member 027	First entry
18 Aug 04:00 / 28 Aug 14:00	Member 017, deferrable	Removal / restoration
08 Sep 00:00	Member 029	First entry
28 Sep 20:00 / 09 Oct 06:00	Member 018, PV	Removal / restoration
19 Oct 16:00	Member 030	First entry
09 Nov 12:00	Member 019	Permanent exit
10 Dec 18:00	Member 008	Permanent exit

Table B.11: Premium-100 dynamic topology calendar. Member ranges denote simultaneous first-entry cohorts.

UTC date/time	**Target**	**Change**
31 Jan / 02 Mar / 01 Apr	Members 061–065 / 081–085 / 091–093	First-entry cohorts
15 Apr / 25 Apr	Member 025, charger 02	Removal / restoration
31 May / 30 Jun	Members 066–070 / 086–090	First-entry cohorts
07 Jul / 17 Jul	Member 012, PV	Removal / restoration
28 Jul / 02 Aug	Member 001, BESS	Removal / restoration
30 Jul / 29 Aug	Members 094–096 / 097–100	First-entry cohorts
07 Aug / 18 Aug	Member 071, charger 02	Removal / restoration
08 Sep / 18 Sep	Member 073, deferrable	Removal / restoration
28 Sep	Members 005, 006, 072	Permanent exits
27 Nov	Members 020–022	Permanent exits

Health and AllIn event files cover missing non-controllable-load observations, biased price forecasts, delayed storage actions, and temporary charger unavailability. Premium-100-AllIn also includes PV telemetry loss, charger-command dropout, a service-charger outage, a stationary-storage outage, and selected grid-outage intervals. Four non-overlapping demand-response requests cover upward and downward directions, different durations, tolerances, activation prices, and shortfall penalties; the Premium power targets are three times the Core-30 targets. These controlled scenario interventions carry no empirical incident-frequency claim.

The final structural audit verifies cadence and daylight saving, explicit civil-time localization policy, exact mapping between session timestamps and quarter-hour indices, the 365 official OMIE source records and their hashes, hourly-to-quarter-hour expansion, publication-aware price forecasts, all member and asset counts, referential integrity, explicit session identity, session occupancy and deliverability, mid-session SoC boundary initialization, non-overlapping deferrable deadlines, Portuguese contracted-power classes, phase sums, topology coverage, clean-twin equality, and every generated-file checksum. Across the suite, 1,817 structural and cross-family checks pass. The diversity audit also measures exact-duplicate traces and schedules, pairwise and cross-archetype profile correlations, peak-clock dispersion, annual demand, PV and BESS distributions, EV arrivals, duration, energy and slack, and deferrable-window dispersion; it reports no finding. Nine week-long schema rollouts complete 6,048 control transitions with finite observations and rewards. Across the regular and compressed rollouts, 31,565 controller-visible price-feature values match their causal inputs; the compressed dynamic timelines apply all 120 topology events and independently verify the corresponding member/asset state, the stationary-battery SoC at activation, and the physical PV balance. The full simulator regression contains 457 passing tests. The scientific audit classifies the suite as ready for benchmarking within the declared scope. The remaining limitations are the absence of a population-level goodness-of-fit claim and of feeder-level electrical physics.

B.4.6 Matched annual baseline scorecard

The final campaign contains two independently identified controller runs for each schema and binds every result to the same dataset-contract version, current family checksum, exact Simulator source digest, runtime attestation, and policy-pair fingerprint. Table B.12 gives the cost and service results. The cost column reports BAU followed by RBC Smart; the percentage column is the matched RBC change. EV values use the feasible-departure denominator, while deferrable values use the topology-valid completed-plus-missed denominator. Exact EV-target rates are omitted because both declared references stop charging within 0.04 SoC of the requirement; minimum and tolerance rates are the relevant service measures.

Table B.12: Annual matched cost and service scorecard. Paired entries are BAU/RBC Smart.

Variant	Cost (EUR) BAU/RBC	RBC Δ (%)	EV minimum BAU/RBC (%)	EV tolerance BAU/RBC (%)	Deferrable BAU/RBC (%)
Micro-4-Q	1229.62/1169.80	−4.86	100.00/100.00	100.00/100.00	100/100
Core-15-Stripped	4078.73/3964.39	−2.80	100.00/100.00	100.00/100.00	100/100
Core-30-Nominal	23497.82/23060.93	−1.86	100.00/100.00	100.00/100.00	100/100
Core-30-Safety	23496.46/23080.49	−1.77	100.00/100.00	100.00/100.00	100/100
Core-30-Health	23498.14/23061.00	−1.86	100.00/100.00	100.00/100.00	100/100
Core-30-Dynamic	12972.56/12676.37	−2.28	100.00/100.00	100.00/100.00	100/100
Core-30-Combined	12971.27/12695.37	−2.13	100.00/100.00	100.00/100.00	100/100
Premium-100-Clean	113639.38/112209.55	−1.26	99.67/99.73	99.66/99.72	100/100
Premium-100-AllIn	113610.37/112170.26	−1.27	99.63/99.65	99.62/99.64	100/100

Premium Clean records 93/77 BAU/RBC minimum misses among 28,108 observable departures. These occur only at Members 091, 095, and 097, which host four, four, and six shared 22 kW

chargers: the BAU distribution is 76, 13, and 4, and RBC Smart yields 62, 14, and 1. AllIn records 107/103 misses and spreads the additional cases over fault-affected service members. The catalogue audit independently found every session to be individually feasible. The annual misses measure simultaneous service, connection, and event pressure, with impossible requests already excluded by the generator audit.

Table B.13 retains the stress-case evidence. Requested pressure is integrated before the feasibility projection; residual violation is reconstructed from the applied total and per-phase trajectories. Robustness counts are identical for the matched policies and record physical asset–time-step unavailability, not the number of internal status fields. Four demand-response events have valid baselines in every declared run.

Table B.13: Annual safety, demand-response, and robustness scorecard for the stress variants. Paired entries are BAU/RBC Smart.

Variant	Requested pressure BAU/RBC (kWh)	Residual BAU/RBC (kWh)	DR compliance BAU/RBC	DR shortfall BAU/RBC (kWh)	Health events/ unavailable steps
Core-30-Safety	6.281/0	0/0	—	—	—
Core-30-Health	0/0	0/0	—	—	4/48
Core-30-Combined	4.696/0	0/0	0.340/0.281	228.923/264.090	4/48
Premium-100-Clean	4235.047/1465.524	0/0	0.469/0.437	705.288/765.391	—
Premium-100-AllIn	4267.160/1494.777	0/0	0.640/0.627	394.175/427.110	8/120

All 18 runs have zero EV energy-accounting shortfall, zero residual electrical violation, and complete every runtime-visible deferrable cycle with zero unserved energy. These integrity results do not erase controller limitations. RBC Smart increases ramping relative to BAU in six schemas and reduces demand-response compliance in Combined and both Premium variants. The detailed machine-readable scorecard retains these outcomes as controller watches and does not relabel them as dataset defects.

The member and asset ranges in this appendix are complemented by the row-level supplementary inventory: the family manifests and Parquet catalogues enumerate every member, physical charger, EV, session, PV system, stationary battery, deferrable appliance, phase assignment, connection level, limit, routine, event target, and source checksum. This machine-readable inventory is authoritative where a 100-member row listing would otherwise duplicate the released schema without adding a scientific distinction.

B.5 Simulator Audits and Demonstrative Results

This section retains the detailed evidence that supports, but would interrupt, the architectural and methodological account in Chapter 5. The results are bound to the archived simulator checkout and declared workloads. They establish implementation consistency and demonstrate that the corresponding mechanisms are executable and observable; they do not by themselves establish external validity or controller optimality. The preceding annual-suite audit and BAU/RBC Smart scorecards remain in Appendix B.4; the current two-year input checks are in Appendix B.3.

B.5.1 Software and contract verification

Table B.14: Verification evidence for the June 2026 simulator-study snapshot and the separately identified historical annual suite.

Check	Failure mode protected against	Reported evidence
Full regression suite	General simulator behaviour and compatibility across the extended codebase	423 passing tests; 18 warnings
Targeted REC requirements	Incorrect KPIs, event semantics, topology propagation, entity interfaces, and multi-REC behaviour	103 passing tests; 8 warnings
Physics and time-unit audit	Energy/power inconsistencies across temporal resolutions	16 of 16 scenarios passed at 60, 300, 900, and 3600 s
Entity/change audit	Unstable identities, relations, masks, action rows, or configuration versions	Strict contract pass
Historical annual-suite audit	Misaligned 2023 clocks or civil-time routines, incorrect OMIE expansion or publication boundary, duplicate profiles or identities, overlapping services, infeasible EV sessions, invalid Portuguese connection classes, incomplete checksums, unmatched scenario twins, or topology events without physical effect	1,817 structural and cross-family checks pass; diversity and deterministic-generation audits pass without findings; zero individually infeasible EV sessions; nine week-long rollouts, 31,565 causal-price checks, and 120 physically verified topology-event applications pass; Core-15 is the declared physical subset of Core-30
BAU/KPI sanity audit	Non-finite values, duplicate keys, or inconsistent reference deltas	3317 rows per export; 1401 BAU-derived rows; no duplicate keys
Thermal/HVAC compatibility	Regressions in inherited thermal and heat-pump behaviour	12 passing tests

The regression and capability results above belong to the simulator-study snapshot. The historical annual-suite row records its later, separately archived data audit. Independently calculable cases complement these tests by checking balances and services directly.

B.5.2 Selected capability demonstrations

The demonstrations below retain the observed values beside the precise interpretation that each case can support.

Table B.15: Selected application evidence and the bounded claim supported by each example.

Example	**Observed evidence**	**Supported interpretation**
Historical 17-member REC/EV reference	BAU: 28.7 kEUR, 166.8 MWh import, 30.0 tCO_2; REC RBC: 22.0 kEUR, 133.3 MWh, 22.8 tCO_2, approximately 91% EV within-tolerance service and 100% flexible-load service	Service, settlement, and constraint indicators prevent a cost-only reading; transparent RBCs are references, not proof of optimality.
Temporal resolution	The same annual trace yields peaks of 96.8, 188.2, and 281.5 kW at 1 h, 15 min, and 15 s	A declared timestep changes the observable capacity problem even when annual energy is preserved.
Historical 17-member settlement	17.0 MWh shared; cost changes from 24.8 to 22.0 kEUR; all 17 members save under the configured rule	Participant and accounting outcomes are observable; equity and legal applicability remain separate questions.
DR	Shortfall changes from 101.1 to 51.6 kWh; compliance from 0.43 to 0.70; cost after settlement from 863.6 to 790.6 EUR	Requests, delivery, shortfall, and settlement can be audited without claiming profitability or controller optimality.
Dynamic topology	Four member/asset events propagate to active counts, identified rows, relations, actions, and logs	Runtime change is represented consistently; controller adaptation must be evaluated separately.
Controlled failures	Missing observations, forecast bias, command dropout, and asset unavailability appear in their respective channels and exports	Disturbances are repeatable and measurable; robustness outside the declared classes is not inferred.
Multiple RECs	Four independent RECs retain local and combined results; 2.45 MWh of simultaneous surplus/demand is identified	Portfolio evidence is preserved without claiming physical coupling or a market-clearing mechanism.

B.5.3 Performance and scalability smoke benchmarks

The smoke benchmarks compare matched configurations and isolate the computational effect of interface detail, REC multiplicity, storage format, and action-feedback retention. The storage comparison contrasts Comma-Separated Values (CSV) files with the columnar Parquet format.

Table B.16: Computational-cost evidence from matched simulator configurations.

Comparison	Observed change	Interpretation
Flat versus basic entity input	5.30 to 5.51 ms/step, approximately +4.1%	Stable identities, relations, masks, and action feedback add a small measured overhead in the matched case.
Flat versus detailed entity input	4.88 to 10.18 ms/step; 165 flat values versus 2538 table values and 302 feature names	The richest interface more than doubles payload processing but remains quantitatively characterised.
One, two, and four RECs	5.02, 7.08, and 11.41 ms/step; four RECs produce 5227 KPI rows, including 107 combined rows	Four RECs take 2.27 times the single-REC step time while retaining local and portfolio evidence.
15-second CSV versus Parquet	Approximately 2.4–2.6 GB becomes 12–13 MB; initialisation falls by 34–38%	Columnar storage addresses data loading and storage; it does not materially change post-load physics.
Entity-feedback memory regression	Approximately +2071 MiB reduced to approximately +5 MiB over 700 targeted steps	The audit exposed and bounded a major implementation leak in the exercised configuration.

The basic entity contract remains close to flat execution in the matched case, while the detailed bundle deliberately exchanges a much richer payload for approximately twice the step time. The four-REC configuration corresponds to approximately 88 steps/s. These observations make the computational cost visible for the reported studies; they do not imply linear scaling to arbitrary population sizes, detailed feeder models, or distributed deployment targets.

B.6 Simulator KPI Catalogue

The core controller scorecard in Chapter 5 is a deliberately small interpretation layer over a broader CityLearn v3 KPI surface. The complete machine-readable output retains the entity or aggregation level, controller condition, baseline condition, unit, and raw value. It is produced through the primary `evaluate_v2()` interface. Table B.17 records the implemented families and the principal quantities used to interpret them. Exact code-level names are versioned with the simulator documentation and exported result, which remain authoritative when the catalogue evolves.

Table B.17: Expanded CityLearn v3 KPI catalogue.

Family	Principal quantities	Typical scope and unit
Energy and grid exchange	Net electricity consumption; total grid import and export; net grid energy; average daily and maximum import/export peaks; ramping; load factor; zero-net-energy indicators	Building, REC, or portfolio; kWh, kW, or ratio

Family	Principal quantities	Typical scope and unit
Cost and settlement	Grid import cost; export revenue; local import/export payment; cost after local and demand-response settlement; grid-only counterfactual; raw, BAU-difference, and valid BAU-ratio rows	Member, REC, or portfolio; EUR or ratio
Participant benefit distribution	Members with lower cost than the matched reference; benefit-coverage fraction; minimum, maximum, median, and dispersion of member savings	Member distribution; count, fraction, or EUR
Emissions	Imported-energy emissions; average carbon intensity; raw and BAU-relative emissions	Building, REC, or portfolio; $kgCO_2$ or $kgCO_2$/kWh
PV and local sharing	PV generation, self-consumption, self-sufficiency, local import/export, matched local exchange, REC surplus and deficit, sharing ratios	Member or REC; kWh or ratio
Stationary storage	Charged/discharged energy, losses, SoC, cycling and degradation-related outputs where enabled, requested/applied power difference	Asset, building, or REC; kWh, kW, ratio, or state
EV and charger service	Connected steps, arrivals, topology-valid departures, target/minimum/tolerance success, departure deficit, charged and V2G energy, required average power, slack, charger clipping, and EV energy-accounting shortfall	EV, charger, member, or REC; count, ratio, kWh, kW, or time
Deferrable service	Requested, started, completed, missed, topology-censored, and interrupted cycles; served/unserved energy; mean and maximum start delay; deadline violations	Asset, member, or REC; count, ratio, kWh, or time
Electrical feasibility	Requested total/per-phase pressure, rejected or reduced power, residual applied violation, violation duration, state-bound violations, and feasible headroom	Asset, member, phase, or REC; kW, kWh, count, or time
DR	Request and active-step counts; requested, delivered, credited, and shortfall energy; compliance; invalid baselines; payment, penalty, and net revenue; up/down delivery	Request, REC, or portfolio; count, kWh, ratio, or EUR
Topology and health	Active member/asset counts and periods; additions/removals; altered observation and forecast values; missing data; changed/lost actions; equipment-unavailable asset–step counts	Entity, member, REC, or portfolio; count, time, or mixed
Controller interface	Entity-table and row counts; relations; feature and action identities; masks; feasible action ranges; requested, clipped, applied, and rejected actions with reasons	Controller, entity, or action port; count or native action unit
Computational effort	Training wall time, inference latency, simulated steps per second, initialisation, reset and export time, peak memory, and the associated hardware, software, population, interface, and training-budget context	Controller or experiment; ms, s, h, steps/s, or MiB
Multiple RECs	Per-REC KPI tables; summed energy, cost, emissions, payments, requests, and events; declared weighted ratios; combined portfolio results	REC or portfolio; inherited unit
Inherited thermal, HVAC, and comfort	Heat-pump electricity, unmet thermal demand, comfort violations, occupant overrides, thermal resilience, and HVAC clipping where the selected scenario enables those inherited models	Building or district; kWh, count, time, or ratio

Three interpretation rules apply across this catalogue. First, reward and KPI are distinct: reward is controller-facing and may combine selected objectives, whereas KPIs remain evaluative outputs. Second, additive quantities may be summed across members or RECs, but ratios require a declared denominator or reporting weight. Third, requested-pressure and applied-residual quantities must not be conflated: the former describes controller intent before feasibility projection, while the latter

establishes whether the executed trajectory violated the configured physical boundary. The archived time series and event logs remain necessary whenever an aggregate KPI cannot explain when or why an outcome occurred.

B.7 Controller, TI-RL, and TI-MARL Configurations

Complete controller definitions cover passive and random operation, rule-based policies, MILP replay policies, strict-local PPO and TD3, IPPO, MAPPO, HAPPO, MADDPG, MATD3, MASAC, the entity-structured PPO and MATD3 variants, hierarchical controllers, and the typed-interface variants introduced in Chapter 7. There is no standalone SAC registry entry: the entropy-regularised implementation is MASAC. `SingleAgentRL` is a configuration placeholder and is not an executable method.

B.7.1 Executable Controller Inventory

Table B.18: Executable Algorithms registry grouped by scientific role.

Role	Registered names	Configuration evidence retained
Passive and diagnostic	`NormalNoBatteryPolicy`, `NormalPolicy`, `RandomPolicy`	Asset availability, service logic, seed, observation/action names, and export contract
Rule-based	`RBCBasicPolicy`, `RBCSmartPolicy`, `RBCSmartLocalPolicy`, `RBCCommunityPolicy`, `RuleBasedPolicy`	Information boundary, price and forecast use, EV/V2G thresholds, deferrable logic, headroom rule, and fallback scope
Signal-aware rules	`SignalAwareRBC`, `SignalAwareRBC SmartLocal`, `FixedPriceSignal`, `CausalPriceSignal`	Signal range, neutral value, update interval, recipient scope, and causal response probe
Oracle replay	`FixedServiceOracle ReplayPolicy`, `TotalHomeOracle ReplayPolicy`, `TotalOracleReplayPolicy`	Solver model, horizon, information assumption, status/gap, source interval, replay tolerance, and fallback

Role	Registered names	Configuration evidence retained
Strict local RL	`PPO`, `TD3`	One learner per member, local profile, reward, network, optimiser, rollout/replay, exploration, safety, seed, and checkpoint
Flat MARL	`IPPO`, `MAPPO`, `HAPPO`, `MADDPG`, `MATD3`, `MASAC`	Actor information, critic/value scope, team-reward mix, update order, entropy or target-noise settings, replay/rollout, and comparison reference
Entity-structured learning	`AgentTransformerPPO`, `AgentTransformerMATD3`	Tokenizer schema, layout signature, known type widths, topology version, homogeneous replay/rollout rule, runtime adapters, and per-topology export
Hierarchical control	`BuildingAgent`, `CommunityCoordinator`, `CCLevel1`, `CCLevel2`	Leaf identity and checkpoint, frozen state, coordinator inputs/outputs, signal interval, price adapter, reward allocation, and neutral-parity run
Typed interface	`TIMARL`	Interface registry, compiler and health-rule hashes, hierarchical typed encoder, grouped hybrid actor, analytic local projector, TI-PPO/TI-MAPPO critic, compatibility signature, and trace versions

B.7.2 Observation, Action, and Training Record

Every neural run archives the raw simulator interface, selected information surface, encoding profile, ordered raw and encoded observation names, ordered action names, physical bounds, building identities, and entity/topology metadata. Fixed-vector campaigns identify one of `minmax_space`, `maddpg_v1`, `maddpg_v2_compact`, `maddpg_v3_operational`, or `maddpg_v3_real time`. Structured campaigns additionally archive the tokenizer configuration, Shared Read-Only (SRO) context, Non-Flexible Consumption (NFC), and Controllable Asset (CA) segments, type feature widths, excluded features, action-to-CA order, and full layout signature.

The action record distinguishes actor output, exploration action, deterministic base, residual correction, composed action, controller projection, simulator request, and applied action whenever the selected path contains those stages. For off-policy learning, current and successor base actions

and the independent cloning target are preserved separately. For PPO, the exact pre-tanh action, old log-probability, value, policy-eligibility flag, and termination/truncation state remain attached to the rollout sample.

The training manifest records the actor and critic classes and layers, learning rates, discount, target-update coefficient, rollout or replay capacity, batch and minibatch sizes, update cadence, PPO epochs and clipping, Generalised Advantage Estimation (GAE), entropy settings, critic loss, target clipping, gradient clipping, target smoothing, actor delay, reward/return normalisation, multi-step return, exploration schedule, semantic noise scales, BC sources and weights, residual authority, local-safety settings, Automatic Mixed Precision (AMP)/device use, seed, checkpoint mode, and export dependencies. A field absent from a method is recorded as not applicable rather than assigned a silent default in the result table.

B.7.3 TI-MARL Reproducibility Contract

The public input to `TIMARL` is one technology-neutral `typed_agent_interface_v1` YAML Ain't Markup Language (YAML) definition for each registered member, together with an explicit simulator-binding registry. The simulator owns runtime entities, topology events, values, fault evidence, and execution feedback. The compiler owns semantic binding, health derivation, dependency closure, mode validity, dynamic bounds, feasibility inputs, and fallback semantics. Generated defaults are expanded into the resolved interface archived with the run.

Table B.19: TI-MARL v1 configuration and evidence fields retained for reproduction.

Area	**Required record**
Interface	Contract version; member definitions; simulator bindings; semantic, sensor, channel, unit, scope, use, and action-group registries; schema and health-rule hashes
Actor	Latent width; attention heads; relational layers; local or action-conditioned group context; categorical modes; Beta-fraction heads; deterministic strategy, gain, deadband, and non-idle margins by group type
Critic	TI-PPO/local or TI-MAPPO/set pairing; optional typed-group critic; value-target normalisation; Huber or Mean Squared Error (MSE) value loss
PPO	Learning rate; reference γ and GAE λ; discount timebase; clip; epochs; rollout length; entropy by group family; advantage-normalisation scope; KL target; gradient limit
Credit and projection	Joint-agent or typed-group credit; intervention-aware masking; intervention distillation; EV service strategy and margins; total/per-phase headroom reserve; deferrable completion margin

Area	Required record
Prior knowledge	BC teacher, demonstration episodes, balancing method, calibration; EV and storage planning coefficients, label parameters, per-reason reservoir capacities, and samples
Population	Stable member order; joining/departing transition treatment; active/registered members; topology version; structural recompilation count
Persistence	`ti_marl_checkpoint_v1`; learning architecture; actor, critic, group critic, optimisers, rollout, Random Number Generator (RNG), compiler, resolved config, versions, hashes, and migration decision
Execution evidence	Raw and projected typed bundles; interventions; encoded commands; `entity_action_execution_v1`; current/next snapshot hashes; trace chunk and deduplication settings
Handoff	Research artefact deployment flag; actor-only bundle format; resolved interfaces; compiler and health rules; projector; normalisation; compatibility signature; omitted critic and optimiser state

The audited implementation state is Algorithms commit `58fb6370` with simulator package version 1.8.0. Its focused TI-MARL suite contains 115 passing tests across compiler, policy runtime, runtime contracts, public typed interfaces, a real-simulator dynamic vertical slice, and a 15-second topology stress case. This count identifies the audited state; later releases must report their own test inventory rather than inheriting it.

B.7.4 Reward Profile Ledger

The profile numbers below are provenance identifiers rather than an ordered claim of improvement. A later profile records a diagnosed change in the control signal; it becomes preferable only when the matched evaluation supports that decision.

Table B.20: Implemented Chapter 5 reward-profile lineage.

Identifier	Registered profile	Change retained for reproduction
Base	Cost Hard Constraint	Configurable local/REC cost, electrical and outage penalties, EV service and V2G, storage bounds/throughput, deferrables, import, peak, ramp, export, emissions, settlement, and timestep scaling

Identifier	Registered profile	Change retained for reproduction
V2	Cost Service Guard	Protects EV service from cost savings created by V2G below the target
V3	Cost Service Cost Balanced	Rebalances service pressure and cost and introduces a light storage-throughput term
Local V1	Local Cost Service	Removes REC settlement and aggregate penalties to enforce a strict-local reward boundary
Local V2	Local Scorecard Guard	Raises electrical, EV, storage, and deferrable failure costs around the frozen scorecard gates
Local V3	Local Economic Safety	Removes dense recoverable urgency when deadline-feasible projection already protects service
Individual V3	Individual Scorecard Aligned	Uses each member's settlement row while actors and critics remain independent; labelled as a market/broadcast reward rather than a strict-local ablation
V4	Community Band	Adds REC settlement and EV over-service tolerance to the service-first objective
V41	Community Storage Band	Strengthens storage discipline and EV target-band pressure
V42	Community Service Band	Raises service pressure after cost-improving under-service and over-service were observed
V43	Community Battery Value	Restores physical storage fallback bounds, uses observed limits, scales by timestep, and lowers the cycling penalty
V44	Community Smooth Service	Uses quadratic dense EV terms so recoverable deficits do not dominate long high-frequency episodes
V45	Community Feasible Service	Computes schedule pressure from effective charging power under building/phase headroom and penalises unrecoverable schedule deficit
V46	Community Feasible Precision	Caps only training penalties for infeasible shortfall and increases over-service precision; raw diagnostics remain uncapped

Identifier	Registered profile	Change retained for reproduction
V47	Community Feasible Precision, tighter band	Retains V46 feasibility and tightens the upper service band
V49	Community Storage Value	Reduces throughput suppression and raises settlement/peak value so useful storage operation remains learnable
V50	Community Deadline Value	Extends bounded deadline shaping while retaining the effective-power feasibility calculation
V51	Community Precision Value	Rebalances minimum service, over-service, and REC value after excessive charging
V52	Community Peak Deadline	Adds stronger explicit peak/export pressure beside deadline service and settlement
V53	Community Residual Constraint	Defines a REC-shape objective for bounded residual learning over an RBC teacher
V54	Community Dense EV Residual	Makes sparse EV consequences visible during connected periods for conservative residual control
V55	Community Storage Residual	Removes uncontrollable service terms only when EV and deferrable actions are copied exactly from the audited teacher
CC	Community Coordinator reward	Price-arbitrage-centred coordinator objective retained for historical reproduction
CC Level 1	Aggregate coordinator reward	Normalised cost, peak excess, ramp, export, violation, and selectable settlement aggregation
CC Level 2	Member-aware coordinator reward	Adds EV harm, member-decomposed settlement and constraint credit, team mixing, neutral counterfactual, and running normalisation
CC Phase 1	Market-making phase reward	Retains the intermediate coordinator formulation used during the hierarchy's historical development

Each final run stores the complete reward class and parameter mapping. The abbreviated identifiers above are insufficient to reproduce a run on their own. Reward component exports, raw

and normalised totals, settlement parameters, seconds per timestep, phase/charger map, and any running state are retained with the experiment evidence.

B.8 Hyperparameters, Seeds, and Statistical Procedures

The supplementary record contains hyperparameter tables, search ranges, selected values, random seeds, training budgets, checkpoint-selection rules, evaluation windows, statistical tests, confidence intervals, and the treatment of failed runs.

B.9 Supplementary and Negative Results

Supplementary evidence includes additional learning curves, participant-level results, scale curves, disturbance-severity curves, ablations, failure cases, methods that did not improve the selected objective, and configurations rejected before deployment. Negative results are retained where they explain a design choice in Chapters 5 or 6.

B.10 Lifecycle and Deployment Records

The supplementary deployment record contains examples of resolved experiment configurations, model manifests, execution-target profiles, and inference-service configuration. These records document the interfaces described in Chapter 4.

B.11 Software, Data, and Archival Record

The archival record lists repositories, releases, licences, archived commits or Digital Object Identifiers (DOIs), container images, external dependencies, datasets and access conditions, partner-restricted components, hardware specifications, and commands used to reproduce the principal figures and tables. Restricted artefacts are accompanied by a stated limitation and the closest reproducible substitute.

B.12 Annual BAU Comparisons and Reference Diagnostics

The paired BAU comparison in Section 6.1.2 covers Core-30 Normal, Topology, Safety, Robustness, and Combined, together with Premium-100 Normal and All-in. Each scenario has a reference without stationary storage and a reference using storage for local PV self-consumption. The 2023 and 2024 accounts remain separate; the 2024 reference accompanies the principal controller comparison.

B.12.1 Development-Year Annual Comparison

Illustrative annual comparison: invented values. The two tables below describe the intended full-year comparison for 2023. Their numbers are manuscript examples, independent of the real Core-30 diagnostic reported later in this section. The chosen year-to-year differences carry no empirical interpretation.

Table B.21: **Illustrative values; invented.** Energy and economic outcomes for the two BAU variants over the complete 2023 window.

BAU variant	**Cost** [EUR]	**CO_2** [kg]	**Import** [kWh]	**Export** [kWh]	**Solar** [%]	**Daily** [kW]	**Max.** [kW]	**Ramp** [kW]
Core-30 Normal								
Without storage	29 160	37 948	205 965	42 840	30.3	65.9	112.7	200 720
With storage	25 920	33 920	186 300	21 420	65.2	61.8	107.6	187 200
Core-30 Topology								
Without storage	27 432	36 040	196 650	39 780	30.7	62.8	110.7	192 400
With storage	24 624	32 542	178 848	23 970	58.2	58.7	104.5	180 960
Core-30 Safety								
Without storage	28 728	37 418	203 895	42 840	30.3	63.9	104.5	191 360
With storage	25 596	33 496	185 265	21 930	64.3	59.7	100.4	179 920
Core-30 Robustness								
Without storage	29 376	38 160	207 000	43 350	29.5	67.0	115.8	212 160
With storage	26 568	34 768	189 922	24 990	59.4	63.9	112.7	201 760
Core-30 Combined								
Without storage	27 540	35 934	195 615	39 984	30.3	61.8	102.5	193 440
With storage	25 056	33 072	180 090	26 316	54.2	58.7	99.4	185 120
Premium-100 Normal								
Without storage	119 880	154 760	807 300	295 800	38.6	262.7	502.2	780 000
With storage	103 680	135 680	705 870	214 200	55.5	236.9	456.1	717 600
Premium-100 All-in								
Without storage	113 400	148 400	755 550	269 280	37.4	253.4	471.5	796 640
With storage	100 440	132 500	673 785	201 960	53.1	231.8	440.7	748 800

Import/export use the external REC boundary; cost sums member purchase bills. All-in cost precedes the separate illustrative DR settlement in Table 6.31. Daily/Max. denote mean daily/maximum peak; Ramp is accumulated upward power change. Each pair uses the same scenario and year; different populations are not ranked by total cost.

Table B.22: **Illustrative values; invented.** Departure service, safety, and storage use for the two BAU variants over the complete 2023 window.

BAU variant	**EV min.** [%]	**Missed EV** [n/N]	**EV band** [%]	**Viol.** [kWh]	**Corrected** [%]	**Battery** [kWh]
Core-30 Normal						
Without storage	100.0	0/10 000	18.0	0	3.4	0
With storage	100.0	0/10 000	18.0	0	3.0	22 440
Core-30 Topology						
Without storage	99.0	90/9 000	18.0	0	4.0	0
With storage	99.2	72/9 000	18.2	0	3.5	19 380
Core-30 Safety						
Without storage	96.0	400/10 000	17.0	0	14.0	0
With storage	97.0	300/10 000	17.5	0	11.0	21 420
Core-30 Robustness						
Without storage	96.5	350/10 000	17.0	0	10.0	0
With storage	97.2	280/10 000	17.5	0	9.0	19 890
Core-30 Combined						
Without storage	93.0	630/9 000	16.0	0	19.0	0
With storage	94.5	495/9 000	16.5	0	16.0	17 850
Premium-100 Normal						
Without storage	99.0	300/30 000	18.0	0	6.0	0
With storage	99.3	210/30 000	18.5	0	4.5	193 800
Premium-100 All-in						
Without storage	91.0	2 250/25 000	16.0	0	23.0	0
With storage	93.0	1 750/25 000	16.5	0	19.0	153 000

Missed EV gives failed minimum-service departures over an invented common eligible cohort within each pair. These counts are invented cohorts, independent of the dataset audit. V2G is disabled. Battery denotes stationary-battery throughput. Appliance, DR, and runtime event outcomes are illustrated separately in Tables 6.30 and 6.31.

The illustrative tables use common invented eligible-departure counts within each scenario pair: 10 000 in Core-30 Normal, Safety, and Robustness; 9 000 in Topology and Combined; 30 000 in Premium Normal; and 25 000 in All-in. They provide arithmetically consistent examples of percentages and missed-service counts. These are not counts from the dataset audit. The annual controller fixtures add an invented unattainable cohort equal to 2% of eligible departures and assume zero incomplete records. These additional requests remain outside the service-rate denominator. Actual comparisons retain both scenario-specific eligibility and the common cohort required for paired disturbance assessment.

B.12.2 Companion Service and Execution Records

Service, runtime, computation, and deployment accompany the annual scorecards in Tables 6.25, 6.29, 6.30, 6.26, and 6.27. The Premium All-in response account appears in Table 6.31. Each companion uses explicitly invented counts and magnitudes. V2G is disabled, stationary-battery throughput is zero without storage, and BAU with storage has zero members saving against itself.

B.12.3 Real Short-Window Reference Diagnostics

The following values were obtained from an actual Core-30 simulator execution. They preserve its original window and accounting. The annual illustrative tables above are independent manuscript examples.

Table B.23: **Measured short-window BAU diagnostic.** Portugal v3.1, 671 executed fifteen-minute steps from 1 January 2023 00:00 through 7 January 2023 23:30. Cost and import sum member-level purchases; peak uses REC net power.

Dataset	**Cost** [EUR]	**Import** [kWh]	**Peak** [kW]	**EV** departures	**Min.** [%]	**Band** [%]
Core-30	240.59	5074.2	85.59	105	100.0	6.7

The storage-enabled BAU execution in Table B.23 provides a concrete reference on the current Core-30 inputs. All 105 observed departures were eligible for minimum-service evaluation, and every one met that criterion. The target-band rate was much lower: 7 of 105 departures. Immediate charging towards full charge explains why ordinary operation can satisfy the minimum requirement while delivering more charge than requested. The run recorded zero residual electrical-violation energy under the simulator's constraint handling. Its exported window contains 671 action intervals; no unexecuted final interval is added to the reported energy or cost.

The import costs were calculated from each member's exported positive net energy and the corresponding realised tariff, retaining negative prices and zero export remuneration. This run establishes actual short-window cost and service values; the full test-window comparison retains its separate scope. It also illustrates why a low bill, a safe executed trajectory, and accurate service are separate observations. The annual scorecard retains these separate outcomes.

The storage-enabled diagnostic used a state-of-charge range of 0.2–0.9, charge and discharge command rates of 0.35, and a local import threshold of 2 kW for discharge. EV charging used rate 1.0 and target SoC 1.0; the appliance start command was 1.0. The no-storage comparison sets stationary-battery charge and discharge to zero while retaining the remaining rules. The resolved configuration is archived with each execution.

B.12.4 Complete Annual Controller Scorecards

The principal controller families and the additional training and hierarchy variants share the seven-scenario scope. Table B.24 first compares the annual cost of the additional configurations. The subsequent paired tables give energy, service, safety, resource use, and member benefit for all 26 configurations. Every value is an invented editorial fixture. The complete-year account includes supervisory fallback when the hypothetical learned controller cannot continue; it does not assert that each implementation has been executed on every scenario.

Table B.24: **Illustrative values; invented.** Annual energy-purchase cost [EUR] of the additional training and hierarchy variants.

Configuration	C30 N	C30 T	C30 S	C30 R	C30 C	P100 N	P100 A
MATD3 unguided	21 840	22 408	21 998	25 520	25 123	87 360	104 095
MATD3 guided exploration	20 880	21 423	21 031	24 398	24 019	83 520	99 519
MATD3 demonstrations	20 400	20 930	20 548	23 837	23 467	81 600	97 232
RBC Smart Local global	20 880	20 332	21 031	22 686	22 202	83 520	91 428
RBC Smart Local per-member	20 640	20 098	20 790	22 425	21 947	82 560	90 377
MATD3 residual global	19 920	20 438	20 064	23 277	22 915	79 680	94 944
MATD3 residual per-member	20 160	20 684	20 306	23 557	23 191	80 640	96 088

C30: Core-30; P100: Premium-100. N: Normal; T: Topology; S: Safety; R: Robustness; C: Combined; A: All-in. Each column covers its complete 2024 sequence, including supervisory fallback. *Generic offline-policy placeholder; the associated method remains to be specified.

All values in the following scorecards are invented manuscript fixtures. Each scenario covers a complete 2024 comparison. Codes retain the same controller configuration throughout the appendix. Operational and computational companions use the same explicitly hypothetical status.

Solar denotes PV self-consumption; Daily and Max. denote REC peaks. Saving counts members with a lower bill than B1. The invented eligible-departure cohorts follow the BAU tables; unattainable requests remain separate. V2G is disabled.

Code	Configuration	Code	Configuration
B0	BAU without storage	MP	MAPPO
B1	BAU with storage	HP	HAPPO
RB	RBC Basic	MS	MASAC
RSL	RBC Smart Local	OF	Offline policy*
RC	RBC Community	TP	Transformer PPO
MLF	MILP local fixed	TT	Transformer MATD3
MCF	MILP REC fixed	TU	MATD3 unguided
MLA	MILP local flexible	TG	MATD3 guided exploration
MCA	MILP REC flexible	TM	MATD3 demonstrations
LP	Local PPO	RG	RBC Smart Local global
LT	Local TD3	RM	RBC Smart Local per-member
DD	MADDPG	HG	MATD3 residual global
TD	MATD3 (residual)	HM	MATD3 residual per-member

Table B.25: **Illustrative values; invented.** Core-30 Normal. Energy and economic outcomes.

Code	Cost [EUR]	CO_2 [kg]	Import [kWh]	Export [kWh]	Solar [%]	Daily [kW]	Max. [kW]	Ramp [kW]
B0	27 000	35 800	199 000	42 000	30.0	64.0	110.0	193 000
B1	24 000	32 000	180 000	21 000	65.0	60.0	105.0	180 000
RB	23 040	30 800	175 000	20 400	66.0	57.0	100.0	174 600
RSL	21 600	28 400	166 000	17 400	71.0	54.0	96.0	167 400
RC	21 120	27 700	163 000	16 800	72.0	51.6	91.0	158 400
MLF	21 480	27 505	169 177	15 717	73.8	53.7	93.9	166 994
MCF	21 000	26 890	165 396	15 366	74.4	52.5	91.8	163 262
MLA	19 560	25 046	154 055	14 312	76.1	48.9	85.5	152 067
MCA	18 720	23 971	147 439	13 698	77.2	46.8	81.8	145 537
LP	22 080	29 200	169 000	19 200	68.0	56.4	99.0	172 800
LT	21 600	28 800	167 000	18 600	69.0	55.2	97.0	169 200
DD	20 880	27 800	163 500	17 400	71.0	52.8	94.0	162 000
TD	19 680	25 200	155 000	14 400	76.0	49.2	86.0	153 000
MP	20 640	26 900	161 000	16 200	73.0	51.6	92.0	160 200
HP	20 400	26 500	159 500	15 600	74.0	51.0	90.0	158 400
MS	20 640	27 100	162 000	16 800	72.0	52.2	93.0	162 000
OF	21 240	28 800	167 000	18 600	69.0	55.2	97.0	169 200
TP	21 360	28 100	164 500	18 000	70.0	54.0	95.0	167 400
TT	20 160	25 700	157 000	15 000	75.0	50.4	88.0	156 600
TU	21 840	27 966	172 012	15 980	73.4	54.6	95.4	169 793
TG	20 880	26 737	164 451	15 278	74.5	52.2	91.2	162 329
TM	20 400	26 122	160 671	14 927	75.1	51.0	89.1	158 598
RG	20 880	27 453	160 467	16 200	73.0	51.0	92.8	158 400
RM	20 640	27 138	158 622	15 600	74.0	49.8	91.7	156 600
HG	19 920	25 507	156 890	13 800	77.0	46.8	87.0	144 000
HM	20 160	25 815	158 780	13 200	78.0	45.6	88.1	140 400

Table B.26: **Illustrative values; invented.** Core-30 Normal. Service, safety and resource use.

Code	EV min. [%]	EV band [%]	Viol. [kWh]	Corr. [%]	Battery [kWh]	V2G [kWh]	Saving [n/N]
B0	100.0	18.0	0	3.4	0	0	0/30
B1	100.0	18.0	0	3.0	22 000	0	0/30
RB	99.2	75.0	0	7.0	23 500	0	21/30
RSL	99.8	92.0	0	4.0	25 500	0	26/30
RC	99.8	93.0	0	3.6	26 800	0	27/30
MLF	100.0	18.0	0	1.2	30 561	0	29/30
MCF	100.0	18.0	0	1.2	29 878	0	29/30
MLA	100.0	98.0	0	1.2	27 829	0	29/30
MCA	100.0	98.0	0	1.2	26 634	0	29/30
LP	98.2	81.0	0	9.5	28 200	0	23/30
LT	98.8	85.0	0	8.0	29 100	0	25/30
DD	98.6	84.0	0	14.0	32 000	0	26/30
TD	99.8	96.0	0	2.3	28 000	0	29/30
MP	99.5	91.0	0	5.5	29 700	0	27/30
HP	99.6	92.5	0	4.8	30 300	0	28/30
MS	99.3	90.0	0	6.5	31 000	0	27/30
OF	99.2	89.0	0	5.8	29 100	0	25/30
TP	99.0	89.0	0	5.0	28 500	0	26/30
TT	99.7	95.5	0	2.0	27 600	0	28/30
TU	96.0	73.0	0	12.4	33 700	0	29/30
TG	98.1	85.0	0	8.2	31 300	0	29/30
TM	99.0	90.0	0	6.1	29 400	0	29/30
RG	99.8	93.0	0	3.8	24 650	0	28/30
RM	99.9	94.0	0	3.5	24 367	0	29/30
HG	99.8	96.0	0	2.5	28 341	0	28/30
HM	99.8	96.0	0	2.7	28 683	0	27/30

Table B.27: **Illustrative values; invented.** Core-30 Topology. Energy and economic outcomes.

Code	Cost [EUR]	CO_2 [kg]	Import [kWh]	Export [kWh]	Solar [%]	Daily [kW]	Max. [kW]	Ramp [kW]
B0	25 400	34 000	190 000	39 000	30.4	61.0	108.0	185 000
B1	22 800	30 700	172 800	23 500	58.0	57.0	102.0	174 000
RB	23 639	29 549	168 000	22 829	59.2	54.1	97.1	168 780
RSL	21 033	27 246	159 360	19 471	65.2	51.3	93.3	161 820
RC	20 566	26 575	156 480	18 800	66.4	49.0	88.4	153 120
MLF	20 610	26 388	162 410	17 588	68.6	51.0	91.2	161 428
MCF	20 150	25 798	158 780	17 195	69.3	49.9	89.2	157 820
MLA	18 768	24 029	147 893	16 016	71.4	46.5	83.1	146 998
MCA	17 962	22 997	141 541	15 329	72.6	44.5	79.5	140 686
LP	22 654	28 014	162 240	21 486	61.6	53.6	96.2	167 040
LT	22 162	27 630	160 320	20 814	62.8	52.4	94.2	163 560
DD	21 423	26 671	156 960	19 471	65.2	50.2	91.3	156 600
TD	20 192	24 176	148 800	16 114	71.2	46.7	83.5	147 900
MP	21 177	25 807	154 560	18 129	67.6	49.0	89.4	154 860
HP	20 930	25 423	153 120	17 457	68.8	48.5	87.4	153 120
MS	21 177	25 999	155 520	18 800	66.4	49.6	90.3	156 600
OF	21 792	27 630	160 320	20 814	62.8	52.4	94.2	163 560
TP	20 799	26 958	157 920	20 143	64.0	51.3	92.3	161 820
TT	19 631	24 656	150 720	16 786	70.0	47.9	85.5	151 380
TU	22 408	26 830	165 132	17 882	68.1	51.9	92.7	164 133
TG	21 423	25 651	157 873	17 097	69.5	49.6	88.6	156 918
TM	20 930	25 061	154 244	16 704	70.2	48.5	86.6	153 311
RG	20 332	26 338	154 048	18 129	67.6	48.5	90.1	153 120
RM	20 098	26 036	152 277	17 457	68.8	47.3	89.1	151 380
HG	20 438	24 471	150 614	15 443	72.4	44.5	84.5	139 200
HM	20 684	24 766	152 429	14 771	73.6	43.3	85.6	135 720

Table B.28: **Illustrative values; invented.** Core-30 Topology. Service, safety and resource use.

Code	EV min. [%]	EV band [%]	Viol. [kWh]	Corr. [%]	Battery [kWh]	V2G [kWh]	Saving [n/N]
B0	99.0	18.0	0	4.0	0	0	0/30
B1	99.2	18.2	0	3.5	19 000	0	0/30
RB	96.2	69.6	0	10.0	20 295	0	7/30
RSL	99.0	90.6	0	7.0	22 023	0	28/30
RC	99.0	91.6	0	6.6	23 145	0	28/30
MLF	99.8	17.6	0	4.2	26 394	0	28/30
MCF	99.8	17.6	0	4.2	25 804	0	28/30
MLA	99.8	97.6	0	4.2	24 034	0	28/30
MCA	99.8	97.6	0	4.2	23 002	0	28/30
LP	95.2	75.6	0	12.5	24 355	0	28/30
LT	95.8	79.6	0	11.0	25 132	0	28/30
DD	95.6	78.6	0	17.0	27 636	0	28/30
TD	96.8	90.6	0	5.3	24 182	0	28/30
MP	96.5	85.6	0	8.5	25 650	0	28/30
HP	96.6	87.1	0	7.8	26 168	0	28/30
MS	96.3	84.6	0	9.5	26 773	0	28/30
OF	96.2	83.6	0	8.8	25 132	0	28/30
TP	98.2	87.6	0	8.0	24 614	0	28/30
TT	98.9	94.1	0	5.0	23 836	0	28/30
TU	93.0	67.6	0	15.4	29 105	0	28/30
TG	95.1	79.6	0	11.2	27 032	0	28/30
TM	96.0	84.6	0	9.1	25 391	0	28/30
RG	99.0	91.6	0	6.8	21 289	0	28/30
RM	99.1	92.6	0	6.5	21 044	0	28/30
HG	96.8	90.6	0	5.5	24 476	0	28/30
HM	96.8	90.6	0	5.7	24 772	0	28/30

Table B.29: **Illustrative values; invented.** Core-30 Safety. Energy and economic outcomes.

Code	Cost [EUR]	CO_2 [kg]	Import [kWh]	Export [kWh]	Solar [%]	Daily [kW]	Max. [kW]	Ramp [kW]
B0	26 600	35 300	197 000	42 000	30.0	62.0	102.0	184 000
B1	23 700	31 600	179 000	21 500	64.2	58.0	98.0	173 000
RB	23 207	30 415	174 028	20 886	65.2	55.1	93.3	167 810
RSL	21 757	28 045	165 078	17 814	70.3	52.2	89.6	160 890
RC	21 273	27 354	162 094	17 200	71.3	49.9	84.9	152 240
MLF	21 318	27 161	168 237	16 091	73.2	51.9	87.6	160 500
MCF	20 841	26 554	164 477	15 732	73.8	50.8	85.7	156 913
MLA	19 412	24 733	153 199	14 653	75.6	47.3	79.8	146 153
MCA	18 578	23 671	146 620	14 024	76.6	45.2	76.3	139 877
LP	22 240	28 835	168 061	19 657	67.2	54.5	92.4	166 080
LT	21 757	28 440	166 072	19 043	68.3	53.4	90.5	162 620
DD	21 031	27 452	162 592	17 814	70.3	51.0	87.7	155 700
TD	19 823	24 885	154 139	14 743	75.4	47.6	80.3	147 050
MP	20 790	26 564	160 106	16 586	72.4	49.9	85.9	153 970
HP	20 548	26 169	158 614	15 971	73.4	49.3	84.0	152 240
MS	20 790	26 761	161 100	17 200	71.3	50.5	86.8	155 700
OF	21 394	28 440	166 072	19 043	68.3	53.4	90.5	162 620
TP	21 515	27 749	163 586	18 429	69.3	52.2	88.7	160 890
TT	20 306	25 379	156 128	15 357	74.4	48.7	82.1	150 510
TU	21 998	27 616	171 056	16 360	72.7	52.8	89.0	163 190
TG	21 031	26 403	163 537	15 642	73.9	50.5	85.1	156 016
TM	20 548	25 795	159 778	15 282	74.5	49.3	83.2	152 430
RG	21 031	27 110	159 576	16 586	72.4	49.3	86.6	152 240
RM	20 790	26 799	157 741	15 971	73.4	48.1	85.6	150 510
HG	20 064	25 188	156 018	14 129	76.5	45.2	81.2	138 400
HM	20 306	25 492	157 898	13 514	77.5	44.1	82.2	134 940

Table B.30: **Illustrative values; invented.** Core-30 Safety. Service, safety and resource use.

Code	EV min. [%]	EV band [%]	Viol. [kWh]	Corr. [%]	Battery [kWh]	V2G [kWh]	Saving [n/N]
B0	96.0	17.0	0	14.0	0	0	0/30
B1	97.0	17.5	0	11.0	21 000	0	0/30
RB	97.2	71.4	0	13.0	22 432	0	28/30
RSL	97.4	88.0	0	9.0	24 341	0	28/30
RC	97.8	89.4	0	9.6	25 582	0	28/30
MLF	99.5	17.1	0	7.2	29 172	0	28/30
MCF	99.5	17.1	0	7.2	28 520	0	28/30
MLA	99.5	97.1	0	7.2	26 564	0	28/30
MCA	99.5	97.1	0	7.2	25 423	0	28/30
LP	96.2	77.4	0	15.5	26 918	0	28/30
LT	96.8	81.4	0	14.0	27 777	0	28/30
DD	92.0	74.0	0	25.0	30 545	0	28/30
TD	98.6	93.0	0	6.8	26 727	0	28/30
MP	97.5	87.4	0	11.5	28 350	0	28/30
HP	97.6	88.9	0	10.8	28 923	0	28/30
MS	97.3	86.4	0	12.5	29 591	0	28/30
OF	97.2	85.4	0	11.8	27 777	0	28/30
TP	97.0	85.4	0	11.0	27 205	0	28/30
TT	98.8	94.0	0	5.1	26 345	0	28/30
TU	94.0	69.4	0	18.4	32 168	0	28/30
TG	96.1	81.4	0	14.2	29 877	0	28/30
TM	97.0	86.4	0	12.1	28 064	0	28/30
RG	97.8	89.4	0	9.8	23 530	0	28/30
RM	97.9	90.4	0	9.5	23 259	0	28/30
HG	97.8	92.4	0	8.5	27 053	0	28/30
HM	97.8	92.4	0	8.7	27 379	0	28/30

Table B.31: **Illustrative values; invented.** Core-30 Robustness. Energy and economic outcomes.

Code	Cost [EUR]	CO_2 [kg]	Import [kWh]	Export [kWh]	Solar [%]	Daily [kW]	Max. [kW]	Ramp [kW]
B0	27 200	36 000	200 000	42 500	29.2	65.0	113.0	204 000
B1	24 600	32 800	183 500	24 500	59.2	62.0	110.0	194 000
RB	26 922	31 570	178 403	23 800	60.3	58.9	104.8	188 180
RSL	23 520	29 110	169 228	20 300	66.2	55.8	100.6	180 420
RC	22 947	28 392	166 169	19 600	67.3	53.3	95.3	170 720
MLF	24 219	28 193	172 467	18 336	69.4	55.5	98.4	179 982
MCF	23 678	27 562	168 612	17 927	70.1	54.2	96.2	175 960
MLA	22 054	25 672	157 051	16 697	72.2	50.5	89.6	163 894
MCA	21 107	24 570	150 306	15 981	73.4	48.4	85.7	156 857
LP	25 800	29 930	172 286	22 400	62.7	58.3	103.7	186 240
LT	25 240	29 520	170 247	21 700	63.8	57.0	101.6	182 360
DD	24 720	28 495	166 679	20 300	66.2	54.6	98.5	174 600
TD	22 080	25 830	158 014	16 800	72.0	50.8	90.1	164 900
MP	23 520	27 572	164 131	18 900	68.5	53.3	96.4	172 660
HP	23 837	27 162	162 601	18 200	69.7	52.7	94.3	170 720
MS	24 118	27 778	165 150	19 600	67.3	53.9	97.4	174 600
OF	24 819	29 520	170 247	21 700	63.8	57.0	101.6	182 360
TP	23 208	28 802	167 699	21 000	65.0	55.8	99.5	180 420
TT	21 840	26 342	160 053	17 500	70.8	52.1	92.2	168 780
TU	25 520	28 665	175 357	18 643	68.9	56.4	99.9	182 999
TG	24 398	27 405	167 649	17 824	70.3	53.9	95.5	174 955
TM	23 837	26 775	163 795	17 415	71.0	52.7	93.3	170 933
RG	22 686	28 139	163 587	18 900	68.5	52.7	97.2	170 720
RM	22 425	27 816	161 706	18 200	69.7	51.5	96.1	168 780
HG	23 277	26 145	159 941	16 100	73.2	48.4	91.1	155 200
HM	23 557	26 460	161 867	15 400	74.3	47.1	92.3	151 320

Table B.32: **Illustrative values; invented.** Core-30 Robustness. Service, safety and resource use.

Code	EV min. [%]	EV band [%]	Viol. [kWh]	Corr. [%]	Battery [kWh]	V2G [kWh]	Saving [n/N]
B0	96.5	17.0	0	10.0	0	0	0/30
B1	97.2	17.5	0	9.0	19 500	0	0/30
RB	95.2	67.8	0	15.0	20 830	0	7/30
RSL	97.7	87.0	0	12.0	22 602	0	27/30
RC	97.5	88.9	0	11.6	23 755	0	27/30
MLF	98.6	15.5	0	9.2	27 088	0	27/30
MCF	98.6	15.5	0	9.2	26 483	0	27/30
MLA	98.6	95.5	0	9.2	24 667	0	27/30
MCA	98.6	95.5	0	9.2	23 607	0	27/30
LP	94.2	73.8	0	17.5	24 995	0	7/30
LT	94.8	77.8	0	16.0	25 793	0	7/30
DD	93.6	71.0	0	29.0	28 364	0	7/30
TD	97.3	90.0	0	8.5	24 818	0	27/30
MP	96.3	83.0	0	15.0	26 325	0	27/30
HP	95.6	85.3	0	12.8	26 857	0	27/30
MS	95.3	82.8	0	14.5	27 477	0	27/30
OF	95.2	81.8	0	13.8	25 793	0	7/30
TP	96.7	84.9	0	13.0	25 261	0	27/30
TT	97.9	92.0	0	6.5	24 464	0	27/30
TU	92.0	65.8	0	20.4	29 870	0	7/30
TG	94.1	77.8	0	16.2	27 743	0	27/30
TM	95.0	82.8	0	14.1	26 059	0	27/30
RG	97.5	88.9	0	11.8	21 849	0	27/30
RM	97.6	89.9	0	11.5	21 598	0	27/30
HG	95.8	88.8	0	10.5	25 120	0	27/30
HM	95.8	88.8	0	10.7	25 424	0	27/30

Table B.33: **Illustrative values; invented.** Core-30 Combined. Energy and economic outcomes.

Code	Cost [EUR]	CO_2 [kg]	Import [kWh]	Export [kWh]	Solar [%]	Daily [kW]	Max. [kW]	Ramp [kW]
B0	25 500	33 900	189 000	39 200	30.0	60.0	100.0	186 000
B1	23 200	31 200	174 000	25 800	53.9	57.0	97.0	178 000
RB	26 504	30 030	169 167	25 063	55.2	54.1	92.4	172 660
RSL	22 968	27 690	160 467	21 377	61.8	51.3	88.7	165 540
RC	22 458	27 008	157 567	20 640	63.1	49.0	84.1	156 640
MLF	22 633	26 817	163 538	19 309	65.5	51.0	86.7	165 139
MCF	22 127	26 218	159 883	18 878	66.3	49.9	84.8	161 448
MLA	20 610	24 420	148 920	17 583	68.6	46.5	79.0	150 377
MCA	19 725	23 372	142 524	16 829	69.9	44.5	75.6	143 920
LP	25 399	28 470	163 367	23 589	57.9	53.6	91.5	170 880
LT	24 847	28 080	161 433	22 851	59.2	52.4	89.6	167 320
DD	24 019	27 105	158 050	21 377	61.8	50.2	86.8	160 200
TD	22 639	24 570	149 833	17 691	68.4	46.7	79.4	151 300
MP	23 743	26 228	155 633	19 903	64.5	49.0	85.0	158 420
HP	23 467	25 838	154 183	19 166	65.8	48.5	83.1	156 640
MS	23 743	26 422	156 600	20 640	63.1	49.6	85.9	160 200
OF	24 433	28 080	161 433	22 851	59.2	52.4	89.6	167 320
TP	22 713	27 398	159 017	22 114	60.5	51.3	87.8	165 540
TT	21 437	25 058	151 767	18 429	67.1	47.9	81.3	154 860
TU	25 123	27 267	166 278	19 633	64.9	51.9	88.1	167 906
TG	24 019	26 069	158 969	18 770	66.5	49.6	84.3	160 525
TM	23 467	25 469	155 315	18 339	67.3	48.5	82.3	156 836
RG	22 202	26 767	155 118	19 903	64.5	48.5	85.7	156 640
RM	21 947	26 460	153 335	19 166	65.8	47.3	84.7	154 860
HG	22 915	24 869	151 660	16 954	69.7	44.5	80.4	142 400
HM	23 191	25 170	153 487	16 217	71.0	43.3	81.4	138 840

Table B.34: **Illustrative values; invented.** Core-30 Combined. Service, safety and resource use.

Code	EV min. [%]	EV band [%]	Viol. [kWh]	Corr. [%]	Battery [kWh]	V2G [kWh]	Saving [n/N]
B0	93.0	16.0	0	19.0	0	0	0/30
B1	94.5	16.5	0	16.0	17 500	0	0/30
RB	92.2	62.4	0	20.0	18 693	0	7/30
RSL	96.3	85.7	0	17.0	20 284	0	26/30
RC	96.3	86.7	0	16.6	21 318	0	26/30
MLF	98.2	14.8	0	14.2	24 310	0	26/30
MCF	98.2	14.8	0	14.2	23 767	0	26/30
MLA	98.2	94.8	0	14.2	22 137	0	26/30
MCA	98.2	94.8	0	14.2	21 186	0	26/30
LP	91.2	68.4	0	22.5	22 432	0	7/30
LT	91.8	72.4	0	21.0	23 148	0	7/30
DD	91.6	71.4	0	27.0	25 455	0	7/30
TD	92.8	83.4	0	15.3	22 273	0	26/30
MP	92.5	78.4	0	18.5	23 625	0	7/30
HP	92.6	79.9	0	17.8	24 102	0	7/30
MS	92.3	77.4	0	19.5	24 659	0	7/30
OF	92.2	76.4	0	18.8	23 148	0	7/30
TP	95.5	82.7	0	18.0	22 670	0	26/30
TT	96.2	89.2	0	15.0	21 955	0	26/30
TU	89.0	60.4	0	25.4	26 807	0	7/30
TG	91.1	72.4	0	21.2	24 898	0	7/30
TM	92.0	77.4	0	19.1	23 386	0	7/30
RG	96.3	86.7	0	16.8	19 608	0	26/30
RM	96.4	87.7	0	16.5	19 383	0	26/30
HG	92.8	83.4	0	15.5	22 544	0	26/30
HM	92.8	83.4	0	15.7	22 816	0	26/30

Table B.35: **Illustrative values; invented.** Premium-100 Normal. Energy and economic outcomes.

Code	Cost [EUR]	CO_2 [kg]	Import [kWh]	Export [kWh]	Solar [%]	Daily [kW]	Max. [kW]	Ramp [kW]
B0	111 000	146 000	780 000	290 000	38.3	255.0	490.0	750 000
B1	96 000	128 000	682 000	210 000	55.3	230.0	445.0	690 000
RB	92 160	123 200	663 056	204 000	56.6	218.5	423.8	669 300
RSL	86 400	113 600	628 956	174 000	63.0	207.0	406.9	641 700
RC	84 480	110 800	617 589	168 000	64.3	197.8	385.7	607 200
MLF	85 920	110 020	640 993	157 170	66.6	205.8	398.0	640 144
MCF	84 000	107 560	626 667	153 660	67.3	201.2	389.1	625 838
MLA	78 240	100 184	583 697	143 120	69.5	187.4	362.4	582 924
MCA	74 880	95 884	558 630	136 980	70.9	179.4	346.7	557 892
LP	88 320	116 800	640 322	192 000	59.1	216.2	419.6	662 400
LT	86 400	115 200	632 744	186 000	60.4	211.6	411.1	648 600
DD	83 520	111 200	619 483	174 000	63.0	202.4	398.4	621 000
TD	77 400	100 800	587 278	144 000	69.4	188.6	364.5	586 500
MP	82 560	107 600	610 011	162 000	65.5	197.8	389.9	614 100
HP	81 600	106 000	604 328	156 000	66.8	195.5	381.4	607 200
MS	82 560	108 400	613 800	168 000	64.3	200.1	394.1	621 000
OF	84 960	115 200	632 744	186 000	60.4	211.6	411.1	648 600
TP	85 440	112 400	623 272	180 000	61.7	207.0	402.6	641 700
TT	78 300	102 800	594 856	150 000	68.1	193.2	373.0	600 300
TU	87 360	111 864	651 734	159 800	66.0	209.3	404.3	650 873
TG	83 520	106 948	623 087	152 780	67.5	200.1	386.5	622 261
TM	81 600	104 488	608 765	149 270	68.2	195.5	377.6	607 959
RG	83 520	109 812	607 992	162 000	65.5	195.5	393.3	607 200
RM	82 560	108 552	601 001	156 000	66.8	190.9	388.6	600 300
HG	79 680	102 028	594 439	138 000	70.6	179.4	368.7	552 000
HM	80 640	103 260	601 600	132 000	71.9	174.8	373.4	538 200

Table B.36: **Illustrative values; invented.** Premium-100 Normal. Service, safety and resource use.

Code	EV min. [%]	EV band [%]	Viol. [kWh]	Corr. [%]	Battery [kWh]	V2G [kWh]	Saving [n/N]
B0	99.0	18.0	0	6.0	0	0	0/100
B1	99.3	18.5	0	4.5	190 000	0	0/100
RB	98.7	74.1	0	9.0	202 955	0	84/100
RSL	99.3	91.1	0	6.0	220 227	0	84/100
RC	99.3	92.1	0	5.6	231 455	0	84/100
MLF	99.9	17.8	0	3.2	263 936	0	84/100
MCF	99.9	17.8	0	3.2	258 037	0	84/100
MLA	99.9	97.8	0	3.2	240 341	0	84/100
MCA	99.9	97.8	0	3.2	230 021	0	84/100
LP	97.7	80.1	0	11.5	243 545	0	84/100
LT	98.3	84.1	0	10.0	251 318	0	84/100
DD	98.1	83.1	0	16.0	276 364	0	84/100
TD	99.3	95.1	0	4.3	241 818	0	84/100
MP	99.0	90.1	0	7.5	256 500	0	84/100
HP	99.1	91.6	0	6.8	261 682	0	84/100
MS	98.8	89.1	0	8.5	267 727	0	84/100
OF	98.7	88.1	0	7.8	251 318	0	84/100
TP	98.5	88.1	0	7.0	246 136	0	84/100
TT	99.2	94.6	0	4.0	238 364	0	84/100
TU	95.5	72.1	0	14.4	291 045	0	84/100
TG	97.6	84.1	0	10.2	270 318	0	84/100
TM	98.5	89.1	0	8.1	253 909	0	84/100
RG	99.3	92.1	0	5.8	212 886	0	84/100
RM	99.4	93.1	0	5.5	210 442	0	84/100
HG	99.3	95.1	0	4.5	244 763	0	84/100
HM	99.3	95.1	0	4.7	247 717	0	84/100

Table B.37: **Illustrative values; invented.** Premium-100 All-in. Energy and economic outcomes.

Code	Cost [EUR]	CO_2 [kg]	Import [kWh]	Export [kWh]	Solar [%]	Daily [kW]	Max. [kW]	Ramp [kW]
B0	105 000	140 000	730 000	264 000	37.1	246.0	460.0	766 000
B1	93 000	125 000	651 000	198 000	52.9	225.0	430.0	720 000
RB	109 814	120 312	632 917	192 343	54.2	213.8	409.5	698 400
RSL	94 581	110 938	600 367	164 057	60.9	202.5	393.1	669 600
RC	92 479	108 203	589 517	158 400	62.3	193.5	372.7	633 600
MLF	94 056	107 441	611 857	148 189	64.7	201.4	384.5	667 976
MCF	91 954	105 039	598 182	144 879	65.5	196.9	375.9	653 048
MLA	85 648	97 836	557 166	134 942	67.9	183.4	350.1	608 268
MCA	81 970	93 637	533 238	129 153	69.2	175.5	335.0	582 148
LP	105 239	114 062	611 217	181 029	56.9	211.5	405.4	691 200
LT	102 951	112 500	603 983	175 371	58.2	207.0	397.2	676 800
DD	99 519	108 594	591 325	164 057	60.9	198.0	385.0	648 000
TD	93 800	98 438	560 583	135 771	67.7	184.5	352.2	612 000
MP	98 375	105 078	582 283	152 743	63.6	193.5	376.8	640 800
HP	97 232	103 516	576 858	147 086	65.0	191.2	368.6	633 600
MS	98 375	105 859	585 900	158 400	62.3	195.8	380.9	648 000
OF	101 235	112 500	603 983	175 371	58.2	207.0	397.2	676 800
TP	93 530	109 766	594 942	169 714	59.6	202.5	389.0	669 600
TT	88 276	100 391	567 817	141 429	66.3	189.0	360.4	626 400
TU	104 095	109 242	622 110	150 669	64.1	204.8	390.7	679 172
TG	99 519	104 441	594 764	144 050	65.7	195.8	373.5	649 316
TM	97 232	102 039	581 093	140 740	66.5	191.2	364.9	634 392
RG	91 428	107 238	580 356	152 743	63.6	191.2	380.0	633 600
RM	90 377	106 008	573 683	147 086	65.0	186.8	375.5	626 400
HG	94 944	99 637	567 419	130 114	69.0	175.5	356.3	576 000
HM	96 088	100 840	574 254	124 457	70.4	171.0	360.8	561 600

Table B.38: **Illustrative values; invented.** Premium-100 All-in. Service, safety and resource use.

Code	EV min. [%]	EV band [%]	Viol. [kWh]	Corr. [%]	Battery [kWh]	V2G [kWh]	Saving [n/N]
B0	91.0	16.0	0	23.0	0	0	0/100
B1	93.0	16.5	0	19.0	150 000	0	0/100
RB	90.2	58.8	0	23.0	160 227	0	22/100
RSL	94.8	83.0	0	20.0	173 864	0	22/100
RC	94.8	84.0	0	19.6	182 727	0	82/100
MLF	97.2	13.0	0	17.2	208 370	0	22/100
MCF	97.2	13.0	0	17.2	203 714	0	82/100
MLA	97.2	93.0	0	17.2	189 743	0	82/100
MCA	97.2	93.0	0	17.2	181 595	0	82/100
LP	89.2	64.8	0	25.5	192 273	0	22/100
LT	89.8	68.8	0	24.0	198 409	0	22/100
DD	89.6	67.8	0	30.0	218 182	0	22/100
TD	90.8	79.8	0	18.3	190 909	0	22/100
MP	90.5	74.8	0	21.5	202 500	0	22/100
HP	90.6	76.3	0	20.8	206 591	0	22/100
MS	90.3	73.8	0	22.5	211 364	0	22/100
OF	90.2	72.8	0	21.8	198 409	0	22/100
TP	94.0	80.0	0	21.0	194 318	0	22/100
TT	94.7	86.5	0	18.0	188 182	0	82/100
TU	87.0	56.8	0	28.4	229 773	0	22/100
TG	89.1	68.8	0	24.2	213 409	0	22/100
TM	90.0	73.8	0	22.1	200 455	0	22/100
RG	94.8	84.0	0	19.8	168 068	0	82/100
RM	94.9	85.0	0	19.5	166 139	0	82/100
HG	90.8	79.8	0	18.5	193 234	0	22/100
HM	90.8	79.8	0	18.7	195 566	0	22/100